

OpenShift Virtualization cookbook

Version 1, 2026-08-12

目次

概要

本書はOpenShift Virtualization のインストールと設定に関するガイドです。

OpenShift

本書はOpenShift Virtualization のインストールと設定に関するガイドです。

[OpenShift Virtualization](#) のインストールと設定に関するガイド

(目次:)

目次

本書はOpenShift Virtualization のインストールと設定に関するガイドです。

- 本書: OpenShift Virtualization Operator のインストール OpenShift 4.18
- 本書: `oc` `kubectrl` `virtctl` (本書: `kustomize` `helm` `ansible`)
- 本書: YAML の CLI のインストール (本書: Kubernetes/OpenShift のインストール)

本書はOpenShift Virtualization のインストールと設定に関するガイドです。

インストールと設定

本書はOpenShift Virtualization のインストールと設定に関するガイドです。

- [インストール](#) - インストール
- [インストール](#) - インストール
- [インストール](#) - インストールIP
- [インストール](#) - VM のDNS
- [VM](#) [インストール](#) - CPU NUMA Hugepages
- [VM](#) [インストール](#) - インストール
- [インストール](#) - インストール
- [API](#) [インストール](#) - API

インストール

- OpenShift Virtualization
 - [インストール](#)
 - [インストール](#)
 - [インストール \(pdf\)](#)
 - [インストール](#)

◦ [OpenShift](#)

- [KubeVirt](#) [OpenShift](#)
- [cloud-init](#) [OpenShift](#)
- [OpenShift](#) - [OpenShift](#)

OpenShift Virtualization

OpenShift Virtualization [Issue](#) ocp-virt-cookbook@redhat.com

OpenShift

OpenShift [OpenShift](#) [Virtualization](#) [Onboarding](#) [Kit](#) [Alexandre Menezes](#) [Andrea Perotti](#) [Daphne Hoover](#) [Josh Manning](#) [Melvin Hillsman](#) [Roshan Warrier](#) [Yash Devan Oza](#)

OpenShift Virtualization [OpenShift](#) [Virtualization](#) [Onboarding](#) [Kit](#)

OpenShift

OpenShift [MIT](#) [OpenShift](#) [OpenShift](#)



OpenShift Virtualization [OpenShift](#) [Virtualization](#) [Onboarding](#) [Kit](#) [AI](#)

OpenShift Virtualization [OpenShift](#) [Virtualization](#) [Onboarding](#) [Kit](#) [AI](#)

OpenShift Virtualization

简介

OpenShift Virtualization 是 OpenShift 平台上的虚拟化功能，允许您在 OpenShift 集群中运行虚拟机 (VM)。

架构

OpenShift Virtualization 架构如下：

- OpenShift Virtualization 由 OpenShift 平台提供。
- 通过 `virtctl` 工具与虚拟机交互。
- 使用 `cloud-init` 配置 VM。

安装

OpenShift Virtualization 安装步骤如下：

- OpenShift Virtualization Operator 是 OpenShift 4.18+ 的一部分。
- 通过 RBAC 配置权限。
- 使用 CLI 工具：`oc` 和 `kubectl`。
- Kubernetes/OpenShift 集群。

使用

安装

OpenShift Virtualization 安装步骤如下：

OpenShift Virtualization 安装

通过 OpenShift Virtualization Operator 安装。

Virtctl 工具

通过 `virtctl` 工具与虚拟机交互。

配置

通过配置 OpenShift Virtualization 来运行 VM。

Web 界面与 VM 管理

通过 OpenShift Web 界面管理 VM。

VM 生命周期

通过 VM 生命周期管理 OpenShift Virtualization。

OpenShift Virtualization VM 安装 cloud-init 包

cloud-init 包是 OpenShift Virtualization 虚拟机中 cloud-init 包的安装包

安装步骤

安装 OpenShift Virtualization 虚拟机

- [安装 OpenShift Virtualization](#) - 安装 OpenShift Virtualization
- [安装 OpenShift Virtualization](#) - VM 安装 OpenShift Virtualization
- [cloud-init 包](#) - cloud-init 包

安装

安装

安装 OpenShift Virtualization 虚拟机

安装

OpenShift 安装

- OpenShift 4.18 安装
- OpenShift Virtualization Operator 安装

安装

安装

- **Cluster Admin:** Operator 安装
- **Namespace Admin:** 安装 namespace 的 VM 安装
- **Developer:** 安装 RBAC 安装 VM 安装

安装 CLI 包

OpenShift CLI (oc)

OpenShift 安装

安装: [OpenShift CLI 安装](#)

安装:

```
oc version
```

Kubernetes CLI (kubectl)

oc 環境を操作するコマンドは **kubectl** を使用します。

環境名: **Kubernetes** 環境

環境名:

```
kubectl version
```

Virtctl

virtctl は KubeVirt/OpenShift Virtualization 環境を操作するコマンドです。

環境名: **OpenShift Virtualization** 環境

環境名:

```
virtctl version
```

環境名

環境名

- **kustomize**: Kubernetes 環境
- **helm**: 環境
- **ansible**: 環境

環境名

環境名

環境名

- 環境 1 の StorageClass 環境
- VM 環境
- 環境

環境名:

```
oc get storageclass
```

環境名

環境名

- [OpenShift Container Platform 4.18+ VM 支持](#)
- Pod 支持 VM 支持
- VM 支持 OpenShift Container Platform 4.18+ VM 支持

❏

OpenShift Container Platform 4.18+ VM 支持

```
# Check cluster status
oc get nodes

# Verify OpenShift Virtualization operator
oc get csv -n openshift-cnv

# Check available VM templates
oc get datasources -n openshift-virtualization-os-images
```

❏

OpenShift Container Platform 4.18+ VM 支持

- [virtctl 支持](#)
- [OpenShift Virtualization 支持](#)

❏

- [OpenShift Virtualization 支持](#)
- [KubeVirt 支持](#)

OpenShift Virtualization 支持

❏

OpenShift Container Platform 4.18+ VM 支持 OpenShift Virtualization Operator KubeVirt

- [Web 支持](#)
- [\[getting-started:install-openshift-virtualization::支持\]](#)

❏

- **OpenShift 4.18+:** 支持 OpenShift Virtualization Operator KubeVirt
- **支持:** Operator 支持 OpenShift Virtualization Operator KubeVirt

□□□□□□□□:

OpenShift 4.18, 4.19, 4.20, 4.21

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

OpenShift

Virtualization

[illegible]

CPU

OpenShift

Virtualization



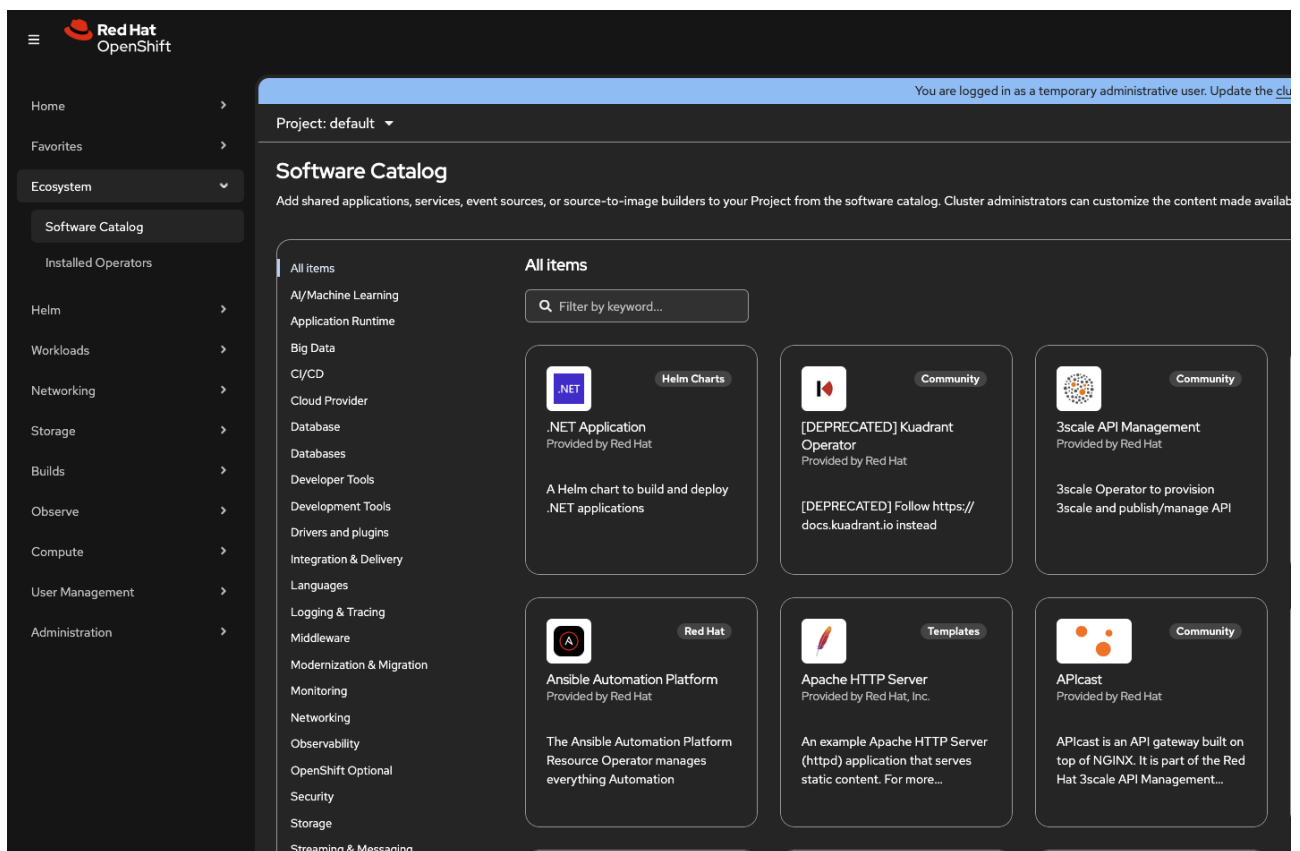
Operator

Web □□□□□□□□□□□□□□□□

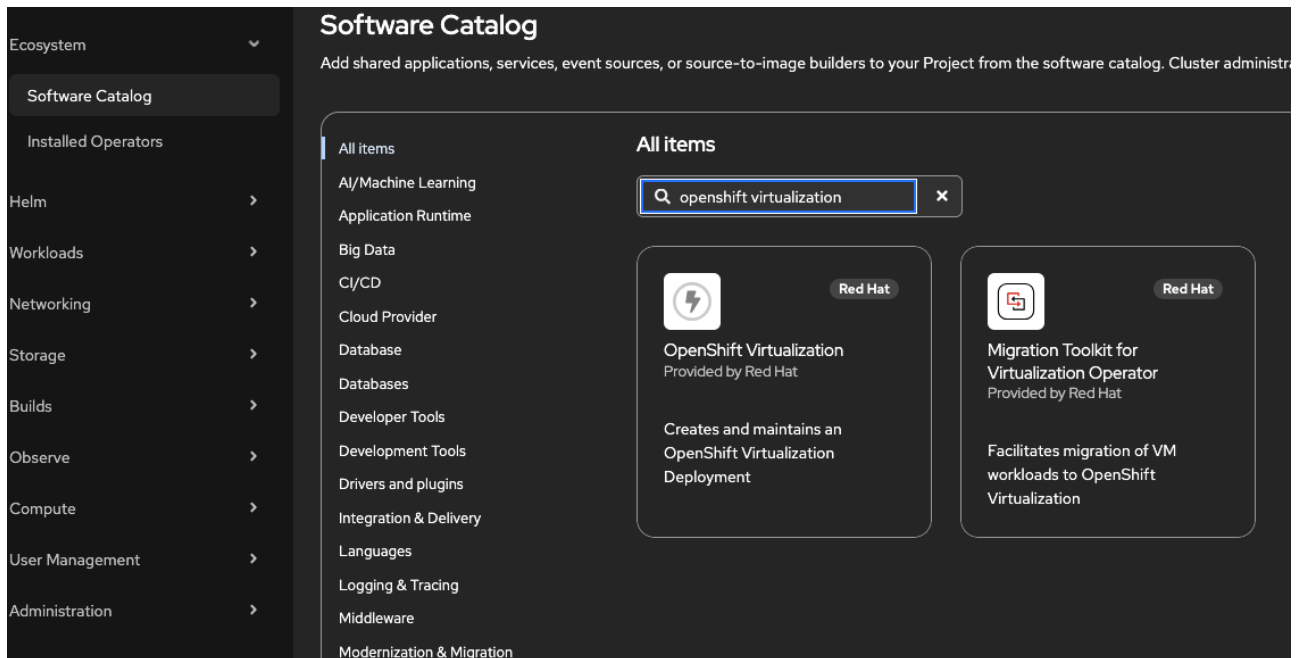
Web OpenShift Virtualization

□□□□ 1: OperatorHub □□□□□□

1. **cluster-admin** 管理 OpenShift Web 控制台
2. **Operators** → **OperatorHub** 管理

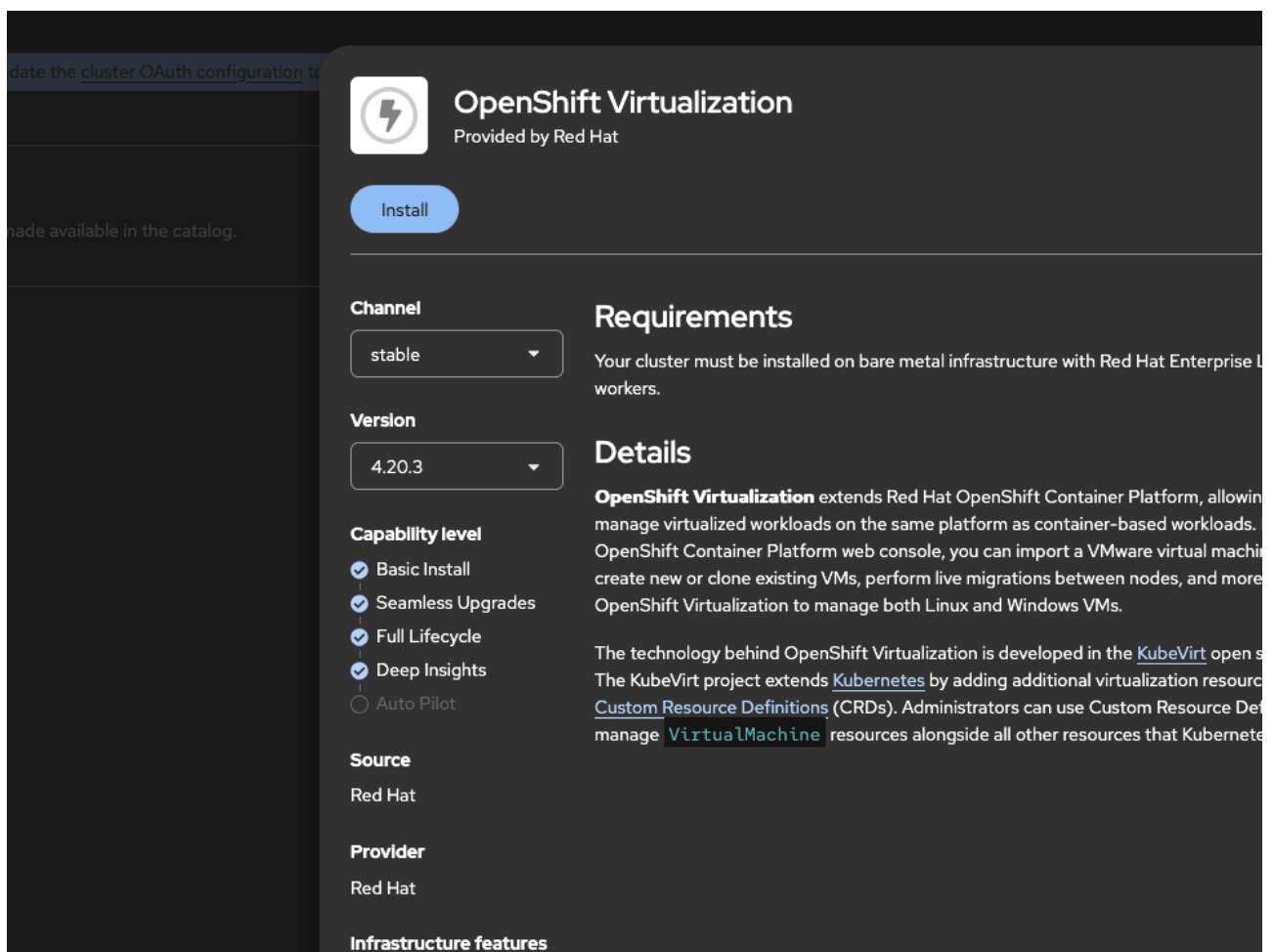


- ### 3. Filter by keyword

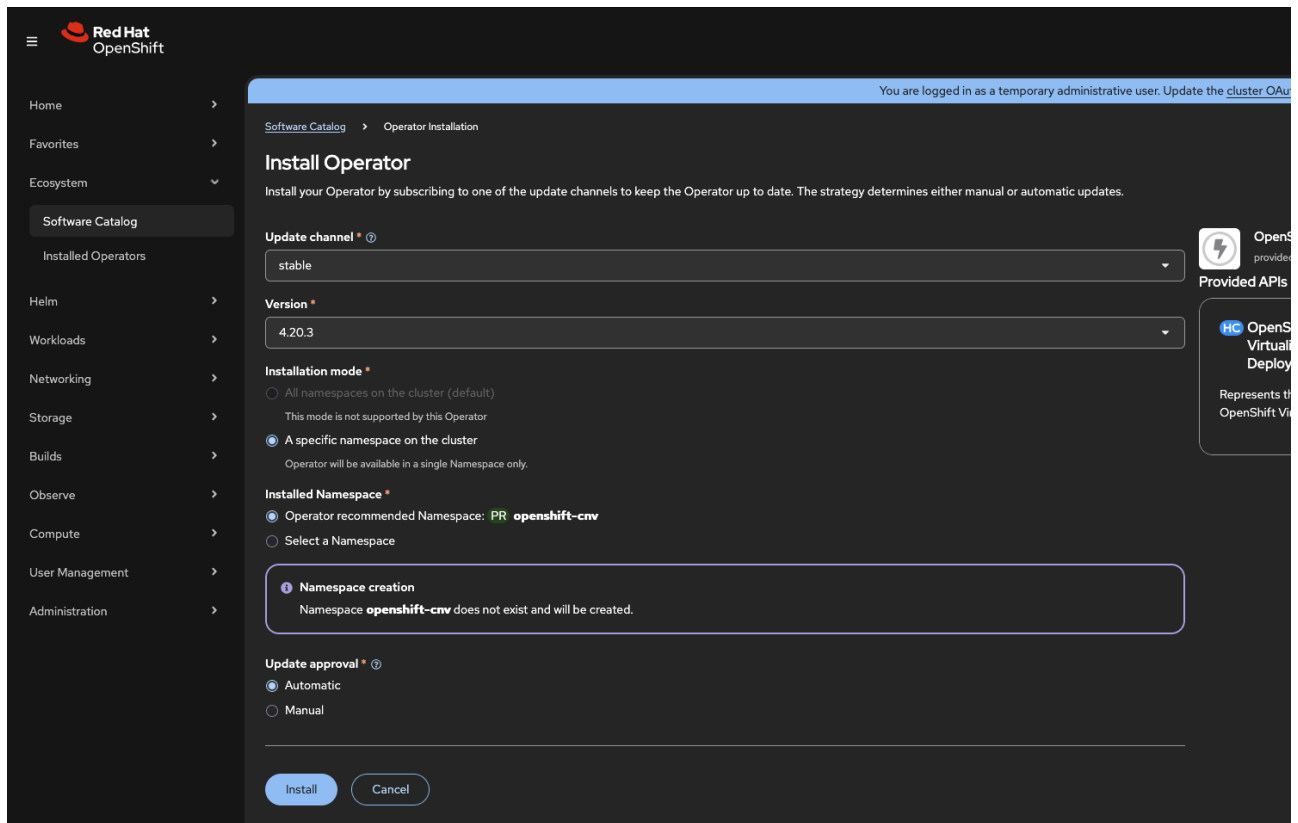


□□□□ 2: Operator □□□□□□□□

1. Red Hat □□□□□□□□□□ **OpenShift Virtualization** □□□□□□□□□□
2. Operator □□□□□□□□**Install** □□□□□□□□□□



3. Install Operator □□□□□□□□□□□□□□ **Install** □□□□□□□□□□



警告

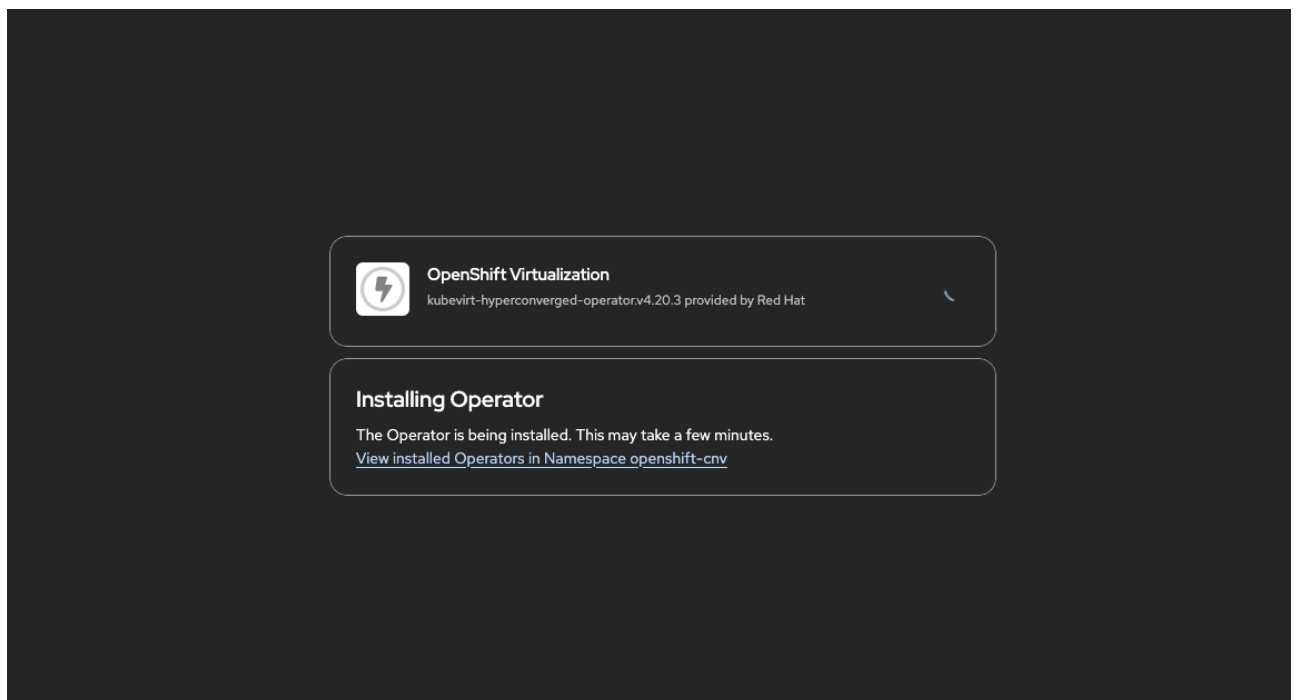
(Manual

approval)

インストール時に、Namespace **openshift-cnv** が存在しないため、自動的に作成されます。

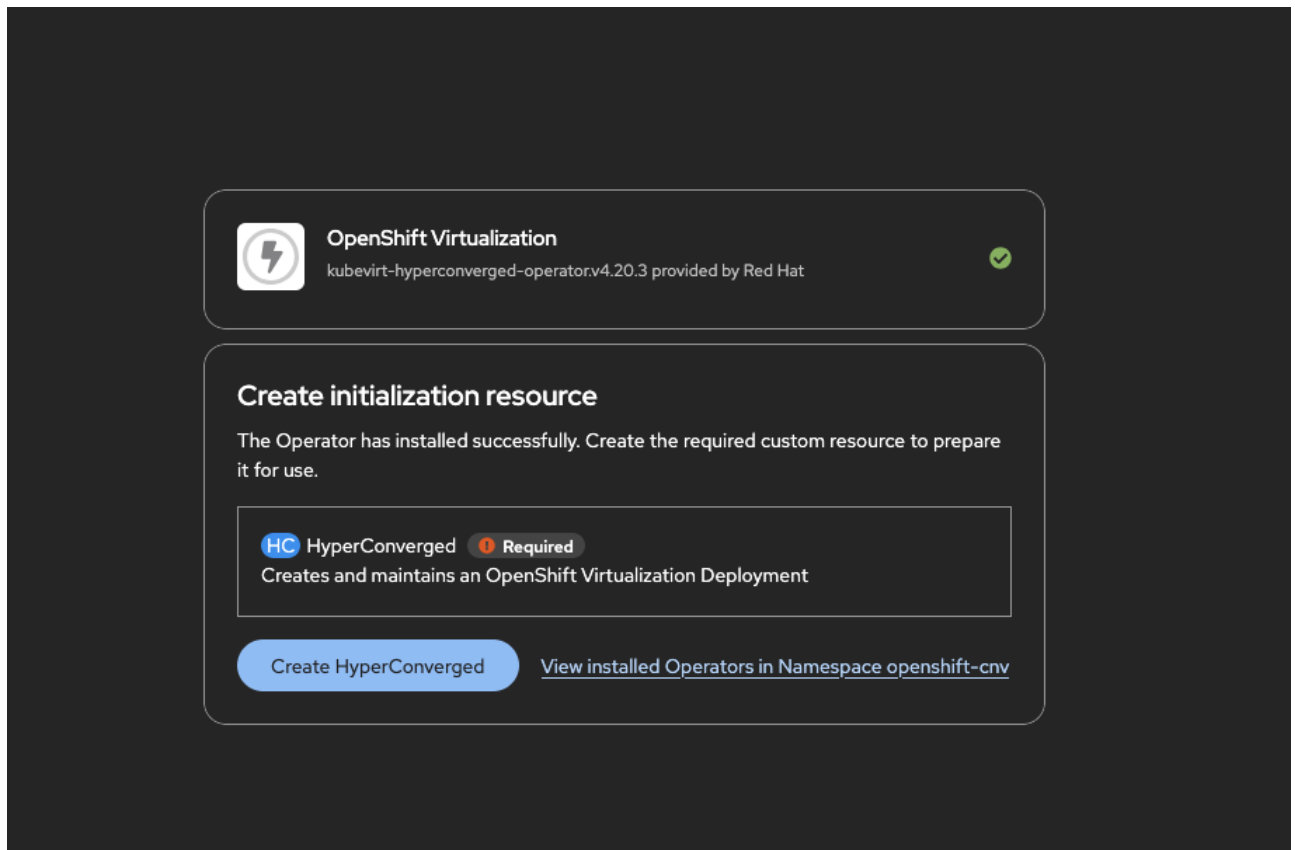
インストールが完了したら、

4. Operator インストール完了確認

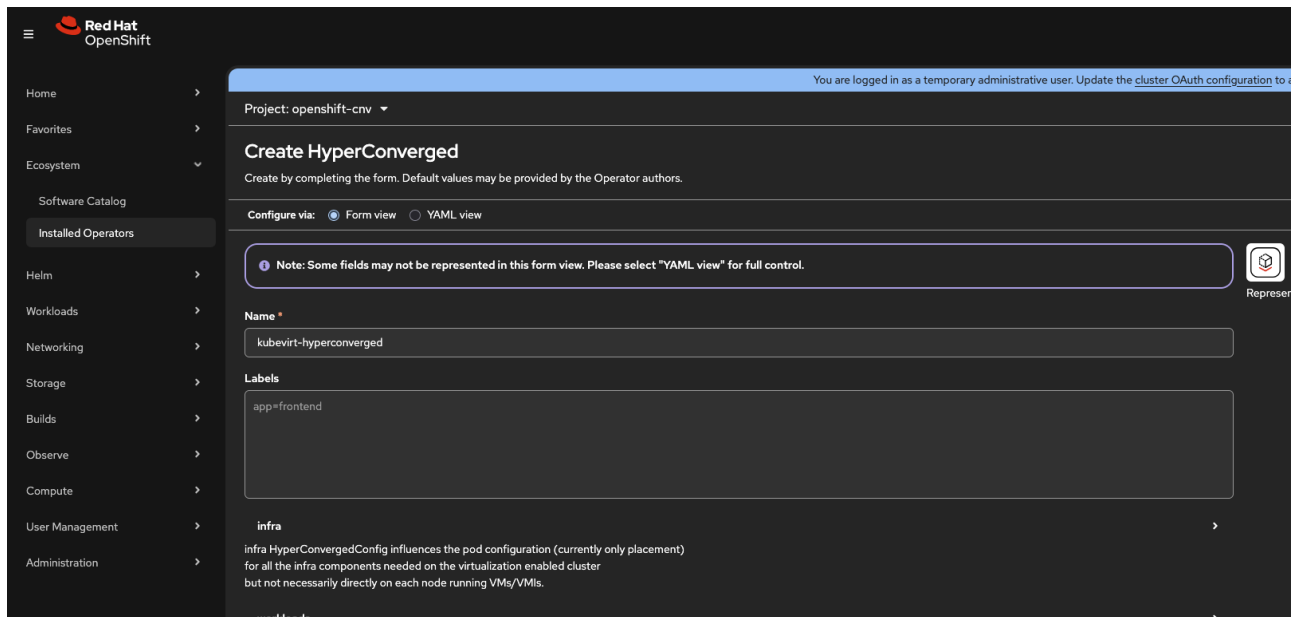


ステップ 3: HyperConverged インストール

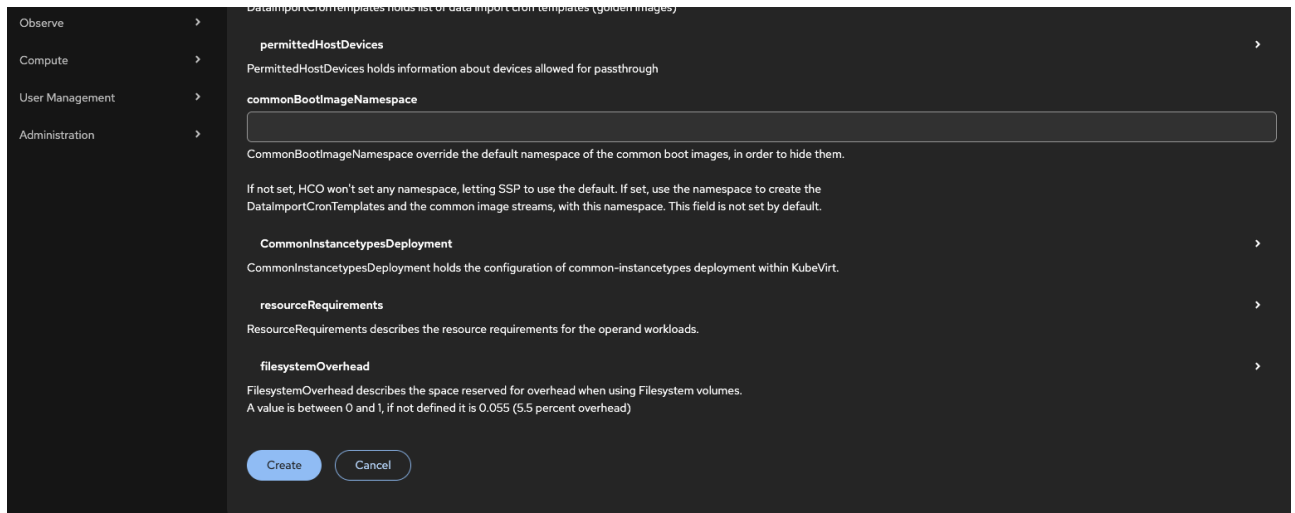
1. Operator インストール完了確認



2. 仮想マシンをインストールする

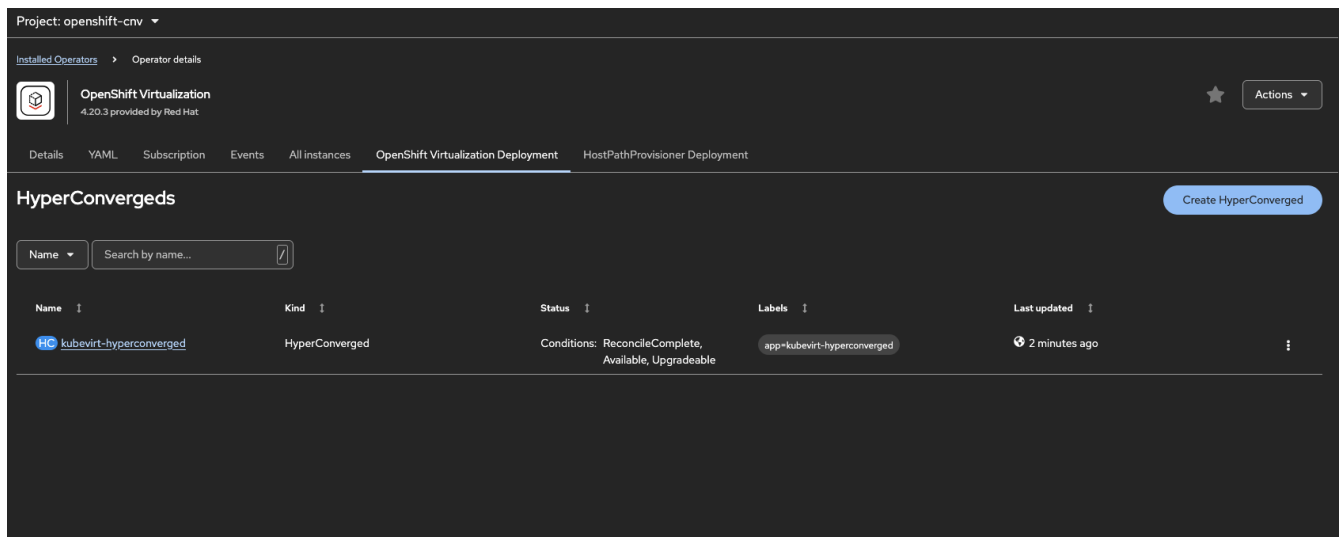


3. Create 仮想マシン OpenShift Virtualization 仮想マシン

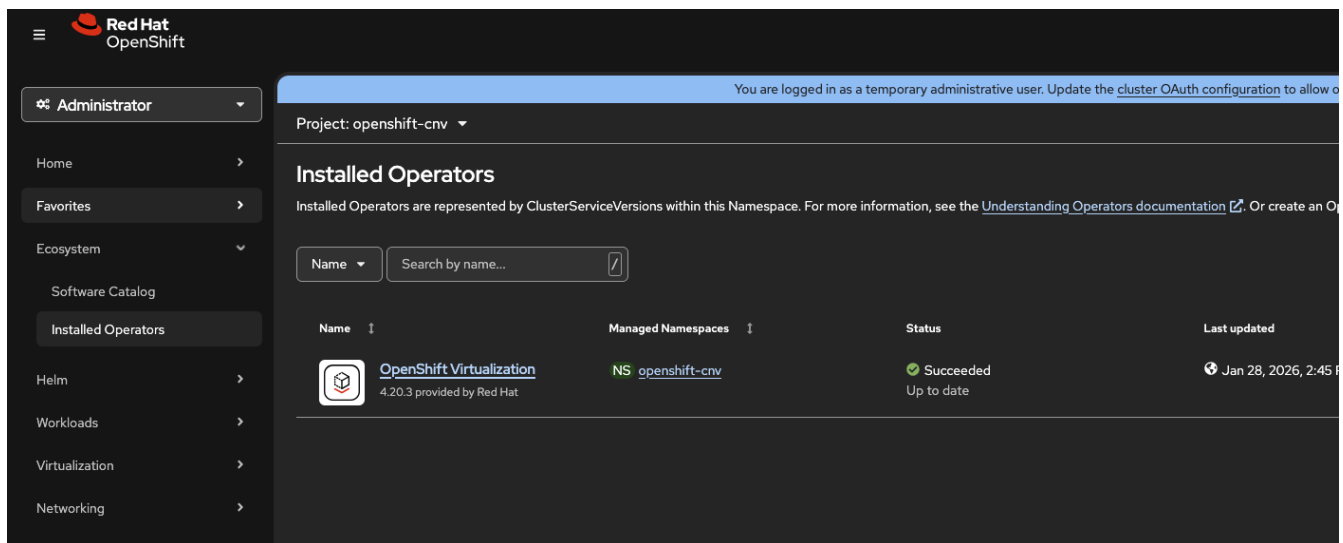


4: HyperConverged

HyperConverged



Operators



12

kubevirt-hyperconverged-operator.v4.20.1 Succeeded

Succeeded `Ctrl+C`

3: HyperConverged

HyperConverged

```
oc apply -f - <<EOF
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
EOF
```



For more information

HyperConverged

https://docs.openshift.com/container-platform/latest/virt/install/installing-virt.html#virt-customizing-storage-profile_installing-virt[`https://docs.openshift.com/container-platform/latest/virt/install/installing-virt.html#virt-customizing-storage-profile_installing-virt`,`window=_blank`]

4: HyperConverged

HyperConverged

```
oc wait hyperconverged kubevirt-hyperconverged -n openshift-cnv
--for=condition=Available --timeout=15m
```

510

5: HyperConverged

HyperConverged

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o jsonpath='{range .status.conditions[*]}.{.type}: {.status}}{"\n"}{end}'
```

:

```
ReconcileComplete: True
Available: True
Progressing: False
Degraded: False
Upgradeable: True
```

OpenShift Virtualization 準備作業

OpenShift

Virtualization

このガイドは、OpenShift Virtualization をインストールするための準備作業を示しています。



Operator は、OpenShift Virtualization のインストールと管理を行います。
HyperConverged は、OpenShift Virtualization のインストールと管理を行います。

ステップ 1: 環境の準備

OpenShift Virtualization VM のインストールと管理を行います。

```
# List all VMs across namespaces
oc get vm -A

# Delete VMs (repeat for each namespace)
oc delete vm --all -n <namespace>
```



OpenShift VM のインストールと管理を行います。

ステップ 2: HyperConverged のインストール

OpenShift Virtualization のインストールと管理を行います。

```
oc delete hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

OpenShift Virtualization のインストールと管理を行います。

```
# Watch pods being terminated (this may take 5-10 minutes)
oc get pods -n openshift-cnv -w
```

OpenShift Pod のインストールと管理を行います。

ステップ 3: Operator のインストール **CSV** のインストール

```
# Delete the subscription
oc delete subscription hco-operatorhub -n openshift-cnv

# Delete the ClusterServiceVersion
oc delete csv -n openshift-cnv -l operators.coreos.com/kubevirt-
hyperconverged.openshift-cnv
```

ステップ 4: Namespace のインストール (オプション)

OpenShift Virtualization Namespace のインストール

```
oc delete namespace openshift-cnv
```

ステップ 5: 確認

確認コマンド

```
# Should return empty or "No resources found"
oc get pods -n openshift-cnv
oc get csv -n openshift-cnv
```

完了

OpenShift Virtualization のインストールとアップグレード

前提条件

- [virtctl](#) のインストール - VM のインストール
- [OpenShift Virtualization](#) - VM のインストール
- [VM の IP](#) - cloud-init/DHCP のインストール、Sysprep のインストール、IP の設定

インストール

OpenShift Virtualization のインストールとアップグレード

- [OpenShift Virtualization](#) のインストール
- [OpenShift Virtualization](#) のアップグレード
- [KubeVirt](#) のインストール

インストールの検証

完了

OpenShift Virtualization のインストールとアップグレードの検証
(Fedora/RHEL/CentOS) DataSource のインストール
Stream/Windows DataSource のインストール (VM)
インストールの検証

インストールの完了:

OpenShift 4.19, 4.20

目標

前提条件

- **OpenShift** 環境: OpenShift Virtualization 環境に OpenShift 環境
- 権限: cluster-admin 権限を有するユーザ
- 容量: 少なくとも 1 GB のメモリ
- 環境: 仮想化環境にインストールされている

ステップ 1: 環境の準備

このステップでは、OpenShift Virtualization 環境に OpenShift 環境をインストールします。

```
oc get sc
```

このコマンドを実行すると、以下の出力が表示されます。

(default)

この出力から、OpenShift Virtualization 環境に OpenShift 環境がインストールされていることが確認できます。

この出力から、OpenShift Virtualization 環境に OpenShift 環境がインストールされていることが確認できます。

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

この出力から、OpenShift Virtualization 環境に OpenShift 環境がインストールされていることが確認できます。

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc (default)	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

この出力から、OpenShift Virtualization 環境に OpenShift 環境がインストールされていることが確認できます。

1. OpenShift Virtualization 環境に OpenShift 環境がインストールされていることを確認します。
2. OpenShift Virtualization 環境に OpenShift 環境がインストールされていることを確認します。

ステップ 2: デフォルトストレージクラス (確認)

デフォルトストレージクラスを確認するには、`kubectl` コマンドを実行します。1. デフォルトストレージクラスを確認

デフォルトストレージクラスを確認するには、`kubectl` コマンドを実行します。

```
oc annotate sc <the-old-default-storage-class-name> storageclass.kubernetes.io/is-default-class-
```



デフォルトストレージクラスを確認するには、`kubectl` コマンドを実行します。

ステップ 3: デフォルトストレージクラスを設定

デフォルトストレージクラスを設定するには、`kubectl` コマンドを実行します。OpenShift のデフォルト

デフォルトストレージクラス: `storageclass.kubernetes.io/is-default-class: "true"`

```
oc annotate sc <your-storage-class-name> storageclass.kubernetes.io/is-default-class="true"
```

デフォルトストレージクラス `ceph-rbd-sc` を確認:

```
oc annotate sc ceph-rbd-sc storageclass.kubernetes.io/is-default-class="true"
```

ステップ 4: デフォルトストレージクラスを確認

デフォルトストレージクラスを確認するには、`kubectl` コマンドを実行します。

```
oc get sc
```

デフォルトストレージクラス:

NAME	PROVISIONER	AGE	RECLAIMPOLICY
VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION		
ceph-rbd-sc (default)	rbd.csi.ceph.com		Delete
Immediate	true	2d	
hostpath-provisioner	kubevirt.io/hostpath-provisioner		Delete
WaitForFirstConsumer	false	1d	
local-storage	kubernetes.io/no-provisioner		Delete
WaitForFirstConsumer	false	1d	

デフォルトストレージクラス (default) デフォルトストレージクラスを確認

例 5: PersistentVolumeClaim の作成

PersistentVolumeClaim (PVC) の作成

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-default-storage
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

PVC の作成を確認する

```
oc get pvc test-default-storage -n default
```

結果:

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
test-default-storage	Pending				lvms-vg1	0s

PVC のステータスが **Pending** の場合は、(ストレージクラス **lvms-vg1**) のリソースが不足している

リソースの不足を確認する

PVC のステータスが **Pending** の場合は、**WaitForFirstConsumer** のリソースが不足している

リソースの種類	リソースの状況	リソースの種類
WaitForFirstConsumer	PVC が Pod を作成するまで PV を作成しない	Pod リソースが不足している場合は、(LVMストレージクラス) を作成する
Immediate	PVC が作成された時点で PV を作成する	リソースが不足している場合は、(NFSやCephストレージクラス) を作成する

リソースの不足を確認する

```
oc get sc <storage-class-name> -o jsonpath='{.volumeBindingMode}'
```

例 6: KubeVirt VM の定義

以下の YAML を使用して、KubeVirt VM を定義します。

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: test-vm-default-storage
  namespace: default
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: test-vm-disk
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
  template:
    metadata:
      labels:
        kubevirt.io/vm: test-vm-default-storage
    spec:
      domain:
        devices:
          disks:
            - name: datavolumedisk
              disk:
                bus: virtio
          resources:
            requests:
              memory: 1Gi
              cpu: 1
          networks:
            - name: default
              pod: {}
          volumes:
            - name: datavolumedisk
              dataVolume:
                name: test-vm-disk
EOF
```


2. `storageclass.kubernetes.io/is-default-class`:
 storageclass.kubernetes.io/is-default-class
3. `storageclass.kubernetes.io/is-default-class`: `storageclass.kubernetes.io/is-default-class`
4. `storageclass.kubernetes.io/is-default-class`: `storageclass.kubernetes.io/is-default-class`

OpenShift

OpenShift Persistent Storage

- `storageclass.kubernetes.io/is-default-class`
- `storageclass.kubernetes.io/is-default-class`
- `storageclass.kubernetes.io/is-default-class` `storageclass.kubernetes.io/is-default-class`
- PVC VM `storageclass.kubernetes.io/is-default-class`
- OpenShift Virtualization `storageclass.kubernetes.io/is-default-class`

OpenShift

- [OpenShift - Persistent Storage](#)
- [OpenShift Virtualization - Storage](#)

Web OpenShift VM

OpenShift

OpenShift Web `storageclass.kubernetes.io/is-default-class` (VM) `storageclass.kubernetes.io/is-default-class` Web `storageclass.kubernetes.io/is-default-class` VM

OpenShift

- OpenShift Virtualization Operator `storageclass.kubernetes.io/is-default-class` OpenShift 4.18 `storageclass.kubernetes.io/is-default-class` ([OpenShift Virtualization](#) `storageclass.kubernetes.io/is-default-class`)
- VM `storageclass.kubernetes.io/is-default-class` OpenShift Web `storageclass.kubernetes.io/is-default-class`
- VM `storageclass.kubernetes.io/is-default-class` default StorageClass
- `storageclass.kubernetes.io/is-default-class` (DataSource)

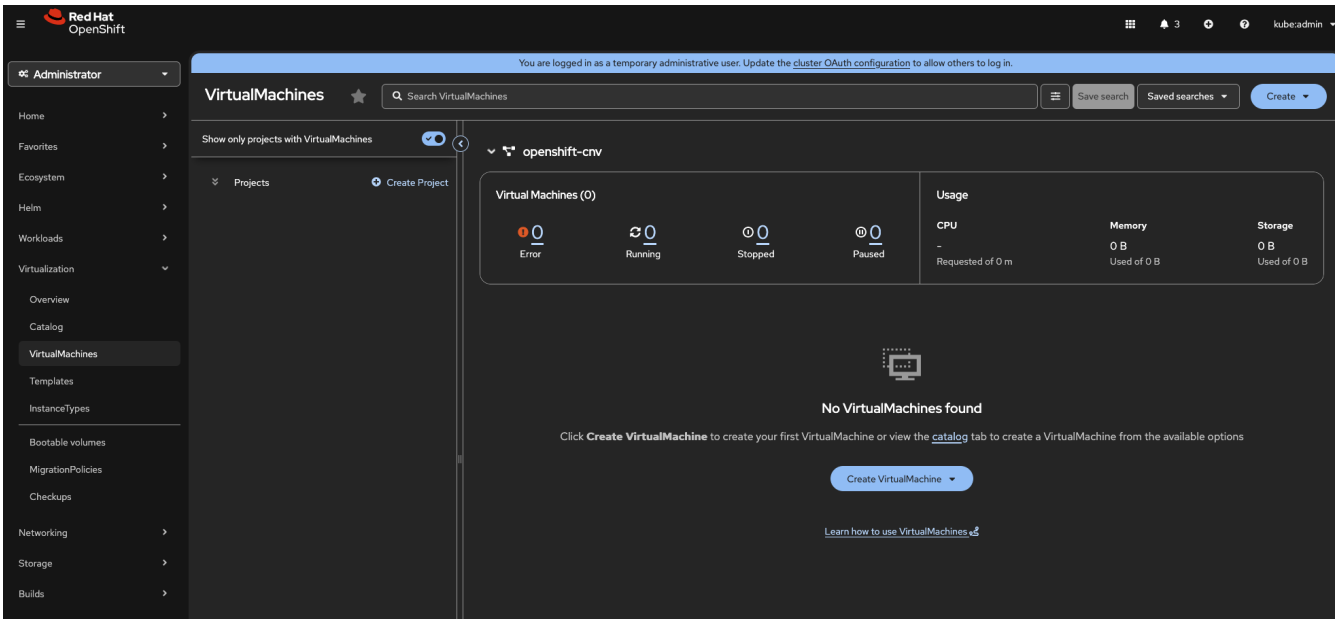
OpenShift

```
oc get datasources -n openshift-virtualization-os-images
```

OpenShift 1: OpenShift Persistent Storage

1. OpenShift Web `storageclass.kubernetes.io/is-default-class`
2. `storageclass.kubernetes.io/is-default-class` **Virtualization** `storageclass.kubernetes.io/is-default-class`

3. VirtualMachines

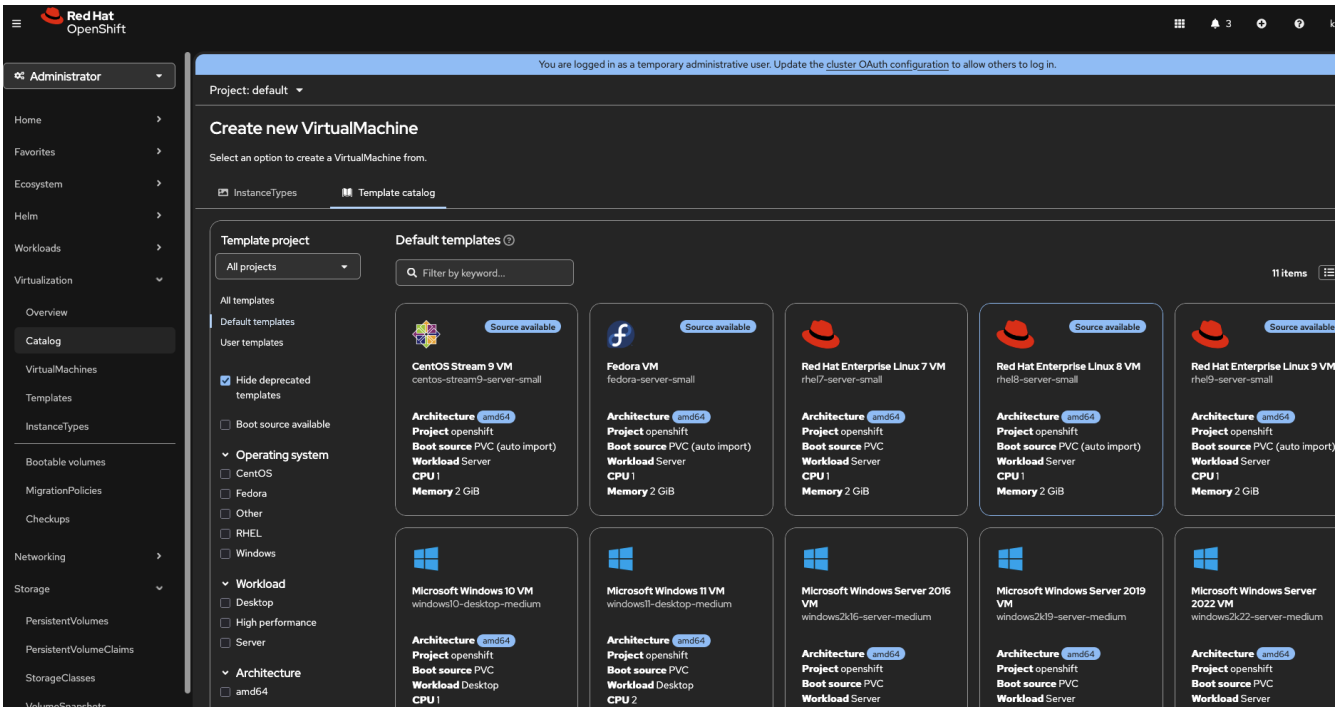


Virtual Machines

Figure 2:

OpenShift Virtualization

1. VirtualMachines **Create** → **From template**
2.
 - **Red Hat Enterprise Linux** - RHEL
 - **Fedora** - Fedora
 - **CentOS Stream** - CentOS Stream
 - **Microsoft Windows** - Windows Server



Source available

default StorageClass

~~~~~

- ~~~~~ CPU ~~~~~
- OS ~~~~~
- ~~~~~ (DataSource)
- cloud-init ~~~~ sysprep ~~~ (~~~~)

### ~~~~~ 3: ~~~~~ VM ~~~

~~~~~ Fedora VM ~~~~~

~~~~~

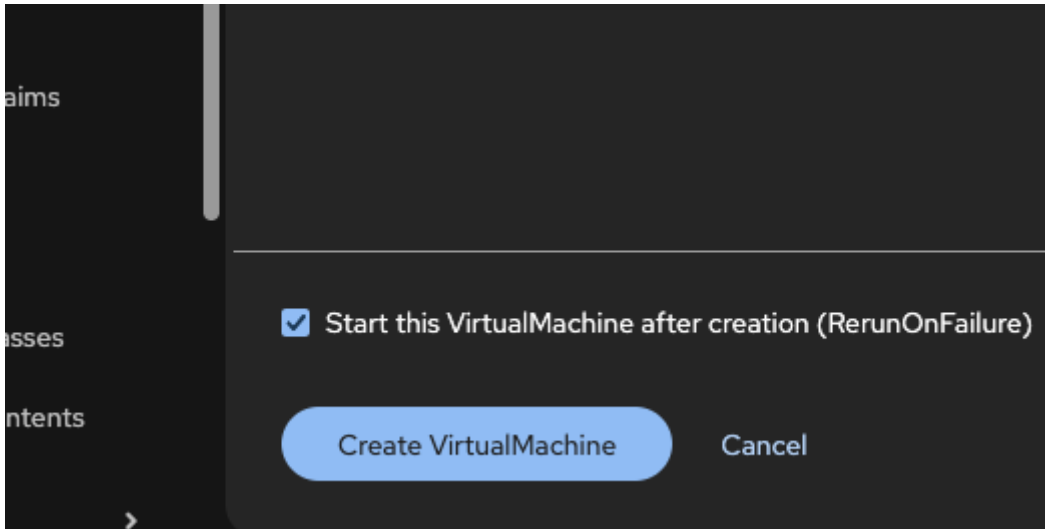
1. **Fedora** ~~~~~
2. ~~~~~
3. **Customize VirtualMachine** ~~~~~





□□□□

1. □□□□□□□□□□□□□□
2. **Create VirtualMachine** □□□□□□□□
3. □□□□□□**Start this VirtualMachine after creation** □□□□□□□□□□□□□□□□□□□□



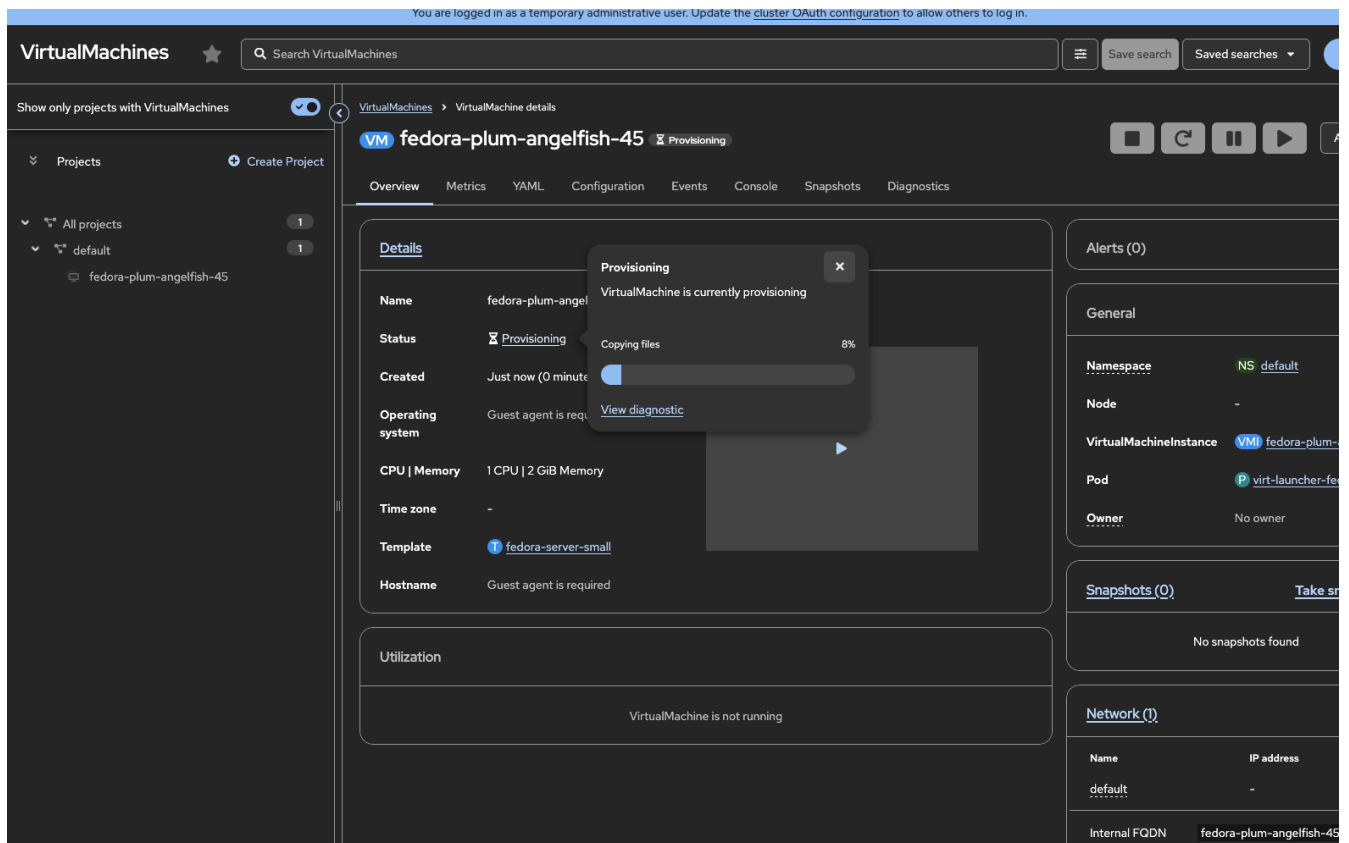
#### □□□□ 4: VM □□□□

□□□□□□□□□□VM □□□□□□□□□□□□□□□□

□□□□□□□□□□□□

VM □□□□□□□□□□□□□□□□□□□□

1. **Provisioning** - □□□□□□□□□□□□□□□□□□□□□□
2. **Starting** - VM □□□□□□□□ (□□□□□□□□□□□□□□□□)
3. **Running** - VM □□□□□□□□□□□□



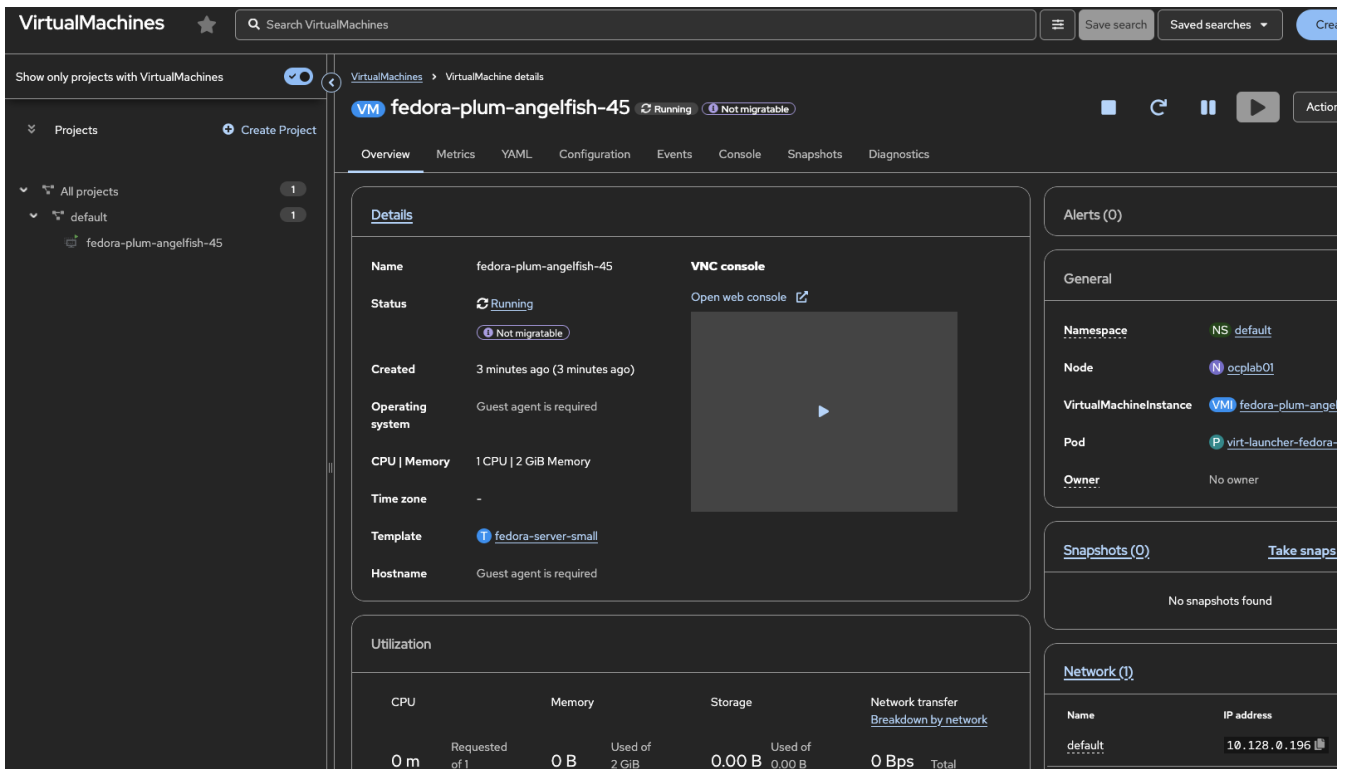
## Events

```
# Alternatively, watch events from CLI
oc get events -n <your-namespace> --field-selector involvedObject.name=<vm-name> -w
```

## VM

Running



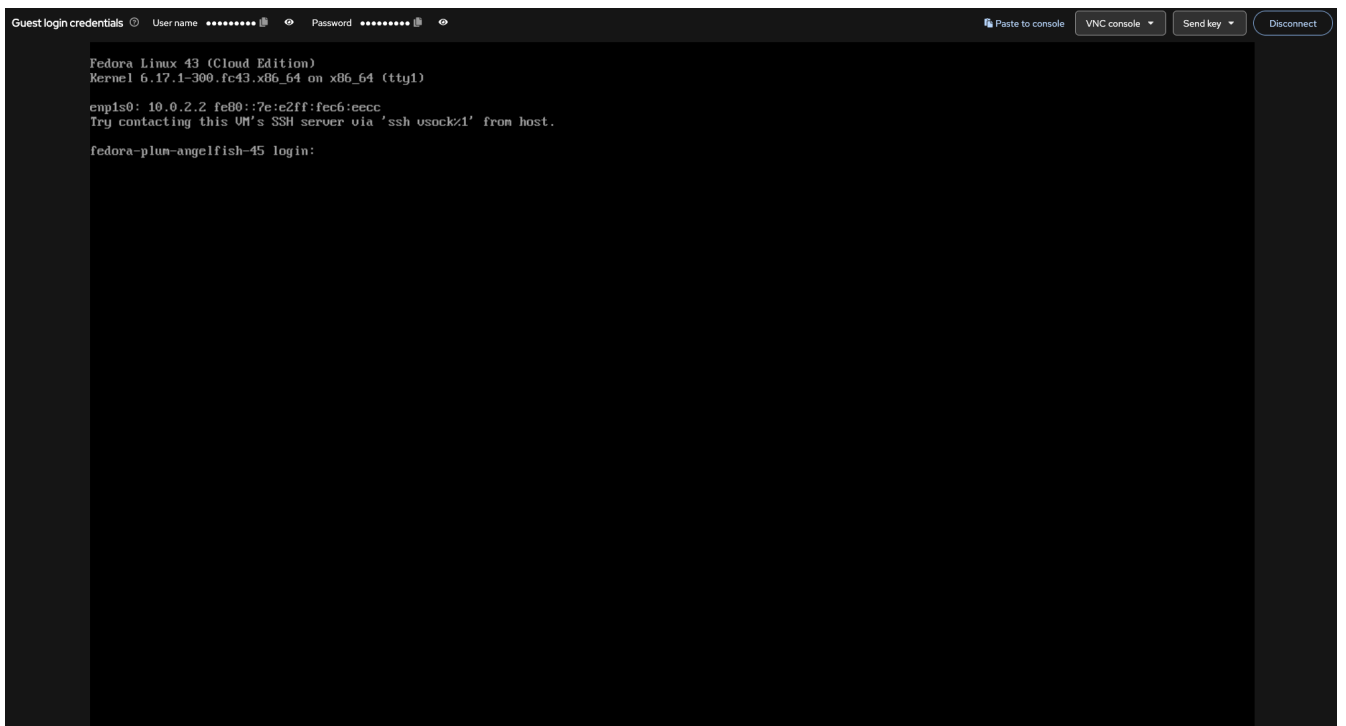


## 5: VM 操作

Web 操作 VM 操作

操作

1. VM Console 操作
2. VNC 操作



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

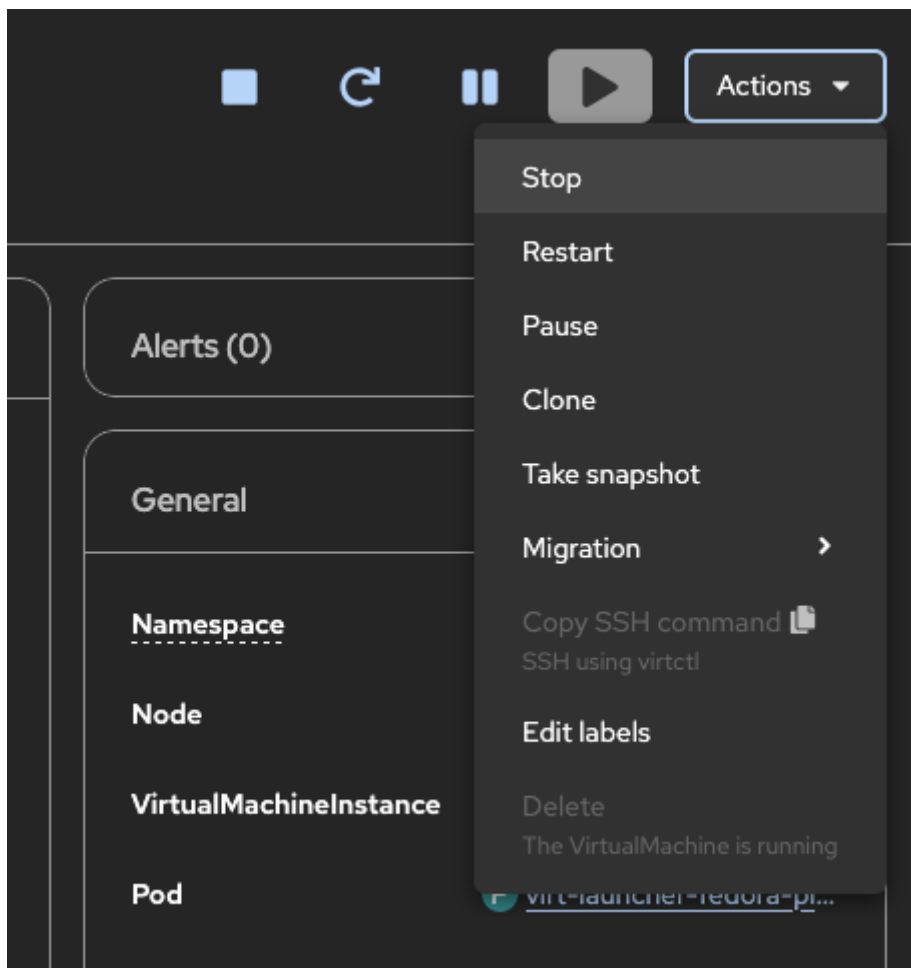
- 

□ □ □ □ □ □ □ □ □

## Figure 6: VMs

VM 0000000000

# VM 虚拟机 VirtualMachines 虚拟机



## VM 停止

1. **Actions** → **Stop** をクリック
2. 確認メッセージが表示されます
3. VM が停止されます

```
# CLI equivalent
virtctl stop <vm-name>
```

## VM 再起動

1. **Actions** → **Restart** をクリック
2. VM が再起動されます

```
# CLI equivalent
virtctl restart <vm-name>
```

## VM 一時停止

## VM 一時停止の確認

1. **Actions** → **Pause** をクリック

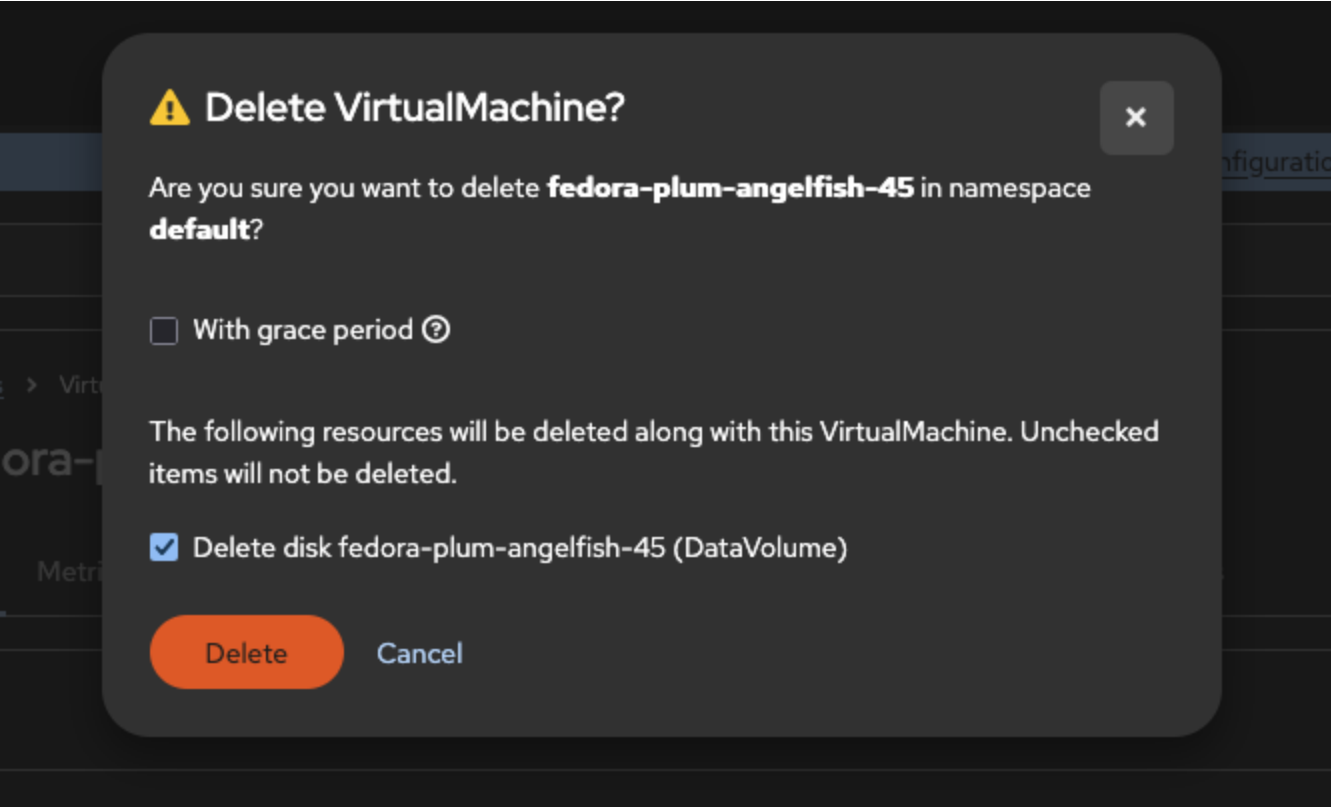
- 2. VM 状態を確認する
- 3. **Actions** → **Unpause** をクリックする



この VM は正常に起動しています。

VM の削除

- 1. **Actions** → **Delete** をクリックする
- 2. 削除対象の PVC (PVC) を確認する
- 3. 削除を確認する





この VM のディスク (DataVolume) を削除するかどうかを確認する。Delete disk



この VM は削除されています。

タスク 7: VM の削除

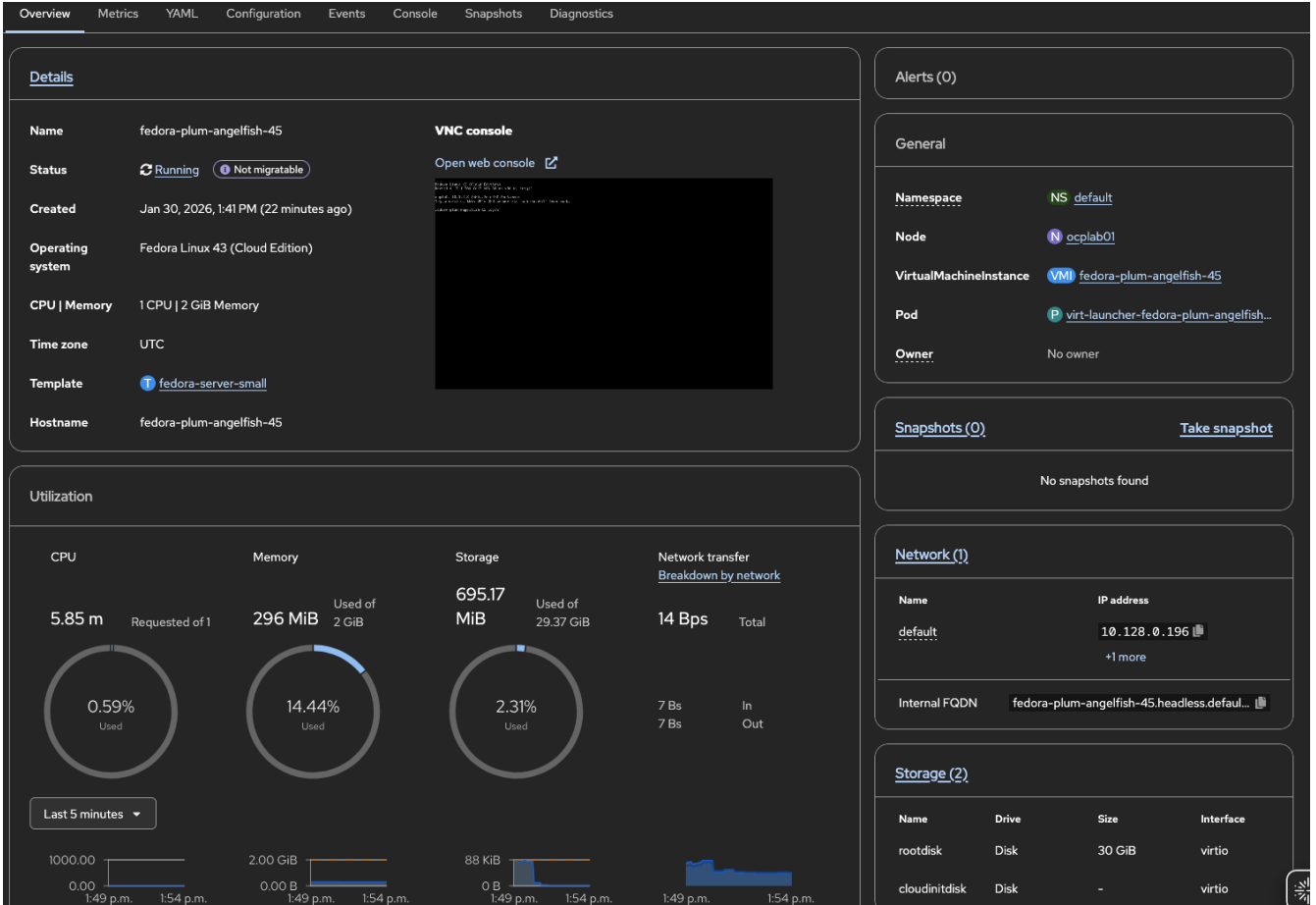
VM の削除を確認する

Overview

この VM の詳細を確認する

- VM の名前
- IP アドレス (パブリック/プライベート)

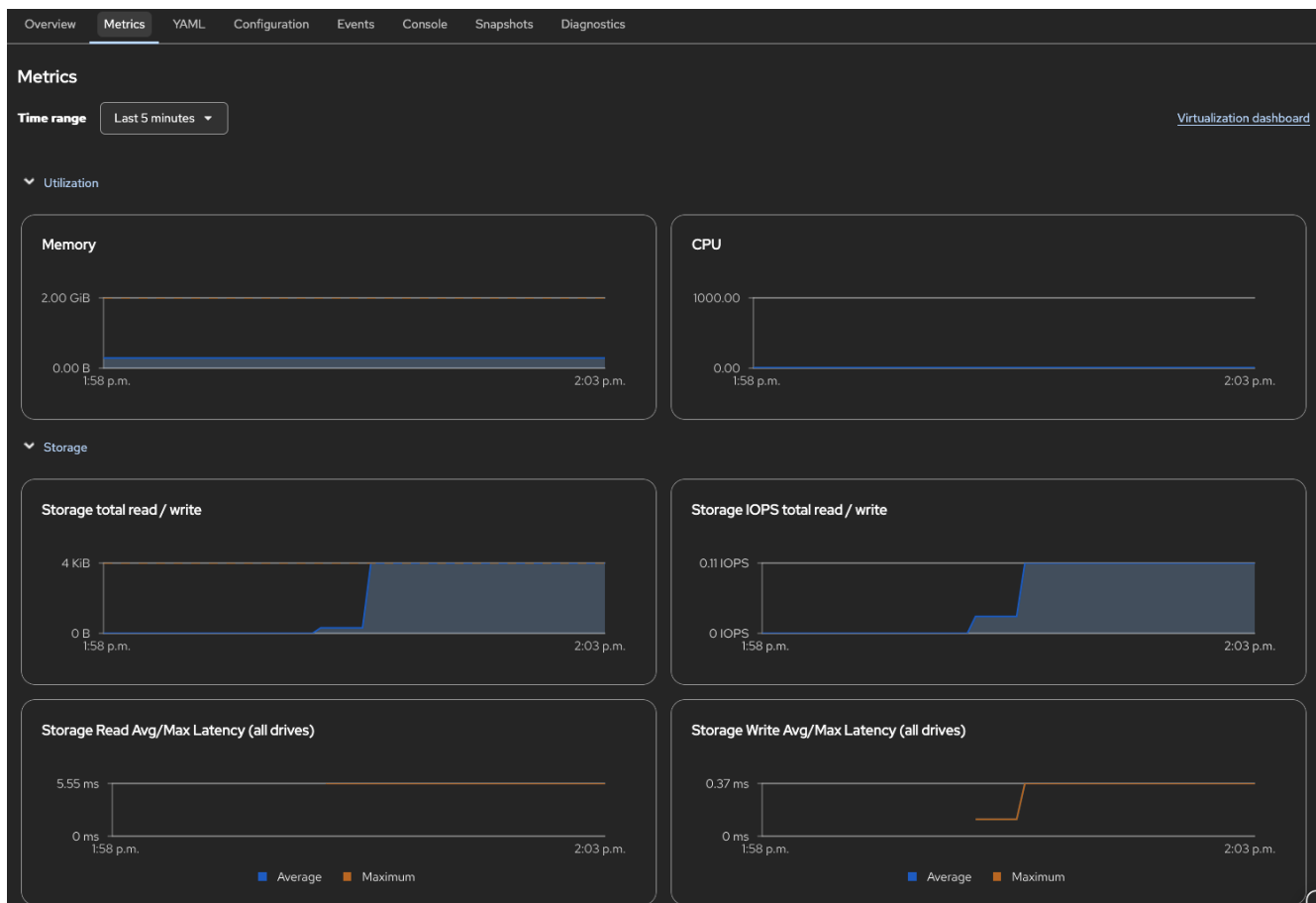
- □□□□□□□□



## Metrics ☐

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- 外部 I/O



この図は、VM の QEMU guest agent が提供するメトリクスを示しています。

## YAML

VM の設定を YAML で定義します。

1. **YAML** の定義
2. **VirtualMachine** の作成
3. **Save** の実行

VirtualMachines > VirtualMachine details

**VM fedora-plum-angelfish-45** Running Not migratable

Overview Metrics **YAML** Configuration Events Console Snapshots Diagnostics

```
1 apiVersion: kubevirt.io/v1
2 kind: VirtualMachine
3 metadata:
4   annotations:
5     kubemacpool.io/transaction-timestamp: '2026-01-30T18:41:32.629136522Z'
6     kubevirt.io/latest-observed-api-version: v1
7     kubevirt.io/storage-observed-api-version: v1
8     vm.kubevirt.io/validations: |
9
10    [
11      {
12        "name": "minimal-required-memory",
13        "path": "jsonpath:..spec.domain.memory.guest",
14        "rule": "integer",
15        "message": "This VM requires more memory.",
16        "min": 2147483648
17      }
18    ]
19 resourceVersion: '1240709'
20 name: fedora-plum-angelfish-45
21 uid: 7142897f-4100-4a71-9230-89b56da92b34
22 creationTimestamp: '2026-01-30T18:41:32Z'
23 generation: 1
24 managedFields: ~
25 namespace: default
26 finalizers:
27   - kubevirt.io/virtualMachineControllerFinalize
28 labels:
29   app: fedora-plum-angelfish-45
30   kubevirt.io/dynamic-credentials-support: 'true'
31   vm.kubevirt.io/template: fedora-server-small
32   vm.kubevirt.io/template.namespace: openshift
33   vm.kubevirt.io/template.revision: '1'
34   vm.kubevirt.io/template.version: v0.34.1
35 spec:
36   dataVolumeTemplates:
37     - apiVersion: cdi.kubevirt.io/v1beta1
38       kind: DataVolume
39       metadata:
40         creationTimestamp: null
41         name: fedora-plum-angelfish-45
42       spec:
43         sourceRef:
```

Save Reload Cancel Download



**Download** `fedora-plum-angelfish-45.yaml` to your local machine

How to create a VirtualMachine

How to create a VirtualMachine from a template

1. VirtualMachines **Create** → **From template**
- 2.
3. **Customize VirtualMachine** **Quick create VirtualMachine**
4. VM

administrative user, U

source available

auto import)

IVM  
medium

Fedora VM  
fedora-server-small

Template info

Operating system

Fedora VM

Workload type

Server (default)

Description

Template for Fedora Linux 40 VM or newer. A PVC with the Fedora disk image must be available.

Documentation

[Refer to documentation](#)

CPU | Memory

1 CPU | 2 GiB Memory

Network interfaces (1)

Name	Network	Type
default	Pod networking	Masquerade

Disks (2)

Name	Drive	Size
rootdisk	Disk	30 GiB
cloudinitdisk	Disk	-

Storage

☐ Boot from CD

Disk source

Template default

Disk size

-

30

+

GiB

Drivers

☐ Mount Windows drivers disk

Optional parameters

Quick create VirtualMachine

VirtualMachine name

fedora-plum-angelfish-45

Project

default

Public SSH key

[Not configured](#)

☒ Start this VirtualMachine after creation (RerunOnFailure)

Quick create VirtualMachine

Customize VirtualMachine

Cancel

□□□□□□□□□□□□□□□□

- □□□□□□□□
- □□□□□□□□□□□□□□□□
- □□□□□□□□□□

1111

□□□□□□□□□□

- Web ــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــ
- VM ــــــــــــــــــــــــــــــــــــــــــــــــــــــــ



- [OpenShift VM 简介](#)
- [VM 简介](#)
- [Web 简介](#) VM 简介
- [OpenShift VM 简介 \(OpenShift VM 简介\)](#)
- [VM 简介](#)

## OpenShift

- [virtctl CLI 简介](#) - OpenShift VM 简介
- [IP 简介](#) - VM 简介 IP 简介
- [SSH 简介](#) - SSH 简介 Linux VM 简介

## OpenShift

- [OpenShift 简介](#) - OpenShift 简介
- [OpenShift 简介](#) - OpenShift 简介

## VM 简介

### 简介

OpenShift Virtualization 是 OpenShift 平台的一部分，它允许您在 OpenShift 平台上运行虚拟机 (VM)。OpenShift Virtualization 使用 `VirtualMachine` 和 `VirtualMachineInstance` 资源来管理 VM。OpenShift Virtualization 是 OpenShift 平台的一部分，它允许您在 OpenShift 平台上运行虚拟机 (VM)。

OpenShift Virtualization:

OpenShift 4.20

### 简介

- OpenShift Virtualization 是 OpenShift 4.18 的一部分
- OpenShift Web 控制台 CLI 工具 (`oc` `virtctl`) 是 OpenShift Virtualization 的一部分
- OpenShift Virtualization VM 简介 1 简介 ([Web 简介](#) VM 简介 简介)

## VirtualMachine 和 VirtualMachineInstance 简介

OpenShift Virtualization 是 OpenShift 平台的一部分，它允许您在 OpenShift 平台上运行虚拟机 (VM)。

### VirtualMachine (VM)

`VirtualMachine` 是 OpenShift VM 资源，它用于管理 VM。

- VM 简介 namespace



번호	상태	설명	VMI 정보
1	Stopped (중지)	VM 프로세스가 완전히 종료된 상태	번호
2	Pending (대기) (예약)	VMI 프로세스가 예약된 상태 예약 대기 중	번호
3	Scheduling (예약) (대기)	Kubernetes 스케줄러가 VM을 할당하는 중	번호
4	Starting (시작)	프로세스가 VM 프로세스를 시작	번호
5	Running (실행) (중)	VM 프로세스가 실행 중	번호
6	Paused (중지) (대기)	VM 프로세스가 일시적으로 중지된 상태	번호
7	Stopping (중지) (대기)	프로세스가 종료 중	번호
8	Terminating (종료) (대기)	VMI 프로세스가 종료 중	번호 (종료)

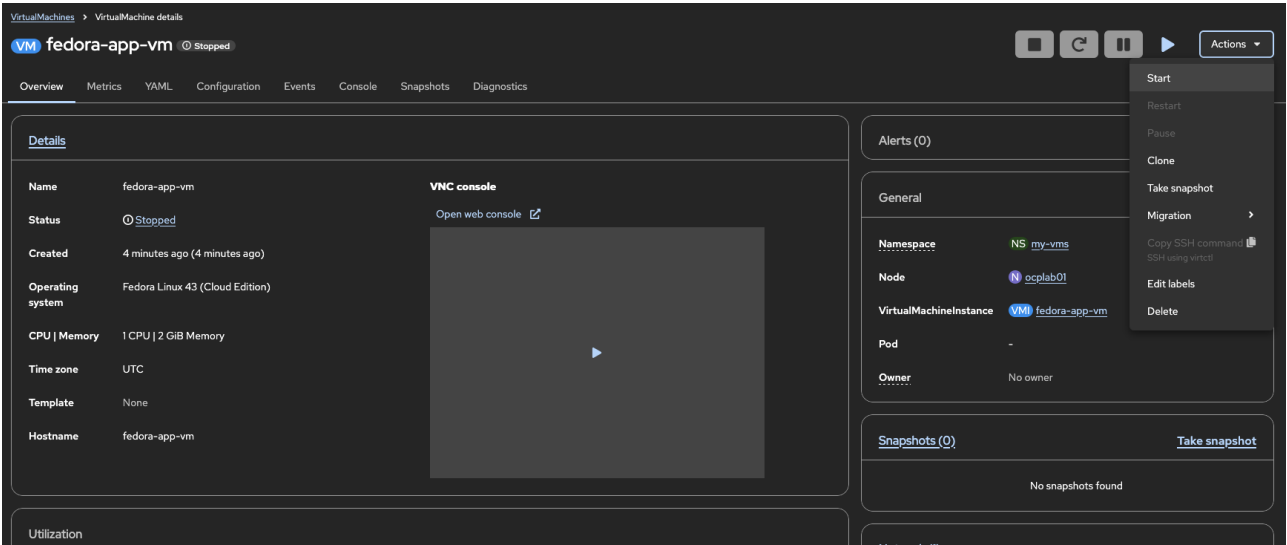
VM 프로세스가 종료

VM 정보

번호 VM 프로세스

Web 콘솔:

- 1. **Virtualization** → **VirtualMachines** 섹션으로 이동
- 2. 섹션에서 VM 프로세스
- 3. VM 프로세스의 **Actions** → **Start** 버튼 클릭
- 4. 프로세스가 (대기 3 초) 후 **Start** 버튼 클릭



CLI:

```
virtctl start fedora-app-vm -n my-vms
```

VM 確認:

```
# Check VM status
oc get vm fedora-app-vm -n my-vms

# Check VMI was created
oc get vmi fedora-app-vm -n my-vms
```

結果:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Running	True

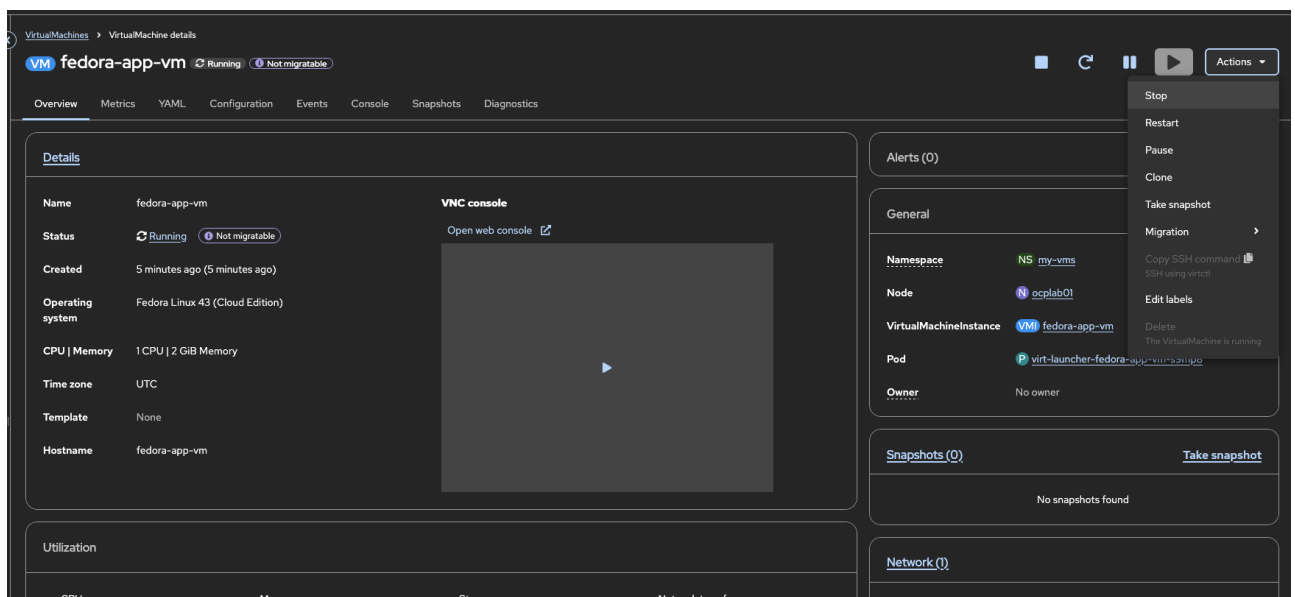
NAME	AGE	PHASE	IP	NODENAME	READY
fedora-app-vm	78s	Running	10.128.0.184	ocplab01	True

VM 削除

VM 削除後 VMI 確認

Web 操作:

1. VM 確認
2. **Actions** → **Stop** 実行



CLI (確認):

```
virtctl stop fedora-app-vm -n my-vms
```

## CLI (ターミナル):

VM を強制停止する

```
virtctl stop fedora-app-vm -n my-vms --force
```



強制停止は、VM の状態を破損する可能性があります。

## VM の状態を確認する:

```
# VM should show Stopped status
oc get vm fedora-app-vm -n my-vms

# VMI should no longer exist
oc get vmi fedora-app-vm -n my-vms
```

結果:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Stopped	False

Error from server (NotFound): virtualmachineinstances.kubevirt.io "fedora-app-vm" not found

## VM を一時停止する

一時停止 (Pause) は VM のメモリをホストに保存し、CPU を一時停止します。

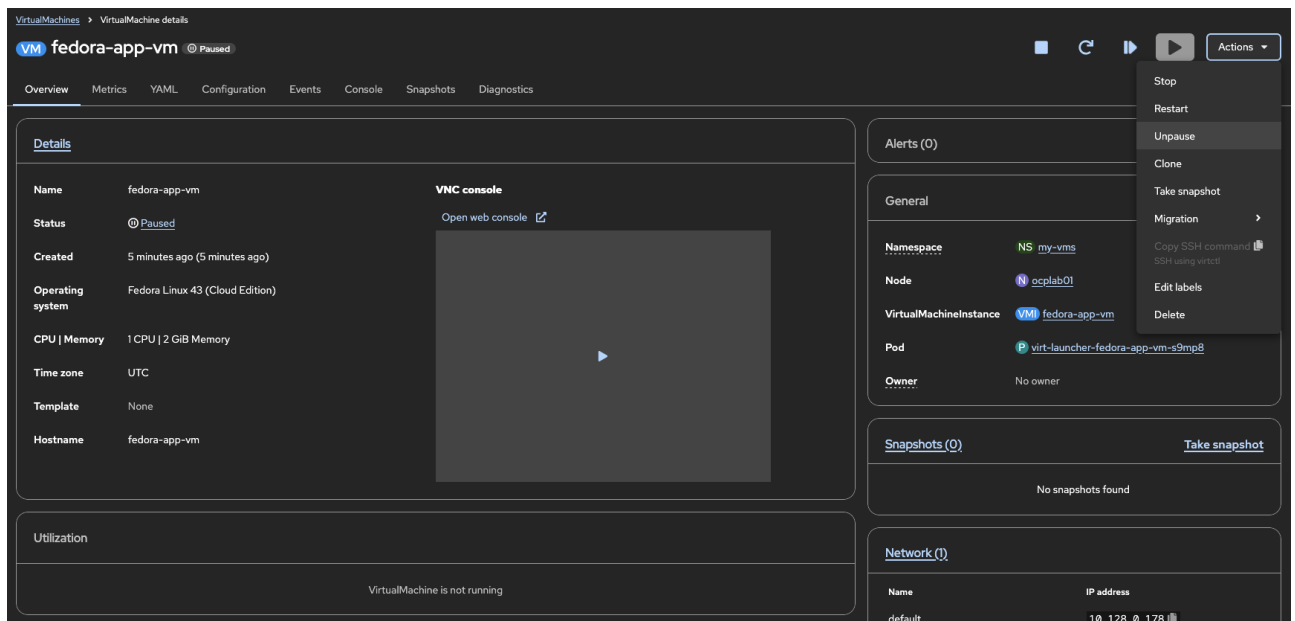
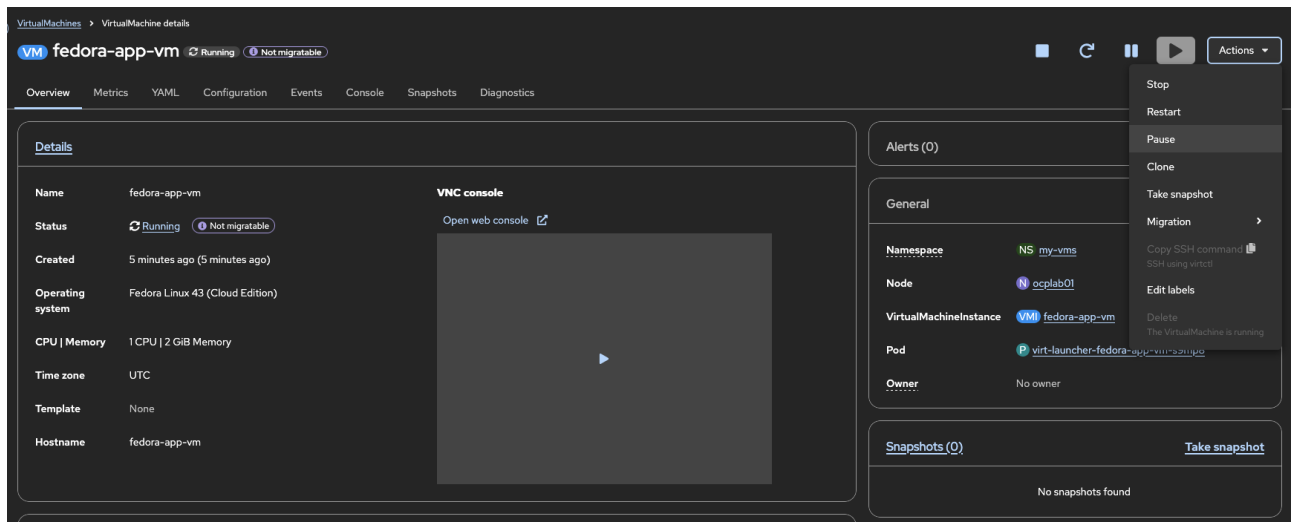
- CPU を一時停止します
- メモリをホストに保存します
- 一時停止



一時停止した VM は、CPU を再起動するまで実行されません。

## Web コンソール:

1. VM を一時停止する
2. **Actions** → **Pause** をクリックする
3. 確認: **Actions** → **Unpause** をクリックする



CLI:

```
# Pause a VM
virtctl pause vm fedora-app-vm -n my-vms

# Unpause a VM
virtctl unpause vm fedora-app-vm -n my-vms
```

oc get vm:

```
oc get vm fedora-app-vm -n my-vms
```

table:

NAME	AGE	STATUS	READY
fedora-app-vm	14m	Paused	False

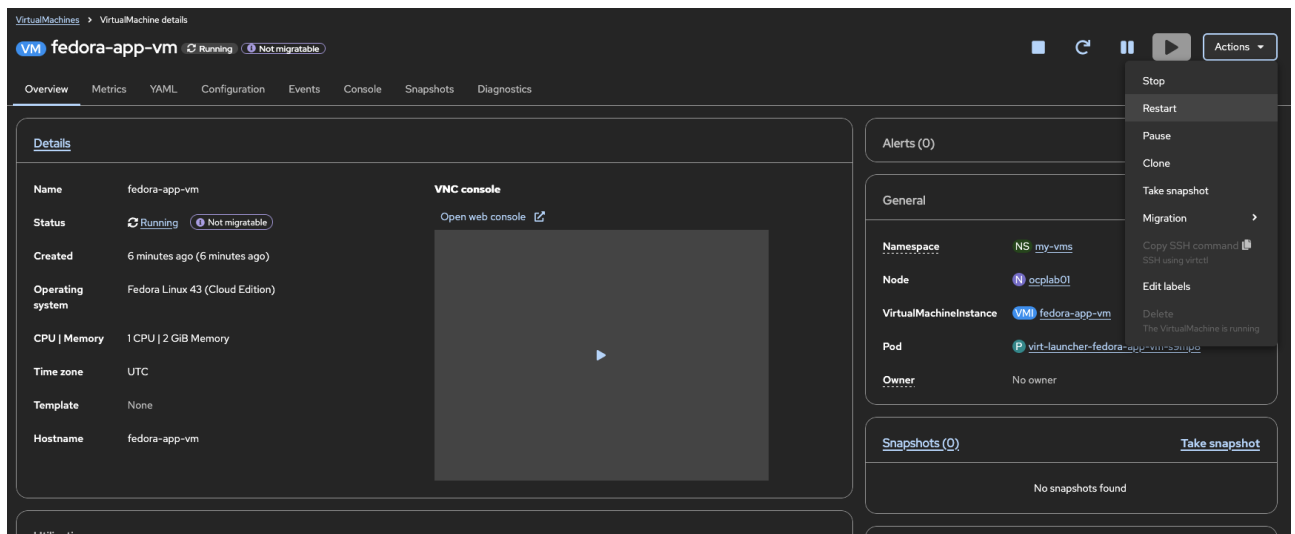
## VM 삭제

VM (Restart) 버튼을 클릭하면 다음과 같은 메시지가 표시됩니다.

- VM을 중지할지 묻는 메시지가 표시됩니다.
- VM OS를 재시작할지 묻는 메시지가 표시됩니다.
- VM을 삭제할지 묻는 메시지가 표시됩니다.

Web 콘솔:

1. VM을 선택합니다.
2. **Actions** → **Restart** 버튼을 클릭합니다.



CLI:

```
virtctl restart fedora-app-vm -n my-vms
```



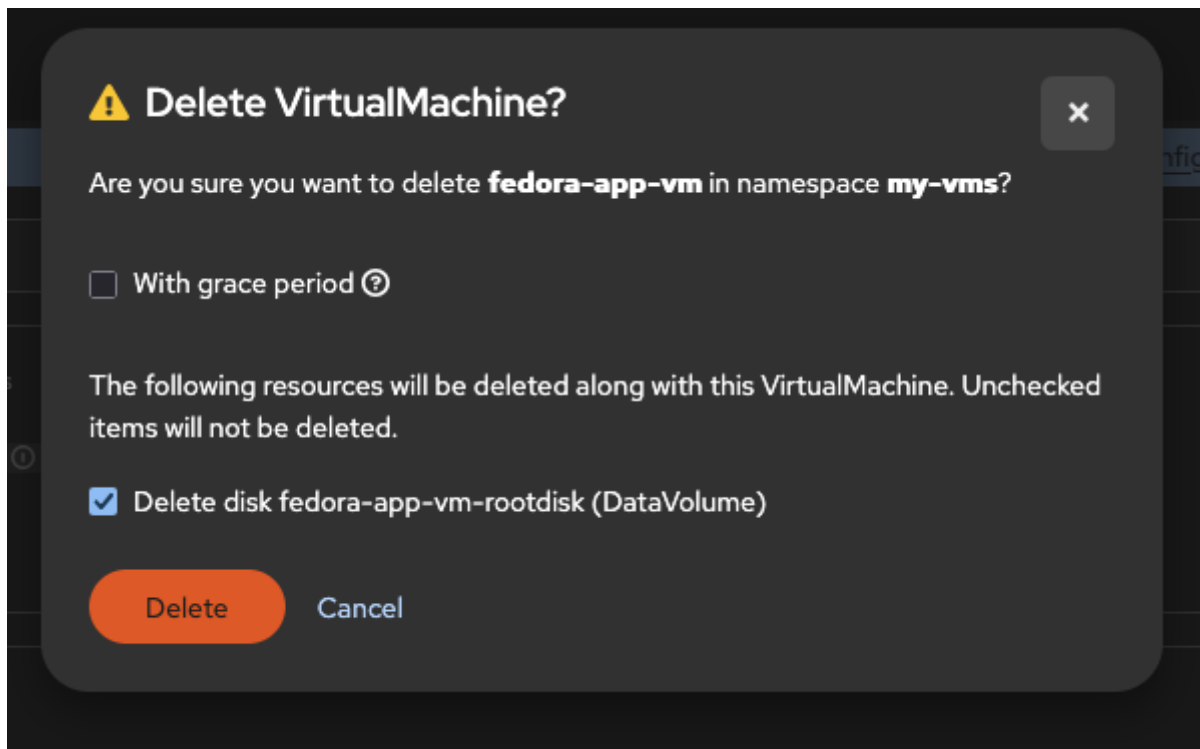
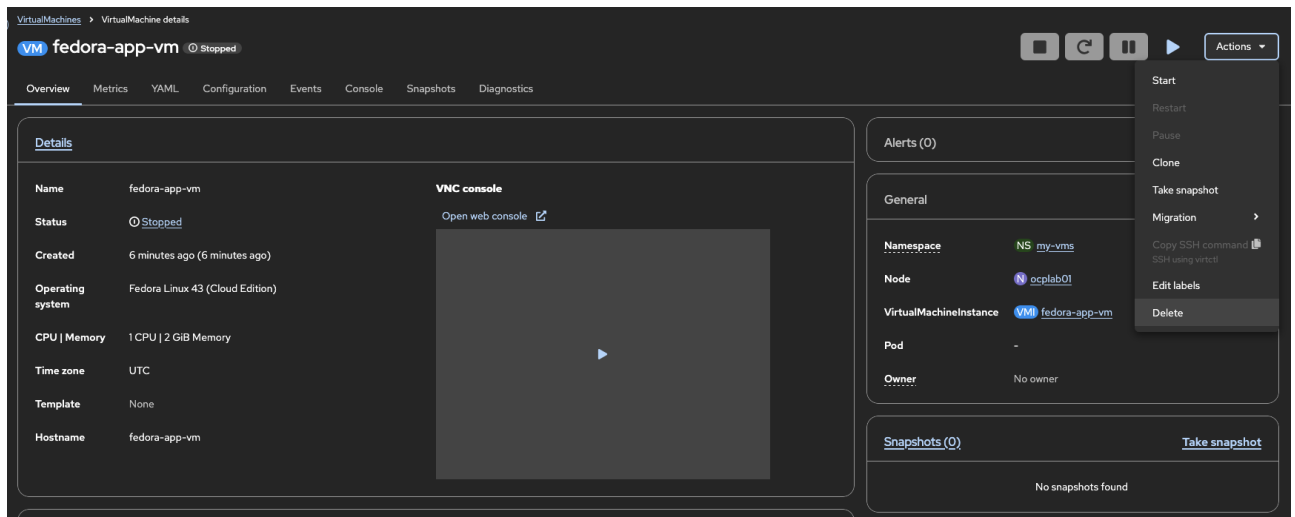
이 메시지는 VM을 삭제할 때만 표시됩니다. OS를 재시작할 때는 표시되지 않습니다.

## VM 삭제

VM을 삭제할 때는 PVC/DataVolume을 삭제할지 묻는 메시지가 표시됩니다.

Web 콘솔:

1. VM을 선택합니다.
2. **Actions** → **Delete** 버튼을 클릭합니다.
3. VM을 삭제할지 묻는 메시지가 표시됩니다.
4. **Delete** 버튼을 클릭합니다.



CLI (VM `oc delete vm`):

```
oc delete vm fedora-app-vm -n my-vms
```

When deleting a VM, the `DataVolume` is also deleted. If you want to preserve the `DataVolume`, you can use the `oc delete vm --keep-data-volume` command.

Verify:

```
# VM should not exist
oc get vm fedora-app-vm -n my-vms

# Check if PVCs were preserved (if desired)
oc get pvc -n my-vms
```

Done:



Error from server (NotFound): virtualmachines.kubevirt.io "fedora-app-vm" not found  
No resources found in my-vms namespace.



VM ทรัพยากรไม่พบ

VM ทรัพยากร

VM ทรัพยากร VMI ทรัพยากร

ทรัพยากร VM ทรัพยากร:

```
oc describe vm fedora-app-vm -n my-vms
```

ทรัพยากร (ทรัพยากร):

```
Name:          fedora-app-vm
Namespace:     my-vms
Labels:        app=fedora-app-vm
...
Status:
  Conditions:
    Last Transition Time:  2026-03-05T17:10:11Z
    Status:                True
    Type:                  Ready
    Message:               All of the VMI's DVs are bound and ready
    Reason:                AllDVsReady
    Status:                True
    Type:                  DataVolumesReady
    Status:                True
    Type:                  AgentConnected
  Printable Status:       Running
  Ready:                  true
  Run Strategy:           Manual
```

ทรัพยากร (ทรัพยากร):

- **Status.Conditions** - ทรัพยากร
- **Status.PrintableStatus** - ทรัพยากร
- **Status.Ready** - VM ทรัพยากร

ทรัพยากร VMI ทรัพยากร:

```
oc describe vmi fedora-app-vm -n my-vms
```

名前 (namespace):

```
Name:      fedora-app-vm
Namespace:  my-vms
...
Status:
  Guest OS Info:
    Id:      fedora
    Kernel Release: 6.17.1-300.fc43.x86_64
    Name:      Fedora Linux
    Pretty Name:  Fedora Linux 43 (Cloud Edition)
    Version Id:  43
  Interfaces:
    Interface Name: enp1s0
    Ip Address:     10.128.0.184
    Mac:            02:7e:e2:0a:7c:df
    Name:            default
  Node Name:      ocplab01
  Phase:          Running
  Phase Transition Timestamps:
    Phase: Pending
    Phase: Scheduling
    Phase: Scheduled
    Phase: Running
```

条件 (conditions):

- **Status.Phase** - 現在のフェーズ
- **Status.Interfaces** - IP アドレス
- **Status.NodeName** - VM ノード名
- **Status.GuestOSInfo** - ゲスト OS 情報 (guest agent 経由)

条件 (Conditions) 確認

VM 条件 (Conditions) 確認コマンド

```
oc get vm fedora-app-vm -n my-vms -o jsonpath='{.status.conditions}' | jq
```

結果:

```
[
  {
    "lastTransitionTime": "2026-03-05T17:10:11Z",
    "status": "True",
    "type": "Ready"
  },
  {
```

```
[{"message": "All of the VMI's DVs are bound and ready",
  "reason": "AllDVsReady",
  "status": "True",
  "type": "DataVolumesReady"},
{"message": "cannot migrate VMI: PVC fedora-app-vm-rootdisk is not shared...",
  "reason": "DisksNotLiveMigratable",
  "status": "False",
  "type": "LiveMigratable"},
{"status": "True",
  "type": "AgentConnected"}]
```

□□□□□□:

VM	VM
Ready	VM 準備完了
Paused	VM 一時停止
AgentConnected	QEMU guest agent 接続済み
Synchronized	VM 同期済み VMI 同期済み

**VM 0000**

000000VM 0000000000000000000000000000

```
# Get events for a specific VM
oc get events -n my-vm --field-selector involvedObject.name=fedora-app-vm
```

□□□:

LAST SEEN	TYPE	REASON	OBJECT
14m	Normal	SuccessfulDataVolumeCreate	virtualmachine/fedora-app-vm
Created DataVolume fedora-app-vm-rootdisk			
90s	Normal	SuccessfulCreate	virtualmachine/fedora-app-vm
Started the virtual machine by creating the new virtual machine instance fedora-app-vm			
90s	Normal	SuccessfulCreate	virtualmachineinstance/fedora-app-vm
Created virtual machine pod virt-launcher-fedora-app-vm-fktmb			
84s	Normal	Created	virtualmachineinstance/fedora-app-vm
VirtualMachineInstance defined.			
84s	Normal	Started	virtualmachineinstance/fedora-app-vm
VirtualMachineInstance started.			

コマンドを実行:

```
oc get events -n my-vms -w
```

結果:

イベント	説明
SuccessfulCreate	VMI 作成成功
Started	VM 起動成功
Stopped	VM 停止 (正常)
Killing	強制終了
FailedScheduling	スケジューリング失敗
FailedCreate	VMI 作成失敗 (エラー)

実行戦略 (Run Strategies)

実行戦略は、VM が失敗した場合の対応方法を定義します。VM が失敗すると、実行戦略に基づいて VM を再実行するか、手動で再実行するか、または停止するかを決定します。

実行戦略の種類:

戦略	説明	結果
Always	VM が失敗すると、常に再実行されます。(再実行回数制限はありません)	再実行されます
RerunOnFailure	VM が失敗すると、再実行されます。再実行回数は 10 回です。10 回失敗すると、VM は停止されます。	再実行されます (10 回まで)
Manual	virtctl コマンドで Web コンソールを開いて、手動で再実行する必要があります。	手動で VM を再実行する必要があります
Halted	VM が失敗すると、VM は永久に停止されます。	再実行されず VM は停止されます

例:

VM の YAML:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  runStrategy: Always # or RerunOnFailure, Manual, Halted
  template:
```

```
# ... VM template spec
```

### Web 操作:

1. VM 操作
2. **YAML** 操作
3. `spec.runStrategy` 操作
4. **Save** 操作

VM

fedora-app-vm

Stopped

Overview

Metrics

YAML

Configuration

Events

Console

Snapshots

Diagnostics

```

3  metadata:
94  labels:
96  spec:
97    dataVolumeTemplates:
98      - metadata:
99          creationTimestamp: null
100         name: fedora-app-vm-rootdisk
101       spec:
102         sourceRef:
103           kind: DataSource
104           name: fedora
105           namespace: openshift-virtualization-os-images
106         storage:
107           resources:
108             requests:
109               storage: 30Gi
110       runStrategy: Manual
111     template:
112       metadata:
113         creationTimestamp: null
114         labels:
115           kubevirt.io/domain: fedora-app-vm
116       spec:
117         architecture: amd64
118         domain:
119           cpu:
120             cores: 1
121             sockets: 1
122             threads: 1
123           devices:
124             disks:
125               - disk:
126                   bus: virtio
127                   name: rootdisk
128               - disk:
129                   bus: virtio
130                   name: cloudinitdisk
131             interfaces:
132               - macAddress: '02:7e:e2:0a:7c:df'
133                 masquerade: {}
134                 name: default
135           firmware:

```

Save

Reload

Cancel



`runStrategy` 是 `running` 的 VM 的 `runStrategy`

Always 的 `RerunOnFailure`

- `virtctl stop` VM

- VM `shutdown -h now` (or: VM `shutdown -h now`) `shutdown -h now` `shutdown -h now`
- VM `shutdown -h now` (or: VM `shutdown -h now`) `shutdown -h now` `shutdown -h now`

VM

VirtualMachine

- VirtualMachine `VirtualMachineInstance` `VirtualMachineInstance`
- VM `VirtualMachineInstance` `VirtualMachineInstance`
- VM `VirtualMachineInstance` `VirtualMachineInstance`
- VM (Conditions) `VirtualMachineInstance` (Events) `VirtualMachineInstance` VM `VirtualMachineInstance`
- VM `VirtualMachineInstance` (Run Strategies) `VirtualMachineInstance`

## Virtctl

VM

`virtctl` CLI `OpenShift` `Virtualization` `Virtualization` (VM) `Virtualization` `Virtualization` VM

OpenShift 4.20

OpenShift 4.20

OpenShift

- OpenShift `Virtualization` `Virtualization` OpenShift 4.18 `Virtualization`
- `virtctl` CLI `Virtualization` `Virtualization`

## virtctl

`virtctl` CLI `OpenShift` Web `Virtualization` `Virtualization`

1. OpenShift Web `Virtualization` `Virtualization` `Virtualization` `Virtualization` `Virtualization`

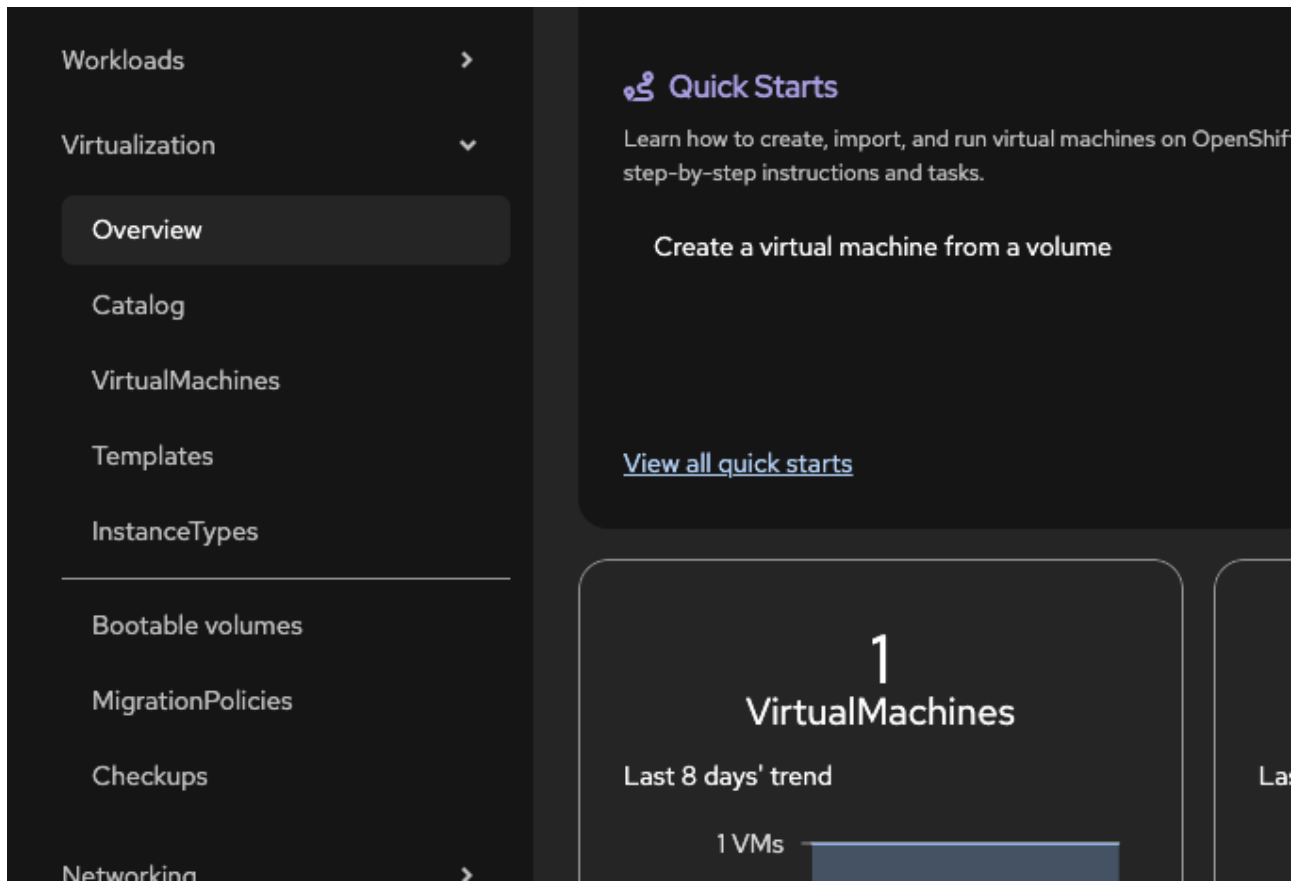


Figure 1. OpenShift **Virtualization Overview**

2. Overview **Download the virtctl command-line utility**

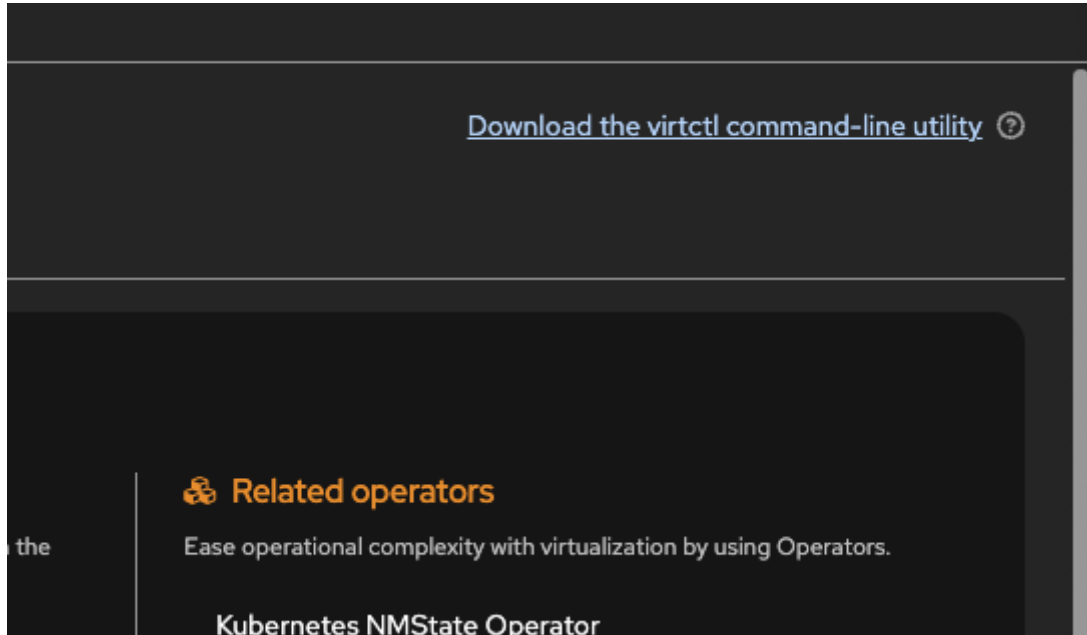


Figure 2. **virtctl**

3. **virtctl**



## virtctl – KubeVirt command line interface

The virtctl client is a supplemental command-line utility for managing virtualization resources from the command line.

- [Download virtctl for Linux for ARM 64](#)
- [Download virtctl for Linux for IBM Z](#)
- [Download virtctl for Linux for x86\\_64](#)
- [Download virtctl for Mac for ARM 64](#)
- [Download virtctl for Mac for x86\\_64](#)
- [Download virtctl for Windows for ARM 64](#)
- [Download virtctl for Windows for x86\\_64](#)

Figure 3. virtctl のインストール

#### 4. PATH 環境変数に追加

```
# Linux/Mac
chmod +x virtctl
sudo mv virtctl /usr/local/bin/

# Verify installation
virtctl version
```

Windows の場合は、PATH 環境変数に追加する。

確認

VM のインストール

VM のインストールは、oc コマンドで実行する。VM のインストール

```
# List all VMs in a specific namespace
oc get vm -n myapp

# List all VMs across all namespaces
oc get vm -A
```

結果:

NAMESPACE	NAME	AGE	STATUS	READY
myapp	fedora-app-vm	13d	Running	True

IP アドレスは、VM の IP (VMI) から取得する。

```
oc get vmi -A
```

□□□:

NAMESPACE	NAME	AGE	PHASE	IP	NODENAME	READY
myapp	fedora-app-vm	13d	Running	10.128.1.225	ocplab01	True



VM □□□□□□□□□□ **oc** □□□□□□□□□□ VM □□□□□□□□□□□□□□□□□□□□□□ **virtctl** □□□□□□□□

**VM** □□□□□□□□

VM □□□□□□□□□□□□□□□□

```
# Start a VM
virtctl start fedora-app-vm -n myapp

# Stop a VM
virtctl stop fedora-app-vm -n myapp

# Restart a VM
virtctl restart fedora-app-vm -n myapp
```

**VM** □□□□□□

VM □□□□□□□□

```
# Console access (press Ctrl+] to exit)
virtctl console fedora-app-vm -n myapp

# SSH access (requires SSH configured)
virtctl ssh user@fedora-app-vm -n myapp

# VNC access
virtctl vnc fedora-app-vm -n myapp
```

**VM** □□□□□□

VM □□□□□□□□□□ (VM □ QEMU guest agent □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□)

```
# Guest OS information
virtctl guestosinfo fedora-app-vm -n myapp
```

□□□:

```
{
  "guestAgentVersion": "10.1.0",
  "hostname": "fedora-app-vm",

```

```

"os": {
  "name": "Fedora Linux",
  "kernelRelease": "6.5.6-300.fc39.x86_64",
  "version": "39",
  "prettyName": "Fedora Linux 39 (Cloud Edition)",
  "versionId": "39",
  "kernelVersion": "#1 SMP PREEMPT_DYNAMIC",
  "machine": "x86_64",
  "id": "fedora"
}
}

```

```

# List filesystems
virtctl fslist fedora-app-vm -n myapp

```

#### (#####):

```

{
  "items": [
    {
      "diskName": "vda2",
      "mountPoint": "/boot/efi",
      "fileSystemType": "vfat",
      "usedBytes": 21723136,
      "totalBytes": 104607744
    },
    {
      "diskName": "vda3",
      "mountPoint": "/boot",
      "fileSystemType": "ext4",
      "usedBytes": 91316224,
      "totalBytes": 1902895104
    },
    {
      "diskName": "vda4",
      "mountPoint": "/",
      "fileSystemType": "btrfs",
      "usedBytes": 687783936,
      "totalBytes": 29525868544
    }
  ]
}

```

```

# List logged-in users
virtctl userlist fedora-app-vm -n myapp

```

□□□□□□

**virtctl scp** □□□□ VM □□□□□□□□□□□□□□□□ VM □ SSH □□□□□□□□□□□□□□□□□□□□□□□□

- **SSH** □□ (□□): □□□□ VM □ `~/.ssh/authorized_keys` □□□□□□
- □□□□□: VM □ SSH □□□□□□□□□□□□□□□□□□□□

```
# Copy to VM (uses default SSH key)
virtctl scp local-file.txt fedora@fedora-app-vm:/tmp/remote.txt -n myapp

# Copy from VM
virtctl scp fedora@fedora-app-vm:/tmp/remote.txt ./local.txt -n myapp

# Specify a different SSH key
virtctl scp -i ~/.ssh/my_key local-file.txt fedora@fedora-app-vm:/tmp/ -n myapp
```



SSH □□□□□□□□□□□□ **virtctl console** □□□□ VM □□□□□□SSH □□□□□□□□□□□□

VM □□□□ SSH □□□□□□□□□□cloud-init □□□□□□□□

```
#cloud-config
user: fedora
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC... your-key-here
```

□□□ SSH □□□□□□□□□□□□[Linux](#) □□□□□□□□□□□□□□□□ □□□□□□□□□□

## VM □□□□□

VM □□□□□□□□□□□□□□□□□□□□□□

```
# Check VM status
oc get vm fedora-app-vm -n myapp
```

□□□□□□□□□□:

NAME	AGE	STATUS	READY
fedora-app-vm	7d3h	Running	True

```
# Check VMI (running instance)
oc get vmi fedora-app-vm -n myapp
```

□□□□□□□□□□: □□□□□□□□ VM□ □□□□□□□□□□□□□□□□ VM□ □□□□□□□□□□



## OpenShift Virtualization

- OpenShift Virtualization **-n** namespace
- OpenShift Virtualization SSH
- OpenShift VM guest agent
- VM **oc** VM **virtctl**
- OpenShift Virtualization **Ctrl+]**
- OpenShift Virtualization **virtctl ssh**

## OpenShift Virtualization

- [virtctl Commands Reference](#)

## OpenShift

### OpenShift Virtualization

- OpenShift Web **virtctl** CLI
- OpenShift VM
- OpenShift SSH/VNC VM
- QEMU guest agent OS
- **virtctl scp** VM

## OpenShift

- [Linux](#) - VM

# OpenShift Virtualization VM cloud-init

## OpenShift

cloud-init OpenShift Virtualization VM cloud-init SSH cloud-init KubeVirt VM cloud-init VirtualMachine cloud-init

## OpenShift

- cloud-init VM
- 3 cloud-init userData networkData metadata
- KubeVirt NoCloud cloud-init VM
- cloud-config
- VirtualMachine cloud-init 2 (Secret)

- cloud-init 設定

インストール

OpenShift 4.18+

インストール

- OpenShift Virtualization をインストール (OpenShift 4.18 以降)
- `oc` から `virtctl` CLI をインストール (Virtctl をインストール)
- YAML をインストール
- VM をインストール (Web コンソールから VM をインストール)

## cloud-init のインストール

cloud-init は、OS のインストール時に、SSH をインストールし、SSH をインストールするためのツールです。

cloud-init は Fedora、RHEL、CentOS、Ubuntu、Debian の Linux のインストール時に、AWS、Azure、GCP、OpenShift、Virtualization、KubeVirt のインストール時にインストールされます。



cloud-init は、VM のインストール時に、cloud-init をインストールするためのツールです。

## KubeVirt から cloud-init をインストール

cloud-init をインストールする VM の IP アドレス (169.254.169.254) は、cloud-init をインストールする KubeVirt の \*NoCloud 設定で指定する必要があります。

NoCloud をインストールする KubeVirt のインストール手順

1. VirtualMachine をインストール (Secret) から cloud-init をインストール
2. ISO をインストールする VM をインストール
3. virtio をインストールする VM をインストール
4. OS から cloud-init をインストールする VM をインストール

cloud-init をインストールする Kubernetes のインストール手順

```
spec:
  template:
    spec:
      volumes:
        - name: cloudinitdisk
          cloudInitNoCloud:
            userData: |
```

①

```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: changeme
```

① **cloudInitNoCloud** VMをNoCloud VMとして起動

## cloud-init のインストール

cloud-init は 3 種類の VM に対応。ここでは **cloudInitNoCloud** をインストール。

### userData (cloud-config)

cloud-init の **userData** は **cloud-config** の形式で、**#cloud-config** から YAML の形式で記述する。VM ごとに異なる。

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_pwauth: true
    user:
      name: fedora
      lock_passwd: false
      plain_text_passwd: s3cur3p4ss
      ssh_authorized_keys:
        - ssh-ed25519 AAAA... user@workstation
    packages:
      - vim
      - tmux
```

②: 1 行の **#cloud-config** から cloud-init の cloud-config を生成。

### networkData (network-config v2)

VM ごとに異なる。ここでは Netplan の形式で **network-config version 2** の形式で記述。IP アドレスと DNS を指定。

ここでは DHCP を指定。

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        dhcp4: true
```



```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        addresses:
          - 192.168.1.100/24           ①
        nameservers:
          addresses:
            - 192.168.1.1
            - 8.8.8.8                 ②
        routes:
          - to: default
            via: 192.168.1.1          ③
          - to: 10.0.0.0/8
            via: 192.168.1.254        ④
```

- ① CIDR 形式で指定する IP 範囲
- ② DNS 設定
- ③ gateway4 設定
- ④ 10.0.0.0/8 範囲の IP 範囲を指定

MAC 設定を指定する

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      primary:
        match:
          macaddress: "52:54:00:12:34:56"
        addresses:
          - 192.168.1.100/24
        routes:
          - to: default
            via: 192.168.1.1
```



KubeVirt VM 設定を指定する **enp1s0** 設定 **enp2s0** 設定  
NetworkAttachmentDefinition 設定 VirtualMachine Spec  
設定

VLAN 設定を指定する IP 設定

## metadata

`instance-id` `local-hostname` VM KubeVirt VirtualMachine

```
cloudInitNoCloud:
  metadata: |
    instance-id: my-vm-001
    local-hostname: my-vm
```



userData `hostname`

## cloud-init

cloud-init 5

Generator	Local	Network	Config	Final
Generator	cloud-init	networkData	SSH	runCmd
Local	networkData	SSH	SSH	runCmd
Network	SSH	SSH	SSH	runCmd
Config	SSH	SSH	SSH	runCmd
Final	runCmd	runCmd	runCmd	runCmd

(networkData) (userData) runCmd

## Cloud-config

VM #cloud-config userData

```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...key-data... user@workstation
```

lock\_passwd: false plain\_text\_passwd ssh\_pwauth: true SSH

cloud-init ユーザーを定義する

```
#cloud-config
users:
  - name: admin
    groups: wheel
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...key-data...
  - name: developer
    shell: /bin/bash
```

パッケージをインストールする

```
#cloud-config
package_update: true
packages:
  - vim
  - git
  - tmux
  - python3
```



cloud-init の **final** ステージで VM を初期化する

cloud-init

```
#cloud-config
runcmd:
  - systemctl enable --now cockpit.socket
  - echo "VM initialized at $(date)" >> /var/log/vm-init.log
```

cloud-init の **final** ステージで `/bin/sh` を実行し、`(date)` を取得する

cloud-init

```
#cloud-config
write_files:
  - path: /etc/motd
    content: |
      Welcome to the OpenShift Virtualization VM.
      Managed by cloud-init.
    permissions: '0644'
  - path: /etc/myapp/config.yaml
    content: |
      database: postgres
      port: 5432
```

```
owner: root:root
permissions: '0600'
```



**write\_files**

□□□□□□□□□□□□□□□□

**config**

□□□□□□□□□□□□ **runCmd**

□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ **runCmd** □□□□□□□□

## VirtualMachine □□□□□□□□ **cloud-init** □□□□

VirtualMachine □ cloud-init □□□□□□□□□□ 2 □□□□□□□□□□□□

□□□□□□□□ (**cloudInitNoCloud**)

□□□□□□□□□□ cloud-init □□□□ VirtualMachine □□□□□□□□□□□□□□□□□□□□

**minimal-cloud-init-vm.yaml** □□□□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cloud-init-demo
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-cloud-init-demo
    spec:
      domain:
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
        resources:
          requests:
            memory: 2Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: fedora-cloud-init-demo
```

```

    name: rootdisk
  - cloudInitNoCloud: ①
    userData: | ②
      #cloud-config
      ssh_pwauth: true
      user:
        name: fedora
        lock_passwd: false
        plain_text_passwd: s3cur3p4ss
        ssh_authorized_keys:
          - 'ssh-ed25519 AAAA...your-key... user@workstation'
      users:
        - name: userone
          lock_passwd: false
          plain_text_passwd: s3cur3p4ss
          ssh_authorized_keys:
            - 'ssh-ed25519 AAAA...your-key... user@workstation'
      packages:
        - vim
        - git
      runcmd:
        - echo "cloud-init complete" > /tmp/cloud-init-done
    name: cloudinitdisk
dataVolumeTemplates:
  - metadata:
      name: fedora-cloud-init-demo
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi

```

① **cloudInitNoCloud** NoCloud

② **userData** #cloud-config cloud-config YAML

Secret

## Secret

cloud-init Kubernetes Secret VirtualMachine Secret

cloud-init-secret-vm.yaml

Secret

```
oc create secret generic my-vm-cloud-init \
  --from-literal=userdata='#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...your-key... user@workstation
users:
  - name: userone
    lock_passwd: false
    plain_text_passwd: s3cur3p4ss
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...your-key... user@workstation
packages:
  - vim
  - git
' \
-n demo
```

VirtualMachine の定義

```
volumes:
  - cloudInitNoCloud:
      secretRef:
        name: my-vm-cloud-init ①
      name: cloudinitdisk
```

① KubeVirt の Secret の **userdata** の値を cloud-init の **userData** に設定

## cloud-init の確認

VM の cloud-init の実行状況を確認

VM のコンソールに接続

```
virtctl console fedora-cloud-init-demo -n demo
```

cloud-init の実行状況を確認

```
cloud-init status --long
```

実行結果

```
status: done
```

```
extended_status: done
boot_status_code: enabled-by-generator
detail:
DataSourceNoCloud [seed=/dev/vdb]
```

cloud-init のインストールと確認

```
cat /var/log/cloud-init.log | tail -50
```

インストールが完了したら、cloud-init のインストールを確認する

確認方法

**#cloud-config** の確認

```
userData 1 の確認 #cloud-config のインストールを確認 cloud-init のインストールを確認 userData
の確認
```

**YAML** の確認

Cloud-config の YAML の確認 (インストール済み) のインストールを確認 VM の cloud-init devel  
schema --config-file /path/to/config.yaml のインストールを確認

インストール済み **userData** の確認

```
インストール済み userData の確認 networkData のインストール済み userData の network-config v2 の YAML
の確認
```

インストール済み **cloud-init** の確認

```
cloud-init のインストールを確認 cloud-init clean のインストールを確認
```

**Secret** の確認

```
secretRef のインストール済み Secret の userdata のインストール済み (インストール済み) のインストール済み userData の user-data
の確認
```

## Windows VM の Sysprep

cloud-init の Linux のインストール済み Windows VM のインストール済み OpenShift Virtualization の Sysprep のインストール済み  
Microsoft System Preparation のインストール済み (インストール済み) のインストール済み

Windows VM のインストール済み **Windows VM** のインストール済み

確認方法

インストール済み

```
oc delete vm fedora-cloud-init-demo -n demo
oc delete vm fedora-cloud-init-secret-demo -n demo
oc delete secret my-vm-cloud-init -n demo
```

```
oc delete namespace demo
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- cloud-init 是 Linux VM 的初始化工具
- KubeVirt 的 NoCloud 镜像使用 cloud-init 来配置 VM 的 virtio 设备
- 3 个主要部分: `userData` (用户数据, `cloud-config`)、`networkData` (网络配置, `network-config v2`)、`metadata` (元数据)
- cloud-init 在 VM 启动时运行, 配置网络、磁盘、用户等
- 2 种部署方式: `cloudInitNoCloud` (无云, 使用本地镜像) 和 `Secret` (使用 Kubernetes Secret)
- `cloud-init status --long` 查看 /var/log/cloud-init.log 日志

- [IP 地址](#) - cloud-init 配置 IP/DHCP 配置和 VLAN 配置
- [cloud-init 配置](#) - cloud-init 配置
- [RDP 连接 Windows VM 配置](#) - Sysprep 配置 Windows VM 配置
- [cloud-init 配置](#)
- [cloud-init 配置](#)
- [KubeVirt cloudInitNoCloud 配置](#)



## OpenShift Virtualization

### Overview

OpenShift Virtualization enables you to run VMs on OpenShift. It uses LVM Storage Operator and HostPath Provisioner to manage storage for VMs.

VMs (Virtual Machines) are managed by the OpenShift Virtualization VM Controller.

### Prerequisites

Before installing OpenShift Virtualization, ensure the following prerequisites are met:

- LVM Storage Operator is installed and configured.
- HostPath Provisioner is installed.
- default StorageClass is configured.
- OpenShift Virtualization VM Controller is installed.

## OpenShift Virtualization Installation

OpenShift Virtualization is installed on OpenShift by using the following steps:

- **LVM Storage:** Logical Volume Manager is used to manage storage (default).
- **HostPath Provisioner:** Used to provision storage (default).
- **Ceph/ODF:** Used for distributed storage.
- **NFS:** Used for shared storage.
- **Cloud Providers:** AWS EBS, Azure Disk, etc.

Single Node OpenShift (SNO) uses LVM Storage and HostPath Provisioner for storage.

### Installation

OpenShift Virtualization is installed on OpenShift by using the following steps:

- OpenShift 4.18 or later.
- Linux operating system.
- 1 GB of free disk space.
- Linux kernel (LVM) is installed.

□ □ □ □ □ □

## LVM Operator

```

##### LVM Storage Operator #####

```

LVM □□□□□□□□□□

[illegible]

## HostPath Provisioner

```

#####HostPath Provisioner#####

```

□□□□□□ VM

## StorageClass を利用して VM を作成する

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

## OpenShift Virtualization

- **DataSource OS** : 데이터소스 OS
- **DataSource VM** : 데이터소스 VM
- **VM** : 가상머신
- **DataSource** : 데이터소스

## ■■■■■■■■LVM Operator ■■■■ Step 5: Set Default StorageClass ■■■■■■■■■■

Page 10 of 10

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- VM □ □□□□□□□
- □□□□□□□□□□

□ □ □ □

- OpenShift LVM Storage

## LVM Operator ☐☐☐☐☐☐

11

OpenShift	LVM	Storage	Operator
LVM	Operator	Logical Volume Manager (LVM)	
Single Node OpenShift (SNO)			

□ □ □ □

- **OpenShift 4.18**
- **Operator:** Kubernetes 1.x.x 버전
- **StorageClass:** Operator StorageClass 생성
- **forceWipeDevicesAndDestroyAllData** 옵션을 사용하여 모든 데이터를 지우고 디바이스를 재사용합니다.

□□□□□□□□:

OpenShift 4.19, 4.20

□ □ □ □ □

[illegible]

**QUESTION 1:**

```
# Get your node name
oc get nodes

# List all block devices on the node
oc debug node/<node-name> -- chroot /host lsblk -o
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

□□□:

```
NAME      SIZE TYPE FSTYPE      MOUNTPOINT MODEL
sda      558.4G disk LVM2_member      PERC H730 Adp
sdc      558.4G disk              PERC H730 Adp
|-sdc1    1M part
|-sdc2  127M part vfat
|-sdc3  384M part ext4      /boot
`-sdc4  557.9G part xfs       /sysroot
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- **sda** - LVM (Logical Volume Manager)
- **sdc** - OS (Operating System)

000 LVM 000000000000

```
oc debug node/<node-name> -- chroot /host pvs
```

□□□:

PV	VG	Fmt	Attr	PSize	PFree
/dev/sda	vg1	lvm2	a--	558.37g	<55.84g

□□□□/dev/sda □□□□ LVM □□□□□□forceWipeDevicesAndDestroyAllData: true □□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-path/ | grep -E  
"pci.*scsi"
```

□□□:

```
lrwxrwxrwx. 1 root root 9 Oct 6 13:30 pci-0000:03:00.0-scsi-0:2:1:0 -> ../../sda
```

□□□□ /dev/disk/by-path/pci-0000:03:00.0-scsi-0:2:1:0 □ /dev/sda □□□□□□□□

## □□□□ 2: LVM Storage Operator □□□□□□□□

namespace □□□□□□LVM Storage Operator □□□□□□□□□□□□

LVM Operator □□ namespace □□□□□□□□

**lvm-namespace.yaml** □□□□□□□□

```
oc apply -f - <<EOF  
apiVersion: v1  
kind: Namespace  
metadata:  
  name: openshift-lvm-storage  
EOF
```

OperatorGroup □□□□□□□□

**lvm-operatorgroup.yaml** □□□□□□□□

```
oc apply -f - <<EOF  
apiVersion: operators.coreos.com/v1  
kind: OperatorGroup  
metadata:  
  name: openshift-lvm-storage-operatorgroup  
  namespace: openshift-lvm-storage
```

```
spec:
  targetNamespaces:
  - openshift-lvm-storage
EOF
```

Operator `Subscription`

`lvm-subscription.yaml`

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: lvms-operator
  namespace: openshift-lvm-storage
spec:
  channel: stable-4.20
  name: lvms-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
EOF
```

Operator

```
# Wait for the operator to be installed (may take 2-3 minutes)
oc get csv -n openshift-lvm-storage

# Check operator pod is running
oc get pods -n openshift-lvm-storage
```

(Operator `LVMCluster`):

NAME	READY	STATUS	RESTARTS	AGE
lvms-operator-864b6975bb-kxt4t	1/1	Running	0	2m



`lvms-operator` Pod `vg-manager` Pod 3 `LVMCluster`

(`3` `LVMCluster`):

NAME	READY	STATUS	RESTARTS	AGE
lvms-operator-864b6975bb-kxt4t	1/1	Running	0	5m
vg-manager-xxxxx	1/1	Running	0	1m

### 例 3: LVMCluster のインストール

LVM のインストールには LVMCluster という Kubernetes の Custom Resource (CR) を利用します。

インストール手順

インストール時に (デフォルト) `forceWipeDevicesAndDestroyAllData: true` を指定します。



**forceWipeDevicesAndDestroyAllData:**

**true**

インストール時に指定されたすべてのストレージデバイス上のデータを削除し、  
LVMCluster をインストールします。

#### lvmcluster-with-wipe.yaml のインストール

```
oc apply -f - <<EOF
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
    - name: vg1
      default: true
      thinPoolConfig:
        name: thin-pool-1
        sizePercent: 90
        overprovisionRatio: 10
      nodeSelector:
        nodeSelectorTerms:
        - matchExpressions:
          - key: node-role.kubernetes.io/worker
            operator: In
            values:
            - ""
      deviceSelector:
        paths:
        - /dev/sda
        forceWipeDevicesAndDestroyAllData: true
EOF
```

インストール (インストール) 完了

インストールを確認するためのコマンド:

#### lvmcluster-clean-disk.yaml のインストール

```
oc apply -f - <<EOF
```



- `oc delete storageclass`: `StorageClass` clean/empty

## 4: LVMCluster

LVMCluster

```
# Check LVMCluster status
oc get lvmcluster -n openshift-lvm-storage

# Verify volume group was created
oc debug node/<node-name> -- chroot /host vgs

# Verify thin pool was created
oc debug node/<node-name> -- chroot /host lvs
```

Results:

- LVMCluster **STATUS: Ready**
- VG **vg1**
- **thin-pool-1**

LVM Storage

## 5: LVM Operator

LVM Operator `lvms-vg1` Storage Class OpenShift Virtualization VM `DataVolume` `lvms-vg1`

Storage Class

```
oc get storageclass
```

Results:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION
lvms-vg1	topolvm.io	Delete	WaitForFirstConsumer	true



(default) StorageClass

LVM Storage Class



```
oc patch storageclass lvms-vg1 -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

□□□□□□□□□□

```
oc describe sc | grep IsDefaultClass
```

**□□□□□□□□□□□□□□:**

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION	AGE		
lvms-vg1 (default)	topolvm.io	Delete	WaitForFirstConsumer
6m			true

```
(default) □□□□□□□□□□□□□□□□□□□□□□□□□□□□
```

## OpenShift Virtualization

OpenShift Virtualization

**DataSource OS**

OpenShift Virtualization □ DataSource OS □□□□ (Fedora□RHEL□CentOS Stream□Windows)  
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
# Check available OS images
oc get datasources -n openshift-virtualization-os-images
```

DataSource VM

**VM**  **PVC**

`dataVolumeTemplates` 指定 VM 的 `storageClassName` 的模板

```
dataVolumeTemplates:
- metadata:
  name: fedora-volume
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi
  sourceRef:
    kind: DataSource
```

```
name: fedora
namespace: openshift-virtualization-os-images
```

lvms-vg1

VM

- VM
- DataSource
- PVC storageClassName
- 

OpenShift Virtualization Operator OS

```
# Check boot source preparation status
oc get DataImportCron -n openshift-virtualization-os-images
```

VM Storage Class

VM

```
dataVolumeTemplates:
- metadata:
  name: fedora-volume
  spec:
    pvc:
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: 30Gi
      storageClassName: lvms-vg1 # Must be explicit
    sourceRef:
      kind: DataSource
      name: fedora
      namespace: openshift-virtualization-os-images
```



OpenShift

Virtualization

## 例 6: PersistentVolumeClaim の作成

PersistentVolumeClaim (PVC) の作成

**test-pvc.yaml** の内容

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: lvms-test-pvc
  namespace: default
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: lvms-vg1
EOF
```

PVC が作成された後 Pod の作成

**test-pod.yaml** の内容

```
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: lvms-test-pod
  namespace: default
spec:
  containers:
  - name: test
    image: registry.access.redhat.com/ubi9/ubi:latest
    command: ["sleep", "3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: lvms-test-pvc
EOF
```

PVC の確認

```
oc get pvc lvms-test-pvc
```

```
# Should show STATUS as "Bound"
```

Pod 创建成功

```
# Wait for pod to start
oc wait --for=condition=Ready pod/lvms-test-pod --timeout=120s

# Test writing to the volume
oc exec lvms-test-pod -- sh -c "echo 'LVM Storage Test' > /data/test.txt && cat /data/test.txt"

# Expected output: LVM Storage Test
```

Pod 删除成功

```
oc delete pod lvms-test-pod
oc delete pvc lvms-test-pvc
```

部署完成

LVM 部署成功，LVM Storage 部署成功

部署成功

- 部署成功
- 部署成功
- 部署成功
- PVC 部署成功
- thin pool 部署
- 部署

## forceWipeDevicesAndDestroyAllData 部署

部署

**forceWipeDevicesAndDestroyAllData: true** 部署成功

1. 部署成功: GPT/MBR 部署成功
2. 部署成功: ext4/xfs/btrfs 部署成功
3. **LVM** 部署成功: LVM 部署成功
4. 部署成功: 部署成功
5. 部署成功: LVM Operator 部署成功

□ □ □ □ □ □ □ □

- 物理卷管理 (物理卷管理)
- 物理卷管理 LVM (PV/VG/LV 管理)
- 物理卷管理 (物理卷管理)
- 物理卷管理 (物理卷管理)

□□□□□□□□□□□□□□□□

- 00000000000000000000000000000000
- 00000000000000000000 (0000000000000000)
- 00000000000000000000000000000000

□ □ □ □ □ □ □ □

[illegible]

```
# 1. Verify the disk path
oc debug node/<node-name> -- chroot /host lsblk -o NAME,SIZE,TYPE,MOUNTPOINT

# 2. Ensure disk is NOT mounted
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda

# 3. Double-check you're not using the system disk
oc debug node/<node-name> -- chroot /host df -h | grep sda

# 4. Review disk serial number
oc debug node/<node-name> -- chroot /host lsblk -d -o NAME,SERIAL /dev/sda
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- LVM `lvcreate` `lvextend` `lvreduce` `lvconvert` `lvchange` `lvremove` `lvmdiskfilter`
- `namespace` `OperatorGroup` `Subscription` `ImageStream` `LVM Storage Operator` `ImageStream`
- `forceWipeDevicesAndDestroyAllData` `LVMCluster` `ImageStream`
- `lvms-vg1` Storage Class `OpenShift Virtualization` `ImageStream`
- PVC `Pod` `LVMCluster` `ImageStream`

11

- LVM 管理物理卷
- 管理逻辑卷

# LVM 操作

1

OpenShift LVM Storage Operator 操作 LVM volume group persistent volume claim 操作

操作

LVM 操作

```
# 1. Check LVMCluster status
oc get lvmcluster -n openshift-lvm-storage

# 2. Check all pods are running
oc get pods -n openshift-lvm-storage

# 3. Check StorageClass exists
oc get storageclass | grep lvms

# 4. Check volume group on node
oc debug node/<node-name> -- chroot /host vgs

# 5. Check thin pool
oc debug node/<node-name> -- chroot /host lvs
```

操作:

- LVMCluster: **STATUS: Ready**
- Pods: **Running**
- StorageClass: **lvms-vg1**
- VG: volume group
- LV: thin provisioning pool

操作

**LVMCluster** 操作

操作

```
oc get lvmcluster -n openshift-lvm-storage
```

操作:

NAME	STATUS
------	--------

```
lvmcluster Ready
```

□□□□□□□:

- □□□□□□ NotReady□Degraded□Error □□□□□□
- □□□□□□□□□□

□□□□□□□□□□

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml
```

□□□□□□□:

```
status:
  conditions:
  - status: "True"
    type: ResourcesAvailable
    message: Reconciliation is complete and all the resources are available
  - status: "True"
    type: VolumeGroupsReady
    message: All the VGs are ready
  deviceClassStatuses:
  - name: vg1
    nodeStatus:
      - devices:
        - /dev/sda
        status: Ready
  ready: true
  state: Ready
```

□□□□□□□:

- status.conditions: ResourcesAvailable: True □□□ VolumeGroupsReady: True □□□□□□□□□□
- deviceClassStatuses[].nodeStatus[].status: Ready □□□□□□□□
- deviceClassStatuses[].nodeStatus[].devices: □□□□□□□□□□□□□□□□□□□□
- ready: true □□□ state: Ready □□□□□□□□

□□□□□□□□□□□□□□

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50
"deviceClassStatuses"
```

□□□:

```
deviceClassStatuses:
```

```
- name: vg1
  nodeStatus:
  - devices:
    - /dev/sda
    excluded:
    - name: /dev/sdc
      reasons:
      - /dev/sdc has children block devices and could not be considered
      - /dev/sdc is not part of the device selector
    status: Ready
```

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

## Pod xxxxxx

xxxx **Pod** xxx

```
oc get pods -n openshift-lvm-storage
```

xxxxxx:

NAME	READY	STATUS	RESTARTS	AGE
lvms-operator-864b6975bb-kxt4t	1/1	Running	0	10m
vg-manager-xxxxx	1/1	Running	0	5m

xxxxxx Pod:

- **lvms-operator:** xxxxxxxxLVM xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
- **vg-manager:** xxxxxxxx 1 xxxxxxxxxxxxxxxx volume group xxxxxxxx
- **topolvm-controller:** Single Node OpenShift (SNO) xxxxxxxxxxxxxxxxxxxx

**Pod** xxxxxx

```
# Get pod details
oc describe pod -n openshift-lvm-storage <pod-name>

# Check specific pod logs
oc logs -n openshift-lvm-storage <pod-name>

# Check vg-manager logs (most useful for device issues)
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

xxxxxxxxxx:

- **"device is in use"** - xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
- **"cannot wipe device"** - **forceWipeDevicesAndDestroyAllData: true** xxxxxxxx



- "device not found" - 装置が見つかりません
- "no devices found" - 装置が見つかりません

## Operator 確認

```
# Check ClusterServiceVersion
oc get csv -n openshift-lvm-storage

# Check subscription
oc get subscription -n openshift-lvm-storage

# Check install plan
oc get installplan -n openshift-lvm-storage
```

## 確認

### StorageClass 確認

```
# List all StorageClasses
oc get storageclass

# Get specific LVM StorageClass
oc get storageclass lvms-vg1 -o yaml
```

## 確認:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION
lvms-vg1	topolvm.io	Delete	WaitForFirstConsumer	true

## 確認:

- **PROVISIONER:** `topolvm.io` - 装置が見つかりません
- **VOLUMEBINDINGMODE:** `WaitForFirstConsumer` - 装置が見つかりません
- **ALLOWVOLUMEEXPANSION:** `true` - 装置が見つかりません

### StorageClass 確認

```
oc get storageclass lvms-vg1 -o
jsonpath='{.metadata.annotations.storageclass\.kubernetes\.io/is-default-class}'
```

## 確認:

- **true** - default StorageClass 確認

- `iscsi` `false` - `iscsi` `no`

## PersistentVolumes

```
# List all PVs
oc get pv

# Check PVs using LVM storage
oc get pv | grep topolvm
```

## volume group

### volume group

```
oc debug node/<node-name> -- chroot /host vgs
```

Output:

```
VG   #PV #LV #SN Attr   VSize   VFree
vg1   1  1  0 wz--n- 558.37g <55.84g
```

Output:

- **VG** `vg1` LVMCluster `vg1` (`vg1`)
- **VSize** `558.37g`
- **VFree** `<55.84g`
- **Attr** `wz--n-` (`wz--n-`) `vg1`

### volume group

```
oc debug node/<node-name> -- chroot /host vgdisplay vg1
```

Output:

```
--- Volume group ---
VG Name                vg1
System ID
Format                 lvm2
Metadata Areas         1
Metadata Sequence No   2
VG Access               read/write
VG Status               resizable
MAX LV                 0
Cur LV                 1
```

```

Open LV          0
Max PV           0
Cur PV          1
Act PV           1
VG Size          558.37 GiB
PE Size          4.00 MiB
Total PE         142942
Alloc PE / Size  128517 / 502.04 GiB
Free PE / Size   14425 / 56.35 GiB
VG UUID          abc123...

```

#####:

- **VG Status:** **resizable**
- **VG Access:** **read/write**
- **Free PE / Size** #####

**thin provisioning pool** []

**thin provisioning pool** #####

```
oc debug node/<node-name> -- chroot /host lvs
```

#####:

LV	VG	Attr	LSize	Pool	Origin	Data%	Meta%
thin-pool-1	vg1	twi-a-tz--	<502.04g			0.00	6.76

##### (twi-a-tz--):

- **t** - thin pool
- **w** - writeable
- **i** - inherited
- **a** - active
- **t** - thin
- **z** - zero

Data% [] Meta% []:

- **Data%:** thin provisioning pool []
  - **< 80%** - []
  - **80-90%** - #####
  - **> 90%** - #####

- **Meta%:**
  - **< 80%** - 低
  - **> 80%** - 高

**thin provisioning pool** □□□□□

```
oc debug node/<node-name> -- chroot /host lvdisplay vg1/thin-pool-1
```

□ □ □ □ □ □ □ □ □ □

10/10/10

```
oc debug node/<node-name> -- chroot /host pvs
```

□□□□□:

```
PV          VG  Fmt  Attr  PSize  PFree
/dev/sda    vg1  lvm2  a--   558.37g  <55.84g
```

□□□□ (a--):

- **a** - allocatable
- - - not exported
- - - not missing

□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host pvdisplay /dev/sda
```

□ □ □ □ □ □ □ □ □

```
# Check device exists
oc debug node/<node-name> -- chroot /host ls -la /dev/sda

# Check device is a block device
oc debug node/<node-name> -- chroot /host lsblk /dev/sda

# Check device is not mounted
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda
```



LVM



```
lvmcluster    NotReady
```

##:

```
# Check detailed status
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 20
"conditions"

# Check vg-manager logs
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

#####:

#####

```
# Verify device exists
oc debug node/<node-name> -- chroot /host ls -la /dev/sda
```

##: ##### LVMCluster #####

#####

```
# Check for existing filesystem
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda
```

##: deviceSelector **forceWipeDevicesAndDestroyAllData: true** #####

#####

```
# Check if mounted
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda
```

##: #####

## **2: volume group** #####

##:

- LVMCluster ##### **vg** volume group #####
- ##### deviceClassStatuses #####

##:

```
# Check if disk was wiped
oc debug node/<node-name> -- chroot /host pvs
```

```
# Check vg-manager logs for errors
oc logs -n openshift-lvm-storage daemonset/vg-manager --tail=100

# Check device visibility
oc debug node/<node-name> -- chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE
```

000:

□ □ □ □ □ □ □ □ □ □ □ □ □ □

- `deviceSelector` `DeviceSelector` のインスタンス
- `deviceClassStatusesExcluded` (`deviceClassStatuses.excluded` の配列)

555

- **vg-manager Pod** `vg-manager`
- **SELinux** `SELinux`

□ □ □ □ □ □ □ □ □ □

- `forceWipeDevicesAndDestroyAllData: true` `boolean`
- `android.support.annotation.NonNull` `Operator` `boolean`

### 3: PVC Pending

□□:

NAME	STATUS	VOLUME	CAPACITY	ACCESS	MODES	STORAGECLASS	AGE
test-pvc	Pending					lvms-vg1	5m

00:

```
# Check PVC events
oc describe pvc <pvc-name>

# Check if pod using PVC exists
oc get pods --all-namespaces -o wide | grep <pvc-name>

# Check available space in VG
oc debug node/<node-name> -- chroot /host vgs
```

□□□□□□:

## WaitForFirstConsumer - Pod 0000000000000000

- **문제:** PVC 삭제 후 Pod 삭제하면 PVC도 Pending 상태가 됨

- **Pod: PVC** Pod **Storage**

Pod:

```
# Check VG free space
oc debug node/<node-name> -- chroot /host vgdisplay vg1 | grep "Free"
```

Pod: **Storage** PV **Storage** volume group **Storage**

**thin provisioning pool** Pod

```
# Check thin pool usage
oc debug node/<node-name> -- chroot /host lvs | grep thin-pool
```

Pod: **Storage** thin provisioning pool **Storage**

Pod:

```
# Check pod node selector vs LVMcluster node selector
oc get pod <pod-name> -o yaml | grep nodeSelector
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 5 nodeSelector
```

**Pod 4: thin provisioning pool Data% Meta% Pod**

Pod:

LV	VG	Attr	LSize	Data%	Meta%
thin-pool-1	vg1	twi-a-tz--	502.04g	92.00	85.00

Pod:

```
# Check thin pool details
oc debug node/<node-name> -- chroot /host lvs -a | grep thin-pool

# List all thin volumes
oc debug node/<node-name> -- chroot /host lvs -a -o
lv_name,lv_size,data_percent,pool_lv | grep thin-pool

# Check which PVs are consuming space
oc get pv -o custom-
columns=NAME:.metadata.name,SIZE:.spec.capacity.storage,STORAGECLASS:.spec.storageClass
sName | grep lvms
```

Pod:



#### PVC #####

```
# Find PVCs not in use
oc get pvc --all-namespaces

# Delete PVC
oc delete pvc <pvc-name> -n <namespace>
```

**thin provisioning pool ##### (VG #####)**

```
# Check VG free space
oc debug node/<node-name> -- chroot /host vgs

# Extend thin pool (if needed manually)
# Usually handled automatically by LVM operator
```

#####

LVMCluster #####overprovisionRatio #####

## 5: #####

##:

- LVMCluster #####
- ### "cannot wipe device" ### "device busy" #####

##:

```
# Check current filesystem/partition
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda

# Check for LVM signatures
oc debug node/<node-name> -- chroot /host pvs | grep /dev/sda

# Check if device is open
oc debug node/<node-name> -- chroot /host lsof | grep /dev/sda
```

###:

```
# Ensure forceWipeDevicesAndDestroyAllData is set
oc edit lvmcluster lvmcluster -n openshift-lvm-storage

# Add under deviceSelector:
forceWipeDevicesAndDestroyAllData: true
```



```
# Check lvms StorageClass annotations
oc get storageclass lvms-vg1 -o yaml | grep is-default-class
```

000:

## LVM StorageClass

```
oc patch storageclass lvms-vg1 -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

## ■■■ LVMCluster ■■■■■■■■■■

```
oc edit lvmcluster lvmcluster -n openshift-lvm-storage

# Add under deviceClasses:
- name: vg1
  default: true
```

□ □ □ □ □ □ □

[illegible]

```
#!/bin/bash

NAMESPACE="openshift-lvm-storage"
NODE_NAME=""

echo "=== LVM Storage Diagnostic Report ==="
echo ""

echo "1. LVMCluster Status:"
oc get lvmcluster -n $NAMESPACE
echo ""

echo "2. Pods Status:"
oc get pods -n $NAMESPACE
echo ""

echo "3. StorageClass:"
oc get storageclass | grep lvms
echo ""

echo "4. PVs:"
oc get pv | grep topolvm
echo ""

echo "5. Volume Groups:"
```

```
oc debug node/${NODE_NAME} -- chroot /host vgs
echo ""

echo "6. Logical Volumes:"
oc debug node/${NODE_NAME} -- chroot /host lvs
echo ""

echo "7. Physical Volumes:"
oc debug node/${NODE_NAME} -- chroot /host pvs
echo ""

echo "8. Block Devices:"
oc debug node/${NODE_NAME} -- chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT
echo ""

echo "9. Recent vg-manager logs:"
oc logs -n $NAMESPACE daemonset/vg-manager --tail=20
echo ""

echo "=== End of Diagnostic Report ==="
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- LVMClusterPodStorageClassvolume groupthin provisioning pool
- LVMCluster NotReadyvolume group Pending PVC
- vgs lvs pvs volume groupthin provisioning pool
- vg-manager CrashLoopBackOffStorageClass
- LVM

11

- LVM Operator  - 

## HostPath Provisioner

11

HostPath	Provisioner	(HPP)	OpenShift	Virtualization
	HPP		OpenShift	Virtualization Operator
		(Persistent Volume)		

HostPath Provisioner ██████

HPP □□□□□□□□□□□□□□□□

- 000000000000: 00000000000000000000000000000000
- 00000000000000: 00000000000000
- 0000000000: 00000000000000000000000000000000
- 000000000000: 0000000000000000

## LVM Storage □□□□

項目	HostPath Provisioner	LVM Storage
インストール方法	パッケージ (OS依存)	パッケージ (OS依存)
インストールコマンド	dnf	dnf
インストール先	/usr	/usr
バージョン	1.0 (最新)	1.0 (最新)
インストール場所	OS (OS依存)	OS (OS依存)
インストール場所	OS (OS依存)	OS
インストール場所	OS/OS依存	OS/OS依存

[illegible]

1111

- HostPathProvisioner 실행
- HPP 실행 StorageClass 실행
- Pod 실행
- PVC 실행 VM 실행

□□□□□□□□:

OpenShift 4.21+  
OpenShift Virtualization 4.21+

□ □ □ □

- **OpenShift 4.21+** (OpenShift Virtualization [インストール](#))
- [インストール](#): HostPathProvisioner [StorageClass](#) [インストール](#)
- [インストール](#): VM [インストール](#) Worker Node [インストール](#)
- **oc CLI**: [インストール](#)
- **virtctl CLI**: VM [インストール](#)

## OpenShift Virtualization

```
oc get csv -n openshift-cnv | grep kubevirt
```

Operator `kubevirt-hyperconverged-operator` (バージョン `v4.x.x`)

`kubevirt-hyperconverged-operator.v4.x.x`    OpenShift Virtualization    4.x.x    Succeeded

ステップ 1: SELinux 設定

HPP は SELinux を使用して、ホストパスプロビジョナーのアクセスを制御します。

ステップ A: **MachineConfig** を作成 (オプション)

MachineConfig は Worker Node に適用されます。

`hpp-machineconfig.yaml` を作成

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 50-hpp-directory
  labels:
    machineconfiguration.openshift.io/role: worker
spec:
  config:
    ignition:
      version: 3.2.0
    systemd:
      units:
        - name: create-hpp-directory.service
          enabled: true
          contents: |
            [Unit]
            Description=Create HostPath Provisioner directory
            Before=kubelet.service

            [Service]
            Type=oneshot
            ExecStart=/bin/mkdir -p /var/hpvolumes
            ExecStart=/bin/chcon -Rt container_file_t /var/hpvolumes
            RemainAfterExit=yes

            [Install]
            WantedBy=multi-user.target
EOF
```



MachineConfig

Worker

Node

ホストパスプロビジョナーのアクセスを制御するための SELinux 設定

MachineConfig を適用

```
oc get mcp worker
```

UPDATED ☒ True ☐ UPDATING ☐ False

NAME	CONFIG	UPDATED	UPDATING
DEGRADED	MACHINECOUNT	READYMACHINECOUNT	
worker	rendered-worker-abc123	True	False
6	6		False

注意 **B**: 注意 (注意)

Worker Node

```
oc get nodes -l node-role.kubernetes.io/worker
```

Worker Node (<worker-node-name> )

```
oc debug node/<worker-node-name> -- chroot /host mkdir -p /var/hpvolumes
```

```
oc debug node/<worker-node-name> -- chroot /host chcon -Rt container_file_t /var/hpvolumes
```



MachineConfig

SELinux

```
oc debug node/<worker-node-name> -- chroot /host ls -laZ /var/hpvolumes
```

:

```
total 0
drwxr-xr-x. 2 root root system_u:object_r:container_file_t:s0 6 Mar 19 10:00 .
drwxr-xr-x. 22 root root system_u:object_r:var_t:s0 4096 Mar 19 10:00 ..
```

container\_file\_t SELinux

## □□□□ 2: HostPathProvisioner □□□

HostPathProvisioner □□□□□□□□□□□□□□□□ CSI □□□□□□□□□□□□

hpp-hostpathprovisioner.yaml □□□□□□□□

```
oc apply -f - <<EOF
apiVersion: hostpathprovisioner.kubevirt.io/v1beta1
kind: HostPathProvisioner
metadata:
  name: hostpath-provisioner
spec:
  imagePullPolicy: IfNotPresent
  storagePools:
  - name: local
    path: /var/hpvolumes
  workload:
    nodeSelector:
      kubernetes.io/os: linux
EOF
```

□□□□□□□□

- **storagePools:** 1 □□□□□□□□□□□□□□□□ (□□□□□□□□□□□□□□□□)
  - **name:** StorageClass □□□□□□□□□□□□□□□□□□□□
  - **path:** □□□□□□□□□□□□□□□□□□□□□□□□
- **workload.nodeSelector:** HPP CSI Pod □□□□□□□□□□□□
- **imagePullPolicy:** □□□□□□□□□□□□□□□□

### HostPathProvisioner □□□

HostPathProvisioner □□□□□□□□□□□□□□□□

```
oc get hostpathprovisioner hostpath-provisioner
```

□□□:

NAME	AGE
hostpath-provisioner	30s

CSI Pod □ Worker Node □□□□□□□□□□□□□□□□□□□□

```
oc get pods -n openshift-cnv -l k8s-app=hostpath-provisioner
```



Worker Node (1 Pod):

NAME	READY	STATUS	RESTARTS	AGE
hostpath-provisioner-csi-xxxxx	4/4	Running	0	2m
hostpath-provisioner-csi-yyyyy	4/4	Running	0	2m
hostpath-provisioner-csi-zzzzz	4/4	Running	0	2m

### 3: StorageClass

HostPath Provisioner StorageClass

[hpp-storageclass.yaml](#)

```
oc apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: hostpath-csi
provisioner: kubevirt.io.hostpath-provisioner
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
parameters:
  storagePool: local
EOF
```

- provisioner:** `kubevirt.io.hostpath-provisioner`
- reclaimPolicy:** `Delete` PVC `Retain` PV
- volumeBindingMode:** `WaitForFirstConsumer` Pod PVC PV
- parameters.storagePool:** HostPathProvisioner

### StorageClass

```
oc get storageclass hostpath-csi
```

:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION	AGE		
hostpath-csi	kubevirt.io.hostpath-provisioner	Delete	WaitForFirstConsumer
false	10s		



5s

Pod

```
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: hpp-test-pod
  namespace: default
spec:
  containers:
  - name: test
    image: registry.access.redhat.com/ubi9/ubi:latest
    command: ["sleep", "3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: hpp-test-pvc
EOF
```

PVC

```
oc get pvc hpp-test-pvc
```

:

NAME	STATUS	VOLUME	CAPACITY	ACCESS
MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE	
hpp-test-pvc	Bound	pvc-abc12345-6789-def0-1234-567890abcdef	5Gi	RWO
hostpath-csi	<unset>	30s		



HPP

hostpath

```
oc exec hpp-test-pod -- sh -c "echo 'HPP Storage Test' > /data/test.txt && cat /data/test.txt"
```

:



```

    pod: {}
  volumes:
  - name: rootdisk
    dataVolume:
      name: hpp-test-vm-rootdisk
  - name: cloudinitdisk
    cloudInitNoCloud:
      userData: |
        #cloud-config
        user: fedora
        password: fedora
        chpasswd:
          expire: false
  dataVolumeTemplates:
  - metadata:
      name: hpp-test-vm-rootdisk
    spec:
      storage:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
        storageClassName: hostpath-csi
      source:
        registry:
          url: docker://quay.io/containerdisks/fedora:latest
EOF

```

VM □ DataVolume □□□□□□□□□□

```

# DataVolume □□□□□□□□□□
oc get dv -n default hpp-test-vm-rootdisk -w

```

□□□□□□:

NAME	PHASE	PROGRESS	RESTARTS	AGE
hpp-test-vm-rootdisk	ImportScheduled	N/A		5s
hpp-test-vm-rootdisk	ImportInProgress	0.00%		10s
hpp-test-vm-rootdisk	ImportInProgress	45.50%		30s
hpp-test-vm-rootdisk	Succeeded	100.0%		1m

VM □□□□□□□□□□

```

oc get vm hpp-test-vm
oc get vmi hpp-test-vm

```

111

NAME	AGE	STATUS	READY		
hpp-test-vm	2m	Running	True		

NAME	AGE	PHASE	IP	NODENAME	READY
hpp-test-vm	2m	Running	10.x.x.x	workerXX	True

VM 0000000000000000

```
virtctl console hpp-test-vm
```

VM □□□□□□□□□□□□□□□□

```
oc delete vm hpp-test-vm
```

□ □ □ □ □ □ □ □ □ □

**PVC** ☐ **Pending** ☐☐☐☐☐

### Pod 创建 PVC 失败 Pending 原因:

```
# PVC
oc describe pvc <pvc-name>

# HPP Pod
oc logs -n openshift-cnv -l k8s-app=hostpath-provisioner -c hostpath-provisioner
```

□□□□□□:

- **Container Engine:** `containerd` / `var/hpvolumes` `containerd`
- **SELinux** `chcon -Rt container_file_t /var/hpvolumes`
- **HPP Pod** `HPP CSI Pod` `Worker Node`

□□□□ (Permission Denied) □□□

[illegible]

```
# SELinux 設定
oc debug node/<worker-node-name> -- chroot /host ls -laZ /var/hpvolumes

# SELinux 設定
oc debug node/<worker-node-name> -- chroot /host chcon -Rt container_file_t
/var/hpvolumes
```

## HPP Pod 確認

HostPathProvisioner 確認コマンド

```
oc describe hostpathprovisioner hostpath-provisioner

# openshift-cnv namespace確認
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | tail -20
```

確認コマンド

**WaitForFirstConsumer** 確認 Pod 確認コマンド:

```
# Pod nodeSelector 確認
spec:
  nodeSelector:
    kubernetes.io/hostname: <worker-node-name>
```

確認

確認 HPP 確認:

```
# StorageClass 確認
oc delete storageclass hostpath-csi

# HostPathProvisioner 確認
oc delete hostpathprovisioner hostpath-provisioner

# 確認: MachineConfig 確認 (確認)
oc delete machineconfig 50-hpp-directory
```



HostPathProvisioner 確認 PV 確認 PVC

確認

確認:

- OpenShift 確認 HostPath Provisioner 確認
- HPP 確認 StorageClass 確認
- SELinux 確認
- Pod VM HPP 確認
- HPP 確認

HPP 確認 [LVM Storage Operator](#)

StorageClass

StorageClass

- [LVM Storage Operator](#) - OpenShift Container Platform (OCP) 4.x (OCv4) StorageClass
- [LVM Storage Operator](#) - OpenShift Container Platform (OCP) 4.x (OCv4) StorageClass
- [OpenShift Virtualization: Configuring Local Storage with HPP](#)

## StorageClass

StorageClass

OpenShift Container Platform (OCP) 4.x (OCv4) StorageClass Container Storage Interface (CSI) OpenShift Virtualization (CNV) StorageClass OpenShift storage class api Container Storage Interface

## StorageClass

OpenShift StorageClass (SC) "fast-ssd", "cheap-hdd")

PersistentVolumeClaim (PVC) StorageClass PersistentVolume (PV)

## StorageClass

- **Provisioner:** AWS ebs.csi.aws.com NFS nfs.csi.k8s.io
- **Parameters:** type: gp3, encrypted: "true"
- **ReclaimPolicy:** PVC
- **VolumeBindingMode:**
- **AllowVolumeExpansion:** true PVC

## Volume Binding Modes

(LVM/HostPath)

### 1. Immediate

- Pod PVC
- AWS us-east-1a, 1b, 1c A Pod B
- PVC



## 2. WaitForFirstConsumer (선택)

- **Pod:** Pod PVC를 사용하여 Pod를 생성할 수 있습니다.
- **Pod:** Pod가 PVC를 생성할 수 있습니다. Pod가 생성될 때 (예: Pod B) PVC를 생성할 수 있습니다. B는 Pod를 생성할 수 있습니다.
- **StorageClass:** StorageClass (LVM) HostPath StorageClass (Multi-AZ)를 생성할 수 있습니다.

## StorageClass (Reclaim Policies)

StorageClass PersistentVolumeClaim (PVC)을 생성할 수 있습니다.

### 1. Delete (선택)

- **Pod:** PVC를 사용하여 PV를 생성할 수 있습니다. (예: AWS EBS StorageClass)를 생성할 수 있습니다.
- **Pod:** Pod가 CI/CD를 생성할 수 있습니다.

### 2. Retain

- **Pod:** PVC를 사용하여 PV를 생성할 수 있습니다. Released StorageClass를 생성할 수 있습니다.
- **Pod:** Pod가 StorageClass를 생성할 수 있습니다. PV를 생성할 수 있습니다.
- **Pod:** Pod가 StorageClass를 생성할 수 있습니다. PV를 생성할 수 있습니다.

## default StorageClass 설정

StorageClass를 생성할 수 있습니다. storageClassName을 사용하여 PVC를 생성할 수 있습니다. 설정:

```
oc patch storageclass <your-class-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

StorageClass를 생성할 수 있습니다:

```
oc patch storageclass <old-default-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

StorageClass: StorageClass를 생성할 수 있습니다. SC 1 StorageClass를 생성할 수 있습니다. [Default Storage Class](#)를 생성할 수 있습니다.

## StorageClass 설정

StorageClass A: AWS EBS (선택) CSI StorageClass를 생성할 수 있습니다. gp3 StorageClass를 생성할 수 있습니다.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-fast
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

```
reclaimPolicy: Delete
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
```

OpenShift 参考: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/using-container-storage-interface-csi#persistent-storage-csi-ebs](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/using-container-storage-interface-csi#persistent-storage-csi-ebs)

項目 B: ローカル / LVM (ローカル/ローカル) Local Storage Operator ローカル LVM Storage Operator  
ローカル LVM Storage Operator

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-lvm-storage
provisioner: topolvm.io # Depends on the specific operator used
volumeBindingMode: WaitForFirstConsumer # Critical for local storage!
reclaimPolicy: Retain
parameters:
  "csi.storage.k8s.io/fstype": "xfs"
```



ローカル StorageClass ローカル LVM Storage Operator

項目 C: NFS (ローカル / RWX) NFS CSI Driver Operator ローカル NFS

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-shared
provisioner: nfs.csi.k8s.io
parameters:
  server: nfs-server.example.com
  share: /exports/data
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - nfsvers=4.1
```

OpenShift 参考: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/persistent-storage-using-local-storage](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/persistent-storage-using-local-storage)

ローカル NFS StorageClass

StorageClass 定義:

```
oc get sc
# Look for the (default) marker next to the name
```

□□□□□□□□:

```
oc describe sc <name>
```

□□□□□□□□□□□□: □□□□ PVC □□□□□□□□:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc
spec:
  accessModes: [ "ReadWriteOnce" ]
  resources: { requests: { storage: 1Gi } }
  storageClassName: gp3-fast
```

OpenShift 文档: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/configuring-persistent-storage](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/configuring-persistent-storage)

[illegible]

- **問題 1: PVC は Pending 状態のまま**
  - **原因 A: volumeBindingMode が WaitForFirstConsumer** になっているため PVC が Pod が起動するまで待っている
    - **確認:** Pod が起動したら PVC が Pending 状態から解除される
  - **原因 B: Pod が起動しないまま**
    - **確認:** `oc get pods` で Pod が Pending 状態のまま
- **問題 2: Pod が起動しない (Error)**
  - **原因:** Volume node affinity conflict
  - **確認:** Pod の `volumeBindingMode: Immediate` になっているため A Pod が起動するまで B Pod が起動しない
  - **確認:** PVC が `WaitForFirstConsumer` になっている StorageClass になっている
- **問題 3: "Provisioner not found"**
  - **原因:** `provisioner:` が CSI プロビジョナーになっている
  - **確認:** Pod の namespace (例: openshift-cluster-csi-drivers) に CSI Operator Pod が存在している
- **問題 4: StorageClass が allowVolumeExpansion が false**
  - **確認:** StorageClass が `allowVolumeExpansion` が `false` になっている
  - **確認:** StorageClass が `true` になっている Pod: Pod が起動しないまま



- `name` `name` `name`

OS `/dev/vda`, `/dev/vdb`

## 2: namespace

```
oc create namespace multi-disk-demo
```

## 3: 2 StorageClass ( )

StorageClass 1 StorageClass  
StorageClass 2 StorageClass

LVM `lvms-vg1` StorageClass  
Retain( ) reclaim policy 2 StorageClass

`sc-data-retain.yaml`

```
oc apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: lvms-vg1-retain
provisioner: topolvm.io
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
EOF
```

StorageClass

```
oc get storageclass
```

:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
lvms-vg1 (default)	topolvm.io	Delete	WaitForFirstConsumer	true	30d
lvms-vg1-retain	topolvm.io	Retain	WaitForFirstConsumer	true	5s

## 4: VM

3 VM

- **rootdisk:** 30Gi (30Gi default StorageClass) Fedora
- **datadisk:** 10Gi (10Gi default StorageClass)
- **cloudinitdisk:** cloud-init



cloud-init  
SSH

cloud-init

SSH

vm-multi-disk.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: multi-disk-vm
  namespace: multi-disk-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: multi-disk-vm
    spec:
      domain:
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
                bootOrder: 1
            - name: datadisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        resources:
          requests:
            memory: 1Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - name: rootdisk
          dataVolume:
            name: multi-disk-rootdisk
        - name: datadisk
```

```

    dataVolume:
      name: multi-disk-datadisk
  - name: cloudinitdisk
    cloudInitNoCloud:
      userData: |
        #cloud-config
        password: redhat
        chpasswd:
          expire: false
dataVolumeTemplates:
- metadata:
  name: multi-disk-rootdisk
  spec:
    pvc:
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: 30Gi
      storageClassName: lvms-vg1
    source:
      registry:
        pullMethod: node
        url: docker://quay.io/containerdisks/fedora:latest
- metadata:
  name: multi-disk-datadisk
  spec:
    pvc:
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: 10Gi
      storageClassName: lvms-vg1-retain
    source:
      blank: {}
EOF

```

□□□□□□□□

- rootdisk □□ **bootOrder: 1** □□VM □ OS □□□□□□□□□□□□□□□□□□
- rootdisk □□ **source.registry** □□Fedora □□□□□□□□□□□□□□ **Pull** □□□□
- datadisk □□ **source.blank: {}** □□□□□□□□□□□□□□□□□□□□□□□□
- □ **dataVolumeTemplates** □□□□□□□□□ **storageClassName** □□□□□□□□
- **devices.disks** □□□□□□□□□ **volumes** □□□□□□□□□□□□□□□□□□ **dataVolumeTemplates** □□□□□□□□□□□□□□□□□□□□

## ステップ 5: VM を起動する

まず DataVolume を作成し VM を起動する

```
oc get dv -n multi-disk-demo -w
```

完了 **Succeeded** として VM が起動する

```
oc get vm -n multi-disk-demo
```

確認:

NAME	AGE	STATUS	READY
multi-disk-vm	3m	Running	True

PVC を作成し StorageClass を指定する

```
oc get pvc -n multi-disk-demo
```

確認:

NAME	AGE	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
multi-disk-rootdisk	3m	Bound	pvc-xxxx	30Gi	RWO	lvms-vg1
multi-disk-datadisk	3m	Bound	pvc-yyyy	10Gi	RWO	lvms-vg1-retain

## ステップ 6: VM を起動しコンソールに接続する

VM を起動しコンソールに接続する

```
virtctl console multi-disk-vm -n multi-disk-demo
```

起動した VM は **fedora** として **redhat** の OS を起動する

起動した VM のコンソールに接続する

```
lsblk
```

確認:



NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
vda	252:0	0	30G	0	disk	
-vda1	252:1	0	1M	0	part	
-vda2	252:2	0	1.9G	0	part	[SWAP]
-vda3	252:3	0	1G	0	part	/boot
`-vda4	252:4	0	27.1G	0	part	/
vdb	252:16	0	10G	0	disk	
vdc	252:32	0	1M	0	disk	

~~~~~ **vdb**(10G~~~~~) ~~~~~ XFS ~~~~~

```
sudo mkfs.xfs /dev/vdb
```

~~~~~

```
sudo mkdir -p /mnt/data
sudo mount /dev/vdb /mnt/data
```

~~~~~

```
df -h /mnt/data
```

~~~~~:

| Filesystem | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|-------|------|------------|
| /dev/vdb   | 10G  | 105M | 9.9G  | 2%   | /mnt/data  |

~~~~~

```
sudo sh -c "echo 'Multi-disk test data' > /mnt/data/test.txt"
```

~~~~~

```
cat /mnt/data/test.txt
```

~~~~~**/etc/fstab** ~~~~~

```
sudo sh -c "echo '/dev/vdb /mnt/data xfs defaults 0 0' >> /etc/fstab"
```

Ctrl+] ~~~~~

ステップ 7: クリーンアップ

1 PVC および StorageClass および PersistentVolume を削除する

```
oc get pvc multi-disk-rootdisk -n multi-disk-demo -o jsonpath='{StorageClass:
{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity:
{.status.capacity.storage}]'
```

```
oc get pvc multi-disk-datadisk -n multi-disk-demo -o jsonpath='{StorageClass:
{.spec.storageClassName}, Volume: {.spec.volumeName}, Capacity:
{.status.capacity.storage}]'
```

クリーンアップ完了 (すべてのリソースが削除された) の確認

VM のクリーンアップ

| リソース | ストレージ | 容量 |
|--------|----------------|-------------|
| OS | SSD (OS + データ) | HDD (100GB) |
| データ | SSD (OS) | SSD (100GB) |
| VM | OS (OS + データ) | データ (100GB) |
| OS/データ | OS (OS) | データ (100GB) |

クリーンアップ

namespace multi-disk-demo、VM、PVC、DataVolume を削除する

```
oc delete namespace multi-disk-demo
```

2 StorageClass を削除する

```
oc delete storageclass lvms-vg1-retain
```

lvms-vg1-retain の `reclaimPolicy: Retain` のため、PVC は PersistentVolume に `Released` の PV として残ります

```
oc get pv | grep Released
```

削除された PV の確認

```
oc delete pv <pv-name>
```


OpenShift Virtualization

Overview

OpenShift Virtualization provides a containerized environment for running Linux-based virtual machines (VMs). It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

Architecture

The architecture consists of the following components:

- cloud-init/DHCP: Provides guest agent IP configuration.
- cloud-init: IP configuration for the VM.
- User Defined Networks (UDN): Layer2 network namespace.
- localnet: Local network namespace.
- VLAN: Virtual Local Area Network.
- Layer2: Layer2 network namespace.
- VM: Linux bridge namespace.
- PCI: PCI device namespace.
- SR-IOV: SR-IOV namespace.
- OVS: Open vSwitch.
- OpenShift Virtualization: OpenShift Virtualization namespace.

OpenShift Virtualization Network Namespace

OpenShift Virtualization provides a containerized environment for running Linux-based virtual machines (VMs). It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

- **User Defined Networks (UDN):** namespace for Layer2 network.
- **Localnet:** Local network namespace.
- **Layer2:** Layer2 network namespace.
- **Linux Bridge:** VM Linux bridge namespace.
- **SR-IOV:** SR-IOV namespace.
- **OVN-Kubernetes Overlay:** Pod network namespace.

OpenShift Virtualization

OpenShift Virtualization provides a containerized environment for running Linux-based virtual machines (VMs). It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

- OpenShift Virtualization: OpenShift 4.19+ namespace.
- NMState Operator: NMState Operator namespace.

- 网络地址转换 (NAT/VLAN) 支持
- 网络策略和防火墙规则支持

网络配置

IP 地址

cloud-init 的 DHCP 支持 QEMU guest agent 的 Windows VM 的 Sysprep 网络配置支持 IP 地址配置支持

cloud-init 网络配置支持

cloud-init 的 IP 地址配置支持

UDN 网络配置支持

User Defined Networks 支持 Layer2 网络配置支持

CUDN 网络配置 Localnet 网络配置支持

ClusterUserDefinedNetwork 支持 localnet 网络配置支持

NAD 网络配置 VLAN 网络配置 localnet 网络配置支持

NetworkAttachmentDefinition 支持 VLAN 网络配置 localnet 网络配置支持

CUDN 网络配置 VLAN 网络配置 localnet 网络配置支持

namespace 支持 ClusterUserDefinedNetwork 支持 VLAN 网络配置 localnet 网络配置支持

Layer 2 网络配置支持

VM 支持 Layer2 网络配置支持

Linux bridge

VM 支持 Linux bridge 网络配置支持

SR-IOV 网络配置支持

(Virtual Functions) 支持 VM 的 PCI 网络配置支持 SR-IOV 网络配置支持

Localnet 网络配置支持

localnet 支持 OVS 网络配置支持

VM Ingress 网络 Routes

OpenShift Ingress 支持 VM 的 HTTP 和 HTTPS 网络配置支持

网络策略和防火墙规则

- 网络策略: Pod 网络策略支持
- 防火墙规则: Pod 防火墙规则支持

OpenShift 网络

localnet 通过 Linux bridge 连接到物理网络

- Worker Node 通过 OpenShift 网络连接到物理网络
- 通过 OpenShift VLAN 连接到物理网络
- 通过 OpenShift 网络连接到物理网络

OpenShift 网络

OpenShift 网络通过 OpenShift 网络连接到物理网络

- OpenShift 网络通过 OpenShift VM 连接到物理网络
- OpenShift 网络通过 OpenShift 网络连接到物理网络

OpenShift 网络

- [Internal DNS for VMs](#) - VM 通过 OpenShift DNS 连接到物理网络
- [OpenShift Virtualization Networking](#)

OpenShift IP 地址

OpenShift

OpenShift Virtualization 通过 OpenShift (VM) 通过 IP 地址连接到物理网络。DHCP 通过 OpenShift 网络连接到物理网络。IP 地址通过 OpenShift 网络连接到物理网络。

OpenShift 5 通过 OpenShift IP 地址连接到物理网络

- **cloud-init networkData** — OpenShift (Linux VM)
- OpenShift IP 地址 UDN 通过 OpenShift — OVN-Kubernetes 通过 OpenShift IP 地址
- **DHCP** 通过 — OpenShift DHCP 通过 OpenShift MAC 通过 OpenShift IP 地址
- **nmcli** 通过 OpenShift **QEMU guest agent** — guest agent 通过 OpenShift IP 地址
- **Sysprep (Windows VM)** — **unattend.xml** 通过 OpenShift

OpenShift 网络通过 OpenShift 网络连接到物理网络

OpenShift

- OpenShift Virtualization 通过 IP 地址连接到物理网络
- cloud-init **networkData** 通过 OpenShift IP 地址
- OpenShift IP 地址 UDN 通过 OpenShift 网络连接到物理网络
- OpenShift MAC 通过 OpenShift DHCP 通过 OpenShift

- QEMU guest agent `nmcli` [NetworkManager](#) IP [NetworkManager](#)
- Sysprep [Windows VM](#) [IP](#) [NetworkManager](#)
- [NetworkManager](#)

NetworkManager

[IP](#) [NetworkManager](#)

[NetworkManager](#)

[NetworkManager](#) IP [NetworkManager](#) VM [NetworkManager](#) Pod [NetworkManager](#) OpenShift [Services](#) [Routes](#)
[NetworkManager](#) (Pod [NetworkManager](#)) [DHCP](#)
[NetworkManager](#)

[NetworkManager](#) **OpenShift** [NetworkManager](#)

[IP](#) [OVN-Kubernetes IPAM](#) [NetworkManager](#) IP
[NetworkManager](#) IP [NetworkManager](#) IPAM [NetworkManager](#)

[NetworkManager](#)

[IP](#) [NetworkManager](#) VM [NetworkManager](#) (Pod [IPAM](#) [NetworkManager](#)) [DHCP](#)
[NetworkManager](#)

[IP](#) [NetworkManager](#)

- [NetworkManager](#)
- IP [NetworkManager](#) ([NetworkManager](#)) [NetworkManager](#)
- [NetworkManager](#) DHCP [NetworkManager](#)
- [NetworkManager](#)

DHCP [OVN-Kubernetes IPAM](#) [NetworkManager](#)

- [VM](#) [NetworkManager](#) VM
- [NetworkManager](#) DHCP [NetworkManager](#)
- Kubernetes [Services](#) [DNS](#) [NetworkManager](#)

NetworkManager

- OpenShift Virtualization Operator [OpenShift 4.18](#) [NetworkManager](#)
- CLI [oc virtctl](#)
- [NetworkManager](#) 1 [NetworkManager](#) (Pod [NetworkManager](#))
- [NetworkManager](#): NetworkAttachmentDefinition (NAD) [NetworkManager](#) (Localnet VLAN [NetworkManager](#))
- DHCP [NetworkManager](#): [DHCP](#) [NetworkManager](#)
- [NetworkManager](#) (IP [NetworkManager](#) DNS)

❏ 1: cloud-init networkData

cloud-init ❏ VM ❏ cloudInitNoCloud ❏ networkData ❏ Netplan
❏ 2 ❏

❏ OpenShift Virtualization ❏ Linux VM ❏

❏

1. VM ❏ cloud-init ❏ NoCloud ❏ networkData ❏
2. cloud-init ❏
3. ❏

❏

cloud-init ❏ 2 (Netplan) ❏ 2 ❏ `ethernets`❏`bonds`❏`bridges`
❏`vlan`s ❏ `addresses`❏`dhcp4`❏`dhcp6`❏`routes`❏`nameservers`(`addresses` ❏ `search` ❏)❏`mtu`
❏`match`(`macaddress`❏`name`❏`driver`) ❏

❏

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: true
    enp2s0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.50/24
      nameservers:
        addresses:
          - 192.168.100.1
```

❏ `enp1s0`(❏ Pod ❏) ❏ DHCP ❏ `enp2s0`(❏) ❏ IP ❏

❏

`gateway4` ❏ `gateway6` ❏ Netplan 0.103 ❏ cloud-init 23.1 ❏
`routes` ❏

```
networkData: |
  version: 2
  ethernets:
    enp2s0:
      dhcp4: false
      addresses:
```



```

- 192.168.100.50/24
routes:
- to: 192.168.100.0/24
  via: 192.168.100.1
nameservers:
addresses:
- 192.168.100.1
search:
- example.com

```

search DNS routes to: default (default) (default)

Pod

1. networkData (Pod)
2. networkData (Pod)
3. DHCP (Pod)

KubeVirt networkData

KubeVirt VM networkData PCI networkData

- enp1s0 — networkData (Pod networkData)
- enp2s0 — 2 networkData (Pod networkData)
- enp3s0 — 3 networkData (2 networkData)

eth0 networkData VM spec networkData (enpXs0) networkData

networkData

cloudInitNoCloud networkData networkData

network: networkData

KubeVirt networkData version: 2 network: networkData NoCloud networkData networkData

user userData

userData cloud-init networkData userData (#cloud-config) userData

Secret networkData

networkDataSecretRef cloud-init networkData networkDataBase64

IP

IP masquerade Pod

Pod IP を指定して IP masquerade を有効にする (10.0.2.0/24) Pod NAT Pod IP を指定する

Pod (enp1s0) に DHCP を有効にする (enp2s0) Pod IP を指定する

Pod: Pod DHCP + Pod IP

Pod DHCP Pod VLAN Pod IP

Pod: Pod (localnet VLAN 100 NetworkAttachmentDefinition (NAD)) Localnet VLAN Pod

Pod namespace

```
oc create namespace static-ip-demo
```

fedora-multi-network-vm.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-multi-vm
  namespace: static-ip-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-multi-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-multi-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
```

```

disks:
  - disk:
      bus: virtio
      name: rootdisk
  - disk:
      bus: virtio
      name: cloudinitdisk
interfaces:
  - masquerade: {}
    name: default
  - bridge: {}
    name: vlan-network
  rng: {}
machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
networks:
  - name: default
    pod: {}
  - name: vlan-network
    multus:
      networkName: localnet-vlan-100
volumes:
  - dataVolume:
      name: fedora-multi-vm-disk
    name: rootdisk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        user: fedora
        password: fedora123
        chpasswd: { expire: False }
        ssh_pwauth: True
      networkData: |-
        version: 2
        ethernet:
          enp1s0:
            dhcp4: true
          enp2s0:
            dhcp4: false
            dhcp6: false
            addresses:
              - 192.168.100.50/24
            nameservers:
              addresses:
                - 192.168.100.1
    name: cloudinitdisk

```

□□□□□□□□□□

```
oc apply -f fedora-multi-network-vm.yaml
```

VM 起動完了

```
virtctl console fedora-multi-vm -n static-ip-demo
```

VM 起動完了

```
ip addr show
```

出力結果

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:7e:e2:23:d7:da brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.2/24 brd 10.0.2.255 scope global dynamic noprefixroute enp1s0
        valid_lft 86313438sec preferred_lft 86313438sec
    inet6 fe80::7e:e2ff:fe23:d7da/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether 02:7e:e2:23:d7:db brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.50/24 brd 192.168.100.255 scope global noprefixroute enp2s0
        valid_lft forever preferred_lft forever
    inet6 fe80::7e:e2ff:fe23:d7db/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

```
ip route show
```

出力結果

```
default via 10.0.2.1 dev enp1s0 proto dhcp src 10.0.2.2 metric 100
10.0.2.0/24 dev enp1s0 proto kernel scope link src 10.0.2.2 metric 100
192.168.100.0/24 dev enp2s0 proto kernel scope link src 192.168.100.50 metric 101
```

□: 仮想ネットワーク (enp1s0) □ DHCP によって自動的に割り当てられた IP 10.0.2.2
□: 仮想ネットワーク (enp2s0) □ 静的に割り当てられた IP 192.168.100.50

cloud-init 確認

cloud-init が VM 上で VirtualMachineInstance 上で VM 上の IP が QEMU guest agent が認識されているか確認

```
oc get vmi fedora-multi-vm -n static-ip-demo -o jsonpath='{.status.interfaces}' |  
python3 -m json.tool
```

確認結果

```
[  
  {  
    "infoSource": "domain, guest-agent",  
    "interfaceName": "enp1s0",  
    "ipAddress": "10.128.1.106",  
    "ipAddresses": [  
      "10.128.1.106"  
    ],  
    "mac": "02:7e:e2:23:d7:da",  
    "name": "default"  
  },  
  {  
    "infoSource": "domain, guest-agent, multus-status",  
    "interfaceName": "enp2s0",  
    "ipAddress": "192.168.100.50",  
    "ipAddresses": [  
      "192.168.100.50",  
      "fe80::7e:e2ff:fe23:d7db"  
    ],  
    "mac": "02:7e:e2:23:d7:db",  
    "name": "vlan-network"  
  }  
]
```

networkData が enp2s0 上の IP 192.168.100.50 が guest agent が認識されているか確認

VM 上で確認

```
virtctl console fedora-multi-vm -n static-ip-demo
```

VM 上:

```
cloud-init status
```



```

spec:
  domain:
    cpu:
      cores: 2
    devices:
      disks:
        - disk:
            bus: virtio
            name: rootdisk
        - disk:
            bus: virtio
            name: cloudinitdisk
      interfaces:
        - masquerade: {}
          name: default
        - bridge: {}
          name: vlan-network
          macAddress: "02:00:00:01:00:01"
      rng: {}
    machine:
      type: pc-q35-rhel9.4.0
    memory:
      guest: 4Gi
    networks:
      - name: default
        pod: {}
      - name: vlan-network
        multus:
          networkName: localnet-vlan-100
    volumes:
      - dataVolume:
          name: fedora-dhcp-reserved-vm-disk
          name: rootdisk
      - cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: fedora
            password: fedora123
            chpasswd: { expire: False }
            ssh_pwauth: True
          name: cloudinitdisk

```

cloud-init `macAddress` `networkData` DHCP IP

DHCP

DHCP MAC DHCP

ISC DHCP (`dhcpd.conf`):


```
host fedora-dhcp-reserved-vm {
  hardware ethernet 02:00:00:01:00:01;
  fixed-address 192.168.100.60;
}
```

dnsmasq 設定:

```
dhcp-host=02:00:00:01:00:01,192.168.100.60,fedora-dhcp-reserved-vm
```

設定した VM を再起動する

```
oc apply -f fedora-dhcp-reserved-vm.yaml
```

確認

VM が静的 IP を取得していることを確認する

```
virtctl console fedora-dhcp-reserved-vm -n static-ip-demo
```

VM 内:

```
ip addr show enp2s0
```

DHCP によって自動的に MAC と IP が割り当てられる

確認

```
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:00:00:01:00:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.60/24 brd 192.168.100.255 scope global dynamic enp2s0
        valid_lft 3600sec preferred_lft 3600sec
```

DHCP によって自動的に IPv6 も割り当てられる。MAC **02:00:00:01:00:01** は `macAddress` フィールドで指定されている。

例: **02:** は MAC の最初のオクテットで、`MAC` フィールドで指定されている。

確認

IP が VM によって自動的に割り当てられていることを確認する

- **(localnet Linux Bridge):** VM が自動的に MAC と IP を取得する。MAC は `MAC` フィールドで指定されている (例: `MAC 02:00:00:01:00:01`)。cloud-

init DHCP によって `nmcli` を使って IP を設定する

- **SR-IOV** について: SR-IOV によって仮想マシンがネットワークカードに直接アクセスできる。OpenShift Virtualization の [SR-IOV について](#) を参照。

Pod は Pod IP を `masquerade` によって OS のネットワークに接続する。

第 4 章: `nmcli` を使って QEMU guest agent

QEMU guest agent (KubeVirt) を使って VM の `nmcli` を使って VM のネットワークを設定する。

VM の `cloud-init` を使って VM のネットワークを設定する。

手順

1. VM の `DHCP` を使って IP を設定する
2. VM に QEMU guest agent をインストールする
3. VM の `(SSH)` を使って `nmcli` を使って IP を設定する
4. NetworkManager を使って VM のネットワークを設定する

手順

- VM の `qemu-guest-agent` をインストールする (Red Hat や Fedora の `qemu-guest-agent` を参照)
- guest agent を VM の `cloud-init` にインストールする

IP の設定

VM の `DHCP` を使って `networkData` を使って VM のネットワークを設定する。

VM の `(例: 2 つの DHCP を使って VM)`

```
virtctl console fedora-dhcp-reserved-vm -n static-ip-demo
```

guest agent のインストール

```
systemctl status qemu-guest-agent
```

guest agent のインストール

```
sudo systemctl enable --now qemu-guest-agent
```

インストールの完了

```
nmcli connection show
```

네트워크 연결 목록 ([: enp2s0 Wired connection 2])

IP 주소 설정

```
sudo nmcli connection modify "Wired connection 2" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.70/24 \
  ipv4.dns 192.168.100.1
```

네트워크 연결 활성화

```
sudo nmcli connection up "Wired connection 2"
```

VM 내부에서 실행

```
ip addr show enp2s0
```

VirtualMachineInstance 객체에서 VM 정보 가져오기

```
oc get vmi fedora-dhcp-reserved-vm -n static-ip-demo -o
jsonpath='{.status.interfaces[*].ipAddress}'
```

NetworkManager 서비스에서 VM 설정 가져오기

참고 사항

- VM 내부에서 실행
- cloud-init 설정을 사용하여 VM 설정
- VM 내부에서 실행
- cloud-init 설정을 사용하여 VM 설정



VM 내부에서 cloud-init `networkData` 설정을 사용하여 cloud-init `nmcli` 설정을 사용하여 (cloud-init 설정) VM spec `networkData` 설정을 사용하여 cloud-init 설정

5: Windows VM Sysprep

Windows VM에서 `unattend.xml` 파일을 사용하여 Microsoft Sysprep를 사용하여 Windows VM (OOBE)에서 `specialize` 단계에서 IP 주소 설정

111

1. 設定する `unattend.xml` のテンプレート
2. テンプレート `sysprep` の VM のインストール
3. Windows のインストールと初期設定

unattend.xml □□□□□□□□□□□□□□

specialize **Ethernet** IP NIC Windows VM
Identifier PowerShell
Get-NetAdapter

```
<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="specialize">
    <component name="Microsoft-Windows-TCPIP"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
      versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State">
      <Interfaces>
        <Interface wcm:action="add">
          <Identifier>Ethernet</Identifier>
          <Ipv4Settings>
            <DhcpEnabled>>false</DhcpEnabled>
          </Ipv4Settings>
          <UnicastIpAddresses>
            <IpAddress wcm:action="add" wcm:keyValue="1">192.168.100.80/24</IpAddress>
          </UnicastIpAddresses>
          <Routes>
            <Route wcm:action="add">
              <Identifier>0</Identifier>
              <NextHopAddress>192.168.100.1</NextHopAddress>
              <Prefix>0.0.0.0/0</Prefix>
            </Route>
          </Routes>
        </Interface>
      </Interfaces>
    </component>
    <component name="Microsoft-Windows-DNS-Client"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
      versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State">
      <Interfaces>
        <Interface wcm:action="add">
          <Identifier>Ethernet</Identifier>
          <DNSServerSearchOrder>
```

```

        <IpAddress wcm:action="add" wcm:keyValue="1">8.8.8.8</IpAddress>
    </DNSServerSearchOrder>
</Interface>
</Interfaces>
</component>
</settings>
</unattend>

```

□ □ □ □ □ □ □ □ □

unattend.xml ConfigMap VM spec sysprep Sysprep Windows VM
 Sysprep Windows VM OpenShift

ConfigMap

```
oc create configmap windows-sysprep --from-file=unattend.xml -n static-ip-demo
```

VM spec □□□□□□□□

```
volumes:
  - name: sysprep
    sysprep:
      configMap:
        name: windows-sysprep
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
devices:
  disks:
    - cdrom:
        bus: sata
        name: sysprep
```

□ □ □ □ □ □ □ □

- IP 地址 Windows Server VM
- 网络配置 Windows 虚拟机
- VMware 与 Hyper-V 的 Windows 虚拟机网络配置

0: Sysprep → Windows → Windows VM → IP → PowerShell → RDP →

□ □ □ □ □ □ □

□ □

| 項目 | cloud-init
networkData | UDN 項目 IP | DHCP 項目 | guest agent /
nmcli | Sysprep |
|-----------------|---------------------------|--------------------|--------------------|------------------------|------------------------|
| 対応 OS | Linux | Linux /
Windows | Linux /
Windows | Linux | Windows |
| インストール | インストール | 項目 (インストール) | 項目 (DHCP) | 項目 | 項目 (OOBE) |
| インストール後 | 項目 | 項目 | 項目 (DHCP 項目) | 項目 | 項目 |
| インストール前 | 項目 | 項目 (IPAMClaim) | 項目 (DHCP 項目) | 項目
(NetworkManager) | 項目 |
| インストール後/インストール前 | 項目 (項目) | 項目 (項目) | 項目 (項目) | 項目 (項目) | 項目 (項目) |
| インストール後/インストール前 | 項目 | 項目 | 項目 | 項目 | 項目 |
| 項目 / 項目 | 項目 | 項目 (UDN 項目) | 項目 (MAC 項目) | 項目 | 項目 (ConfigMap) |
| 項目 | VM 項目 | 項目 | DHCP 項目 | 項目 VM 項目 | 項目
(RDP/PowerShell) |

インストール後

network: インストール後 networkData

項目: **network:** インストール後cloud-init 項目 networkData インストール後

項目: KubeVirt 項目 cloudInitNoCloud インストール後networkData インストール後 version: 2 インストール後
network: インストール後 network:\n version: 2\n ethernet:\n … インストール後version:
2\nethernet:\n … インストール後

インストール前

項目: networkData 項目 enp1s0 インストール後 eth0 インストール後

項目: 項目 OS インストール後 Linux インストール後 (RHEL 8+項目Fedora項目Ubuntu
16.04+) 項目 enp1s0項目enp2s0 インストール後 VM spec インストール後

インストール後/インストール前

項目: インストール後 DHCP インストール後

項目: cloud-init 項目 networkData 項目 IP インストール後 dhcp4: false インストール後 dhcp6: false インストール後

インストール後/インストール前

項目: インストール後 インストール後

項目: インストール後 (項目 enp1s0) インストール後

cloud-init

YAML cloud-init

VM `/var/log/cloud-init.log` cloud-init YAML `cloud-init status`

DNS

`networkData` DNS

`gateway4`

`networkData` `gateway4: 192.168.100.1` cloud-init (23.1)

`gateway4` `routes` `routes: [{to: default, via: 192.168.100.1}]`
gateway6 Netplan 0.103

DHCP

DHCP VM IP

VM interface spec `macAddress` MAC KubeVirt
MAC DHCP MAC

cloud-init cloud-init

VM

```
oc delete vm fedora-multi-vm -n static-ip-demo --ignore-not-found
```

```
oc delete vm fedora-dhcp-reserved-vm -n static-ip-demo --ignore-not-found
```

namespace

```
oc delete namespace static-ip-demo
```

- OpenShift Virtualization IP

- Linux VM 通过 cloud-init `networkData` 设置 IP 地址
- 通过云提供商 IP 地址设置网络 IP 通过 UDN 地址设置网络
- 通过 IP 地址设置网络 MAC 地址设置 DHCP 地址
- QEMU guest agent 通过 `nmcli` 设置网络 IP 地址
- Sysprep 通过 `unattend.xml` 设置 Windows VM 网络 IP 地址
- 通过云提供商网络地址设置
- 通过云提供商网络地址设置

网络

- [cloud-init 网络](#)
- [cloud-init 网络 2 \(Netplan 网络\)](#)
- [cloud-init NoCloud 网络](#)
- [KubeVirt 网络 \(cloud-init\)](#)
- [KubeVirt 网络](#)
- [VM 网络 Pod 网络](#)
- [OpenShift Virtualization 网络](#)
- [OpenShift Virtualization 网络](#)
- [Microsoft Sysprep 网络 Unattend 网络](#)
- [Netplan YAML 网络](#)
- [cloud-init 网络](#)
- [Localnet VLAN 网络](#)
- [UDN 网络 Layer2 网络](#)

cloud-init 网络

通过 OpenShift Virtualization 网络 cloud-init 网络 IP 地址设置网络

网络

- [网络](#)
- [cloud-init 网络](#)
- [网络](#)
- [网络](#)

网络

VM 操作

```
oc get vm <vm-name> -n <namespace>
oc get vmi <vm-name> -n <namespace>
```

VM の IP 取得

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces[*].ipAddress}'
```

Guest Agent 操作

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.guestOSInfo}'
```

Guest Agent 操作 (Guest Agent 操作)

cloud-init 操作

VM 操作

```
virtctl console <vm-name> -n <namespace>
```

cloud-init 操作

VM 操作:

```
# cloud-init 操作
cloud-init status

# 操作:
# status: done

# 操作
cloud-init status --long
```

cloud-init 操作

```
# cloud-init 操作
cat /var/log/cloud-init.log

# 操作 (操作)
cat /var/log/cloud-init-output.log

# 操作
grep -i error /var/log/cloud-init.log
```

```
grep -i failed /var/log/cloud-init.log
```

cloud-init

```
# userdata
sudo cat /var/lib/cloud/instance/user-data.txt

# networkData (networkData)
sudo cat /var/lib/cloud/instance/network-config.txt

# cloud-init
sudo cat /var/lib/cloud/instance/cloud-config.txt
```

□ □ □ □ □ □ □ □ □ □

```
# 確認 IP アドレス
ip addr show

# 確認 ルーティング表
ip route show

# DNS 確認
cat /etc/resolv.conf

# NetworkManager 確認 (Fedora, RHEL)
nmcli device status
nmcli connection show
```

□ □ □ □ □ □ □

```
# 外部ネットワークに接続
ping -c 3 8.8.8.8

# DNS 確認
nslookup google.com

# Pod IP 確認
curl http://<vm-ip>
```

□ □ □ □ □ □ □ □ □ □ □ □

□□□□ 1: DHCP □□

00: DHCP 000 VM 0 IP 00000000

00:

```
$ oc get vmi fedora-dhcp-vm -n vms -o jsonpath='{.status.interfaces[0].ipAddress}'  
# IP address
```

root:

```
# VM console  
virtctl console fedora-dhcp-vm -n vms  
  
# IP address  
ip addr show enp1s0  
  
# DHCP logs  
journalctl -u NetworkManager | grep -i dhcp
```

root:

- DHCP logs
- IP address
- cloud-init DHCP logs

root:

1. VM cloud-init DHCP logs:

```
cloudInitNoCloud:  
  networkData: |  
    version: 2  
    ethernets:  
      enp1s0:  
        dhcp4: true
```

2. NetworkManager logs:

```
sudo nmcli connection down "Wired connection 1"  
sudo nmcli connection up "Wired connection 1"
```

3. NetworkManager logs:

```
sudo journalctl -u NetworkManager --since "5 minutes ago"
```

root: cloud-init logs

root: NetworkManager logs: device eth0 not found

00:

```
# VM 0 - 0000000000000000
ip link show
```

□□□□□□:

- OS 介面 (eth0, eth1) 介面

□□□:

1. OS 安装配置 (Linux 发行版: RHEL 8+, Fedora, Ubuntu 16.04+)
 - 网络接口: `enp1s0`
 - 2 网络接口: `enp2s0`
 - 3 网络接口: `enp3s0`
2. cloud-init 中 networkData 配置:

```
networkData: |
  version: 2
  ethernet:
    enp1s0: # eth0
      dhcp4: true
```

□□□□ 2: □□ IP □□

□□: □□ **IP** □□□□□□□□

□□: □□□□□ IP □□□□□VM □ DHCP IP □□□□□□□□

00:

```
# VM
ip addr show enp1s0

# cloud-init
grep -i "network" /var/log/cloud-init.log
```

□□□□□□:

- networkData 网络数据
- DHCP 动态主机配置协议
- networkData YAML 配置文件
- cloud-init 与 networkData 网络数据

□□□:

1. networkData □□□ (□□□□□□YAML □□) □□□:

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false # DHCP □□□□
      addresses:
        - 192.168.1.100/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

2. cloud-init □□□□□□□:

```
cloud-init status --long
cat /var/log/cloud-init.log | grep -A 5 -B 5 "network"
```

3. □□□□□□□□□□□□□:

```
sudo nmcli connection modify "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.1.100/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "8.8.8.8 8.8.4.4"

sudo nmcli connection up "Wired connection 1"
```

□□: □□ **IP** □□□□□□□□□□□□□□

□□:

```
# VM □
ip addr show enp1s0
# □□□ IP □□□□□□□

ping 192.168.1.1
# Network unreachable
```

□□:

```
# 確認コマンド
ip route show

# 確認コマンド
ping -c 1 192.168.1.1
```

確認:

- 確認コマンド
- 確認コマンド
- IP 確認コマンド
- VLAN 確認

確認:

1. cloud-init 確認コマンド
2. 確認コマンド
3. 確認コマンド IP 確認コマンド
4. VLAN 確認コマンドNetworkAttachmentDefinition (NAD) 確認コマンド

確認 3: 確認コマンド

確認: 確認コマンド

確認: 確認コマンド IP 確認コマンド

確認:

```
# VM
ip addr show

# NetworkManager 確認
nmcli connection show
```

確認:

- networkData 確認コマンド
- 確認コマンド
- 確認コマンド

確認:

1. VM 確認コマンド:

spec:

```
template:
  spec:
    networks:
      - name: default
        pod: {}
      - name: secondary-net
    multus:
      networkName: my-nad
```

2. networkData 配置:

```
networkData: |
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
    enp2s0: # 配置 IP 地址
      dhcp4: false
      addresses:
        - 192.168.100.10/24
```

3. NAD 配置:

```
oc get network-attachment-definition -n <namespace>
oc describe network-attachment-definition <nad-name> -n <namespace>
```

配置:

配置: 配置 IP 地址和 DHCP 配置

配置:

```
# VM 配置
ip route show

# 配置 IP 地址
# default via 10.128.0.1 dev enp1s0
# default via 192.168.100.1 dev enp2s0
```

配置:

- 配置 IP 地址和 DHCP 配置
- DHCP 配置 IP 地址和 DHCP 配置
- 配置 IP 地址和 DHCP 配置

配置:

1. networkData 配置示例:

```
enp2s0:
  dhcp4: false
  addresses:
    - 192.168.100.10/24
  # 'routes' 配置 - 配置示例
```

2. 配置示例:

```
enp1s0:
  dhcp4: true
  dhcp4-overrides:
    route-metric: 100 # 配置示例

enp2s0:
  dhcp4: false
  addresses:
    - 192.168.100.10/24
  routes:
    - to: default
      via: 192.168.100.1
      metric: 200 # 配置 = 配置示例
```

配置 4: VLAN 配置示例

配置: VLAN 配置示例

配置: VLAN 配置示例 (配置: **enp2s0.100**) 配置示例

配置:

```
# VM 配置
ip link show

# cloud-init 配置
grep -i vlan /var/log/cloud-init.log
```

配置:

- networkData 配置 VLAN 配置示例
- 配置示例
- VLAN 配置示例

配置:

1. networkData 配置 VLAN 配置:


```
networkData: |
  version: 2
  ethernet:
    enp2s0: {} #
  vlans:
    vlan100:
      id: 100
      link: enp2s0
      addresses:
        - 192.168.100.10/24
```

2. VLAN :

```
sudo nmcli connection add type vlan \
  con-name vlan100 \
  ifname enp2s0.100 \
  dev enp2s0 \
  id 100 \
  ip4 192.168.100.10/24
```

: **VLAN**

: **VLAN**

:

```
# VLAN
ip addr show enp2s0.100

#
tcpdump -i enp2s0 -n vlan 100
```

:

-
- **VLAN**
- **VLAN ID**
- **NAD** **VLAN**

:

1. **VLAN 100**
2. **NAD** **VLAN** :

```
oc get network-attachment-definition <nad-name> -n <namespace> -o yaml
```

"vlan": 100

3. localnet 与 OVN 配置:

```
oc get nncp -A
```

步骤 5: IPv6 配置

配置 IPv6 地址池:

配置 IPv4 地址池和 IPv6 地址池:

配置:

```
# VM 配置  
ip -6 addr show
```

配置:

- cloud-init 配置 IPv6 地址池
- OS 配置 IPv6 地址池

配置:

1. networkData 配置 IPv6 地址池:

```
networkData: |  
  version: 2  
  ethernet:  
    enp1s0:  
      dhcp4: true  
      dhcp6: true  
      # 配置 IPv6:  
      addresses:  
        - 2001:db8::10/64
```

2. IPv6 地址池配置:

```
cat /proc/sys/net/ipv6/conf/all/disable_ipv6  
# 0 (配置) 配置 IPv6
```

步骤 6: DNS 配置

配置 DNS 地址池:

配置:

```
# VM
ping 8.8.8.8
#

ping google.com
# Unknown host
```

□□:

```
cat /etc/resolv.conf

# DNS
nslookup google.com
```

□□□□□□:

- cloud-init □□□□□□□□□□□□□□□□
- DHCP □ DNS □□□□□□□□□□□□
- **/etc/resolv.conf** □□□□□□□□□□□□

□□□:

1. networkData □□□□□□□□□□□□:

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false
      addresses:
        - 192.168.1.100/24
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
      search:
        - example.com
```

2. DNS □□□□□□:

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

□□□□□□

□□□□□□□□□□□□□□

```
# VM □ - □□□□□□□□□□
echo "=== Interfaces ==="
ip addr show

echo "=== Routes ==="
ip route show

echo "=== DNS ==="
cat /etc/resolv.conf

echo "=== cloud-init Status ==="
cloud-init status --long

echo "=== cloud-init Network Config ==="
sudo cat /var/lib/cloud/instance/network-config.txt

echo "=== NetworkManager Connections ==="
nmcli connection show

echo "=== Connectivity Tests ==="
ping -c 2 8.8.8.8 && echo "IP connectivity: OK" || echo "IP connectivity: FAILED"
nslookup google.com && echo "DNS: OK" || echo "DNS: FAILED"
```

OpenShift □□□□□□□□□□

```
# VM □□□□□□□□
oc get vm,vmi <vm-name> -n <namespace>

# VM IP □□□□□□□□
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces}' | jq

# □□□□□□□□□□□□
oc get vm <vm-name> -n <namespace> -o jsonpath='{.spec.template.spec.networks}' | jq

# NADs □□□
oc get network-attachment-definition -n <namespace>

# VM □□□□□□□□
oc get events -n <namespace> --field-selector involvedObject.name=<vm-name> --sort
-by='.lastTimestamp'

# VM □□□ cloud-init □□□□□□□□
oc get vm <vm-name> -n <namespace> -o yaml | grep -A 50 cloudInitNoCloud
```

cloud-init 実行 (実行)

```
# VM 1 - cloud-init 実行 (実行)
sudo cloud-init clean --logs
sudo cloud-init init
sudo cloud-init modules --mode config
sudo cloud-init modules --mode final

# 実行確認
cloud-init status --long
```

実行結果を確認

VM 1 Linux 実行結果を確認PCI 実行結果を確認

| 実行結果 | 実行結果 | 実行結果 |
|-------------------|--------|-------------|
| 1 実行 (実行 Pod 実行) | enp1s0 | 実行結果 (DHCP) |
| 2 実行 (Multus/NAD) | enp2s0 | 実行結果 |
| 3 実行 (Multus/NAD) | enp3s0 | 3 実行結果 |
| 4 実行 (Multus/NAD) | enp4s0 | 4 実行結果 |

実行 OS 実行結果を確認 Linux 実行結果を確認 (RHEL 8+ Fedora Ubuntu 16.04+) 実行結果 enpXsY 実行結果 eth0 eth1 実行結果

cloud-init 実行結果

cloud-init v1 vs v2 実行結果

Version 1 (実行結果):

```
networkData: |
  version: 1
  config:
    - type: physical
      name: eth0
      subnets:
        - type: dhcp
```

Version 2 (実行結果):

```
networkData: |
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
```



```
[fedora@fedora ~]$ cloud-init status
status: done
```

DHCP

UDN

OpenShift Virtualization User Defined Networks (UDN) Layer2 IP namespace (Network Attachment Definition) 5

OpenShift 4.19, 4.20, 4.21

- OpenShift Virtualization Operator OpenShift 4.19
- OVN-Kubernetes (OpenShift)
- namespace UserDefinedNetwork
- CLI: `oc virtctl`

1: UDN namespace

UDN namespace User Defined Network namespace k8s.ovn.org/primary-user-defined-network OpenShift namespace



namespace

`udn-primary-namespace.yaml`

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: udn-primary-vm-guests
  labels:
    k8s.ovn.org/primary-user-defined-network: ""
EOF
```

□□□□ 2: UserDefinedNetwork □□□□□□□

Layer2 □□□□□□□□□□□□□□□□□□□□ UDN □□□□□□□□□□□□□□□□ IP □□□□ IPAM
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ namespace □□ Pod □□□□□□□□□□□□□□□□□□□□□□□□□□□□

udn-primary.yaml □□□□□□□

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: UserDefinedNetwork
metadata:
  name: udn-primary
  namespace: udn-primary-vm-guests
spec:
  topology: Layer2
  layer2:
    role: Primary
    subnets:
      - "192.168.100.0/24"
  ipam:
    lifecycle: Persistent
EOF
```

□□□□ 3: □□□□ UDN □□□□□□□□□□□□□□□□

□□□□ UDN □□□□□□□□□□□□□□□□□□□□ Python HTTP □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ UDN □ IPAM □□□□
192.168.100.0/24 □□□□□□□□□□ IP □□□□□□□□□□ UDN □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□



□□□□ cloud-init □□□ SSH □□□□□□□□□□□□□□□□

fedora-vm-with-udn.yaml □□□□□□□

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vm-with-udn
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
    network-type: udn-primary
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-udn-volume
      spec:
        pvc:
          accessModes:
```



```

- ReadOnlyOnce
resources:
  requests:
    storage: 33Gi
sourceRef:
  kind: DataSource
  name: fedora
  namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-web-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            binding:
              name: l2bridge
      resources:
        requests:
          memory: 2Gi
          cpu: 1
    networks:
      - name: default
        pod: {}
    volumes:
      - name: datavolumedisk
        dataVolume:
          name: fedora-udn-volume
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            write_files:
              - path: /etc/resolv.conf
                content: |
                  nameserver 172.30.0.10      # cluster DNS
                  search udn-primary-vm-guests.svc.cluster.local svc.cluster.local
                  options ndots:5
          user: fedora
          password: fedora
          chpasswd: { expire: False }
          packages:

```

```
- python3
runcmd:
- echo "<h1>Welcome to OpenShift Virtualization !!!</h1>" >
/root/index.html
- cd /root && nohup python3 -m http.server 80 > /dev/null 2>&1 &
EOF
```

□□□□ 4: VM □□□□□□□□□□

Kubernetes LoadBalancer VM Python HTTP



AWS

MetalLB

fedora-vm-web-service.yaml

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: fedora-vm-web-service
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
spec:
  selector:
    app: fedora-web-vm
  ports:
    - name: http
      port: 80
      targetPort: 80
      protocol: TCP
  type: LoadBalancer
EOF
```

Box 5: *Boxed text*

[illegible]

```
oc get svc
```

111

| NAME | TYPE | CLUSTER-IP | EXTERNAL-IP |
|--|--------------|---------------|-------------|
| PORT(S) | AGE | | |
| fedora-vm-web-service | LoadBalancer | 172.30.72.140 | |
| aa32981df19ad4959ae9a82fb9990db9-1996929606.ca-central-1.elb.amazonaws.com | | | |

80:32291/TCP 45m

curl コマンドを実行すると:

```
curl aa32981df19ad4959ae9a82fb9990db9-1996929606.ca-central-1.elb.amazonaws.com
```

返される:

```
<h1>Welcome to OpenShift Virtualization !!!</h1>
```

例

ネットワーク構成:

- k8s.ovn.org/primary-user-defined-network ユーザー定義 UDN namespace
- Layer2 ユーザー定義 **UserDefinedNetwork**
- IPAM ユーザー定義 UDN VM
- UDN VM LoadBalancer
- UDN IP

例

- [OpenShift -](#)
-

CUDN Localnet

例:

OpenShift 4.20

OpenShift Virtualization ClusterUserDefinedNetwork (CUDN) localnet Localnet Pod VM L2

例

- **NMState Operator:**
- **NMState CR:** Operator nmstate
- **Worker Node:** bridge mapping Worker Node

例:

ステップ 1: VM の namespace を作成

localnet を使用して VM の namespace を作成

localnet-secondary-namespace.yaml を作成

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-localnet
EOF
```

ステップ 2: bridge mapping を作成

physnet を **br-ex** を使用して bridge mapping を作成 localnet を使用して OVN-Kubernetes を作成

localnet-bridge-mapping-nncp.yaml を作成

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: localnet-bridge-mapping
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
      - localnet: localnet1
        bridge: br-ex
        state: present
EOF
```

注: localnet1 を Worker Node の **br-ex** を使用して VM を作成

ステップ 3: ClusterUserDefinedNetwork を作成

localnet を使用して ClusterUserDefinedNetwork を作成

cudn-localnet.yaml を作成

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: cudn-localnet
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
        values: ["vm-guests-localnet"]
  network:
    topology: Localnet
    localnet:
      role: Secondary
      physicalNetworkName: localnet1
      ipam:
        mode: Disabled
EOF
```

□□□□ 4: □□□□□□□□□□□□□□ VM □□□□□□

Pod □□□□□□ localnet □□□□□□□□□□□□□□□□□□□□□□□□ VM □□□□□□



□□□□ cloud-init □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ SSH □□□□□□□□□□□□

fedora-localnet-vm.yaml □□□□□□□□

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-localnet-vm
  namespace: vm-guests-localnet
  labels:
    app: fedora-localnet-vm
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-localnet-volume
      spec:
        pvc:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 33Gi
```

```

    sourceRef:
      kind: DataSource
      name: fedora
      namespace: openshift-virtualization-os-images
  template:
    metadata:
      labels:
        app: fedora-localnet-vm
    spec:
      domain:
        devices:
          disks:
            - name: datavolumedisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              bridge: {}
            - name: secondary_localnet
              bridge: {}
          resources:
            requests:
              memory: 2Gi
              cpu: 1
        networks:
          - name: default
            pod: {}
          - name: secondary_localnet
            multus:
              networkName: cudn-localnet
        volumes:
          - name: datavolumedisk
            dataVolume:
              name: fedora-localnet-volume
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                user: fedora
                password: fedora
                chpasswd: { expire: False }
                packages:
                  - python3
              runcmd:
                - echo "<h1>Welcome to OpenShift Virtualization Localnet!</h1>" >
                  /root/index.html
                - cd /root && nohup python3 -m http.server 8080 > /dev/null 2>&1 &

```

ステップ 5: VM を起動する

VM を起動するコマンドを実行する

```
# VM を起動
oc get vm -n vm-guests-localnet

# VM の IP アドレスを確認
oc get vmi -n vm-guests-localnet

# VM のコンソールに接続 (cloud-init で fedora/fedora)
virtctl console -n vm-guests-localnet fedora-localnet-vm
```

VM の IP アドレスを確認する

```
# IP アドレスを確認 - Pod (Pod) の IP アドレス (localnet)
ip addr show

# Pod - 2つのネットワーク (Pod)
# Pod: Pod: 100, Pod: 101
ip route show

# Pod:
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100
default via 192.168.2.1 dev enp2s0 proto dhcp src 192.168.2.128 metric 101

# Pod
ping 8.8.8.8

# Web
curl localhost:8080

# localnet (localnet-ip) の IP アドレス
curl <localnet-ip>:8080
```

完了

ネットワークポリシーを設定する

- **NodeNetworkConfigurationPolicy** を使用して OVN-Kubernetes の **br-ex** を bridge mapping する
- **localnet** を **ClusterUserDefinedNetwork** として定義する
- Pod が **localnet** を使用して VM と通信できるようにする


```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan
EOF
```

2: bridge mapping

localnet-vlan Worker Node br-ex bridge mapping
OVN-Kubernetes VLAN localnet
VM VLAN

localnet-vlan-bridge-mapping-nncp.yaml

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: localnet-vlan-bridge-mapping
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
      - localnet: localnet-vlan
        bridge: br-ex
        state: present
EOF
```

3: VLAN NetworkAttachmentDefinition

VLAN localnet NetworkAttachmentDefinition

localnet-vlan-100-nad.yaml

```
oc apply -f - <<EOF
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: localnet-vlan-100
  namespace: vm-guests-vlan
spec:
  config: '{
    "cniVersion": "0.3.1",
    "name": "localnet-vlan-100",

```



```

sourceRef:
  kind: DataSource
  name: fedora
  namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-vlan-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            bridge: {}
          - name: vlan_network
            bridge: {}
        resources:
          requests:
            memory: 2Gi
            cpu: 1
      networks:
        - name: default
          pod: {}
        - name: vlan_network
          multus:
            networkName: localnet-vlan-100
      volumes:
        - name: datavolumedisk
          dataVolume:
            name: fedora-vlan-volume
        - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: False }
          packages:
            - python3
            - tcpdump
            - net-tools
        runcmd:
          - echo "<h1>Welcome to OpenShift Virtualization VLAN Network!</h1>" >
/root/index.html

```

```
- echo "<p>VLAN ID: 100</p>" >> /root/index.html
- echo "<p>Network: 192.168.100.0/24</p>" >> /root/index.html
- cd /root && nohup python3 -m http.server 8080 > /dev/null 2>&1 &
```

EOF

05: VLAN 000000000000

VM 000000000000VLAN 0000000000000000000000

```
# Check the VM status
oc get vm -n vm-guests-vlan

# Check the running VM instance
oc get vmi -n vm-guests-vlan

# Access the VM console (credentials: fedora/fedora via cloud-init)
virtctl console -n vm-guests-vlan fedora-vlan-vm
```

VM 00000000VLAN 00000000000000:

```
# Check network interfaces - primary (pod network) and secondary (VLAN network)
ip addr show

# Check routes - two default gateways (lower metric prioritized)
# Primary network: metric 100, Secondary VLAN: metric 101
ip route show

# Expected output example:
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100
default via 192.168.100.1 dev enp2s0 proto dhcp src 192.168.100.50 metric 101

# Test external connectivity
ping 8.8.8.8

# Verify the web service is running
curl localhost:8080

# Test VLAN interface connectivity (replace <vlan-ip> with actual IP)
curl <vlan-ip>:8080

# Verify VLAN tagging (if tcpdump is available)
# This will show VLAN-tagged traffic on the secondary interface
sudo tcpdump -i enp2s0 -nn vlan 100
```

000000000000

□ □ □ □ □ □

1. VM VLAN 0000000000000000:

- **Worker Node** → **bridge mapping** → **VLAN ID**

2. IP 地址：

- VLAN 00000000 DHCP 00000000000000000000

3.

- 配置 VLAN 成员
- 配置 VLAN 成员

□□□□□□

```
# Check NetworkAttachmentDefinition status
oc get network-attachment-definitions -n vm-guests-vlan

# Verify bridge mapping on worker nodes
oc get nncp localnet-vlan-bridge-mapping -o yaml

# Check VM network status
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks

# View OVN-Kubernetes logs for troubleshooting
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node
```

111

[illegible]

- Worker Node `NodeNetworkConfigurationPolicy` `VLAN` bridge mapping
- localnet `VLAN` OVN-Kubernetes IPAM `NetworkAttachmentDefinition`
- `VLAN`
- Pod `VLAN` localnet VM
- `VLAN` `VLAN`

11

- OpenShift - [Container Management](#)
- OCP Virtualization [on localnet](#) [Network](#)

CUDN 設定 VLAN 設定 localnet 設定

OpenShift Virtualization ClusterUserDefinedNetwork (CUDN) VLAN Localnet namespace VLAN Pod (VM) VLAN CUDN namespace VLAN

- **NMState Operator:**
- **NMState CR:** Operator nmstate
- **Worker Node:** bridge mapping Worker Node
- **VLAN :** VLAN
- **:** OpenShift Worker Node VLAN

OpenShift 4.19

(TRUNK MODE)

Localnet VLAN OpenShift Worker Node

- : VLAN
- **VLAN :** VLAN ID (100, 200, 300)
- : VLAN

1: VM namespace

VLAN Localnet VM namespace ClusterUserDefinedNetwork (CUDN) namespace

```
# Create first namespace
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan-prod
EOF

# Create second namespace
oc apply -f - <<EOF
```

```

apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan-dev
EOF

```

ステップ 2: NIC を作成して OVS をインストール

このステップでは、VLAN を作成し、NIC (NIC) を作成して OVS をインストールします。

```

oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: br-vlan-creation
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    interfaces:
      - name: enp11s0f0np0 # Replace with your physical interface name
        type: ethernet
        state: up
        ipv4:
          enabled: false
        ipv6:
          enabled: false
        description: "Physical interface for VLAN localnet"
      - name: br-vlan
        type: ovs-bridge
        state: up
        bridge:
          port:
            - name: enp11s0f0np0
        description: "OVS bridge for VLAN traffic"
EOF

```



enp11s0f0np0 は、物理インターフェースの名前です。

```
oc debug node/<node-name> -- ip link show
```

確認

```

oc get nncp br-vlan-creation
# Expected output: STATUS: Available

```

3: bridge mapping

localnet-vlan Worker Node br-vlan OVS bridge mapping
VLAN Localnet OVN-Kubernetes

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: cudn-localnet-vlan-bridge-mapping
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
      - localnet: localnet-vlan
        bridge: br-vlan
        state: present
EOF
```

Figure 4: VLAN ClusterUserDefinedNetwork

```

namespace 00000000000000000000000000000000VLAN 00000000 Localnet 0000000000
ClusterUserDefinedNetwork 00000000

```

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: cudn-localnet-vlan-100
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
        values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"]
  network:
    topology: Localnet
    localnet:
      role: Secondary
      physicalNetworkName: localnet-vlan
      vlanID: 100
      ipam:
        mode: Disabled
EOF
```

□□□□□:

- **vlanID: 100:** 100번의 VLAN ID를 지정합니다.
- **physicalNetworkName: bridge mapping** 100번의 VLAN ID를 지정합니다 (**localnet-vlan**)
- **role: Secondary:** VM의 Pod에 Secondary 역할을 부여합니다.
- **namespaceSelector:** namespace를 지정합니다.
- **IPAM Disabled:** IP를 cloud-init (DHCP)로 설정합니다.

5: VLAN 설정을 위한 IP를 VM에 할당

namespace 1 Pod에 VLAN Localnet을 설정하여 VM에 할당



cloud-init을 사용하여 SSH 접속을 가능하게 합니다.

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cudn-vlan-vm
  namespace: vm-guests-vlan-prod
  labels:
    app: fedora-cudn-vlan-vm
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-cudn-vlan-volume
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 33Gi
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
    template:
      metadata:
        labels:
          app: fedora-cudn-vlan-vm
      spec:
        domain:
          devices:
            disks:
            - name: datavolumedisk
              disk:
                bus: virtio
            - name: cloudinitdisk
```

```

    disk:
      bus: virtio
    interfaces:
      - name: default
        bridge: {}
      - name: cudn_vlan_network
        bridge: {}
    resources:
      requests:
        memory: 2Gi
        cpu: 1
    networks:
      - name: default
        pod: {}
      - name: cudn_vlan_network
    multus:
      networkName: cudn-localnet-vlan-100
    volumes:
      - name: datavolumedisk
        dataVolume:
          name: fedora-cudn-vlan-volume
      - name: cloudinitdisk
    cloudInitNoCloud:
      networkData: |
        version: 2
        ethernet:
          enp2s0:
            addresses: ["192.168.100.10/24"]
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }
      packages:
        - python3
        - tcpdump
        - net-tools
    write_files:
      - path: /root/index.html
        content: |
          <h1>Welcome to OpenShift Virtualization CUDN VLAN Network!</h1>
          <p>VLAN ID: 100</p>
          <p>Static IP: 192.168.100.10/24</p>
          <p>Network: CUDN Localnet</p>
          <p>Namespace: vm-guests-vlan-prod</p>
        permissions: '0644'
    runcmd:
      - nohup python3 -m http.server 8080 --directory /root > /dev/null 2>&1 &

```

EOF


```

    cpu: 1
networks:
- name: default
  pod: {}
- name: cudn_vlan_network
  multus:
    networkName: cudn-localnet-vlan-100
volumes:
- name: datavolumedisk
  dataVolume:
    name: fedora-cudn-vlan-volume-dev
- name: cloudinitdisk
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernet:
      enp2s0:
        addresses: ["192.168.100.20/24"]
userData: |
  #cloud-config
  user: fedora
  password: fedora
  chpasswd: { expire: False }
  packages:
  - python3
  - tcpdump
  - net-tools
  write_files:
  - path: /root/index.html
    content: |
      <h1>Welcome to OpenShift Virtualization CUDN VLAN Network!</h1>
      <p>VLAN ID: 100</p>
      <p>Static IP: 192.168.100.20/24</p>
      <p>Network: CUDN Localnet</p>
      <p>Namespace: vm-guests-vlan-dev</p>
    permissions: '0644'
  runcmd:
  - nohup python3 -m http.server 8080 --directory /root > /dev/null 2>&1 &
EOF

```

7: VLAN

VM namespace VLAN

```

# Check VMs in production namespace
oc get vm -n vm-guests-vlan-prod
oc get vmi -n vm-guests-vlan-prod

# Check VMs in development namespace
oc get vm -n vm-guests-vlan-dev

```

```
oc get vmi -n vm-guests-vlan-dev
```

```
# Access production VM console (credentials: fedora/fedora via cloud-init)
virtctl console -n vm-guests-vlan-prod fedora-cudn-vlan-vm
```

```
# Access development VM console (credentials: fedora/fedora via cloud-init)
virtctl console -n vm-guests-vlan-dev fedora-cudn-vlan-vm-dev
```

□ VM □□□□□□VLAN □□□□□□□□□□□□

```
# Check network interfaces - primary (pod network) and secondary (VLAN network)
ip addr show
```

```
# Check routes
ip route show
```

```
# Expected output example (primary network via DHCP):
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.174 metric 100
```

```
# Test external connectivity
ping 8.8.8.8
```

```
# Verify the web service is running
curl localhost:8080
```

```
# Verify static IP assignment on VLAN interface
ip addr show enp2s0
# Expected: inet 192.168.100.10/24 (prod) or 192.168.100.20/24 (dev)
```

```
# Test VLAN interface connectivity between VMs
# From production VM (192.168.100.10) ping development VM:
ping 192.168.100.20
```

```
# Verify VLAN tagging (if tcpdump is available)
# This will show VLAN-tagged traffic on the secondary interface
sudo tcpdump -i enp2s0 -nn vlan 100
```

□□□ 8: namespace □□□□□□

□□ namespace □□ VM □□□□□ CUDN VLAN □□□□□□□□□□□□□□□□□□

```
# From production VM (192.168.100.10), test connectivity to development VM
ping 192.168.100.20
```

```
# Test web service connectivity between namespaces
curl 192.168.100.20:8080
```

```
# From development VM (192.168.100.20), test connectivity to production VM
```



```
# Check ClusterUserDefinedNetwork status
oc get clusteruserdefinednetwork

# Verify bridge mapping on worker nodes
oc get nncp cudn-localnet-vlan-bridge-mapping -o yaml

# Check VM network status in both namespaces
oc get vmi -n vm-guests-vlan-prod -o yaml | grep -A 10 networks
oc get vmi -n vm-guests-vlan-dev -o yaml | grep -A 10 networks

# View OVN-Kubernetes logs for troubleshooting
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node

# Check CUDN network attachment across namespaces
oc get network-attachment-definitions -A | grep cudn-localnet-vlan-100
```

CUDN □ NetworkAttachmentDefinition □□□

| □□ | CUDN | NetworkAttachmentDefinitio
n |
|-----------|----------------------|---------------------------------|
| □□□□ | □□□□□□□□□□ namespace | namespace □□ |
| □□ | □□□□□□ | namespace □□□□□□ |
| VLAN □□□□ | □□□□□□ vlanID □□□□□□ | □□□□□□ vlanID □□□□□□ |
| IP □□ | □□/Disabled □□□ | IPAM □□□□□□/DHCP |
| □□□□□□ | □□□□□□□□□□□□□□□□□□ | □□□□□□□□□□□□□□□□ |

□□□

□□□□□□□□□□□□□□□□□□□□□□

- □□□□□ NIC □□□□□□ OVS □□□□□□□□□□VLAN □□□□□□□□□□ bridge mapping □□□□□□□□
- □□□ namespace □□□□□□ VLAN □□□□□ □ClusterUserDefinedNetwork □□□□□□□□
- CUDN VLAN □□□□□□□□□□□□□□□□cloud-init □□□□□□□□ IP □□□□□□□□ VM □□□□□□□□□□
- □□ CUDN VLAN □□□□□□□□□□□□ VM □□□namespace □□□□□□□□□□□□
- VLAN □□□□□□□□□□□□□□ CUDN □ NetworkAttachmentDefinition □□□□□□□□□□

□□

- [OpenShift - □□□□□□□□□□□□ \(□□\)](#)
- [ClusterUserDefinedNetwork □□□ \(□□\)](#)

Layer 2 仮想ネットワーク

概要

仮想ネットワークは、**VirtualMachines** を介して **layer2** 仮想ネットワークを実現します。

VirtualMachines は **Pod** を介して仮想ネットワークを実現します。

- **namespace** (仮想ネットワーク) VM 仮想ネットワーク
- **namespace** VM 仮想ネットワーク
- VM 仮想ネットワーク **namespace** 仮想ネットワーク
- 仮想ネットワーク仮想ネットワーク仮想ネットワーク
- 仮想ネットワーク仮想ネットワーク仮想ネットワーク仮想ネットワーク
- 仮想ネットワーク仮想ネットワーク仮想ネットワーク

インストール:

OpenShift 4.20,4.21

インストール

- OpenShift Virtualization Operator を介して OpenShift 4.20 をインストール
- **NetworkAttachmentDefinition** を介して仮想ネットワーク
- CLI を介して: **oc**, **virtctl**
- Kubernetes を介して仮想ネットワーク

概要

仮想ネットワーク 3 仮想ネットワーク

- **namespace** 仮想
- **NetworkAttachmentDefinition** 仮想
- **VirtualMachines** 仮想ネットワーク
 - 仮想 **VirtualMachine** 仮想
 - 2 仮想 **VirtualMachine** 仮想
- 仮想ネットワーク仮想ネットワーク

namespace 仮想

仮想ネットワーク仮想ネットワーク **NetworkAttachmentDefinition** 仮想ネットワーク仮想ネットワーク **VirtualMachines** 仮想ネットワーク namespace 仮想 (仮想ネットワーク)仮想ネットワーク仮想ネットワーク仮想ネットワーク

YAML を介して **oc** を介して namespace 仮想ネットワーク


```
oc create namespace example-l2-secondary
```

この namespace を kube context (namespace) の namespace (OpenShift project) に設定する

```
oc project example-l2-secondary
```

namespace の context を `NetworkAttachmentDefinition` に設定する

NetworkAttachmentDefinition

2 namespace の `layer2` の `NetworkAttachmentDefinition` を定義する

`l2-network.nad.yaml` の YAML を定義する

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: l2-network
  namespace: example-l2-secondary
spec:
  config: |-
    {
      "cniVersion": "0.3.1",
      "name": "l2-network",
      "type": "ovn-k8s-cni-overlay",
      "topology": "layer2",
      "mtu": 1400,
      "netAttachDefName": "example-l2-secondary/l2-network"
    }
```

`oc apply` を使って OpenShift の namespace `example-l2-secondary` に `NetworkAttachmentDefinition` を定義する

```
oc apply -f l2-network.nad.yaml
```

`NetworkAttachmentDefinition` の `l2-network` を namespace `example-l2-secondary` に定義する

```
oc get net-attach-def l2-network -n example-l2-secondary
```

`l2-network` の `VirtualMachines` を定義する

VirtualMachines

VirtualMachines layer2 OpenShift (Kubernetes)
VirtualMachine API

l2-network 2 VM

VirtualMachine

VirtualMachine YAML (fedora-l2-secondary.vm.yaml)



cloud-init SSH

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-l2-secondary
  labels:
    app: fedora-l2-secondary
spec:
  dataVolumeTemplates:
    - metadata:
        name: fedora-l2-secondary-root
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  instancetype:
    name: u1.large
  preference:
    name: fedora
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-l2-secondary
    spec:
      domain:
        devices:
          interfaces:
            - name: default
              masquerade: {}
            - name: secondary
              bridge: {}
```

```

resources: {}
terminationGracePeriodSeconds: 180
networks:
- name: default
  pod: {}
- name: secondary
  multus:
    networkName: l2-network
volumes:
- name: fedora-l2-secondary-root
  dataVolume:
    name: fedora-l2-secondary-root
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd:
        expire: False
    networkData: |
      version: 2
      ethernets:
        enp2s0:
          addresses:
            - 192.168.20.10/24

```

VM 2

(`spec.template.spec`)

domain.devices.interfaces - 2

- **name** - `networks` `secondary`
- **bridge** `masquerade` - `bridge` Pod `{}`



networks - `interfaces` 2

- **name** - `default` `secondary`
- **pod** `multus` - `multus` `pod` `multus` `networkName`
 - **networkName** (`multus`) - `NetworkAttachmentDefinition` `l2-network`

volumes - `fedora-l2-secondary-root` VM `cloudinitdisk` 2

`oc create` namespace (example-l2-secondary) VirtualMachine

```
oc create -f fedora-l2-secondary.vm.yaml
```

example-l2-secondary VM example-l2-secondary namespace Provisioning

```
oc get vm fedora-l2-secondary -n example-l2-secondary
```



VirtualMachine

Pending VM

- DataVolume (DataVolume)
- l2-network NetworkAttachmentDefinition
- OpenShift vCPU VM

(vmi datavolume pvc) oc describe oc get -o yaml Events (oc describe) Status (oc get -o yaml)

2 VirtualMachine

2 VM

VirtualMachine YAML fedora-l2-secondary2.vm.yaml VM IP

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-l2-secondary2
  labels:
    app: fedora-l2-secondary2
spec:
  dataVolumeTemplates:
    - metadata:
        name: fedora-l2-secondary2-root
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
```

```

    requests:
      storage: 30Gi
instancetype:
  name: u1.large
preference:
  name: fedora
runStrategy: Always
template:
  metadata:
    labels:
      app: fedora-l2-secondary2
  spec:
    domain:
      devices:
        interfaces:
          - name: default
            masquerade: {}
          - name: secondary
            bridge: {}
        resources: {}
    terminationGracePeriodSeconds: 180
    networks:
      - name: default
        pod: {}
      - name: secondary
        multus:
          networkName: l2-network
    volumes:
      - name: fedora-l2-secondary2-root
        dataVolume:
          name: fedora-l2-secondary2-root
      - name: cloudinitdisk
    cloudInitNoCloud:
      userData: |
        #cloud-config
        user: fedora
        password: fedora
        chpasswd:
          expire: False
      networkData: |
        version: 2
        ethernets:
          enp2s0:
            addresses:
              - 192.168.20.20/24

```

OpenShift `openshift` namespace `VirtualMachine`

```
oc apply -f fedora-l2-secondary2.vm.yaml
```

`fedora-l2-secondary2` 2 개의 VM 이 `example-l2-secondary` namespace 00000000000000000000000000000000 VirtualMachines 00000000000000000000000000000000

```
oc get vm fedora-l2-secondary2 -n example-l2-secondary
```

000000000000000000000000

000000000000 layer2 000000000000 namespace 00000000000000000000000000000000 VM 0000 VM 00000000000000000000000000000000

0000000000example-l2-secondary namespace 00000000000000000000000000000000

- NetworkAttachmentDefinition: l2-network
- VirtualMachine: fedora-l2-secondary (000000 NIC IP 0000: 192.168.20.10)
- VirtualMachine: fedora-l2-secondary2 (000000 NIC IP 0000: 192.168.20.20)

0000000000000000 VM (000000 VM) 00000000000000000000000000000000 VM ping 000000000000

`virtctl` 000000000000 VirtualMachine 00000000000000000000000000000000: `fedora` 00000000: `fedora` 00000000 (00000000000000000000000000000000 ENTER 0000000000)0

```
virtctl console fedora-l2-secondary
```

VirtualMachine 0 IP 00000000000000000000000000000000enp2s0 00000000000000000000000000000000 192.168.20.10 000000000000

```
ip addr show
```

000 VM 0000000000 fedora-l2-secondary2 (IP 0000: 192.168.20.20) VM ping 00000000000000000000000000000000 (CTRL+C 0000 ping 000000000000)0

```
ping 192.168.20.20
```

VM 000000000000CTRL+] 00000000

00000000000000000000000000000000 namespace 000000 example-l2-secondary namespace 000000000000

```
oc project default
oc delete namespace example-l2-secondary
```

000

00000000000000000000000000000000

- ovn-k8s-cni-overlay CNI 000000000000 layer2 00000000 NetworkAttachmentDefinition 00000000

- `masquerade` `bridge` `VirtualMachines` `IP`
- `cloud-init networkData` `IP`
- `OVN` `VM` `Layer 2`
- `Linux bridge` `localnet` `Layer 2`

OpenShift

- [OpenShift Virtualization - OVN](#)
- [OpenShift -](#)

Linux bridge

OpenShift 4.20,4.21

OpenShift 4.20,4.21

`Linux bridge` `Kubernetes NMState Operator` `Linux bridge` `Multus CNI` `bridge CNI` `Pod` `VirtualMachine (VM)`

`Linux bridge` `namespace` `Pod` `VM` `2` `Linux bridge` `VM` `Pod`

`Linux bridge`

- `OpenShift` `bridge CNI`
- `VM` `NAT`
- `VirtualMachine` `VLAN`



`Linux bridge` `NetworkAttachmentDefinition` `VirtualMachine` `VLAN`

OpenShift

- `OpenShift Virtualization Operator` `OpenShift 4.20`
- `NMState Operator`
- `NodeNetworkConfigurationPolicy` `NetworkAttachmentDefinition`
- `CLI` `oc, virtctl`
- `Worker Node`

- **lnbr0** (linux bridge 0): **Linux bridge** **interface**
- **ipv4**: **IP** **address**
- **options.stp.enabled**: **STP** (Spanning Tree Protocol) **enabled**
- **port** (interface): **port** (1 to 16)
 - **name**: (interface) **name** **ens1**



```
oc create -f lnbr0-bridge.nncp.yaml
```

```

NodeNetworkConfigurationPolicy  NodeNetworkConfigurationEnactment
(ncnc)

```

oc get nnce

```
lnbr0 00000000000000000000000000000000 NodeNetworkState 000000 (000000 ip addr show 0000000000)
00000000000000000000000000000000
```

```
oc describe nns | grep lnbr0
```



Linux bridge NetworkAttachmentDefinition

```
NetworkAttachmentDefinition                                VirtualMachine
|
+----- nncp -----+
|
+----- NIC -----+ VM
|
+----- VM -----+
```

188

```
oc create namespace bridge-demo
```

namespace namespace

```
oc project bridge-demo
```

NetworkAttachmentDefinition lnbr0-bridge.net-attach-def.yaml nncp vlan ID

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: lnbr0-bridge
  namespace: bridge-demo
spec:
  config: '{
    "name": "lnbr0-bridge",
    "cniVersion": "0.3.1",
    "type": "bridge",
    "bridge": "lnbr0",
    "ipam": {},
    "vlan": 1,
    "macspoofchk": false,
    "disableContainerInterface": false,
    "portIsolation": false
  }'
```

(NetworkAttachmentDefinition lnbr0-bridge.net-attach-def.yaml)

- **metadata.name:** NetworkAttachmentDefinition Kubernetes
- **metadata.namespace:** Pod/VM namespace (bridge-demo) namespace
- **spec.config:** JSON
 - **"name":** metadata.name
 - **"cniVersion":** CNI "0.3.1"
 - **"type" (type):** Linux bridge "bridge"
 - **"bridge" (bridge):** NNCNCP/bridge CNI
 - **"ipam" (ipam):** OpenShift Virtualization bridge CNI IP (layer2)
 - **"vlan" (vlan):** VLAN ID (1)
 - **"macspoofchk" (macspoofchk):** MAC (false)

- **"disableContainerInterface"** (boolean): `disableContainerInterface` (veth peer) `disableContainerInterface` (boolean `false`)
- **"portIsolation"** (boolean): `portIsolation` Isolation `portIsolation` (boolean `false`)

Linux bridge CNI `bridge` (OpenShift Virtualization `bridge`) `bridge` CNI `bridge` `OpenShift` `bridge`



IP `IP`

- `VirtualMachine` `NetworkAttachmentDefinition` `IP` `IPAM` `OpenShift Virtualization` `"ipam": {}`
- Linux bridge `VM` `IP`
 - **cloud-init** `networkData`: `VM` `cloud-init` `IP` (boolean `IP`)
 - **DHCP** `IP`: `DHCP` `VM` `IP`
 - `nmcli` `IP`
- OpenShift Virtualization `Linux bridge bonding mode` `00506`

`lnbr0-bridge.net-attach-def.yaml` `network-attachment-definition` `OpenShift`

```
oc create -f lnbr0-bridge.net-attach-def.yaml
```

`network-attachment-definition`

```
oc get net-attach-def lnbr0-bridge
```

3 `VirtualMachine`

VirtualMachine

`VirtualMachine`

- `kubernetes-nmstate-operator` `nmstate` (boolean)
- `NodeNetworkConfigurationPolicy` `OpenShift` `Linux bridge`
- `NodeNetworkConfigurationEnactment`
- `NodeNetworkState`
- (Multus) `NetworkAttachmentDefinition` `VirtualMachine` `Linux bridge`

`VirtualMachine`

VirtualMachine

VirtualMachine YAML (kind: VirtualMachine) oc apply

VirtualMachine YAML fedora-bridge-demo.vm.yaml



cloud-init SSH

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-bridge-demo
  namespace: bridge-demo
  labels:
    app: fedora-bridge-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-bridge-demo-root
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi
 instancetype:
    name: u1.large
  preference:
    name: fedora
  template:
    metadata:
      labels:
        app: fedora-bridge-demo
    spec:
      domain:
        devices:
          disks:
            - name: fedora-bridge-demo-root
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
```

```

- masquerade: {}
  name: default
- bridge: {}
  name: nic-linux-bridge
networks:
- name: default
  pod: {}
- multus:
  networkName: lnbr0-bridge
  name: nic-linux-bridge
volumes:
- name: fedora-bridge-demo-root
  dataVolume:
    name: fedora-bridge-demo-root
- name: cloudinitdisk
  cloudInitNoCloud:
    networkData: |
      version: 2
      ethernets:
        enp2s0:
          addresses:
            - 192.168.50.10/24
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }

```

이 YAML 파일은 (spec.template.spec)에 정의된 VM의 사양을 나타냅니다.

- **domain.devices.interfaces:** 이 항목은 default NIC를 nic-linux-bridge로 설정하고 2개의 네트워크 인터페이스를 정의합니다.
 - **bridge:** VM의 NIC를 2개의 네트워크 인터페이스로 연결합니다. (1은 IP masquerade)로 vm-to-container 네트워크를 lnbr0로 연결하고 container-to-node 네트워크를 (lnbr0)로 연결합니다.
 - **name:** 이 네트워크의 이름을 지정합니다.
- **networks: interfaces** 이 항목은 네트워크 인터페이스를 정의합니다.
 - **name: interfaces** 이 항목은 네트워크 인터페이스의 이름을 지정합니다.
 - **multus.networkName:** Pod/VM가 NetworkAttachmentDefinition (NAD)을 사용하여 네트워크에 연결하는 데 사용됩니다.
- **volumes:** 2개의 볼륨이 정의됩니다. 1은 datavolumedisk로, 1은 cloudinitdisk로.
 - **CloudInitNoCloud:** 이 항목은 cloud-init을 사용하여 NoCloud를 설정합니다. KubeVirt에서는 이 항목이 사용됩니다.
 - **networkData:** 이 항목은 (enp2s0)에 IP (192.168.50.10/24)를 설정합니다.
 - **ethernets:** 이 항목은 enp2s0을 사용하여 KubeVirt에서 VM의 네트워크 인터페이스를 설정합니다.



(enp1s0)에 Masquerade를 사용하여 Pod가 IP를 할당받도록 설정합니다.

VirtualMachine 사양은 fedora-bridge-demo.vm.yaml 파일에 정의되어 OpenShift 클러스터에 배포됩니다.

```
oc apply -f fedora-bridge-demo.vm.yaml
```

VirtualMachine 仮想マシン VM によって **DataVolume** によって VM が **Running** 状態にあることを確認する。2 つの VM が起動していることを確認する。

```
oc get vm fedora-bridge-demo
```

2 つの VirtualMachine を作成

2 つの **VirtualMachine** を作成する。vm によって (仮想マシン) が IP によって識別される。この YAML ファイルは **fedora-bridge-demo2.vm.yaml** によって定義されている。

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-bridge-demo2
  namespace: bridge-demo
  labels:
    app: fedora-bridge-demo2
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-bridge-demo2-root
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  instancetype:
    name: u1.large
  preference:
    name: fedora
  template:
    metadata:
      labels:
        app: fedora-bridge-demo2
    spec:
      domain:
        devices:
          disks:
            - name: fedora-bridge-demo2-root
              disk:
```

```

        bus: virtio
      - name: cloudinitdisk
        disk:
          bus: virtio
        interfaces:
          - masquerade: {}
            name: default
          - bridge: {}
            name: nic-linux-bridge
    networks:
      - name: default
        pod: {}
      - multus:
          networkName: lnbr0-bridge
          name: nic-linux-bridge
    volumes:
      - name: fedora-bridge-demo2-root
        dataVolume:
          name: fedora-bridge-demo2-root
      - name: cloudinitdisk
    cloudInitNoCloud:
      networkData: |
        version: 2
        ethernet:
          enp2s0:
            addresses:
              - 192.168.50.20/24
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }

```

다음은 `fedora-bridge-demo2.vm.yaml` 파일의 내용을 2개의 `VirtualMachine` OpenShift 클러스터에 배포합니다.

```
oc create -f fedora-bridge-demo2.vm.yaml
```

다음 VM 클러스터에 배포된 `VirtualMachineInstance (vmi)`의 `interfaces`와 `status`를 확인합니다.

```

oc get vm -n bridge-demo
oc get vmi fedora-bridge-demo -n bridge-demo -o jsonpath='{.status.interfaces}' | jq .
oc get vmi fedora-bridge-demo2 -n bridge-demo -o jsonpath='{.status.interfaces}' | jq .

```

2개의 VM 클러스터에 배포된 VM의 상태를 확인합니다.

Linux bridge 設定手順

Linux bridge 設定手順 (Pod/VM/namespace 設定)

Linux bridge 設定手順 VirtualMachine 設定 VirtualMachine 設定

bridge-demo namespace 設定

- NetworkAttachmentDefinition: lnbr0-bridge
- VirtualMachine: fedora-bridge-demo (NIC IP: 192.168.50.10)
- VirtualMachine: fedora-bridge-demo2 (NIC IP: 192.168.50.20)

VM (NIC IP) ping

virtctl console fedora-bridge-demo user: `fedora` password: `fedora` (ENTER)

```
virtctl console fedora-bridge-demo
```

VirtualMachine IP enp2s0 192.168.50.10

```
ip addr show
```

VM fedora-bridge-demo2 (IP: 192.168.50.20) VirtualMachine ping (CTRL+C ping)

```
ping 192.168.50.20
```

VM CTRL+]

namespace bridge-demo namespace

```
oc project default
oc delete namespace bridge-demo
```

- NMState Operator NodeNetworkConfigurationPolicy Linux bridge
- bridge CNI NetworkAttachmentDefinition VM
- masquerade VirtualMachine
- cloud-init networkData IP

11

```
oc get csv -n openshift-sriov-network-operator
```

Operator CSV (openshift-sriov-network-operator):

| NAME
PHASE | DISPLAY | VERSION |
|---|-------------------------|---------|
| sriov-network-operator.v4.21.0-202605200241
202605200241 Succeeded | SR-IOV Network Operator | 4.21.0- |

Operator Pod 正在运行:

```
oc get pods -n openshift-sriov-network-operator
```

Operator webhook injector Pod 正在 Running 正在运行 SR-IOV 正在运行 Worker Node
正在运行 sriov-network-config-daemon Pod 正在运行

2: SR-IOV 正在运行

SR-IOV Network Operator 正在运行 SR-IOV 正在运行 SriovNetworkNodeState
正在运行

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator
```

Operator 正在运行:

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o yaml
```

.status.interfaces 正在运行 totalvfs 0 正在运行 PCI 正在运行
ID 正在运行 VF 正在运行

正在运行 Mellanox ConnectX-4 Lx 正在运行:

```
- deviceID: "1015"  
  driver: mlx5_core  
  linkSpeed: 25000 Mb/s  
  linkType: ETH  
  name: enp11s0f0np0  
  pciAddress: "0000:0b:00.0"  
  totalvfs: 6  
  vendor: "15b3"
```



正在运行 IOMMU 正在运行 IOMMU
正在运行:

```
oc debug node/<node-name> -- chroot /host ls /sys/kernel/iommu_groups/ | wc -l
```

0 00000000 (0000000000) 000000IOMMU 000000000000000000000000 000000 0 000000BIOS/UEFI 0 IOMMU 000000MachineConfig 000000000000000000000000 intel_iommu=on iommu=pt (Intel) 0000 amd_iommu=on iommu=pt (AMD) 000000000000

0000 3: SrioNetworkNodePolicy 0000

SrioNetworkNodePolicy 0000000000 (PF) 00000000 VF 000000000000000000000000 Operator 00000000 OpenShift Virtualization 0 VM 0000deviceType: vfio-pci 00000000VF 0 VFIO 000000000000000000000000 PCI 0000000000000000

0000000000000000 4 00 VF 00000000 00000000 VF 0000 VLAN 0000000000VLAN 0000000000 SrioNetwork 000000000000 0000VF 000000000000 VLAN 0000000000000000

srio-network-node-policy.yaml 00000000

```
oc apply -f - <<EOF
apiVersion: srio.network.openshift.io/v1
kind: SrioNetworkNodePolicy
metadata:
  name: mlx-srio-policy
  namespace: openshift-srio-network-operator
spec:
  resourceName: mlxsriovnic
  nodeSelector:
    kubernetes.io/hostname: <node-name>
  numVfs: 4
  nicSelector:
    vendor: "15b3"
    deviceID: "1015"
    pfNames:
      - enp11s0f0np0
  deviceType: vfio-pci
  isRdma: false
EOF
```

<node-name> 00SR-IOV NIC 0000000000000000 (0: ocplab01) 0000000000000000

0000000000:

- resourceName — VF 00000000 Kubernetes 0000000000 (0: openshift.io/mlxsriovnic)0
- numVfs —0000 VF 00 (0000000000 totalvfs 000000000000)0000 VM 0 2 00 VF (VLAN 0000 1 0) 0000000000 2 000000 VM 00000000
- nicSelector.vendor / deviceID — 0000 NIC 0000000000000000Mellanox ConnectX-4 Lx 0000 15b3 / 1015 00Intel NIC 000000000000 ID 000000000000
- nicSelector.pfNames — 0000000000000000VF 0000000000000000000000

sriov-demo-namespace.yaml

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: sriov-demo
EOF
```

VLAN 100

sriov-net-data.yaml

```
oc apply -f - <<EOF
apiVersion: sriovnetwork.openshift.io/v1
kind: SriovNetwork
metadata:
  name: sriov-net-data
  namespace: openshift-sriov-network-operator
spec:
  networkNamespace: sriov-demo
  resourceName: mlxsrionnic
  vlan: 100
  spoofChk: "off"
  trust: "on"
EOF
```

VLAN 200

sriov-net-mgmt.yaml

```
oc apply -f - <<EOF
apiVersion: sriovnetwork.openshift.io/v1
kind: SriovNetwork
metadata:
  name: sriov-net-mgmt
  namespace: openshift-sriov-network-operator
spec:
  networkNamespace: sriov-demo
  resourceName: mlxsrionnic
  vlan: 200
  spoofChk: "off"
  trust: "on"
EOF
```

- **vlan**— VF 802.1Q VLAN ID NIC OS


```

    name: fedora
    namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-sriov-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            masquerade: {}
          - name: data-plane
            sriov: {}
          - name: mgmt-plane
            sriov: {}
        resources:
          requests:
            memory: 4Gi
            cpu: 2
    networks:
      - name: default
        pod: {}
      - name: data-plane
        multus:
          networkName: sriov-net-data
      - name: mgmt-plane
        multus:
          networkName: sriov-net-mgmt
    volumes:
      - name: datavolumedisk
        dataVolume:
          name: fedora-sriov-volume
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            user: fedora
            password: fedora
            chpasswd: { expire: False }
EOF

```

VM 3

- **default** (masquerade: {}) — Pod IP address translation (Pod) address translation
- **data-plane** (sriov: {}) — SR-IOV VF address translation VLAN 100
- **mgmt-plane** (sriov: {}) — SR-IOV VF address translation VLAN 200

SR-IOV address translation 1 VF address translation VM 4 address translation VF 2 address translation VM address translation

VM address translation:

```
oc get vm -n sriov-demo
```

```
oc get vmi -n sriov-demo
```

VMI is **Running** address translation **READY: True** address translation

7: VLAN SR-IOV address translation

SR-IOV VF address translation PCI address translation address translation VLAN address translation

PCI address translation

VM address translation:

```
virtctl console -n sriov-demo fedora-sriov-vm
```

fedora / fedora address translation VF is PCI address translation address translation:

```
lspci | grep -i ethernet
```

Pod address translation virtio NIC address translation Mellanox (Intel) Ethernet VF address translation address translation VF address translation address translation PCI address translation

address translation

address translation:

```
ip link show
```

address translation:

- **enp1s0** — Pod address translation (virtio address translation)
- **enp7s0** — SR-IOV VF (address translation VLAN 100)
- **enp8s0** — 2 SR-IOV VF (address translation VLAN 200)



このセクションではPCI 対応のネットワークカード **udev** を使用してネットワークカードの **ip link show** コマンドで SR-IOV VF を確認 (virtio 対応の Mellanox 対応 Intel ネットワークカード)。

IP アドレスを割り当てる

SR-IOV ネットワークカード IP アドレスを VLAN ネットワークカードに割り当てる。

ネットワークカード (VLAN 100) を確認:

```
sudo ip addr add 10.100.0.10/24 dev enp7s0
```

```
sudo ip link set enp7s0 up
```

ネットワークカード (VLAN 200) を確認:

```
sudo ip addr add 10.200.0.10/24 dev enp8s0
```

```
sudo ip link set enp8s0 up
```

ネットワークカードの IP アドレスを確認:

```
ip -br addr show
```

結果:

| | | |
|--------|---------|---------------------|
| lo | UNKNOWN | 127.0.0.1/8 ::1/128 |
| enp1s0 | UP | 10.0.2.2/24 |
| enp7s0 | UP | 10.100.0.10/24 |
| enp8s0 | UP | 10.200.0.10/24 |

コマンド: `ip addr add` ネットワークカードの IP アドレスを割り当てる。

- **DHCP** — VLAN の DHCP を NetworkManager を使用して設定。
- **cloud-init** — VM の cloud-init を使用して `networkData` を使用して IP アドレスを割り当てる。
- **NetworkManager** — cloud-init の `write_files` を使用して NetworkManager を設定。

VLAN 配置

配置 VLAN 100 和 VLAN 200 并 ping 测试

配置 VLAN (100) 配置:

```
ping -c 3 -I enp7s0 10.100.0.1
```

配置 VLAN (200) 配置:

```
ping -c 3 -I enp8s0 10.200.0.1
```

配置 ping 测试 VLAN 100 和 VLAN 200 配置

配置 VLAN 200 配置 (配置) 配置

配置 `Ctrl+]` 配置

配置

配置 VF 配置

- `SriovNetworkNodeState` 配置 `Failed` 配置 NIC 配置
- BIOS/UEFI 配置 IOMMU 配置
- 配置 `nicSelector` 配置 (配置 ID PF) 配置

VM 配置

- VF 配置: `oc get node <node-name> -o jsonpath='{.status.allocatable}'`
- 配置 0 配置 VF 配置
- 配置 `deviceType: vfio-pci` 配置 VM 配置 VFIO 配置
- `Insufficient openshift.io/mlxsriovnic` 配置 VM 配置 VF 配置 VM 配置 SR-IOV 配置 `numVfs` 配置

VM 配置 SR-IOV 配置

- `SriovNetwork` 配置 `resourceName` 配置 `SriovNetworkNodePolicy` 配置 `resourceName` 配置
- 配置 NAD 配置 `namespace` 配置: `oc get net-attach-def -n sriov-demo`
- `virt-launcher` Pod 配置: `oc describe pod -n sriov-demo -l app=fedora-sriov-vm`

SR-IOV 配置

- 配置 NIC 配置: `oc debug node/<node-name>-chroot /host ip link show <pf-name>`
- 配置 VLAN 100 配置 200 配置

- Mellanox NIC 設定を完了します。

VLAN 設定と VM 設定

- **SriovNetwork** の **vlan** 設定を **VLAN** 設定に反映させます。
- 設定を反映させる (リロード) するには **VLAN ID** を設定します。
- 設定を反映させる **VF** 設定を **SriovNetwork** の **spoofChk** を **"off"**、**trust** を **"on"** に設定します。

削除

削除するコマンド:

```
oc delete vm fedora-sriov-vm -n sriov-demo
```

```
oc delete sriovnetwork sriov-net-data -n openshift-sriov-network-operator
```

```
oc delete sriovnetwork sriov-net-mgmt -n openshift-sriov-network-operator
```

```
oc delete sriovnetworknodepolicy mlx-sriov-policy -n openshift-sriov-network-operator
```

SR-IOV 設定を確認するには (VF 設定を確認するには):

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o
jsonpath='{.items[0].status.syncStatus}'
```

namespace 削除:

```
oc delete namespace sriov-demo
```

確認

確認するコマンド:

- SR-IOV の PCI 設定を VF から VM 設定に反映させます。
- **SriovNetworkNodeState** を確認して SR-IOV 設定を確認します。
- VM 設定を **deviceType: vfio-pci** に設定して **SriovNetworkNodePolicy** を確認します。
- SR-IOV の VLAN 設定を確認 (1 つの VLAN を設定して **SriovNetwork** を確認して VF の VLAN を確認)
- 設定 (VLAN 100) を確認 (VLAN 200) を確認して **SriovNetwork** を確認します。

- 透過 **VLAN** 實現 **SR-IOV** 透過網路連接 **VM** 透過網路
- 透過 **PCI** 實現 **VLAN** 透過網路連接

11

- [OpenShift](#) [ネットワーク](#)
- [VM](#) [SR-IOV](#) [ネットワーク](#)
- [KubeVirt](#) [SR-IOV](#) [ネットワーク](#)
- [Localnet](#) [ネットワーク](#) — [ネットワーク](#)
- [Localnet](#) [VLAN](#) [ネットワーク](#) — [localnet](#) [VLAN](#) [ネットワーク](#) ([ネットワーク](#))
- [Linux bridge](#) — [ネットワーク](#)

VM Ingress ☐☐☐ Routes



OpenShift Virtualization VM OpenShift Routes Service
Web VM ClusterIP Service TLS OpenShift Route

OpenShift Ingress VM Pod

- **OpenShift 4.18+** (OpenShift Virtualization Operator)
- (system:admin)
- **oc CLI** or **virtctl CLI**
- **default StorageClass**
- **Ingress Controller** (OpenShift)

OpenShift 4.20

1: namespace

```
oc create namespace ingress-demo
```

2: Web VM

cloud-init Nginx Web Fedora VM Service VM



cloud-init SSH

vm-webserver.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  namespace: ingress-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
```

```

    kubevirt.io/domain: web-vm
    app: web-vm
spec:
  domain:
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 1Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        containerDisk:
          image: quay.io/containerdisks/fedora:latest
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            password: redhat
            chpasswd:
              expire: false
            packages:
              - nginx
            write_files:
              - path: /usr/share/nginx/html/index.html
                content: |
                  <html>
                  <head><title>OpenShift Virt - Ingress Demo</title></head>
                  <body>
                  <h1>Hello from OpenShift Virtualization</h1>
                  <p>This page is served by Nginx inside a VM, exposed via an OpenShift
Route.</p>
                  </body>
                  </html>
            runcmd:
              - systemctl enable nginx
              - systemctl start nginx
EOF

```

`spec.template.metadata.labels` `app: web-vm` `virt-launcher` `Pod` `Service`
○○○○○○○○○○○○○○○○○○

VM ○○○○○○○○○○

```
oc get vm -n ingress-demo -w
```

VM `Running` `Ready=True` ○○○○○○○○○○○○○○○○○○○`Ctrl+C` ○○○○○○○○○○○○

○○○○ **3: Web** ○○○○○○○○

cloud-init `Nginx` ○○○○○○○○○○○○○○○○○○○ (VM `Running` ○○○○○○○○ 102 ○○○○○○○○○○○○)

VM ○○○○○○○○○○○○

```
virtctl console web-vm -n ingress-demo
```

○○○○ `fedora` ○○○○○ `redhat` ○○○○○○○○○○○○○○○○○○○

```
curl -s http://localhost
```

○○○○ `HTML` ○○○○○○○○○○○○○○○○○`Ctrl+]` ○○○○○○○○○○○○○○○○○

○○○○ **4: Service** ○○○○

VM ○○○○ 80 ○○○○○○○○○ `ClusterIP Service` ○○○○○○○Service ○○○○○○○VM ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

`service-web-vm.yaml` ○○○○○○○○

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: web-vm-svc
  namespace: ingress-demo
spec:
  selector:
    app: web-vm
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
EOF
```

Service ○○○○○○○○○○○○○○

```
oc get svc web-vm-svc -n ingress-demo
```

□□□□□□□□□□□□□□□□

```
oc get endpoints web-vm-svc -n ingress-demo
```

□□□□□□:

| NAME | TYPE | CLUSTER-IP | EXTERNAL-IP | PORT(S) | AGE |
|------------|-----------|------------|-------------|---------|-----|
| web-vm-svc | ClusterIP | 172.30.x.x | <none> | 80/TCP | 5s |

| NAME | ENDPOINTS | AGE |
|------------|---------------|-----|
| web-vm-svc | 10.128.x.x:80 | 5s |

| NAME | ENDPOINTS | AGE |
|------------|---------------|-----|
| web-vm-svc | 10.128.x.x:80 | 5s |

□□□□□□ IP □ VM □ Pod IP □□□□□□Service □ VM □□□□□□□□□□□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc run curl-test --rm -it --restart=Never
--image=registry.access.redhat.com/ubi9/ubi:latest -n ingress-demo -- curl -s
http://web-vm-svc.ingress-demo.svc.cluster.local
```

HTML 00000000000000000000

□□□□ 5: OpenShift Route □□□□ Service □□

HTTP Route (□□□□□□□□□□□□□□□□)

TLS Route OpenShift Route

route-web-vm.yaml

```
oc apply -f - <<EOF
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web-vm-route
  namespace: ingress-demo
spec:
  to:
    kind: Service
    name: web-vm-svc
    weight: 100
  port:
    targetPort: 80
  tls:
```



```

termination: edge
insecureEdgeTerminationPolicy: Redirect
wildcardPolicy: None
EOF

```

□ □ □ □ □ □ □ □

- **spec.to.name:** 容器 4 个 Service 容器
- **spec.port.targetPort:** Service 端口 (80) 容器端口
- **tls.termination: edge:** TLS 在 Router 容器 Router 的 VM 容器 HTTP 容器
- **insecureEdgeTerminationPolicy: Redirect:** HTTP 容器 HTTPS 容器

Route

```
oc get route web-vm-route -n ingress-demo
```

□□□□□□:

| NAME | HOST/PORT | PATH | SERVICES |
|--------------|---|----------|------------|
| PORT | TERMINATION | WILDCARD | |
| web-vm-route | web-vm-route-ingress-demo.apps.ocplab01.example.com | | web-vm-svc |
| 80 | edge/Redirect | None | |

□ □ □ □ □ □ □ □ □ □

Route □□□□□□□□□□

```
ROUTE_HOST=$(oc get route web-vm-route -n ingress-demo -o jsonpath='{.spec.host}')
echo "Route URL: https://$ROUTE_HOST"
```

```
curl -sk https://$ROUTE_HOST
```

VM 上 Nginx 安装 HTML 静态资源



DNS 配置 Route 配置 Ingress IP 配置

`/etc/hosts` `curl --resolve`

図表 6: 図表 TLS (図表)

VM TLS Router VM

```
oc apply -f - <<EOF
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web-vm-passthrough
  namespace: ingress-demo
spec:
  to:
    kind: Service
    name: web-vm-svc
    weight: 100
  port:
    targetPort: 443
  tls:
    termination: passthrough
    wildcardPolicy: None
EOF
```



OpenShift VM 通过 OpenShift Ingress Controller 的 HTTPS 路由（Route）通过 Nginx 代理 HTTP 流量。Route 通过 OpenShift Ingress Controller 的 VM 的 TLS 代理 Nginx 代理流量。

□□□

OpenShift Route 通过 OpenShift VM 代理流量。

1. OpenShift Route 通过 OpenShift HTTPS 代理流量。
2. OpenShift Ingress Controller (HAProxy Router) 通过 TLS 代理流量 (OpenShift)。
3. Router 通过 OpenShift ClusterIP Service 通过 80 代理流量。
4. Service 通过 OpenShift virt-launcher Pod 通过 IP 通过 80 代理流量。
5. virt-launcher Pod 通过 OpenShift masquerade 代理流量。OpenShift VM 通过 Nginx 代理流量。

OpenShift Ingress Controller 通过 OpenShift VM 代理流量。

OpenShift Ingress Controller

Route 503 Service Unavailable

Service 通过 OpenShift Ingress Controller 代理流量。

```
oc get endpoints web-vm-svc -n ingress-demo
```

OpenShift Service Selector 通过 VM 代理流量。

```
oc get pods -n ingress-demo -l app=web-vm
```

Route 504 Gateway Timeout

VM 504 Gateway Timeout VM 504 Gateway Timeout

```
virtctl console web-vm -n ingress-demo
```

VM Nginx 504 Gateway Timeout

```
systemctl status nginx  
curl -s http://localhost
```

DNS

Route Ingress DNS

```
oc get ingresscontroller default -n openshift-ingress-operator -o  
jsonpath='{.status.domain}'
```

Route `<route-name>-<namespace>.<ingress-domain>`

```
oc delete namespace ingress-demo
```

- cloud-init Web VM
- Kubernetes Service VM
- HTTPS TLS OpenShift Route
- VM TLS
- VM Service Route

- [Exposing a VM by Using a Service - OpenShift](#)
- [Route Configuration - OpenShift](#)

Localnet 設定

環境:

OpenShift 4.20

OpenShift Virtualization 環境 Localnet 環境 OVS (Open vSwitch) bridge 環境

OVS bridge と Linux bridge

概要: OVS bridge と Linux bridge

- OVS bridge `ovs-vsctl` で設定
- OVS bridge `ip addr show` と `ip link show` で確認
- OVS bridge と Linux bridge の違い

手順

ステップ 1: NodeNetworkConfigurationPolicy

NNCP の設定

```
# NNCP の取得
oc get nncp

# NNCP の設定
oc get nncp br-vlan-creation
```

結果:

| NAME | STATUS | REASON |
|------------------|-----------|------------------------|
| br-vlan-creation | Available | SuccessfullyConfigured |

ステップ 2: NNCP

NNCP の設定

```
oc get nncp br-vlan-creation -o yaml
```

結果:

```
status:
conditions:
```

```
- lastHeartbeatTime: "2025-10-06T20:38:57Z"
  lastTransitionTime: "2025-10-06T20:38:57Z"
  message: 1/1 nodes successfully configured
  reason: SuccessfullyConfigured
  status: "True"
  type: Available
- reason: SuccessfullyConfigured
  status: "False"
  type: Degraded
```

- **Available: True** == **SuccessfullyConfigured** = 正常
- **Degraded: False** = 不正常
- **message: X/X nodes successfully configured** = 成功配置了X/X个节点

实验 3: OVS bridge

实验 OVS bridge 配置:

```
# 查看节点
oc get nodes

# 查看OVS bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-br
```

结果:

```
Starting pod/ocplab01-debug-k7bfc ...
To use host binaries, run `chroot /host`
br-ex
br-int
br-vlan

Removing debug pod ...
```

可以看到 **br-vlan** 是 OVS bridge 的一部分

实验 4: bridge

OVS bridge 配置:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl show
```

结果 (输出):

```
Bridge br-vlan
```

```
Port enp11s0f0np0
Interface enp11s0f0np0
type: system
```

□□□□:

- bridge □ (**br-vlan**) □□□□□□□□
- □□□□□□□□ (□: **enp11s0f0np0**) □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□ **system** □□□□□

□□□□ 5: □□□□□□□□□□□□□□□□

□□□□□□□□□□ **bridge** □□□□□□□□□□**UP** □□□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host ip link show <interface-name>
```

□□□:

```
4: enp11s0f0np0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master ovs-system
state UP mode DEFAULT group default qlen 1000
    link/ether e8:eb:d3:13:06:16 brd ff:ff:ff:ff:ff:ff
```

□□□□:

- **UP,LOWER_UP** - □□□□□□□□□□□□□□□□
- **master ovs-system** - □□□□□□□□ **OVS** □□□□□□□□□□□□
- **state UP** - □□□□□□□□□□□□□□□□□□

□□□□ 6: "**ip addr show br-vlan**" □□□□□□□□

□□□□□□□□□□:

```
oc debug node/<node-name> -- chroot /host ip addr show br-vlan
```

□□□□□□□□:

```
Device "br-vlan" does not exist.
```

□□:

- **OVS bridge** □□□□ **Linux** □□□□□□□□□□□□□□□□**OVS** □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□ **ip** □□□□□□□□□□□□□□
- □□□□ **bridge** □□□□□□□□□□□□□□□□□□

- OVS bridge `ovs-vsctl` `bridge`

1. `ovs-vsctl list-br`

1: `ovs-vsctl list-br` `bridge`

2:

- NNCP `Available: True` `bridge` `ovs-vsctl list-br`

3:

```
# NNCP
oc get nncp br-vlan-creation -o
jsonpath='{.status.conditions[?(@.type=="Available")]}'|jq

# NMState Operator
oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator

# 
oc get nns -A
```

4:

- `ip link show`
- OVS
- NMState Operator

2: `bridge`

3:

- `bridge` `ovs-vsctl show`

4:

```
# bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-ports br-vlan

# 
oc debug node/<node-name> -- chroot /host ip link show <interface-name>

# NNCP
oc get nncp br-vlan-creation -o yaml | grep -A 5 "name: <interface-name>"
```

5:

1. NNCP

2. `bridge` `bridge` `bridge`

3. `bridge` `bridge` `bridge`

3: `NodeSelector` `NodeSelector`

`NodeSelector`:

- `NNCP` `bridge` `bridge`

`NodeSelector`:

```
# NodeSelector
oc get nodes --show-labels

# NNCP NodeSelector
oc get nncp br-vlan-creation -o jsonpath='{.spec.nodeSelector}'
```

`NodeSelector`:

- `Single Node OpenShift (SNO)` `control-plane, master, worker` `NodeSelector`
- `node-role.kubernetes.io/worker: ""` `NNCP` `NodeSelector`
- `NNCP` `"X/X nodes successfully configured"` `NodeSelector`

4: `bridge` `bridge`

`bridge`:

- `bridge` `bridge` `bridge`

`bridge`:

```
# bridge
oc debug node/<node-name> -- chroot /host cat /sys/class/net/<interface-name>/carrier

# bridge
oc debug node/<node-name> -- chroot /host ethtool -S <interface-name>

# bridge
oc debug node/<node-name> -- chroot /host ethtool <interface-name> | grep "Link detected"
```

`bridge`:

- `bridge` `bridge`
- `bridge` `bridge` `bridge`
- `bridge` `bridge` `bridge`

Localnet の bridge mapping

OVS bridge を利用して OVN-Kubernetes の bridge mapping を設定します。

bridge mapping

bridge mapping (NetworkAttachmentDefinitions) を OVS bridge に設定します。

- 名前: `localnet-vlan`
- bridge: `br-vlan`
- mapping: `localnet-vlan:br-vlan`

ステップ 1: bridge mapping の NNCP

bridge mapping の NNCP を設定します。

```
# NNCP を取得
oc get nncp | grep mapping

# NNCP の bridge mapping を取得
oc get nncp localnet-vlan-bridge-mapping
```

結果:

```
localnet-vlan-bridge-mapping  Available  SuccessfullyConfigured
```

ステップ 2: bridge mapping の YAML

```
oc get nncp localnet-vlan-bridge-mapping -o yaml
```

結果:

```
spec:
  desiredState:
    ovn:
      bridge-mappings:
      - bridge: br-vlan
        localnet: localnet-vlan
        state: present
  nodeSelector:
    node-role.kubernetes.io/worker: ""
status:
  conditions:
  - message: 1/1 nodes successfully configured
    reason: SuccessfullyConfigured
    status: "True"
```

type: Available

Figure 3: OVN bridge mapping

OVN-Kubernetes bridge mapping:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .  
external_ids:ovn-bridge-mappings
```

:

```
"localnet-vlan:br-vlan,physnet:br-ex"
```

:

- **localnet-vlan:br-vlan** = mapping
- **physnet:br-ex** = OpenShift mapping
- mapping

Figure 4: OVS External ID

OVS external ID:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list Open_vSwitch | grep -A 2  
external_ids
```

:

```
external_ids      : {hostname=ocplab01, ovn-bridge-mappings="localnet-vlan:br-  
vlan,physnet:br-ex", ovn-bridge-remote-probe-interval="0", ovn-enable-lflow-  
cache="true", ovn-encap-ip="192.168.2.241", ovn-encap-type=geneve, ovn-is-  
interconn="true", ovn-memlimit-lflow-cache-kb="1048576", ovn-monitor-all="true", ovn-  
ofctrl-wait-before-clear="0", ovn-remote="unix:/var/run/ovn/ovnsb_db.sock", ovn-  
remote-probe-interval="180000", ovn-set-local-ip="true",  
rundir="/var/run/openvswitch", system-id="1b41fc3f-f597-4cd9-b508-e8247c209d97"}
```

:

- **ovn-bridge-mappings** mapping
- **hostname**
- **ovn-encap-type=geneve**

Figure 5: Bridge configuration

Configure the bridge configuration:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list-ports br-vlan
```

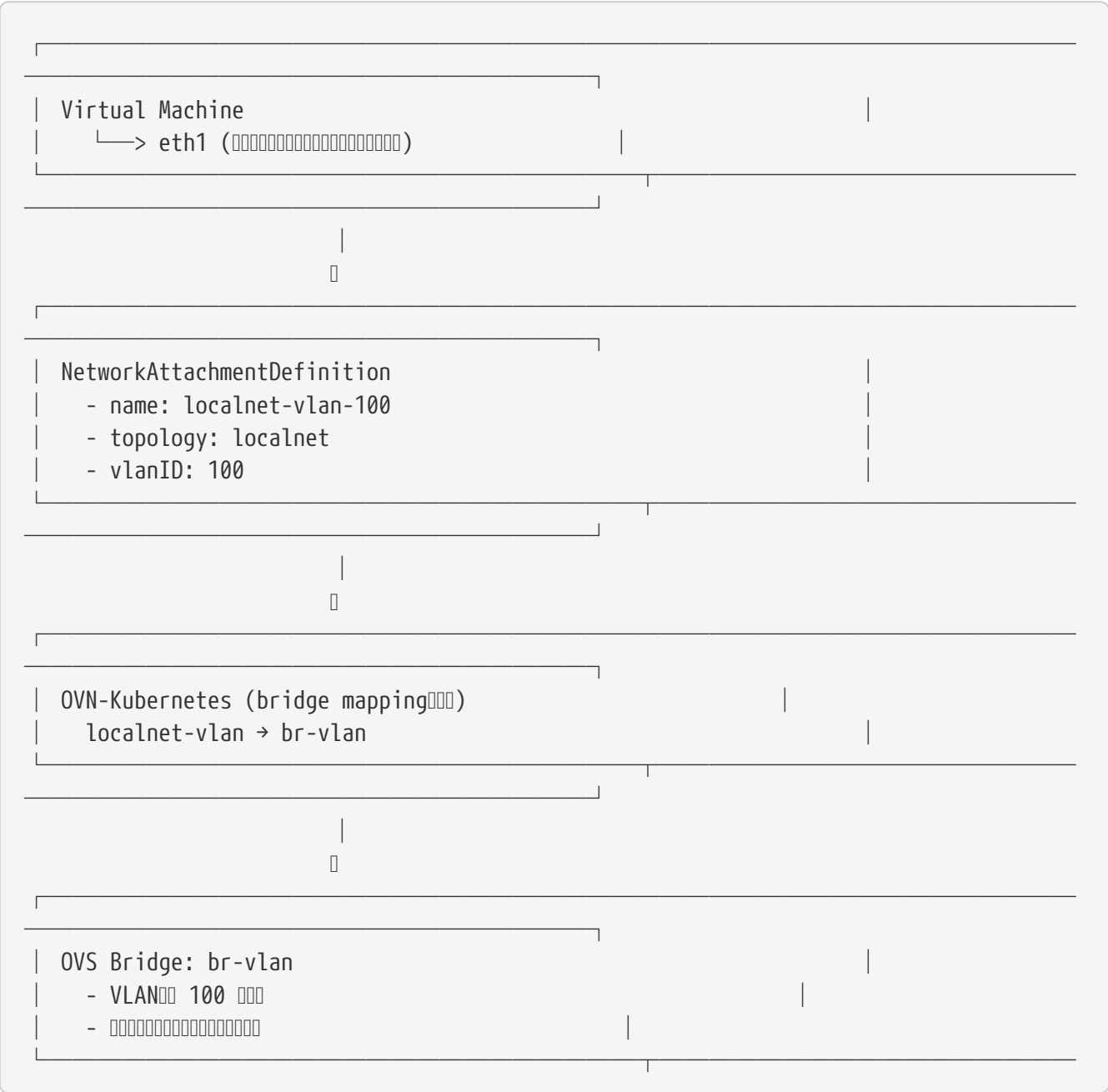
Output:

```
enp11s0f0np0
```

Configure the bridge configuration:

bridge mapping configuration

bridge mapping configuration:



```
Physical Interface: enp11s0f0np0
- VLAN
```

bridge mapping

1: bridge mapping OVN External ID

:

- NNCP `ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings mapping`

:

```
# NNCP
oc get nncp localnet-vlan-bridge-mapping -o yaml

# NMState Operator
oc delete pod -n openshift-nmstate -l app=kubernetes-nmstate-operator

# NMState
oc logs -n openshift-nmstate -l component=kubernetes-nmstate-handler
```

:

- bridge NNCP
- NMState
- OVS

:

bridge bridge mapping NNCP

2: mapping bridge

:

- bridge mapping bridge

:

```
# 查看bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-br

# mapping
oc get nncp localnet-vlan-bridge-mapping -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].bridge}'
```

结果:

mappingNCP 与 bridge 关联 OVS bridge

图 3: 关联

图:

- bridge mapping 与 VM 关联

图:

```
# bridge mapping
oc get nncp localnet-vlan-bridge-mapping -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].localnet}'

# NetworkAttachmentDefinition 与 ClusterUserDefinedNetwork (CUDN) 关联
oc get network-attachment-definitions <nad-name> -n <namespace> -o yaml
```

图:

bridge mapping 与 NetworkAttachmentDefinition 与 ClusterUserDefinedNetwork 关联

图 4: mapping

图:

- bridge mapping 与 bridge 关联

图:

```
# 查看 bridge mapping
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings
```

结果:

```
"mapping1:bridge1,mapping2:bridge2,mapping3:bridge3"
```

111

```

##### bridge → mapping #####OVN ##### mapping #####

```

□ □ □ □ □ □ □ □ □ □ □ □

```

#####OVS bridge - bridge mapping #####

```

bridge □□

- ☐ NNCP `Available: True` `SuccessfullyConfigured`
- ☐ NNCP
- ☐ `ovs-vsctl list-br` `bridge`
- ☐ `ovs-vsctl show` `bridge`
- ☐ `UP,LOWER_UP`
- ☐ `master ovs-system`
- ☐ (
- ☐ NMState Operator

bridge mapping

- ❑ bridge mapping NNCP ❑ Available: True
- ❑ ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings ❑ mapping
- ❑ mapping : logical-name:bridge-name
- ❑ NetworkAttachmentDefinition (NAD) / CUDN
- ❑ mapping ❑ bridge OVS bridge
- ❑ bridge (ovs-vsctl list-ports)
- ❑ ❑ bridge mapping

1010101010

OVS □□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host systemctl status openvswitch
```

OVS □□□□□

```
oc debug node/<node-name> -- chroot /host journalctl -u openvswitch -n 50
```

□□□□ **OVS** □□□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list interface
```

OVS `ovs-vsctl`

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list bridge
```

OVS bridge `ovs-ofctl` (브릿지)

```
# 브릿지 정보 출력
oc debug node/<node-name> -- chroot /host ovs-ofctl dump-ports br-vlan
```

ClusterUserDefinedNetwork (CUDN) `oc`

NetworkAttachmentDefinition `ClusterUserDefinedNetwork` `NetworkAttachmentDefinition` 생성

예제 1: CUDN 생성

ClusterUserDefinedNetwork `ClusterUserDefinedNetwork` 생성:

```
# CUDN 생성
oc get clusteruserdefinednetwork

# CUDN 정보 출력
oc get clusteruserdefinednetwork <cudn-name> -o yaml
```

출력 결과:

| NAME | AGE |
|------------------------|-----|
| cudn-localnet-vlan-100 | 5m |

예제 2: CUDN 정보 출력

CUDN spec `ClusterUserDefinedNetwork` 정보 출력:

```
oc get clusteruserdefinednetwork cudn-localnet-vlan-100 -o yaml
```

출력 결과:

```
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
```

```

    values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"]
network:
  topology: Localnet
  localnet:
    role: Secondary
    physicalNetworkName: localnet-vlan # bridge mapping
    vlanID: 100
    ipam:
      mode: Disabled # Static

```

Figure 3: NetworkAttachmentDefinition

CUDN namespace NetworkAttachmentDefinition:

```

# namespaceNAD
oc get network-attachment-definitions -A | grep <cudn-name>

# namespace
oc get network-attachment-definitions -n <namespace>

```

Output:

| NAMESPACE | NAME | AGE |
|---------------------|------------------------|-----|
| vm-guests-vlan-dev | cudn-localnet-vlan-100 | 5m |
| vm-guests-vlan-prod | cudn-localnet-vlan-100 | 5m |

Figure 4: physicalNetworkName and bridge mapping

CUDN physicalNetworkName and bridge mapping:

```

# CUDN physicalNetworkName
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.physicalNetworkName}'

# bridge mapping
oc get nncp <bridge-mapping-nncp> -o jsonpath='{.spec.desiredState.ovn.bridge-
mappings[0].localnet}'

# OVN
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings

```

Output (localnet-vlan):

Figure 5: namespace

namespace VM CUDN:


```
# namespace VM
oc get vmi -n vm-guests-vlan-prod -o
jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'

# 2 namespace VM
oc get vmi -n vm-guests-vlan-dev -o
jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'
```

VM の CUDN 設定を修正して IP 設定を修正する

CUDN の修正

1. namespace の NAD の修正:

- 例: CUDN の修正 `oc get network-attachment-definitions -n <namespace>` の結果
- 例: namespace の修正 namespace の修正
- 例: namespace の CUDN の修正

```
# namespace
oc get namespace <namespace-name> -o jsonpath='{.metadata.labels}'

# CUDN
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.namespaceSelector}'
```

2. VM の "failed bridge mapping validation" の修正:

- 例: CUDN の修正 VM Pod の修正
- 例: CUDN の `physicalNetworkName` の bridge mapping の修正
- 例: 例 (例) の修正

```
#
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.physicalNetworkName}'
oc get nncp <bridge-mapping-nncp> -o jsonpath='{.spec.desiredState.ovn.bridge-
mappings[0].localnet}'
```

3. IPAM の修正:

- 例: VM の IP の修正
- 例: IPAM の修正 cloud-init の修正
- cloud-init の IP: CUDN の `ipam.mode: Disabled` の修正
- OVN IPAM の修正: `staticIPAMConfig` の修正 `ipam.mode: Static` の修正
- 例:

```
# IPAM操作
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.ipam.mode}'

# VM cloud-init操作
virtctl console -n <namespace> <vm-name>
# VM: cat /var/log/cloud-init.log
```

Localnet VLAN 操作

操作

1. VLAN 操作 VM 操作:

- 操作 VLAN 操作
- Worker Node 的 bridge mapping 操作
- VLAN ID 操作
- 的 NIC 操作UP 操作
- OVS bridge 操作

2. IP 操作:

- DHCP 操作 VLAN 操作

3. 操作:

- 操作 VLAN 操作
- 操作 VLAN 操作
- 操作

4. 操作 bridge 操作:

- 的: OVS bridge 的 `ip addr show` 操作 `ovs-vsctl list-br` 操作
- 操作
- NMState Operator 操作

5. bridge mapping 操作:

- mapping 的 bridge 操作 bridge 操作
- OVN bridge mapping 的: `ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings`
- mapping 的 localnet 的 NAD 操作

6. VM Pod 的 "failed bridge mapping validation" 操作:

- 的: VM Pod 操作: `failed to find OVN bridge-mapping for network: "localnet-vlan-100"`
- 的: NetworkAttachmentDefinition 的 `physicalNetworkName` 操作
- 的: NAD 的 bridge mapping 操作 `"physicalNetworkName": "localnet-vlan"` 操作
- 的: NAD 操作 `oc delete` 的 `oc apply` 操作

```
# NetworkAttachmentDefinition
oc get network-attachment-definitions -n vm-guests-vlan

# NodeNetworkConfigurationPolicies
oc get nncp

# bridge (bridge)
oc get nncp br-vlan-creation -o yaml

# Worker Node bridge mapping
oc get nncp localnet-vlan-bridge-mapping -o yaml

# Worker Node
oc debug node/<worker-node-name> -- chroot /host ip link show ens224

# OVS bridge
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl show

# OVNbridge mapping
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings

# bridge
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl list-ports br-vlan

# VM
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks

# OVN-Kubernetes
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node

# NMState Operator
oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator
```

111

[illegible]

- OVS bridge → Linux bridge → ip addr show →
- ovs-vsctl → OVS bridge → bridge →
- OVS bridge → OVN bridge mapping →
- bridge →
- ClusterUserDefinedNetwork (CUDN) → NAD namespace →

□□

- [NMState Documentation](#)
- [Open vSwitch Documentation](#)
- [OpenShift NMState Operator](#)

OpenShift Virtualization

Overview

OpenShift Virtualization is a Kubernetes extension that enables running VMs on OpenShift. It provides a unified interface for managing VMs and integrates with OpenShift's security and networking features.

Architecture

OpenShift Virtualization architecture consists of the following components:

- VMs are managed by Pod VMs, which are managed by DNS.
- Linux and Windows VMs are supported.
- Selector, Affinity, and Anti-affinity are supported.
- VMs are managed by the OpenShift Virtualization Operator.
- VMs are managed by the OpenShift Virtualization Operator.
- VMs are managed by the OpenShift Virtualization Operator.

Getting Started

VMs are managed by the OpenShift Virtualization Operator.

- OpenShift 4.18 includes the OpenShift Virtualization Operator.
- VMs are managed by the OpenShift Virtualization Operator.
- YAML is used to manage VMs.
- CLI tools for managing VMs: `oc`, `virtctl`, `kubectl`.

VMs and DNS

VMs and DNS

VMs are managed by the OpenShift Virtualization Operator. The OpenShift Virtualization Operator manages VMs and DNS.

Linux VMs

Linux VMs are managed by the OpenShift Virtualization Operator. The OpenShift Virtualization Operator manages VMs and DNS. The OpenShift Virtualization Operator manages VMs and DNS.

Linux VMs

Linux VMs are managed by the OpenShift Virtualization Operator. The OpenShift Virtualization Operator manages VMs and DNS.

RDP and Windows VMs

LoadBalancer and NodePort are used to manage Windows VMs. The OpenShift Virtualization Operator manages VMs and DNS.

Infrastructure

HyperConverged OpenShift Virtualization
(virt-controller, virt-api, virt-handler, CDI)

Affinity

NodeSelector NodeAffinity Pod Affinity/Anti-Affinity rule
VM

virtctl VM

OpenShift Virtualization VM

-
- VM
-
- CPU NUMA Hugepages VM
- VM

- OpenShift Virtualization
- cloud-init
- KubeVirt

VM DNS

OpenShift Virtualization DNS
Kubernetes Service VM DNS IP Pod
VM VM VM

- Kubernetes Service VM
- VM DNS
- OpenShift Virtualization VM DNS

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18](#)
- OpenShift CLI `oc` `virtctl`
- Kubernetes Service `DNS`

□□□ **1:** □□□□□□□□□□

1: namespace

namespace

```
oc new-project myapp
```

□□□□ 2: Fedora VM □□□□

Fedora VM 環境構築



```

##### cloud-init #####
##### SSH #####

```

fedora-app-vm.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-app-vm
  namespace: myapp
  labels:
    app: myapp
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-app-vm-volume
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-app-vm
```

```

    app: myapp
spec:
  domain:
    devices:
      disks:
        - disk:
            bus: virtio
            name: rootdisk
        - disk:
            bus: virtio
            name: cloudinitdisk
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 2Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: fedora-app-vm-volume
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: false }
          ssh_pwauth: true
        name: cloudinitdisk

```

EOF

VM 0000000000000000

```
watch 'oc get vm,vmi,dv -n myapp'
```

VM 0 Running 00 Ready: True 0000000000000000 Ctrl+C 000000

0000 3: VM 0 Service 000

VM 0000000000 Headless Service 00000000

headless-service.yaml 00000000

```

oc apply -f - <<EOF
apiVersion: v1
kind: Service

```



```

restartPolicy: Never
EOF

# Pod
oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s

# DNS
oc exec -n myapp debug-dns -- getent hosts myapp-service.myapp.svc.cluster.local

# 
oc delete pod debug-dns -n myapp

```

：

- DNS VM Pod IP (10.128.0.169 myapp-service.myapp.svc.cluster.local)

Service DNS VM IP

2:

DNS

Pod VM Service DNS



- **Service** DNS : Service `metadata.name` `<service-name>.<namespace>.svc.cluster.local` CoreDNS
- **Headless Service** VM IP : Headless Service (`clusterIP: None`) VM Pod IP VM
- **Service** : ClusterIP Service IP VM
- : `/etc/resolv.conf` `search` namespace Service (FQDN) `myapp-service`

3:

Service **EndpointSlice**:

```
# Service
oc get svc myapp-service -n myapp

# EndpointSlice (VM Pod IP)
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service

# Service
oc describe svc myapp-service -n myapp
```

DNS:

```
# Pod ( )
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: debug-dns
  namespace: myapp
spec:
  containers:
  - name: debug
    image: registry.access.redhat.com/ubi9/ubi
    command: ["sleep", "3600"]
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      runAsUser: 1000
    capabilities:
      drop:
      - ALL
    seccompProfile:
      type: RuntimeDefault
  restartPolicy: Never
<<EOF>>
```

EOF

```
# Pod
oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s

# DNS
oc exec -n myapp debug-dns -- getent hosts myapp-service.myapp.svc.cluster.local

# 
oc delete pod debug-dns -n myapp
```

VM :

```
# VMService
oc get vm fedora-app-vm -n myapp -o jsonpath='{.metadata.labels}'
oc get svc myapp-service -n myapp -o jsonpath='{.spec.selector}'
```

DNS

:

```
oc get svc myapp-service -n myapp
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service
```

- Service VM
- VM (EndpointSlice)
- namespace

:

1. Service VM `kubevirt.io/domain`
2. VM : `oc get vmi -n myapp`
3. Service VM namespace

namespace DNS

:

```
# namespacePod
oc new-project test-namespace 2>/dev/null || true

oc apply -f - <<EOF
```

```

apiVersion: v1
kind: Pod
metadata:
  name: debug-cross-ns
  namespace: test-namespace
spec:
  containers:
  - name: debug
    image: registry.access.redhat.com/ubi9/ubi
    command: ["sleep", "300"]
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      runAsUser: 1000
      capabilities:
        drop:
        - ALL
      seccompProfile:
        type: RuntimeDefault
    restartPolicy: Never
EOF

```

```

# 確認
oc wait --for=condition=Ready pod/debug-cross-ns -n test-namespace --timeout=60s
oc exec -n test-namespace debug-cross-ns -- \
  getent hosts myapp-service.myapp.svc.cluster.local

# 削除
oc delete pod debug-cross-ns -n test-namespace

```

確認:

- FQDN 確認
- NetworkPolicy 確認 namespace 確認

手順:

1. namespace 確認 FQDN 確認: `<service>.<namespace>.svc.cluster.local`
2. NetworkPolicy 確認

確認:

- [Kubernetes DNS for Services and Pods](#)
- [FQDN 確認 VM 確認 - OpenShift Virtualization](#)
- [FQDN 確認 VM 確認 - OpenShift Virtualization](#)

確認

確認

- DNS `Headless Kubernetes Service` `VM` `Pod`
- `CoreDNS` `Service` `VM` `Pod IP`
- `Headless Service` (`VM IP`) `ClusterIP Service` (`IP`)
- `FQDN` `getent hosts` `Pod` `DNS`
- `DNS` `Service`

- `IP`
- `Linux VM`

Linux

OpenShift Virtualization Linux

| | | SSH |
|------------------|-----|-----|
| | | |
| VNC | GUI | |
| virtctl SSH | | |
| NodePort Service | | |
| | | |
| | | |

- OpenShift Virtualization Operator OpenShift 4.18
- CLI: `oc``virtctl``kubectl`
- Linux VM 1 ()
- SSH

namespace: VM

namespace Fedora VM

```
oc create namespace vm-access-demo
```



cloud-init
接続

接続 SSH

fedora-access-vm.yaml 接続

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-access-vm
  namespace: vm-access-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-access-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-access-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
      machine:
        type: pc-q35-rhel9.4.0
```


- VM へ SSH 接続可能
- 接続可能な仮想マシンを指定

詳細

- 接続/接続可能な (接続)
- 接続可能な
- 接続可能な接続可能な
- 接続 1 接続可能な

ステップ 2: VNC 接続

接続可能な仮想マシン VM 接続

VNC 接続可能

```
virtctl vnc fedora-access-vm -n vm-access-demo
```

接続可能な VM 接続可能な VNC 接続可能な

接続可能な

- 接続可能な Linux VM 接続可能な
- GUI 接続可能な
- GUI 接続可能な接続可能な
- 接続可能な接続可能な

接続

- 接続可能な VNC 接続可能な接続可能な
- 接続可能な (GNOME/KDE 接続) 接続可能な VM

ステップ 3: virtctl 接続 SSH 接続

virtctl 接続可能な SSH 接続可能な接続可能な VM へ SSH 接続可能な

接続

- VM へ SSH 接続可能な (cloud-init 接続)
- VM へ SSH 接続可能な

SSH 接続可能

```
virtctl ssh fedora@fedora-access-vm -n vm-access-demo
```

ssh 접속 (ssh)

```
virtctl ssh fedora@fedora-access-vm -n vm-access-demo --local-ssh-opts="-o
StrictHostKeyChecking=no"
```

ssh 접속

- ssh 접속
- ssh 접속
- ssh 접속
- **oc** **kubectl** ssh 접속

ssh

- ssh 접속
- ssh 접속

4: NodePort Service SSH

NodePort Service SSH 접속

NodePort Service

ssh-nodeport-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: fedora-ssh-nodeport
  namespace: vm-access-demo
spec:
  type: NodePort
  selector:
    kubevirt.io/domain: fedora-access-vm
  ports:
    - protocol: TCP
      port: 22
      targetPort: 22
      nodePort: 30022
```

Service

```
oc apply -f fedora-ssh-nodeport.yaml
```

Service 操作

```
oc get svc fedora-ssh-nodeport -n vm-access-demo
```

SSH 操作

获取节点的 IP 地址

```
oc get nodes -o wide
```

NodePort 操作

```
ssh fedora@<node-ip> -p 30022
```

操作

- 创建服务
- 创建服务端口
- 创建/更新服务
- 删除服务

操作

- 创建 SSH 服务
- 创建 SSH (SSH 服务)
- 创建服务端口
- 创建服务
- 删除服务

第 5 章: 创建 SSH 服务

kubectl 使用 `oc port-forward` 创建 SSH 服务

VM Pod 操作

```
oc get pods -n vm-access-demo
```

输出:

| NAME | READY | STATUS | RESTARTS | AGE |
|--------------------------------------|-------|---------|----------|-----|
| virt-launcher-fedora-access-vm-xxxxx | 2/2 | Running | 0 | 10m |

SSH 接続

```
oc port-forward -n vm-access-demo virt-launcher-fedora-access-vm-xxxxx 2222:22
```

SSH 接続 (ポートフォワード)

```
ssh fedora@localhost -p 2222
```

接続方法

- 接続方法
- 接続方法
- VM 接続方法
- NodePort 接続方法

接続

- 接続方法
- 1 接続方法
- 接続方法
- 接続方法

6: 接続方法

bridge 接続 localnet 接続 VM 接続 IP 接続

接続

- VM 接続 (bridge localnet VLAN) 接続
- 接続
- IP 接続 DHCP 接続

接続 VM

fedora-vlan-access-vm.yaml 接続

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vlan-vm
  namespace: vm-access-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
```

```

kind: DataVolume
metadata:
  name: fedora-vlan-vm-disk
spec:
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
runStrategy: RerunOnFailure
template:
  metadata:
    labels:
      kubevirt.io/domain: fedora-vlan-vm
  spec:
    domain:
      cpu:
        cores: 2
      devices:
        disks:
          - disk:
              bus: virtio
              name: rootdisk
          - disk:
              bus: virtio
              name: cloudinitdisk
        interfaces:
          - masquerade: {}
            name: default
          - bridge: {}
            name: vlan-network
        rng: {}
      machine:
        type: pc-q35-rhel9.4.0
      memory:
        guest: 4Gi
    networks:
      - name: default
        pod: {}
      - name: vlan-network
        multus:
          networkName: localnet-vlan-100
    volumes:
      - dataVolume:
          name: fedora-vlan-vm-disk
          name: rootdisk
      - cloudInitNoCloud:
          userData: |-

```

```
#cloud-config
user: fedora
password: fedora123
chpasswd: { expire: false }
ssh_pwauth: true
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ... your-public-key-here
networkData: |-
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
    enp2s0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.100/24
name: cloudinitdisk
```

SSH 接続

```
ssh fedora@192.168.100.100
```

接続方法

- 接続
- 接続（パスワード入力）
- 接続（パスワード入力） VM
- 接続（パスワード入力）
- 接続（パスワード入力） VM

接続

- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）

SSH 接続

SSH 接続（パスワード入力）

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/ocp-virt-key
```

cloud-init

VM の `userData`

```
ssh_authorized_keys:
- ssh-rsa AAAAB3NzaC... your-key-here
```

root cloud-init の `userData`

```
volumes:
- cloudInitNoCloud:
  userData: |-
    #cloud-config
    user: fedora
    ssh_pwauth: false
    disable_root: true
    ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC... your-key-here
  name: cloudinitdisk
```

- `ssh_pwauth: false` - SSH
- `disable_root: true` - SSH root
- SSH

VM

```
# VirtualMachine
oc get vm -n vm-access-demo

# VirtualMachineInstance
oc get vmi -n vm-access-demo

# VM
oc describe vmi fedora-access-vm -n vm-access-demo
```

VM SSH


```
virtctl console fedora-access-vm -n vm-access-demo
```

VM 内部:

```
# SSH 接続確認
sudo systemctl status sshd

# SSH 接続ポート確認
sudo ss -tlnp | grep :22

# ファイアウォール確認
sudo firewall-cmd --list-all
```

接続確認

```
# VM 内部から ping
ping -c 3 8.8.8.8

# 外部から ping (ホスト IP)
ping -c 3 192.168.100.100
```

接続確認

接続確認が完了したら、Linux VM 内部で SSH を有効化し、接続を確認します。

ホスト: **virtctl** 内部で接続を確認します。

ホスト: VM 内部で接続を確認します。

ホスト: 内部 VM 内部で接続を確認します。 VM 内部で接続を確認します。

ホスト: **SSH** 接続を確認します。

ホスト: SSH 接続を確認します。

ホスト:

- sshd サービスの状態: `sudo systemctl status sshd`
- sshd サービスの起動: `sudo systemctl start sshd`
- ファイアウォールのサービス: `sudo firewall-cmd --list-services`

ホスト: **virtctl ssh** 接続を確認します。

ホスト: SSH 接続を確認します。

ホスト:

- VM 的 SSH 连接
- SSH 连接
- VM 连接
- 连接 SSH 连接

的: **NodePort** 连接

的: 连接

的: 连接NodePort 连接

的: 连接 **IP** 连接

的: 连接

的: 连接VLAN/连接

连接

的

- SSH 连接
- 连接 SSH 连接
- 连接
- 连接

连接

- 连接NodePort 连接 IP 连接
- NetworkPolicy 连接 VM 连接
- VM 连接
- 连接

连接

- virtctl 连接 RBAC 连接
- namespace 连接
- VM 连接
- 连接

连接

连接

VM

```
oc delete vm fedora-access-vm fedora-vlan-vm -n vm-access-demo
```

```
# 删除服务
```

```
oc delete svc fedora-ssh-nodeport -n vm-access-demo
```

```
# namespace删除
```

```
oc delete namespace vm-access-demo
```

删除

删除虚拟机和虚拟机镜像

- OpenShift Virtualization 中 Linux VM 删除虚拟机和虚拟机镜像
- 删除虚拟机和虚拟机镜像
- cloud-init 配置 SSH 访问
- 删除虚拟机和虚拟机镜像
- 删除虚拟机和虚拟机镜像
- 删除虚拟机和虚拟机镜像
- VM 删除虚拟机和虚拟机镜像
- 删除虚拟机和虚拟机镜像

删除

- [Managing VMs](#)

Linux VM 删除虚拟机和虚拟机镜像

删除虚拟机和虚拟机镜像 OpenShift Virtualization 删除 Linux VM

删除

- [\[vm-configuration:remote-access-linux-troubleshooting::删除虚拟机和虚拟机镜像\]](#)
- 步骤 1: 删除虚拟机和虚拟机镜像
- 步骤 2: VNC 访问
- 步骤 3: virtctl 访问 SSH
- 步骤 4: NodePort Service 访问 SSH
- 步骤 5: LoadBalancer 访问 SSH
- 步骤 6: 删除虚拟机和虚拟机镜像 SSH
- 步骤 7: 删除虚拟机和虚拟机镜像
- [\[vm-configuration:remote-access-linux-troubleshooting::删除虚拟机和虚拟机镜像\]](#)

□ □ □ □ □ □ □ □ □ □

VM □□□□□□□□

```
oc get vm <vm-name> -n <namespace>
oc get vmi <vm-name> -n <namespace>
```

0000 VM 000000000:

| NAME | AGE | STATUS | READY |
|-------|-----|---------|-------|
| my-vm | 5m | Running | True |

VM IP 0000000000000000

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces[*].ipAddress}'
```

□□□□ 1 □□ IP □□□□□□□□□□□□□□□□

VM Pod □□□□□□

```
oc get pods -n <namespace> -l kubevirt.io/domain=<vm-name>
```

□□□□□□:

| NAME | READY | STATUS | RESTARTS | AGE |
|--|-------|---------|----------|-----|
| virt-launcher- <code><vm>-xxxxx</code> | 3/3 | Running | 0 | 5m |

guest agent □□□□□□□□

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.guestOSInfo.id}'
```

```

guest agent
(SSHNodePortLoadBalancer) guest agent
agent OS SSH

```

□□□□ **1:** □□□□□□□□□□

□□: □□□□□□□□□□

00:

```
$ virtctl console my-vm -n my-namespace
```

Error: unable to connect to console

□□:

```
# Check VM is running
oc get vm my-vm -n my-namespace

# Check VMI exists
oc get vmi my-vm -n my-namespace

# Check virt-launcher pod
oc get pods -n my-namespace -l kubevirt.io/domain=my-vm
```

□□□□□□: - VM □□□□□□□□ - VMI □□□□□□□□ - virt-launcher Pod □□□□□□□□ - □□□□□□ API □□□□□□□□□□□□

□□□:

1. VM □□□□□□□□□□□□□□:

```
virtctl start my-vm -n my-namespace
```

- ## 2. VMI 0000000000000000:

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace --timeout=60s
```

- ### 3. virt-launcher Pod 配置:

```
oc logs -n my-namespace -l kubevirt.io/domain=my-vm
```

4. virtctl
-
- :

```
virtctl version
oc get kubevirt -n openshift-cnv -o
jsonpath='{.items[0].status.observedKubeVirtVersion}'
```

00: 00000000000000000000000000000000

:

□□:

```
# Try sending Enter key
# Press Enter multiple times
```

□□□:

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.phase}'
```

- RHEL/Fedora 检查: `systemctl status serial-getty@ttyS0.service`

3. □□□□□□□□□□:

00: 0000000000000000

```
root@kali: ~# ssh -o StrictHostKeyChecking=no root@10.10.10.10 - cloud-init - SSH root@10.10.10.10 - root
```

000



1. cloud-init 安装配置:

2. 通过SSH 连接远程主机

3. VM cloud-init

2: VNC

```
00: virtctl vnc 000000000000
```

00:

```
$ virtctl vnc my-vm -n my-namespace
Error: VNC is not configured for this VM
```

00:

```
# Check if graphics is enabled
oc get vm my-vm -n my-namespace -o
jsonpath='{.spec.template.spec.domain.devices.autoattachGraphicsDevice}'

# Check VM devices configuration
oc get vm my-vm -n my-namespace -o yaml | grep -A 10 "devices:"
```

```

00000000:      -      autoattachGraphicsDevice:      false      00000000000000000000000000000000      -      VM
00000000000000000000000000000000

```

[illegible]

00: VNC 00000000000000

```
root@kali:~# virtctl vnc --listen=:5900 --wgl-driver=none --ssl-cert=/dev/null
```

□□:

```
# Check virtctl output for port number
virtctl vnc my-vm -n my-namespace
# Shows: "VNC connection on port 5900"

# Check if port is listening
netstat -tuln | grep 5900
```

[illegible]

000:

1. VNC 接続方法:

```
# Linux
sudo dnf install tigervnc

# macOS
brew install tiger-vnc
```

2. localhost:5900 □□□ 127.0.0.1:5900 □□□□□□

3. `virtctl` (5901, 5902 行) のインストール

00: **VNC** 0000000000000000

00: **VNC** 000000000000 (000000) 000000000000

000000: - **VM** 000000 - 000000000000000000000000 - **VM** 000000000000 (**GUI** 00)

000:

1. **VM** 0000000000000000
2. **VM** 0 **GUI** 00000000000000000000:

```
# Inside VM
systemctl status gdm # For GNOME
systemctl status lightdm # For other DEs
```

3. **GUI** 000 **VM** 000000000000000000000000

0000 **3: virtctl** 000 **SSH**

00: **virtctl ssh** 0000000000

00:

```
$ virtctl ssh fedora@my-vm -n my-namespace
Error: SSH is not available for this VM
```

00:

```
# Check VM has IP address
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}'

# Check SSH service in VM (via console)
virtctl console my-vm -n my-namespace
# Inside VM: systemctl status sshd
```

000000: - **VM** 00 **SSH** 0000000000000000 - **VM** 0 **IP** 0000000000000000 - **VM** 0 **SSH** 0000000000000000 - 000000000000

000:

1. **VM** 0 **SSH** 0000000000000000:

```
# Inside VM via console
sudo dnf install openssh-server
sudo systemctl enable --now sshd
```

2. **VM** 0 **IP** 0000000000000000:


```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces[0].ipAddress}'
```

3. SSH 接続:

```
# Try port-forward method first
oc port-forward vmi/my-vm -n my-namespace 2222:22 &
ssh -p 2222 fedora@localhost
```

□□: **virtctl ssh** 接続:

□□:

```
$ virtctl ssh fedora@my-vm -n my-namespace
fedora@my-vm: Permission denied (publickey,gssapi-keyex,gssapi-with-mic) .
```

□□:

```
# Check SSH keys
ls -la ~/.ssh/

# Try with password authentication
virtctl ssh --local-ssh-opts="-o PreferredAuthentications=password" fedora@my-vm -n
my-namespace
```

□□□□□□: - SSH 接続 VM の `authorized_keys` 接続 - □□□□□□□□□□ - SSH 接続

□□□:

1. cloud-init の SSH 接続:

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC1yc2EA... user@host
```

2. □□□□VM の SSH 接続:

```
mkdir -p ~/.ssh
echo "ssh-rsa AAAAB3..." >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3. □□□□□□□□□□Fedora □□□□□□ `fedora`□□RHEL □□□□ `ccloud-user` □□□ `ec2-user` □□□

root: virtctl ssh

root: 30

root: - DNS - GSSAPI

root: SSH

```
virtctl ssh --local-ssh-opts="-o GSSAPIAuthentication=no -o UseDNS=no" fedora@my-vm -n my-namespace
```

4: NodePort Service SSH

root: NodePort Service

root:

```
$ ssh -p 30022 fedora@<node-ip>
ssh: connect to host <node-ip> port 30022: Connection refused
```

root:

```
# Check service exists
oc get svc -n my-namespace

# Check service endpoints
oc get endpoints my-vm-ssh -n my-namespace

# Check NodePort assigned
oc get svc my-vm-ssh -n my-namespace -o jsonpath='{.spec.ports[0].nodePort}'
```

root: - NodePort - VM Pod - NodePort
root: - VM SSH (listen)

root:

1.

```
apiVersion: v1
kind: Service
metadata:
  name: my-vm-ssh
spec:
  type: NodePort
  selector:
    kubevirt.io/domain: my-vm
  ports:
```

```
- protocol: TCP
  port: 22
  targetPort: 22
  nodePort: 30022 # Optional, will be auto-assigned if omitted
```

2. Selector $\square$ VM $\square\square\square\square\square\square\square\square\square\square$:

```
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.metadata.labels}'
```

3. VM $\square$ SSH $\square\square\square\square\square\square\square\square\square\square\square\square$:

```
virtctl console my-vm -n my-namespace
# Inside VM:
systemctl status sshd
```

$\square\square$: **NodePort** $\square\square\square\square\square\square\square\square\square$ **VM** $\square\square\square\square\square$

$\square\square$: SSH $\square\square\square\square\square\square\square\square$ VM $\square\square\square\square\square\square\square\square\square\square\square\square$

$\square\square$:

```
# Check service endpoints
oc describe svc my-vm-ssh -n my-namespace

# Look at Endpoints field - should show VM pod IP
```

$\square\square\square\square\square$: - Service $\square$ Selector $\square\square\square\square$ VM $\square\square\square\square\square\square$ - Service $\square$ Selector $\square\square\square$ VM $\square\square\square\square\square\square\square\square$

$\square\square\square$:

1. Service Selector $\square\square\square\square\square\square\square\square\square\square\square$:

```
selector:
  kubevirt.io/domain: my-vm # Unique per VM
# OR
  app: my-specific-vm-app
```

2. $\square\square\square\square\square\square\square$ 1 $\square\square\square\square\square\square\square\square\square\square\square$:

```
oc get endpoints my-vm-ssh -n my-namespace -o yaml
```

$\square\square$: **Node IP** $\square\square\square\square\square\square$

$\square\square$: **NodePort** $\square\square\square\square\square\square\square\square$ **Node IP** $\square\square\square\square\square\square$

root:

```
# Get node IPs
oc get nodes -o wide

# Use any node's INTERNAL-IP or EXTERNAL-IP (if available)

# Or get specific node where VM is running
VM_NODE=$(oc get vmi my-vm -n my-namespace -o jsonpath='{.status.nodeName}')
oc get node $VM_NODE -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}'
```

5: LoadBalancer SSH

root: LoadBalancer Service IP address

root:

```
$ oc get svc my-vm-ssh -n my-namespace
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)
my-vm-ssh     LoadBalancer  172.30.x.x      <pending>        22:30022/TCP
```

root:

```
# Check if LoadBalancer provider is available
oc get svc -A | grep LoadBalancer

# Check cluster has LoadBalancer support
oc describe svc my-vm-ssh -n my-namespace
```

root: - LoadBalancer address (MetalLB address) - LoadBalancer address - LoadBalancer address

root:

1. MetalLB address
2. MetalLB address
3. LoadBalancer address:

```
oc describe svc my-vm-ssh -n my-namespace
```

4. root: LoadBalancer NodePort address

00: **LoadBalancer IP** 00000000

00: 00 IP 000000000000SSH 000000000000

00:

```
# Check LoadBalancer IP
LB_IP=$(oc get svc my-vm-ssh -n my-namespace -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')

# Try ping
ping -c 3 $LB_IP

# Check from within cluster
oc run test-curl --image=curlimages/curl --rm -it --restart=Never -- curl -v
telnet://$LB_IP:22
```

000000: - LoadBalancer IP 000000000000 - 00000000000000000000000000000000 - LoadBalancer 000000 - VM 0 SSH
0000000000

000:

1. LoadBalancer IP 00000000000000000000000000000000
2. 0000000000 LoadBalancer IP 00000000000000000000000000000000
3. VM 0 SSH 000000000000000000 (00000 4 00000000) 0

0000 6: 00000000000000 SSH

00: **oc port-forward** 00000000

00:

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22
Error: unable to forward port
```

00:

```
# Check VMI exists
oc get vmi my-vm -n my-namespace

# Check local port not in use
netstat -tuln | grep 2222
lsof -i :2222
```

000000: - 00000000000000000000 - VMI 0000000000 - NetworkPolicy 00000000000000000000

000:

1. `oc port-forward vmi/my-vm -n my-namespace 12345:22`

```
oc port-forward vmi/my-vm -n my-namespace 12345:22
```

2. `oc port-forward vmi/my-vm -n my-namespace 2222:22`

```
# Find process
lsof -ti :2222 | xargs kill -9
```

3. `oc wait --for=condition=Ready vmi/my-vm -n my-namespace`

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace
```

`oc port-forward vmi/my-vm -n my-namespace 2222:22` **SSH** `ssh`

`oc port-forward vmi/my-vm -n my-namespace 2222:22`

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22 &
Forwarding from 127.0.0.1:2222 -> 22

$ ssh -p 2222 fedora@localhost
ssh: connect to host localhost port 2222: Connection refused
```

`oc port-forward vmi/my-vm -n my-namespace 2222:22`

```
# Check SSH is listening in VM
virtctl console my-vm -n my-namespace
# Inside VM:
ss -tuln | grep :22
```

`oc port-forward vmi/my-vm -n my-namespace 2222:22` - VM `SSH` `ssh` - `SSH` `ssh` - VM `SSH` `ssh`

`oc port-forward vmi/my-vm -n my-namespace 2222:22`

1. `oc port-forward vmi/my-vm -n my-namespace 2222:22`

```
# Inside VM via console
sudo dnf install openssh-server
sudo systemctl enable --now sshd
```

2. `oc port-forward vmi/my-vm -n my-namespace 2222:22`

```
# Inside VM
```

```
sudo ss -tuln | grep ssh
```

3. SSH 確認:

```
# Inside VM
sudo cat /etc/ssh/sshd_config | grep ^Port
```

00: 確認

00: 確認

000000: - 確認 - API 確認 - 確認

000:

1. SSH 保持 alive 確認:

```
ssh -o ServerAliveInterval=60 -p 2222 fedora@localhost
```

2. 確認:

```
while true; do
  oc port-forward vmi/my-vm -n my-namespace 2222:22
  sleep 5
done &
```

0000 7: 確認

00: 確認 **VM** 確認

00:

```
$ ping 192.168.100.10
Request timeout
```

00:

```
# Check VM has secondary interface configured
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq

# Check NetworkAttachmentDefinition (NAD) exists
oc get network-attachment-definition -n my-namespace

# Check VM network configuration
```

```
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.spec.networks}' | jq
```

00000000: - NetworkAttachmentDefinition 0000000000 - VM 000000000000000000000000 - 000000000000 IP
 0000000000 - 00000000000000000000 - VLAN 00000000

□□□:

1. NAD □□□□□□□□□□□□□□□□:

```
oc describe network-attachment-definition my-nad -n my-namespace
```

2. VM □□□□□□□□□□□□□□□□□□□□:

```
spec:
  template:
    spec:
      networks:
        - name: default
          pod: {}
        - name: secondary-net
          multus:
            networkName: my-nad
      domain:
        devices:
          interfaces:
            - name: default
              masquerade: {}
            - name: secondary-net
              bridge: {}
```

- ### 3. VM IP:

```
virtctl console my-vm -n my-namespace
# Inside VM:
ip addr show enp2s0 # Secondary interface
```

4. cloud-init 配置 IP 地址 (cloud-init IP 地址配置) 。

- ## 5. VLAN 与 OVS bridge

00: 000000000000000000000000 **VM** 00000000

00: 000000000000 (enp1s0) 00000000enp2s0 0000

□□:

```
# Inside VM
```



```
ip link show

# Check from OpenShift
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq
```

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq
```

111

- ```
oc get network-attachment-definition -n my-namespace
```

3. multus 多:

#### 4. virt-launcher Pod 启动失败:

□ □ □ □ □ □

[illegible]

268

```

echo ""

echo ""
echo "=== VM Networks ==="
oc get vm $VM_NAME -n $NAMESPACE -o jsonpath='{.spec.template.spec.networks}' | jq

echo ""
echo "=== Services for VM ==="
oc get svc -n $NAMESPACE -l kubevirt.io/domain=$VM_NAME

echo ""
echo "=== NetworkAttachmentDefinitions ==="
oc get network-attachment-definition -n $NAMESPACE

echo ""
echo "=== Recent Events ==="
oc get events -n $NAMESPACE --field-selector involvedObject.name=$VM_NAME --sort
-by='.lastTimestamp' | tail -10

```

## VM □□□□□□□□□□□□□□□□

```

#!/bin/bash
echo "=== Network Interfaces ==="
ip addr show

echo ""
echo "=== Routing Table ==="
ip route show

echo ""
echo "=== DNS Configuration ==="
cat /etc/resolv.conf

echo ""
echo "=== SSH Service Status ==="
systemctl status sshd

echo ""
echo "=== SSH Listening Ports ==="
ss -tuln | grep :22

echo ""
echo "=== Guest Agent Status ==="
systemctl status qemu-guest-agent

echo ""
echo "=== Firewall Status ==="
systemctl status firewalld 2>/dev/null || echo "Firewall not active"

echo ""

```

```
echo "=== Connectivity Test ==="
ping -c 2 8.8.8.8 && echo "External connectivity: OK" || echo "External connectivity:
FAILED"
```

## Network Troubleshooting - VM Access

Need access to VM?

```
|
| — VM not running? → virtctl start VM
|
| — Just need console? → virtctl console
| — Not working? → Check VM running, check virt-launcher pod
|
| — Need GUI? → virtctl vnc
| — Not working? → Check VNC graphics device, install VNC viewer
|
| — Need SSH?
| — From workstation with virtctl? → virtctl ssh
| — Not working? → Check SSH service running, SSH keys configured
|
| — Persistent access needed? → NodePort Service
| — Not working? → Check service selector, NodePort number, node firewall
|
| — Public access needed? → LoadBalancer Service
| — Not working? → Check LoadBalancer provider installed, IP assigned
|
| — Temporary access? → oc port-forward
| — Not working? → Check local port free, VMI running, SSH in VM
|
| — Direct network access? → Secondary Network
| — Not working? → Check NAD, VM secondary interface, IP config, network
routing
```

## Resources

- [OpenShift Virtualization Networking](#)
- [KubeVirt Network Access](#)
- [Virtctl CLI Reference](#)
- [Kubernetes Services](#)
- [MetalLB](#)
- [cloud-init IP Configuration Troubleshooting](#)
- [OVS Bridge Verification](#)

コンテナ: **ocplab01** コンテナ

SNO コンテナ (ocplab01) コンテナ:

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.interfaces}' |
jq
[
 {
 "infoSource": "domain, guest-agent, multus-status",
 "interfaceName": "eth0",
 "ipAddress": "10.128.0.171",
 "ipAddresses": [
 "10.128.0.171",
 "fd02::62"
],
 "mac": "02:92:c2:00:00:08",
 "name": "default",
 "queueCount": 1
 }
]
```

guest agent コンテナ:

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.guestOSInfo}' |
jq
{
 "id": "fedora",
 "kernelRelease": "6.11.4-301.fc41.x86_64",
 "kernelVersion": "#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024",
 "name": "Fedora Linux",
 "prettyName": "Fedora Linux 41 (Cloud Edition) ",
 "version": "41",
 "versionId": "41"
}
```

コンテナVM コンテナguest agent コンテナguest agent コンテナ OS コンテナSSH  
コンテナコンテナ

## RDP コンテナ Windows VM コンテナコンテナ

コンテナ

コンテナOpenShift コンテナ Windows 11 コンテナコンテナ (RDP)  
コンテナWindows コンテナ VM コンテナVNC  
コンテナコンテナ

OpenShift Virtualization

Red Hat OpenShift Virtualization Windows VM Microsoft

OpenShift Virtualization

OpenShift Virtualization

OpenShift IP Address

- **10.0.2.2**: Windows OS VM IP
- **10.128.1.132**: OpenShift Pod IP
- OpenShift IP Address - OpenShift IP

OpenShift

- OpenShift Windows 11 VM IP
- OpenShift Web console oc CLI
- VNC OpenShift VM IP
- Windows 11 VM IP
- OpenShift IP Address Windows IP

OpenShift

OpenShift

OpenShift 1: Windows 11 IP

## 1. VM IP

OpenShift VNC OpenShift Windows 11 VM IP

## 2. OpenShift

OpenShift IP

OpenShift IP

## 3. OpenShift

OpenShift IP

OpenShift IP

OpenShift IP

OpenShift IP PC IP

ステップ 2: VM を RDP 接続可能にする

RDP ポート 3389 を VM に公開する

タスク A: OpenShift Web コンソール (GUI)

1. ナビゲーション

OpenShift Console → Your Project → Networking → Services

2. サービスの作成

サービス名を指定する

サービスタイプを選択する

| フィールド       | 値                                   | 説明                   |
|-------------|-------------------------------------|----------------------|
| Service タイプ | LoadBalancer                        | 公開 IP を取得            |
| Service 名   | win11-rdp-service                   | サービス名                |
| Selector    | vm.kubevirt.io/name: <your-vm-name> | VM の名前を指定            |
| ポート         | 3389                                | 公開ポート                |
| ターゲットポート    | 3389                                | Windows VM の RDP ポート |
| プロトコル       | TCP                                 | RDP 接続               |



LoadBalancer は NodePort を使用して (ポート 30000-32767 の範囲内) 公開されます

3. Service の作成

コマンドを実行する

NodePort を指定する

タスク B: OpenShift CLI (CLI)

1. Service の定義

rdp-service.yaml を作成する

```
apiVersion: v1
kind: Service
metadata:
 name: win11-rdp-service
spec:
 selector:
 vm.kubevirt.io/name: your-vm-name # REPLACE with your VM's actual name
 ports:
```

```
- protocol: TCP
 port: 3389
 targetPort: 3389
type: LoadBalancer # or NodePort
```

## 2. 確認

```
oc apply -f rdp-service.yaml
```

## ステップ 3: 確認

### LoadBalancer 確認:

1. **Networking** → **service** 確認
2. **win11-rdp-service** 確認
3. **IP** 確認

### NodePort 確認:

1. Service 確認 **NodePort** (例: **31234**)
2. Worker Node **IP** 確認

```
oc get nodes -o wide
```

### EXTERNAL-IP 確認

## ステップ 4: 確認

1. 確認

Windows 確認

2. 確認

| 確認項目                | 確認内容                          | 結果                 |
|---------------------|-------------------------------|--------------------|
| <b>LoadBalancer</b> | <External-IP>                 | 203.0.113.5        |
| <b>NodePort</b>     | <Node-External-IP>:<NodePort> | 203.0.113.10:31234 |

3. 確認

確認

Windows 11 VM 確認

Windows VM RDP 接続方法



RDP

Windows VM への RDP 接続方法

Windows VM への RDP 接続方法

VPN (NLA)

Windows VM への RDP 接続方法

Windows VM への RDP 接続方法

接続方法

接続方法

- Windows 11 VM への RDP 接続方法
- Windows 11 VM への RDP 接続方法
- Windows 11 VM への RDP 接続方法
- Windows 11 VM への RDP 接続方法 (詳細情報)

接続方法

| 接続方法         | 接続方法                | 接続方法          |
|--------------|---------------------|---------------|
| 接続方法         | 接続方法                | 接続方法 IP 接続方法  |
| 接続方法         | Windows VM RDP 接続方法 | Windows 接続方法  |
| 接続方法         | VM 接続方法             | VNC 接続方法/接続方法 |
| 接続方法 IP 接続方法 | LoadBalancer 接続方法   | NodePort 接続方法 |

接続方法

接続方法

- OpenShift Virtualization 接続方法 Windows 11 VM 接続方法 (RDP) 接続方法
- LoadBalancer 接続 NodePort 接続方法 VM RDP 接続 (3389) 接続方法
- 接続方法 (接続 IP 接続 NodePort 接続 IP) 接続方法
- 接続方法 Windows VM 接続方法
- RDP 接続方法

接続方法

- [OpenShift Virtualization Documentation](#)
- [KubeVirt User Guide](#)
- [Windows Remote Desktop Best Practices](#)

VirtIO 接続方法 Windows VM 接続方法





```
accessModes:
 - ReadWriteOnce
source:
 upload: {}
EOF
```

VirtIO `virtio-win.iso` `virtio-drivers-iso`

```
virtctl image-upload dv virtio-drivers-iso --image-path=virtio-win.iso --insecure
```

CDI `--insecure` `virtio-drivers-iso`

## 2: Windows `windows.iso` `windows-installation-iso`

Windows `windows.iso` `DataVolume` `windows-installation-iso`

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: windows-installation-iso
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 storage:
 resources:
 requests:
 storage: 6Gi
 accessModes:
 - ReadWriteOnce
 source:
 upload: {}
EOF
```

Windows `windows.iso` `windows-installation-iso` (`windows.iso` `windows-installation-iso`)

```
virtctl image-upload dv windows-installation-iso --image-path=windows.iso --insecure
```

## 3: VirtIO `virtio-win.iso` Windows VM

VirtIO `virtio-win.iso` Windows `virtio-win.iso`

`windows-vm-virtio.yaml` `windows-vm-virtio.yaml`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
```

```

kind: VirtualMachine
metadata:
 name: windows-vm-virtio
 labels:
 app: windows-vm-virtio
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/size: medium
 kubevirt.io/domain: windows-vm-virtio
 vm.kubevirt.io/name: windows-vm-virtio
 spec:
 domain:
 cpu:
 cores: 2
 sockets: 1
 threads: 2
 resources:
 requests:
 memory: 4Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 bootOrder: 2
 - name: windows-installation-iso
 cdrom:
 bus: sata
 bootOrder: 1
 - name: virtio-drivers-iso
 cdrom:
 bus: sata
 interfaces:
 - name: default
 masquerade: {}
 model: virtio
 tpm: {}
 rng: {}
 clock:
 utc: {}
 timer:
 hpet:
 present: false
 pit:
 tickPolicy: delay
 rtc:
 tickPolicy: catchup
 hyperv: {}

```

```

features:
 acpi: {}
 apic: {}
 hyperv:
 relaxed: {}
 vpic: {}
 spinlocks:
 spinlocks: 8191
 vpinx: {}
 runtime: {}
 synic: {}
 stimer: {}
 reset: {}
 frequencies: {}
 reenlightenment: {}
 tlbflush: {}
 ipi: {}
 smm:
 enabled: true
firmware:
 bootloader:
 efi:
 secureBoot: true
networks:
 - name: default
 pod: {}
volumes:
 - name: rootdisk
 dataVolume:
 name: windows-vm-virtio-rootdisk
 - name: windows-installation-iso
 dataVolume:
 name: windows-installation-iso
 - name: virtio-drivers-iso
 dataVolume:
 name: virtio-drivers-iso
dataVolumeTemplates:
 - metadata:
 name: windows-vm-virtio-rootdisk
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 storage:
 resources:
 requests:
 storage: 60Gi
 accessModes:
 - ReadWriteOnce
 source:
 blank: {}

```

EOF

VM を再起動します

```
oc get vm windows-vm-virtio -w
```

STATUS=Running かつ READY=True になったら **Ctrl+C** を入力して終了します

## ステップ 4: VirtIO をインストールして Windows をインストール

VM を再起動して Windows をインストールします

```
virtctl vnc windows-vm-virtio
```

OpenShift Web コンソール: **Virtualization** → **VirtualMachines** → **windows-vm-virtio** → **Console** を開く

### 1. Windows をインストール:

VM の `bootable` 属性を `true` に設定し、`device` 属性に `found` 属性を `UEFI` に設定します。

- UEFI をインストールする場合は `boot from other media or DVD` を選択します
- `Boot from DVD/CD` を選択します
- ISO イメージ (Windows ISO) を選択します
- Windows をインストールする場所を選択します

### 2. VirtIO をインストール: Windows をインストールする場所を選択します

- `viostor` を選択します
- VirtIO の CD (D: または E: を選択) を選択します
- `viostor{windows-version}\amd64\` を選択します
- Windows Server 2022 の場合は `viostor\2k22\amd64\`
- Windows 11 の場合は `viostor\w11\amd64\`
- **Red Hat VirtIO SCSI Controller** をインストールします

### 3. インストール: VirtIO をインストールして Windows をインストール

### 4. インストール: Windows をインストール

## ステップ 5: VirtIO をインストール

Windows をインストールする場所を選択します

1. インストール: **PC** → **インストール** → **インストール**
2. インストール:

- 仮想マシンを起動する
- 仮想マシン → 仮想マシンを起動する
- VirtIO CD として `NetKVM{windows-version}\amd64\` を使用する
- 仮想マシンを起動する

3. 仮想マシンを起動する:

- **Balloon** 設定: `PCI` 設定 → `Balloon{windows-version}\amd64\`
- 仮想マシン設定: `PCI` 設定 → `vioserial{windows-version}\amd64\`
- 仮想マシン設定: `PCI` 設定 → `vioinput{windows-version}\amd64\`

4. QEMU guest agent 設定:

VirtIO 仮想マシン CD として `guest-agent\qemu-ga-x86_64.msi` を使用する

## 6: 仮想マシン

仮想マシン VirtIO 仮想マシンを起動する

1. 仮想マシンを起動する: 仮想マシンを起動する
2. 仮想マシンを起動する: サービス (services.msc) から **QEMU Guest Agent** をインストールする
3. 仮想マシン: OpenShift 仮想マシンを起動する

```
oc get vmi windows-vm-virtio -o jsonpath='{.status.guestOSInfo}'
```

VM 仮想マシンを起動する

```
oc get vmi windows-vm-virtio -o jsonpath='{.status.interfaces[0].ipAddress}'
```

## VirtIO 仮想マシン

VirtIO 仮想マシンを起動するWindows VM 仮想マシンを起動する

| 仮想マシン            | 仮想マシン               |
|------------------|---------------------|
| 仮想マシン (NetKVM)   | 仮想マシンを起動する 10 分     |
| 仮想マシン (viosstor) | 仮想マシンを起動する IOPS 100 |
| 仮想マシン (vioser)   | 仮想マシンを起動する          |
| 仮想マシン            | 仮想マシンを起動する          |

## 仮想マシン

仮想マシン

| OS                 | Driver | Notes                                 |
|--------------------|--------|---------------------------------------|
| Windows 11 TPM 2.0 | TPM    | VM Windows 11 TPM                     |
|                    |        |                                       |
|                    |        | Windows ISO <code>bootOrder: 1</code> |
|                    |        | <code>bootOrder: 2</code>             |
|                    | VirtIO | viostor                               |
|                    | NetKVM | Windows                               |

ISO

ISO TLS

```
--insecure
virtctl image-upload dv <datavolume-name> --image-path=<iso-file> --insecure
```

ISO

ISO

```
oc delete vm windows-vm-virtio
oc delete dv windows-installation-iso virtio-drivers-iso
```

ISO

ISO

- DataVolume VirtIO Windows ISO
- VirtIO Windows VM
- Windows VirtIO
- VirtIO QEMU guest agent
- OpenShift Virtualization VM
- 

Windows VM VirtIO OpenShift Virtualization

ISO

- OpenShift VirtIO
- KubeVirt

- [Windows VM](#) [インストール](#)

# Infrastructure [インストール](#)

[インストール](#)

OpenShift Virtualization [インストール](#) (virt-controller、virt-api、virt-handler、CDI) [openshift-cnv](#) namespace [インストール](#) Pod [インストール](#) VM [インストール](#) M [インストール](#)

[HyperConverged](#) [OpenShift](#) Virtualization [インストール](#)

[インストール](#)

- OpenShift Virtualization [インストール](#)
- HyperConverged CR [インストール](#)
- Infrastructure Pod [NodeSelector](#) [Affinity](#) [インストール](#)
- [インストール](#)
- [Taint](#) [Toleration](#) [インストール](#)

[インストール](#)

- OpenShift Virtualization [OpenShift 4.18](#) [インストール](#)
- HyperConverged CR [インストール](#)
- [oc CLI](#) [インストール](#)
- [Worker Node](#) ([インストール](#))

[インストール](#)

OpenShift Virtualization [インストール](#)

| <a href="#">インストール</a> | <a href="#">インストール</a>                                             | <a href="#">インストール</a>                 |
|------------------------|--------------------------------------------------------------------|----------------------------------------|
| virt-controller        | VM <a href="#">インストール</a>                                          | Deployment (2 <a href="#">インストール</a> ) |
| virt-api               | <a href="#">API</a> <a href="#">インストール</a>                         | Deployment (2 <a href="#">インストール</a> ) |
| virt-handler           | <a href="#">VM</a> <a href="#">インストール</a>                          | DaemonSet                              |
| cdi-controller         | <a href="#">Containerized Data Importer</a> <a href="#">インストール</a> | Deployment (1 <a href="#">インストール</a> ) |
| cdi-uploadproxy        | Web <a href="#">VM</a> <a href="#">インストール</a>                      | Deployment (1 <a href="#">インストール</a> ) |
| cdi-apiserver          | CDI API <a href="#">インストール</a>                                     | Deployment (1 <a href="#">インストール</a> ) |
| hco-webhook            | HyperConverged CR <a href="#">インストール</a>                           | Deployment (1 <a href="#">インストール</a> ) |





|                              |                                                                           |
|------------------------------|---------------------------------------------------------------------------|
| spec.infra.nodePlacement     | Control plane 组件: virt-api virt-controller cdi-controller cdi-uploadproxy |
| spec.workloads.nodePlacement | 组件: virt-handler DaemonSet                                                |



HyperConverged

CR

OpenShift

Virtualization

Operator

Operator 部署在 OpenShift 上，用于管理虚拟化基础设施。

部署虚拟化基础设施

**NodeSelector** 配置

NodeSelector 配置用于指定节点选择器

```
Label nodes for virtualization infrastructure
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=
oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=
```

HyperConverged CR 配置

```
oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

NodeSelector 配置

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
 name: kubevirt-hyperconverged
 namespace: openshift-cnv
spec:
 infra:
 nodePlacement:
 nodeSelector:
 node-role.kubernetes.io/infra: ""
 workloads:
 nodePlacement:
 nodeSelector:
 node-role.kubernetes.io/worker: ""
```

部署虚拟化基础设施

- **Control plane** 组件 (virt-api virt-controller CDI) 部署在 `node-role.kubernetes.io/infra` 节点上
- **virt-handler DaemonSet** 部署在 `node-role.kubernetes.io/worker` 节点上

## NodeAffinity

NodeAffinity

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
 name: kubevirt-hyperconverged
 namespace: openshift-cnv
spec:
 infra:
 nodePlacement:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: node-role.kubernetes.io/infra
 operator: Exists
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 preference:
 matchExpressions:
 - key: topology.kubernetes.io/zone
 operator: In
 values:
 - us-east-1a
 - us-east-1b
```

- ( )
- ( )

## Taint Toleration

Taint

```
Taint infrastructure nodes
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
oc adm taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
```

Toleration HyperConverged CR

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
 name: kubevirt-hyperconverged
```

```

namespace: openshift-cnv
spec:
 infra:
 nodePlacement:
 nodeSelector:
 node-role.kubernetes.io/infra: ""
 tolerations:
 - key: node-role.kubernetes.io/infra
 operator: Exists
 effect: NoSchedule
 workloads:
 nodePlacement:
 nodeSelector:
 node-role.kubernetes.io/worker: ""

```

이제 Tolerations (Virtualization Infrastructure) Pod Taints를 지정할 수 있습니다.

**Pod Anti-Affinity** 지정

이제 Pod Anti-Affinity를 지정할 수 있습니다.

```

apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
 name: kubevirt-hyperconverged
 namespace: openshift-cnv
spec:
 infra:
 nodePlacement:
 affinity:
 podAntiAffinity:
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 podAffinityTerm:
 labelSelector:
 matchLabels:
 kubevirt.io: virt-api
 topologyKey: topology.kubernetes.io/zone
 - weight: 100
 podAffinityTerm:
 labelSelector:
 matchLabels:
 kubevirt.io: virt-controller
 topologyKey: topology.kubernetes.io/zone

```

이제 virt-api와 virt-controller Pod를 지정할 수 있습니다.



```
nodeSelector:
 node-role.kubernetes.io/infra: ""
tolerations:
- key: node-role.kubernetes.io/infra
 operator: Exists
 effect: NoSchedule
workloads:
 nodePlacement:
 nodeSelector:
 kubevirt.io/schedulable: "true"
```

**VM**

### Figure 3: $\mathcal{L}_{\text{train}}$ vs. $\mathcal{L}_{\text{val}}$

[illegible]

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
 name: kubevirt-hyperconverged
 namespace: openshift-cnv
spec:
 infra:
 nodePlacement:
 nodeSelector:
 node-role.kubernetes.io/infra: ""
 affinity:
 nodeAffinity:
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 preference:
 matchExpressions:
 - key: topology.kubernetes.io/zone
 operator: In
 values:
 - us-east-1a
 - us-east-1b
 - us-east-1c
 podAntiAffinity:
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 podAffinityTerm:
 labelSelector:
 matchLabels:
 kubevirt.io: virt-api
 topologyKey: topology.kubernetes.io/zone
 - weight: 100
 podAffinityTerm:
 labelSelector:
```

```

 matchLabels:
 kubevirt.io: virt-controller
 topologyKey: topology.kubernetes.io/zone
 tolerations:
 - key: node-role.kubernetes.io/infra
 operator: Exists
 effect: NoSchedule
 workloads:
 nodePlacement:
 nodeSelector:
 kubevirt.io/schedulable: "true"

```

注意: 默认情况下，Kubernetes 会根据 SLA 来调度 Pod。

部署 Pod

### HyperConverged CR 部署

```
oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

### HyperConverged CR 配置

```

oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type=merge -p '
{
 "spec": {
 "infra": {
 "nodePlacement": {
 "nodeSelector": {
 "node-role.kubernetes.io/infra": ""
 }
 }
 }
 }
}'

```

部署 Pod

### Infrastructure Pod 部署

```

Watch pod changes in real-time
oc get pods -n openshift-cnv -w

Verify new placement after changes settle
oc get pods -n openshift-cnv -o wide

Check specific component placement
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o jsonpath='{range

```

```
.items[*]]{.metadata.name}{"\t"}{.spec.nodeName}{"\n"}{end}'
```

□□□□□□:

- Infrastructure Pod □□□□□□□□□□□□□□□□□□□□
- virt-handler DaemonSet □□NodeSelector □□□□□□□□ Pod □□□/□□□□□□
- □□□□□ 205 □□□□□□□□□□

□□□□□□□□□□

**Pod** □□□□□□□□

```
Group pods by node
oc get pods -n openshift-cnv -o jsonpath='{range
.items[*]]{.spec.nodeName}{"\t"}{.metadata.name}{"\n"}{end}' | sort | column -t

Count pods per node
oc get pods -n openshift-cnv -o jsonpath='{range
.items[*]]{.spec.nodeName}{"\n"}{end}' | sort | uniq -c
```

**Toleration** □□□

```
Check if virt-api pods have correct tolerations
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o
jsonpath='{.items[0].spec.tolerations}' | jq
```

**NodeAffinity** □□□□□□□□

```
View node affinity for virt-controller
oc get pods -n openshift-cnv -l kubevirt.io=virt-controller -o
jsonpath='{.items[0].spec.affinity}' | jq
```

□□□□□□□□□□□□

**Infrastructure Pod** □□□ (Pending) □□□□□□□□□□

□□: □□□□□□□□□□Infrastructure Pod □ Pending □□□□□□□□□□

□□:

```
Check pod events
oc describe pod <pod-name> -n openshift-cnv

Look for scheduling errors
```



```
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | grep -i failedscheduling
```

□□□□□□:

- NodeSelector □ Affinity rule □□□□□□□□□□
- □□□□ Taint □□□□□□□□Toleration □□□
- □□□□□□□□□□□□□□□□□□

□□□:

```
Verify nodes have required labels
oc get nodes -L node-role.kubernetes.io/infra

Check node taints
oc get nodes -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.spec.taints}{"\n"}{end}{'

Verify node capacity
oc describe nodes | grep -A 5 "Allocated resources"
```

**virt-handler** □□□□□□□□□□□□□□□□

□□: virt-handler DaemonSet □ Pod □□□□□□□□□□□□□□□□

□□:

```
Check virt-handler DaemonSet status
oc get daemonset virt-handler -n openshift-cnv

Compare desired vs current pods
oc describe daemonset virt-handler -n openshift-cnv | grep -E "Desired|Current|Ready"
```

□□□: **spec.workloads.nodePlacement** Selector □□□□□□□□□□□□□□□□

```
View workloads node selector
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o
jsonpath='{.spec.workloads.nodePlacement}' | jq

Verify nodes match the selector
oc get nodes --selector=<label-from-workloads-nodeSelector>
```

□□□□□□□□□□□□

□□: HyperConverged CR □□□□□□□□Infrastructure Pod □□□□□□□□

**HyperConverged CR** □□□□□□□□:



111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- OpenShift Virtualization
- HyperConverged CR
- `spec.infra.nodePlacement` `spec.workloads.nodePlacement`
- Infrastructure Pod `NodeSelector``Affinity``Toleration`
- 
- Infrastructure

□ □ □ □

- VM のインストール
- VM の起動
- インストール済みの VM の管理

## □□□□□□ Affinity

11

OpenShift  
Kubernetes  
VM  
VM

|              |                |                                |
|--------------|----------------|--------------------------------|
| OpenShift    | Virtualization | VM                             |
| NodeSelector | NodeAffinity   | PodAffinity/Anti-Affinity rule |

□ □ □ □

- `nodeNameSelector` `NodeSelector` `nodeNameSelector`
- `NodeAffinity` rule (`required`) `preferred`) `NodeAffinity`
- VM `PodAffinity/Anti-Affinity` `PodAffinity/Anti-Affinity`
- `PodAffinity/Anti-Affinity` `PodAffinity/Anti-Affinity`
- `PodAffinity/Anti-Affinity` `PodAffinity/Anti-Affinity`

□ □ □ □

- OpenShift 4.18 需要 OpenShift Virtualization 容器虚拟化支持
- 容器虚拟化支持需要 KVM 支持
- `oc` 需要 `virtctl` CLI 容器虚拟化支持
- 容器虚拟化需要 Worker Node 支持 (容器虚拟化支持)

## OpenShift Virtualization

OpenShift Virtualization VM 3

| NodeSelector              | NodeAffinity             | PodAffinity/Anti-Affinity |
|---------------------------|--------------------------|---------------------------|
| NodeSelector              | VM                       |                           |
| NodeAffinity              | (required) / (preferred) |                           |
| PodAffinity/Anti-Affinity | VM Pod VM                | VM                        |



VM VM **virt-launcher** Pod Kubernetes Pod

## Selector & Affinity rule

Selector & Affinity rule

```
oc get nodes --show-labels
```

:

```
NAME STATUS ROLES AGE VERSION LABELS
ip-10-0-8-25.ec2.internal Ready control-plane,master 23d v1.31.14 beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-8-25.ec2.internal,...
ip-10-0-24-89.ec2.internal Ready worker 22d v1.31.14 beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-24-89.ec2.internal,...
ip-10-0-43-58.ec2.internal Ready worker 22d v1.31.14 beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-43-58.ec2.internal,...
ip-10-0-82-27.ec2.internal Ready worker 22d v1.31.14 beta.kubernetes.io/arch=amd64,kubernetes.io/hostname=ip-10-0-82-27.ec2.internal,...
```

( )

```
Get worker node names
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[0].metadata.name}')
WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[1].metadata.name}')
WORKER3=$(oc get nodes -l node-role.kubernetes.io/worker -o jsonpath='{.items[2].metadata.name}')
```

```
Label nodes by environment
oc label node $WORKER1 env=development
oc label node $WORKER2 env=production

Label nodes by GPU type
oc label node $WORKER1 gpu=nvidia-t4
oc label node $WORKER2 gpu=nvidia-a100

Label nodes by disk type
oc label node $WORKER1 storage=ssd
oc label node $WORKER3 storage=nvme

Label nodes by workload type
oc label node $WORKER1 workload=database
```

□□□□□□□□

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
oc get node $WORKER1 -o jsonpath='{.metadata.labels}' | jq
```

□□□:

```
{
 "beta.kubernetes.io/arch": "amd64",
 "beta.kubernetes.io/os": "linux",
 "env": "development",
 "gpu": "nvidia-t4",
 "kubernetes.io/arch": "amd64",
 "kubernetes.io/hostname": "ip-10-0-24-89.ec2.internal",
 "kubernetes.io/os": "linux",
 "node-role.kubernetes.io/worker": "",
 "storage": "ssd",
 "workload": "database"
}
```

□□□□□□

```
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
oc label node $WORKER1 gpu-
```

## NodeSelector □□□

NodeSelector □□□□□□□□□□□□□□□□ VM □□□□□□□□□□□□□□□□

namespace

namespace

```
oc create namespace vm-placement
```

NodeSelector

storage=ssd VM



cloud-init SSH

Download [vm-node-selector.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: database-vm
 namespace: vm-placement
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 app: database
 spec:
 nodeSelector:
 storage: ssd # VM will only run on nodes with storage=ssd label
 domain:
 cpu:
 cores: 2
 memory:
 guest: 4Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:43
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
```

```
user: fedora
password: fedora
chpasswd: { expire: False }
```

VM を起動する

```
oc apply -f vm-node-selector.yaml
virtctl start database-vm -n vm-placement
```

確認する

VM のステータスを確認する

```
oc get vmi database-vm -n vm-placement -o wide
```

結果:

| NAME        | AGE | PHASE   | IP           | NODENAME | READY |
|-------------|-----|---------|--------------|----------|-------|
| database-vm | 45s | Running | 10.128.0.200 | ocplab02 | True  |

ノードのステータスを確認する

```
oc get vmi database-vm -n vm-placement -o jsonpath='{.status.nodeName}' | xargs oc get node -L storage
```

結果:

| NAME                       | STATUS | ROLES  | AGE | VERSION | STORAGE |
|----------------------------|--------|--------|-----|---------|---------|
| ip-10-0-24-89.ec2.internal | Ready  | worker | 30d | v1.30.0 | ssd     |

NodeSelector を設定する

NodeSelector を設定する (NodeSelector を設定する)

```
spec:
 template:
 spec:
 nodeSelector:
 storage: ssd
 env: production
 gpu: nvidia-t4
```

VM を 3 つ起動する

## NodeAffinity

NodeAffinity (required) (preferred)

### NodeAffinity (required)

required Affinity (required)

[Download vm-node-affinity-required.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: gpu-workload-vm
 namespace: vm-placement
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 app: ml-training
 spec:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: gpu
 operator: In
 values:
 - nvidia-t4
 - nvidia-a100
 - key: env
 operator: In
 values:
 - production
 domain:
 cpu:
 cores: 4
 memory:
 guest: 8Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 volumes:
 - name: rootdisk
```



```

 containerDisk:
 image: quay.io/containerdisks/fedora:43
- name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 user: fedora
 password: fedora
 chpasswd: { expire: False }

```

선택:

- **requiredDuringSchedulingIgnoredDuringExecution:** VM에 반드시 실행되어야 하는 조건
- **matchExpressions:** In, NotIn, Exists, DoesNotExist, Gt, Lt 조건
- **nodeSelectorTerms:** OR (1개 이상의 matchExpressions) 또는 AND (2개 이상의 matchExpressions)
- 1개의 matchExpressions AND 2개의 matchExpressions

**NodeAffinity** (preferred) 섹션

preferred Affinity (선택) 섹션은 VM이 특정 노드에 배치될 수 있도록 합니다.

[Download vm-node-affinity-preferred.yaml](#)

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: web-vm
 namespace: vm-placement
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 app: webserver
 spec:
 affinity:
 nodeAffinity:
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 preference:
 matchExpressions:
 - key: storage
 operator: In
 values:
 - nvme
 - weight: 50
 preference:
 matchExpressions:
 - key: storage

```

```

 operator: In
 values:
 - ssd
 domain:
 cpu:
 cores: 2
 memory:
 guest: 2Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:43
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 user: fedora
 password: fedora
 chpasswd: { expire: False }

```

affinity:

- **preferredDuringSchedulingIgnoredDuringExecution:** `preferredDuringSchedulingIgnoredDuringExecution`
- **weight:** 1-100 `weight`
- `preference` (array) `preference`

**required** **preferred** **Affinity**

```

spec:
 template:
 spec:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: env
 operator: In
 values:
 - production
 preferredDuringSchedulingIgnoredDuringExecution:

```

```

- weight: 100
 preference:
 matchExpressions:
 - key: storage
 operator: In
 values:
 - nvme

```

VM NVMe

## PodAffinity Anti-Affinity

PodAffinity Anti-Affinity VM Pod VM

**PodAffinity: VM**

PodAffinity VM

[Download vm-pod-affinity.yaml](#)

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: web-frontend-vm
 namespace: vm-placement
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 app: web-frontend
 tier: frontend
 spec:
 affinity:
 podAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - database
 topologyKey: kubernetes.io/hostname
 domain:
 cpu:
 cores: 2
 memory:
 guest: 2Gi
 devices:
 disks:

```

```

- name: rootdisk
 disk:
 bus: virtio
- name: cloudinitdisk
 disk:
 bus: virtio
volumes:
- name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:43
- name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 user: fedora
 password: fedora
 chpasswd: { expire: False }

```

VM `app=database` Pod/VM (`topologyKey: kubernetes.io/hostname`)

Pod/VM:

| Pod/VM                                     | Pod/VM |
|--------------------------------------------|--------|
| <code>kubernetes.io/hostname</code>        | Pod/VM |
| <code>topology.kubernetes.io/zone</code>   | Pod/VM |
| <code>topology.kubernetes.io/region</code> | Pod/VM |

**PodAnti-Affinity: VM**

PodAnti-Affinity VM

[Download vm-pod-anti-affinity.yaml](#)

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: cache-vm-01
 namespace: vm-placement
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 app: redis-cache
 role: cache
 spec:
 affinity:
 podAntiAffinity:

```



```

 operator: In
 values:
 - redis-cache
 topologyKey: kubernetes.io/hostname

```

### Affinity rule

```

spec:
 template:
 spec:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: env
 operator: In
 values:
 - production
 podAffinity:
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 100
 podAffinityTerm:
 labelSelector:
 matchExpressions:
 - key: tier
 operator: In
 values:
 - database
 topologyKey: kubernetes.io/hostname
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - web-frontend
 topologyKey: kubernetes.io/hostname

```

VM :

- 
- VM
- Web VM

## 실습 1: PostgreSQL VM 배포

실습 1: PostgreSQL VM 배포

PostgreSQL VM 배포:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: postgres-vm-01
 labels:
 app: postgres-cluster
 instance: "01"
spec:
 template:
 metadata:
 labels:
 app: postgres-cluster
 spec:
 affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchLabels:
 app: postgres-cluster
 topologyKey: kubernetes.io/hostname
```

PostgreSQL VM 배포 완료

실습 2: GPU VM 배포

GPU VM 배포:

```
spec:
 template:
 spec:
 nodeSelector:
 gpu: "true"
 tolerations:
 - key: "nvidia.com/gpu"
 operator: "Exists"
 effect: "NoSchedule"
```

NodeSelector 및 Taints/Toleration을 사용하여 GPU VM을 GPU 노드에 배포합니다.

실습 3: PostgreSQL VM 배포

PostgreSQL VM 배포:







**□□: preferred Affinity □□□□□□□□□□□□□□□□□□□□□□□□:**

- 物理内存地址空间
- 虚拟内存地址空间
- 每个 VM 物理地址空间

□□:

```
Check where VM is running
oc get vmi <vm-name> -n vm-placement -o jsonpath='{.status.nodeName}'

Check that node's labels
oc get node <node-name> --show-labels
```

□□□:

- preferred Affinity  $\neq$  required Affinity
- Affinity is not a requirement
- Affinity is a preference

### Anti-Affinity ☐☐☐☐☐

□□: Anti-Affinity □□□□□□□□ VM □□□□□□□□□□□□□□

□□:

```
List all VMs and their nodes
oc get vmi -n vm-placement -o wide

Check anti-affinity configuration
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 15 "podAntiAffinity:"
```

□□□□□□:

- **Pod Selector:** Anti-Affinity Pod Selector → VM 分散配置
- **Pod Disruption Budget:** Pod 分散配置
- **Pod (preferred) vs Pod (required):** preferred Pod 分散配置
- **Pod:** Pod 分散配置 Anti-Affinity Pod Selector

111

[illegible]

```
Check VM labels
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.metadata.labels}' |
jq
```

```
Verify anti-affinity selector matches those labels
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "labelSelector:"
```

## NodeSelector

VM: VM

NodeSelector:

```
View VM spec
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "nodeSelector:"

Verify it's under spec.template.spec, not just spec
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.spec.nodeSelector}' |
jq
```

YAML nodeSelector:

```
WRONG - nodeSelector at VM spec level
spec:
 nodeSelector: # This is ignored!
 storage: ssd
 template:
 spec:
 domain:
 ...

CORRECT - nodeSelector at pod spec level
spec:
 template:
 spec:
 nodeSelector: # This works!
 storage: ssd
 domain:
 ...
```

```
Delete VMs
oc delete vm --all -n vm-placement

Remove custom node labels from all workers
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
```

```
WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[1].metadata.name}')
WORKER3=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[2].metadata.name}')

oc label node $WORKER1 storage- env- gpu- workload-
oc label node $WORKER2 storage- env- gpu-
oc label node $WORKER3 storage-
```

[illegible]

```
oc get vm -n vm-placement
oc get nodes --show-labels | grep -E "storage|workload"
```

111

[illegible]

- `NodeSelector` `NodeAffinity` `PodAffinity`/`Anti-Affinity` `PodDisruptionBudget`
- `PodDisruptionBudget` `PodAffinity`/`Anti-Affinity` `PodDisruptionBudget`
- `PodDisruptionBudget` `NodeSelector` `PodDisruptionBudget`
- `PodDisruptionBudget` (required) `PodDisruptionBudget` (preferred) `NodeAffinity` `PodDisruptionBudget`
- VM `PodDisruptionBudget` `PodAffinity`/`Anti-Affinity` `PodDisruptionBudget`
- HA `PodDisruptionBudget` `PodAffinity`/`Anti-Affinity` `PodDisruptionBudget`
- `PodDisruptionBudget` `PodAffinity`/`Anti-Affinity` `PodDisruptionBudget`

11

- VM 管理 - VM 管理
- Virtctl 管理 - VM 管理 CLI 管理
- 管理 - Red Hat 管理
- Pod 管理 - Kubernetes 管理
- Taint & Toleration - Kubernetes 管理

11

[illegible]

```
virtctl addvolume POD PVC VM OS virtctl
removevolume POD
```

□□□□

- **OpenShift 4.18** □□ (OpenShift Virtualization Operator □□□□□□□□□□)
- □□□□□□□□ (system:admin □□□□□□□□)
- **oc CLI** □□ **virtctl CLI** □□□□□□□□□□□□
- **default StorageClass** □□□□□□□□

□□□□□□□□□□:

OpenShift 4.20, 4.21

## □□ 1: namespace □□□□ VM □□□

namespace □□□□□□□□□□□□□□□□ VM □□□□□□□□

```
oc create namespace hotplug-demo
```



□□□□□□□□ cloud-init □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□SSH □□□□□□□□□□□□

**vm-hotplug.yaml** □□□□□□□

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: hotplug-vm
 namespace: hotplug-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: hotplug-vm
 spec:
 domain:
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
```

```

resources:
 requests:
 memory: 1Gi
networks:
- name: default
 pod: {}
volumes:
- name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
- name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false

```

EOF

VM 创建成功

```
oc get vm -n hotplug-demo -w
```

STATUS=Running 且 READY=True 时即可使用 **Ctrl+C** 强制结束

查看 VM 状态

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}{.name}: {.target}{"\n"}{end}'
```

输出结果:

```

rootdisk: vda
cloudinitdisk: vdb

```

第 2 步: 创建 **PVC**

通过 VM 创建 PersistentVolumeClaim

**hotplug-pvc.yaml** 文件内容

```

oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: hotplug-disk

```

```

namespace: hotplug-demo
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 5Gi
 storageClassName: lvms-vg1
 volumeMode: Block
EOF

```



volumeMode: Block Filesystem PVC disk.img

PVC WaitForFirstConsumer binding mode Pending

```
oc get pvc hotplug-disk -n hotplug-demo
```

3:

virtctl addvolume VM PVC

```

virtctl addvolume hotplug-vm --volume-name=hotplug-disk --namespace=hotplug-demo
--persist

```

:

Successfully submitted addvolume request to VM hotplug-vm for volume hotplug-disk

--persist VM

4:

VMI

```

oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}{.name}: {.target} (hotplugVolume:
{.hotplugVolume}){"\n"}{end}'

```

:

```

rootdisk: vda (hotplugVolume:)
cloudinitdisk: vdb (hotplugVolume:)

```

```
hotplug-disk: sda (hotplugVolume: {"attachPodName":"...", "attachPodUID":"..."})
```

Pod 状态: Ready 磁盘: SCSI (sda) 容量: hotplugVolume 大小: Pod 名称: hotplug-disk

PVC 名称: hotplug-disk

```
oc get pvc hotplug-disk -n hotplug-demo
```

输出:

| NAME         | STATUS | VOLUME   | CAPACITY | ACCESS MODES | STORAGECLASS | AGE |
|--------------|--------|----------|----------|--------------|--------------|-----|
| hotplug-disk | Bound  | pvc-xxxx | 5Gi      | RWO          | lvms-vg1     | 1m  |

Pod OS 名称: VM 名称: hotplug-vm

```
virtctl console hotplug-vm -n hotplug-demo
```

Pod 名称: fedora 磁盘: redhat 容量: 5Gi

```
lsblk
```

PVC 名称: hotplug-disk (sda) 容量: 5Gi 访问模式: RWO

Pod 名称: hotplug-vm (fedora)

VM 名称: hotplug-vm

```
sudo mkfs.xfs /dev/sda
```

Pod 名称: hotplug-vm

```
sudo mkdir -p /mnt/hotplug
sudo mount /dev/sda /mnt/hotplug
```

Pod 名称: hotplug-vm

```
echo "Hotplug test" | sudo tee /mnt/hotplug/test.txt
```

```
cat /mnt/hotplug/test.txt
```



□□□□□□:

## Hotplug test

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
sudo umount /mnt/hotplug
```

Ctrl+] □□□□□□□□□□□□□□□□

## □□ 6: □□□□□□□□□□□□□□□□

0000 VM 0000000000000000

```
virtctl removevolume hotplug-vm --volume-name=hotplug-disk --namespace=hotplug-demo
```

□□□□□□:

Successfully submitted removevolume request to VM hotplug-vm for volume hotplug-disk

[illegible]

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}{.name}: {.target}}{"\n"}{end}'
```

□□□□□□:

```
rootdisk: vda
cloudinitdisk: vdb
```

## hotplug-disk 磁碟機PVC 磁碟機

```
oc get pvc hotplug-disk -n hotplug-demo
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
--persist [][]
```

|                        |                       |
|------------------------|-----------------------|
| 名前                     | 値                     |
| <code>--persist</code> | VM spec の VM spec の名前 |
| (オプション)                | VM VMI の名前 (オプション)    |

VM spec の VM spec の名前 `--persist` の VM spec の名前

VM spec の VM spec の名前

VM spec の VM spec の名前:

- PersistentVolumeClaims (PVCs)
- DataVolumes

VM spec の VM spec の名前:

- containerDisk オプション
- cloudInitNoCloud / cloudInitConfigDrive オプション
- emptyDir オプション

VM spec の VM spec の名前

VM spec の VM spec の名前 SCSI オプション (sda-sdb オプション) virtio オプション PCI  
VM spec の VM spec の名前 SCSI オプション

VM spec の VM spec の名前 (オプション)

OpenShift

Virtualization

VM spec の VM spec の名前

VM

- **Multus CNI thick plugin:** Multus の thick plugin
- **Multus Dynamic Networks Controller:** Multus の Dynamic Networks Controller
- **Multi-node cluster:** Single Node Cluster

VM spec

VM spec (virtctl addvolume) の VM spec

1. NetworkAttachmentDefinition (NAD) の名前
2. VM spec の VM spec の名前
3. VM (thick Multus) の VM spec の名前

VM spec

VM spec の state: absent

```
interfaces:
- name: dyniface1
 state: absent
 bridge: {}
```



bridge SR-IOV

[KubeVirt Hotplug Network Interfaces documentation](#)

```
oc delete namespace hotplug-demo
```

- `virtctl addvolume` VM PVC
- `--persist`
- OS
- VM
- `virtctl removevolume`
- 

- [Hotplug Volumes - KubeVirt User Guide](#)
- [Hotplug Network Interfaces - KubeVirt User Guide](#)
- 

`VirtualMachines` `VirtualMachine`

- (vCPU RAM)
- (CPU)
-

- `label` Annotation

OpenShift Virtualization Machine spec

VirtualMachine OpenShift Virtualization rhel9-server-large fedora-desktop-medium VM

VirtualMachine

:

OpenShift 4.21

- 
- 1:
- 2:
- 3:
- 4:
- 
- 

- bash (zsh)
- OpenShift oc
- OpenShift Virtualization OpenShift
- (IDE)

1:

1

openshift project templates

oc get templates -n openshift

OpenShift OpenShift Virtualization

rhel のインストールテンプレート (インストールテンプレート) のインストール

```
...
rhel8-desktop-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-large Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-medium Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-small Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-large Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-medium Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-small Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-tiny Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-large Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-medium Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-small Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-tiny Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-large Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-medium Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-small Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-tiny Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-large Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-medium Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-small Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-tiny Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
...
```

openshift project の rhel9-server-medium のインストールテンプレート (インストールテンプレート -o yaml) のインストール

```
oc get template rhel9-server-medium -n openshift -o yaml > rhel9-server.template.yaml
```

### rhel9-server.template.yaml ##### (creationTimestamp uid resourceVersion #####)

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
 annotations:
 defaults.template.kubevirt.io/disk: rootdisk
 description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
 RHEL disk image must be available.
 iconClass: icon-rhel
 name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
 openshift.io/display-name: Red Hat Enterprise Linux 9 VM
 openshift.io/documentation-url: https://github.com/kubevirt/common-templates
 openshift.io/provider-display-name: Red Hat
 openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
 openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
 operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
 operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
 tags: hidden,kubevirt,virtualmachine,linux,rhel
 template.kubevirt.io/containerdisks: |
 registry.redhat.io/rhel9/rhel-guest-image
 template.kubevirt.io/editable: |
 /objects[0].spec.template.spec.domain.cpu.sockets
 /objects[0].spec.template.spec.domain.cpu.cores
 /objects[0].spec.template.spec.domain.cpu.threads
 /objects[0].spec.template.spec.domain.memory.guest
 /objects[0].spec.template.spec.domain.devices.disks
 /objects[0].spec.template.spec.volumes
 /objects[0].spec.template.spec.networks
 template.kubevirt.io/images: |
 https://access.redhat.com/downloads/content/479/ver=/rhel---
9/9.0/x86_64/product-software
 template.kubevirt.io/provider: Red Hat
 template.kubevirt.io/provider-support-level: Full
 template.kubevirt.io/provider-url: https://www.redhat.com
 template.kubevirt.io/version: v1alpha1
 template.openshift.io/bindable: "false"
 labels:
 app.kubernetes.io/component: templating
 app.kubernetes.io/managed-by: ssp-operator
```

```

app.kubernetes.io/name: common-templates
app.kubernetes.io/part-of: hyperconverged-cluster
app.kubernetes.io/version: 4.21.1
flavor.template.kubvirt.io/medium: "true"
os.template.kubvirt.io/rhel9.0: "true"
os.template.kubvirt.io/rhel9.1: "true"
os.template.kubvirt.io/rhel9.2: "true"
os.template.kubvirt.io/rhel9.3: "true"
os.template.kubvirt.io/rhel9.4: "true"
os.template.kubvirt.io/rhel9.5: "true"
os.template.kubvirt.io/rhel9.6: "true"
os.template.kubvirt.io/rhel9.7: "true"
template.kubvirt.io/architecture: amd64
template.kubvirt.io/type: base
template.kubvirt.io/version: v0.34.1
workload.template.kubvirt.io/server: "true"
name: rhel9-server-medium
namespace: openshift
objects:
- apiVersion: kubvirt.io/v1
 kind: VirtualMachine
 metadata:
 annotations:
 vm.kubvirt.io/validations: |
 [
 {
 "name": "minimal-required-memory",
 "path": "jsonpath::.spec.domain.memory.guest",
 "rule": "integer",
 "message": "This VM requires more memory.",
 "min": 1610612736
 }
]
 labels:
 app: ${NAME}
 kubvirt.io/dynamic-credentials-support: "true"
 vm.kubvirt.io/template: rhel9-server-medium
 vm.kubvirt.io/template.revision: "1"
 vm.kubvirt.io/template.version: v0.34.1
 name: ${NAME}
 spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubvirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: ${NAME}
 spec:
 sourceRef:
 kind: DataSource
 name: ${DATA_SOURCE_NAME}
 namespace: ${DATA_SOURCE_NAMESPACE}

```

```

storage:
 resources:
 requests:
 storage: 30Gi
runStrategy: Halted
template:
 metadata:
 annotations:
 vm.kubevirt.io/flavor: medium
 vm.kubevirt.io/os: rhel9
 vm.kubevirt.io/workload: server
 labels:
 kubevirt.io/domain: ${NAME}
 kubevirt.io/size: medium
 spec:
 architecture: amd64
 domain:
 cpu:
 cores: 1
 sockets: 1
 threads: 1
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 model: virtio
 name: default
 rng: {}
 features:
 smm:
 enabled: true
 firmware:
 bootloader:
 efi: {}
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 terminationGracePeriodSeconds: 180
 volumes:
 - dataVolume:
 name: ${NAME}
 name: rootdisk
 - cloudInitNoCloud:

```



```

 userData: |-
 #cloud-config
 user: cloud-user
 password: ${CLOUD_USER_PASSWORD}
 chpasswd: { expire: False }
 name: cloudinitdisk

parameters:
- description: VM name
 from: rhel9-[a-z0-9]{16}
 generate: expression
 name: NAME
- description: Name of the DataSource to clone
 name: DATA_SOURCE_NAME
 value: rhel9
- description: Namespace of the DataSource
 name: DATA_SOURCE_NAMESPACE
 value: openshift-virtualization-os-images
- description: Randomized password for the cloud-init user cloud-user
 from: '[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}'
 generate: expression
 name: CLOUD_USER_PASSWORD

```

parameters parameter field value description

- **NAME** - VM ([a-z0-9]{16})
- **DATA\_SOURCE\_NAME** - VM DataSource
- **DATA\_SOURCE\_NAMESPACE** - DataSource namespace openshift-virtualization-os-images project
- **CLOUD\_USER\_PASSWORD** - cloud-user NAME (4)



- 
- bash

rhel9-server.template.yaml oc process

```
oc process -f rhel9-server.template.yaml -o yaml
```

yaml List VirtualMachine



- OpenShift 1 Kubernetes

- OpenShift Virtualization のインストールと VirtualMachine のインストール

パート 2 のインストール

## パート 2: OpenShift Virtualization のインストール

OpenShift CLI を使用して VirtualMachine をインストールするための `oc process` コマンドを使用して `oc create` (または `oc apply`) を使用してインストールします。

OpenShift RHEL 9 VirtualMachine をインストールする `project (namespace)` をインストールします。

```
oc new-project template-example
```

`rhel9-server.template.yaml` を使用して `oc apply` を使用して (または VirtualMachine をインストールする `oc create` を使用して)。

```
oc process -f rhel9-server.template.yaml | oc create -n template-example -f -
```

RHEL 9 のインストールを namespace にインストールします。

```
oc get all -n template-example
```

DataVolume は PendingPopulation をインストールする VirtualMachine は Stopped をインストールします。

インストールした `rhel9-server` VM をインストールします。

```
export RHEL9_SERVER_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n template-example)
```

インストールした VM をインストールする `rhel9-abcdef0987654321` をインストールします。

```
echo $RHEL9_SERVER_VM
```

`virtctl` を使用して VM をインストールする (または VM をインストールする)。

```
virtctl start $RHEL9_SERVER_VM
```

`oc get vm` を使用して (または `-w` を使用して) VM をインストールする `Stopped` は `Provisioning` をインストールする `Running` をインストールします。

VM をインストールする CLI をインストールする [パート 4](#) をインストールします。

### □□□□ 3: □□□□□□□□□□□□

□□□□□□□□□□□□□□□□ `rhel9` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□ 2 □□□□ `project/namespace` □□□□□□□□□□

```
oc project template-example
```

`rhel9-server.template.yaml` □□□□□□□□ `rhel9-custom.template.yaml` □□□□□□□□□□

```
cp rhel9-{server,custom}.template.yaml
```

□□□□`rhel9-custom.template.yaml` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
 annotations:
 defaults.template.kubevirt.io/disk: rootdisk
 description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
 RHEL disk image must be available.
 iconClass: icon-rhel
 name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
 name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
 openshift.io/display-name: Red Hat Enterprise Linux 9 VM
 openshift.io/documentation-url: https://github.com/kubevirt/common-templates
 openshift.io/provider-display-name: Red Hat
 openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
 openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
 operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
 operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
 tags: hidden,kubevirt,virtualmachine,linux,rhel
 template.kubevirt.io/containerdisks: |
 registry.redhat.io/rhel9/rhel-guest-image
 template.kubevirt.io/editable: |
 /objects[0].spec.template.spec.domain.cpu.sockets
 /objects[0].spec.template.spec.domain.cpu.cores
 /objects[0].spec.template.spec.domain.cpu.threads
 /objects[0].spec.template.spec.domain.memory.guest
 /objects[0].spec.template.spec.domain.devices.disks
 /objects[0].spec.template.spec.volumes
 /objects[0].spec.template.spec.networks
```

```

 template.kubevirt.io/images: |
 https://access.redhat.com/downloads/content/479/ver=/rhel---
9/9.0/x86_64/product-software
 template.kubevirt.io/provider: Red Hat
 template.kubevirt.io/provider-support-level: Full
 template.kubevirt.io/provider-url: https://www.redhat.com
 template.kubevirt.io/version: v1alpha1
 template.openshift.io/bindable: "false"
labels:
 app.kubernetes.io/component: templating
 app.kubernetes.io/managed-by: ssp-operator
 app.kubernetes.io/name: common-templates
 app.kubernetes.io/part-of: hyperconverged-cluster
 app.kubernetes.io/version: 4.21.1
 flavor.template.kubevirt.io/medium: "true"
 os.template.kubevirt.io/rhel9.0: "true"
 os.template.kubevirt.io/rhel9.1: "true"
 os.template.kubevirt.io/rhel9.2: "true"
 os.template.kubevirt.io/rhel9.3: "true"
 os.template.kubevirt.io/rhel9.4: "true"
 os.template.kubevirt.io/rhel9.5: "true"
 os.template.kubevirt.io/rhel9.6: "true"
 os.template.kubevirt.io/rhel9.7: "true"
 template.kubevirt.io/architecture: amd64
 template.kubevirt.io/type: base
 template.kubevirt.io/version: v0.34.1
 workload.template.kubevirt.io/server: "true"
name: rhel9-custom
namespace: openshift
objects:
- apiVersion: kubevirt.io/v1
 kind: VirtualMachine
 metadata:
 annotations:
 vm.kubevirt.io/validations: |
 [
 {
 "name": "minimal-required-memory",
 "path": "jsonpath::.spec.domain.memory.guest",
 "rule": "integer",
 "message": "This VM requires more memory.",
 "min": 1610612736
 }
]
 labels:
 app: ${NAME}
 kubevirt.io/dynamic-credentials-support: "true"
 vm.kubevirt.io/template: rhel9-server-medium
 vm.kubevirt.io/template.revision: "1"
 vm.kubevirt.io/template.version: v0.34.1
 name: ${NAME}

```

```

spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: ${NAME}
 spec:
 sourceRef:
 kind: DataSource
 name: ${DATA_SOURCE_NAME}
 namespace: ${DATA_SOURCE_NAMESPACE}
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: Always
 template:
 metadata:
 annotations:
 vm.kubevirt.io/flavor: medium
 vm.kubevirt.io/os: rhel9
 vm.kubevirt.io/workload: server
 labels:
 kubevirt.io/domain: ${NAME}
 kubevirt.io/size: medium
 spec:
 architecture: amd64
 domain:
 cpu:
 cores: 1
 sockets: 1
 threads: 1
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 model: virtio
 name: default
 rng: {}
 features:
 smm:
 enabled: true
 firmware:
 bootloader:
 efi: {}

```



```
oc process -f rhel9-custom.template.yaml -o yaml
```

[illegible]

□□□□ **4:** □□□□□□□□□□□□□□

[illegible]

```
rhel9-custom 0000000000 VirtualMachine 0000000000
```

```
oc process -f rhel9-custom.template.yaml | oc apply -n template-example -f -
```

```

#####rhel9-custom VM #####oc get vm #####

```

```
oc get vm -n template-example
```

```
export RHEL9_CUSTOM_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n
template-example | grep custom)
```

```
0: rhel9 VM $RHEL9_CUSTOM_VM RHEL9_SERVER_VM
```

## VM 的 yml 和 cloud-init 的配置文件

```
oc describe vm $RHEL9_CUSTOM_VM | grep password
```

```

$ virtctl exec vm username --vm shadowman -- cloud-user
cloud-user$

```

```
virtctl console $RHEL9_CUSTOM_VM
```

**ENTER** [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] **CTRL+]** [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]



- OpenShift Virtualization 
-  openshift project 

```

$ kubectl get pods --context kube-context --namespace (kubectl get namespace default -o jsonpath='{.metadata.namespace}') --no-headers

```

```
oc project default
```

namespace namespace

```
oc delete namespace template-example
```

- 
- OpenShift Virtualization VirtualMachine
- 
- VirtualMachine
- VirtualMachine



# VM ทรัพยากรและการจัดการ

## VM

OpenShift Virtualization ทรัพยากรและการจัดการ VM ทรัพยากรและการจัดการ CPU NUMA ทรัพยากรและการจัดการ

## ทรัพยากร

- CPU/หน่วย (requests) (limits) QoS ทรัพยากร
- vCPU CPU (pinning) ทรัพยากร
- Hugepage NUMA ทรัพยากร
- ทรัพยากร

## การตั้งค่า

VM ทรัพยากรและการจัดการ

- OpenShift 4.18+ OpenShift Virtualization Operator ทรัพยากรและการจัดการ
- Kubernetes ทรัพยากรและการจัดการ
- CLI (oc, virtctl, kubectl) ทรัพยากรและการจัดการ
- (CPU Manager hugepages) ทรัพยากรและการจัดการ

## การตั้งค่า

### Resource limits & QoS

CPU/หน่วย (requests) (limits) QoS namespace ResourceQuota & LimitRange ทรัพยากรและการจัดการ

### CPU

Kubernetes CPU Manager static vCPU CPU (pinning) CPU (CPU steal) ทรัพยากรและการจัดการ

### NUMA & Hugepages

Hugepage NUMA TLB NUMA ทรัพยากรและการจัดการ

### VM

CPU NUMA Hugepages RT (SCHED\_FIFO) ทรัพยากรและการจัดการ

## Resource limits and QoS

OpenShift Virtualization provides the following QoS classes:

1. **Resource Limits and QoS** — CPU Guaranteed QoS
2. **CPU Pinning** — CPU Manager static vCPU
3. **NUMA and Hugepages** — hugepage NUMA
4. **Real-time VMs** — RT SCHED\_FIFO

## Configuration

- [VM Configuration](#) -
- [Storage Configuration](#) -

## OpenShift Virtualization

- [OpenShift Virtualization](#)
- [KubeVirt](#)

# Resource limits & QoS

OpenShift Virtualization provides the following QoS classes: requests/limits/QoS

KubeVirt VM spec requests & limits Kubernetes QoS class VM ResourceQuota & LimitRange namespace

- **OpenShift 4.18** (OpenShift Virtualization Operator)
- **system:admin**
- **oc CLI** **virtctl CLI**
- **default StorageClass** (VM)

OpenShift 4.20, 4.21

## VM

OpenShift Virtualization VM KubeVirt VM spec VM virt-launcher

[illegible]

## CPU □□□□

- `cpuAllocationRatio` CPU request (10) 1 vCPU VM 100m CPU
- CPU limits VM request
- VM CPU request limits
- vCPU `sockets * cores * threads` 1

□ □ □ □ □ □ □

- VM `spec.domain.memory.guest` `spec.domain.resources.requests.memory` `memory`
- KubeVirt `memory` (QEMU `memory`) `Memory request`
- `Memory limits`
- `Memory limits` `500 MiB` `2%`

[illegible]

## □□□□ 1: □□□□ namespace □□□

```
oc create namespace resource-qos-demo
```

## Figure 2: QoS class VM

Kubernetes `Pod` 3 `QoS class` 1 `VM` `Pod`

## Burstable QoS (整形整形)

requests limits VM Burstable QoS OpenShift Virtualization VM



cloud-init SSH

## vm-burstable.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-burstable
 namespace: resource-qos-demo
```

```

spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-burstable
 spec:
 domain:
 memory:
 guest: 512Mi
 resources:
 requests:
 memory: 512Mi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false

```

EOF

## Guaranteed QoS

CPU requests limits VM Guaranteed QoS VM  
 (evict)

**vm-guaranteed.yaml**

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine

```

```

metadata:
 name: vm-guaranteed
 namespace: resource-qos-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-guaranteed
 spec:
 domain:
 memory:
 guest: 512Mi
 resources:
 requests:
 memory: 1Gi
 cpu: "1"
 limits:
 memory: 1Gi
 cpu: "1"
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false

```

EOF



Memory limits (1Gi) (512Mi)  
limits Pod

## BestEffort QoS

requests limits VM BestEffort QoS VM (evict)

**vm-besteffort.yaml**

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-besteffort
 namespace: resource-qos-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-besteffort
 spec:
 domain:
 memory:
 guest: 512Mi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
```

EOF

### Chapter 3: QoS Class

VM QoS Class

```
oc get vm -n resource-qos-demo
```

Output

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| vm-besteffort | 30s | Running | True  |
| vm-burstable  | 60s | Running | True  |
| vm-guaranteed | 45s | Running | True  |

VM Pod QoS class

Burstable VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o
jsonpath='{.items[0].status.qosClass}'
```

Output

Burstable

Guaranteed VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o
jsonpath='{.items[0].status.qosClass}'
```

Output

Guaranteed

BestEffort VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o
jsonpath='{.items[0].status.qosClass}'
```

Output

Burstable



**virt-launcher** VMs are managed by KubeVirt and have Resource requests and limits. **vm-besteffort** VMs have BestEffort requests and limits. **Burstable** VMs have requests and limits.

**virt-launcher** Pod requests and limits

Burstable VM requests

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o yaml
```

Guaranteed VM requests

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o yaml
```

BestEffort VM requests

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o yaml
```

## 4: ResourceQuota

ResourceQuota is used to limit the number of VMs and Pods in a namespace. It also limits the total amount of resources (CPU, memory, etc.) that can be used by the VMs and Pods in the namespace. ResourceQuota is managed by KubeVirt and VMs have Resource limits.

**resource-quota.yaml** file

```
oc apply -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata:
 name: vm-resource-quota
 namespace: resource-qos-demo
spec:
 hard:
 requests.cpu: "4"
 requests.memory: 8Gi
 limits.cpu: "8"
 limits.memory: 16Gi
 pods: "10"
EOF
```

Quota requests

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

namespace requests limits namespace requests VM quota limits



□□□□□□□□□□□□□□ CPU limits (1 vCPU □□□ 1) □ Memory limits (requests □ 2 □) □□□□□□□□

□□ **Memory Limits Ratio** □□□□□□□

□□□□□□□□□ Memory limits □ requests □□□□ 2 □□□□□□□□□□ namespace □□□□□□□□□□□□

```
oc label namespace resource-qos-demo alpha.kubevirt.io/auto-memory-limits-ratio=1.5
```

□□□□□□ namespace □□□□□□□□ VM □□□□□□□ Memory limits ratio □ 1.5 □□□□□□□□□

## □□□□ 5: **LimitRange** □□□

LimitRange □□ namespace □□ Pod □□□□□□□□□□□□□□□□□□□□□□□□□□□□ VM □□□□□□□□□□□□□□□□□□□□□□□□□□□□

**limit-range.yaml** □□□□□□□

```
oc apply -f - <<EOF
apiVersion: v1
kind: LimitRange
metadata:
 name: vm-limit-range
 namespace: resource-qos-demo
spec:
 limits:
 - type: Pod
 max:
 cpu: "4"
 memory: 8Gi
 min:
 cpu: 100m
 memory: 256Mi
 - type: Container
 default:
 cpu: 500m
 memory: 1Gi
 defaultRequest:
 cpu: 100m
 memory: 512Mi
 max:
 cpu: "4"
 memory: 8Gi
 min:
 cpu: 50m
 memory: 128Mi
EOF
```

□□□□□□□

```
oc describe limitrange vm-limit-range -n resource-qos-demo
```

LimitRange 限制 Pod (VM 即 **virt-launcher** Pod) 的 Resource requests 和 limits

## 图 6: 限制 VM 的 CPU

VM Pod 的 CPU 限制

```
oc adm top pods -n resource-qos-demo
```

**virt-launcher** Pod 的 CPU requests 和 limits

Quota 限制 namespace 的资源

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

## QoS class 限制

| QoS class  | 限制                     | Eviction Priority  | 限制          |
|------------|------------------------|--------------------|-------------|
| Guaranteed | CPU requests == limits | Low (evict)        | 限制 VM 的 CPU |
| Burstable  | requests < limits      | BestEffort (evict) | 限制 VM 的 CPU |
| BestEffort | requests > limits      | High (evict)       | 限制 VM 的 CPU |

## 限制 VM 的内存

- 限制 VM 的 **Burstable** 内存: Memory request 限制 VM 的内存
- 限制 VM 的 **(evict)** 内存: VM 的 **Guaranteed** 内存
- Memory limits** 限制: 限制 VM 的内存
- limits** 限制: 限制 VM 的内存
- namespace** 限制 **ResourceQuota** 限制: 限制 namespace 的资源
- LimitRange** 限制 VM 的内存

## 限制 VM 的 CPU

限制 VM 的 CPU

```
oc delete namespace resource-qos-demo
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- VM CPU requests & limits
- KubeVirt VM
- Guaranteed/Burstable/BestEffort QoS class VM
- ResourceQuota namespace
- LimitRange Pod
- VM
- namespace Memory limits ratio

□ □ □ □

- [KubeVirt Resource Requests and Limits \(KubeVirt 日本語ドキュメント\)](#)
- [Working with Resource Quotas for VMs - OpenShift 日本語ドキュメント](#)
- [Configure QoS for Pods - Kubernetes 日本語ドキュメント](#)
- [Resource Quotas Per Project - OpenShift 日本語ドキュメント](#)

□□ CPU □□

11

OpenShift Virtualization CPU vCPU  
CPU CPU vCPU CPU (pinning) CPU  
(jitter)

**NFV/HPC**

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18](#) [OS \(4.21\)](#)
- [Operator](#) (`system:admin` [RBAC](#))
- `oc` [CLI](#) `virtctl` CLI [CLI](#)
- [CPU](#) [Worker Node](#) ([4](#) [vCPU](#))
- Kubernetes [QoS](#) [Resource limits](#) [QoS](#)

□□ CPU □□□□□

OpenShift → Kubernetes → CPU Manager → Pod → CPU → CPU Manager → **static**  
 → Pod → CPU → CPU →

KubeVirt  $\rightarrow$  **dedicatedCpuPlacement**  $\rightarrow$  **CPU Manager**  $\rightarrow$  **virt-launcher** Pod  $\rightarrow$  vCPU  $\rightarrow$  CPU  
 $\rightarrow$  **Pod**  $\rightarrow$  vCPU  $\rightarrow$  CPU  $\rightarrow$

2

- CPU Manager `static`
- VM QoS **Guaranteed** requests limits CPU

|                               | vCPU         |         |
|-------------------------------|--------------|---------|
| (shared)                      | vCPU CPU     | VM      |
| (dedicatedCpuPlacement: true) | vCPU CPU CPU | NFV/HPC |

## 1: CPU Manager Static Policy

CPU Manager KubeletConfig MachineConfigPool

Worker MachineConfigPool

```
oc label machineconfigpool worker custom-kubelet=cpumanager-enabled
```



OpenShift (SNO) worker master MachineConfigPool KubeletConfig machineConfigPoolSelector worker master

kubeletconfig-static-policy.yaml

KubeletConfig Static CPU manager policy

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
 name: cpumanager-enabled
spec:
 machineConfigPoolSelector:
 matchLabels:
 custom-kubelet: cpumanager-enabled
 kubeletConfig:
 cpuManagerPolicy: static
 cpuManagerReconcilePeriod: 5s
EOF
```

Machine Config Operator kubelet

```
oc get mcp worker -w
```

UPDATING ☐ False ☐ READYMACHINECOUNT ☐ MACHINECOUNT ☐

| NAME     | CONFIG                                               | UPDATED | UPDATING |
|----------|------------------------------------------------------|---------|----------|
| DEGRADED |                                                      |         |          |
| worker   | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True    | False    |

## CPU Manager Policy ☐

☐ Worker Node ☐ kubelet ☐ static ☐

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

☐

```
cpuManagerPolicy: static
cpuManagerReconcilePeriod: 5s
```

## ☐ 2: ☐ CPU ☐

☐ CPU ☐ VM ☐ CPU Manager ☐ NodeSelector ☐ Static Policy  
☐ Worker Node ☐ cpumanager=true ☐

```
oc label node <node-name> cpumanager=true
```

☐

```
oc get nodes -L cpumanager
```

☐

| NAME      | STATUS | ROLES  | AGE | VERSION | CPUMANAGER |
|-----------|--------|--------|-----|---------|------------|
| worker-01 | Ready  | worker | 10d | v1.34.5 | true       |
| worker-02 | Ready  | worker | 10d | v1.34.5 | true       |



KubeVirt CR ☐ CPUManager ☐ KubeVirt ☐ cpumanager=true  
☐ KubeVirt ☐ Static CPU manager policy  
☐

## ☐ 3: namespace ☐

```
oc create namespace cpu-pinning-demo
```

#### ステップ 4: CPU 専用 VM を作成する

この VM は `dedicatedCpuPlacement: true` を指定して 2 個の vCPU を確保し、`nodeSelector` で VM の `cpumanager=true` のノードにスケジューリングされるように設定します。また、`requests` と `limits` を指定して Guaranteed QoS を確保します。



この cloud-init は、SSH を使用して VM にアクセスできるように設定されています。

`vm-dedicated-cpu.yaml` の内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-dedicated-cpu
 namespace: cpu-pinning-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-dedicated-cpu
 spec:
 nodeSelector:
 cpumanager: "true"
 domain:
 cpu:
 sockets: 2
 cores: 1
 threads: 1
 dedicatedCpuPlacement: true
 memory:
 guest: 1Gi
 resources:
 requests:
 memory: 2Gi
 limits:
 memory: 2Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
```

```

 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
EOF

```



QEMU VM (2Gi) KubeVirt (1Gi) Resource limits QoS

VM Running

```
oc get vm vm-dedicated-cpu -n cpu-pinning-demo -w
```

| NAME             | AGE | STATUS  | READY |
|------------------|-----|---------|-------|
| vm-dedicated-cpu | 30s | Running | True  |

## 5: CPU pinning

VM

```
oc get vmi vm-dedicated-cpu -n cpu-pinning-demo -o wide
```

| NAME             | AGE | PHASE   | IP          | NODENAME  | READY |
|------------------|-----|---------|-------------|-----------|-------|
| vm-dedicated-cpu | 60s | Running | 10.128.0.45 | worker-01 | True  |

```
oc debug node/<node-name>
```

確認CPU Manager state file 確認CPU CPU 確認

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

JSON 確認entries 確認 virt-launcher Pod の UID 確認 Pod の compute  
確認 CPU ID 確認

```
{"policyName":"static","defaultCpuSet":"0-1,3-41,43-79","entries":{"<pod-
uid>":{"compute":"2,42"}}, "checksum":...}
```

確認CPU 2 の 42 の VM の 2 の vCPU 確認 defaultCpuSet 確認VM  
確認

確認

```
exit
```

確認

VM 確認

```
virtctl console vm-dedicated-cpu -n cpu-pinning-demo
```

確認 fedora 確認 redhat 確認 OS 確認 vCPU 確認

```
lscpu | grep -E "^CPU\(s\)|^Socket|^Core|^Thread"
```

確認

```
CPU(s): 2
Thread(s) per core: 1
Core(s) per socket: 1
Socket(s): 2
```

確認 2 の vCPU (2 の 1 の 1 の) 確認 Topology 確認

Ctrl+] 確認



## Emulator Thread

QEMU I/O threads are pinned to the Emulator Thread. VM vCPU CPU threads are pinned to the `isolateEmulatorThread` CPU thread.

Emulator Thread threads VM threads CPU threads `dedicatedCpuPlacement: true` `isolateEmulatorThread: true`

```
domain:
 cpu:
 sockets: 2
 cores: 1
 threads: 1
 dedicatedCpuPlacement: true
 isolateEmulatorThread: true
```

KubeVirt has 3 CPU threads (2 vCPU 1 Emulator Thread) threads.



Simultaneous Multithreading (SMT) `full-pcpus-only` CPU Manager option  
vCPU `isolateEmulatorThread`  
KubeVirt CR `AlignCPUs`

VM is **Pending**

VMI

```
oc describe vmi vm-dedicated-cpu -n cpu-pinning-demo
```

```
Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector
```

- `cpumanager=true` `oc get nodes -L cpumanager` Static CPU manager policy
- CPU `oc describe node <node-name>` CPU

## VM CPU pinning

KubeletConfig

```
oc get kubeletconfig cpumanager-enabled
```

MachineConfigPool

```
oc get mcp worker
```

UPDATED **False** DEGRADED **True** MCO

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-operator --tail=50
```

## VM "insufficient dedicated CPU resources"

**dedicatedCpuPlacement: true** QoS **Guaranteed** CPU requests limits

virt-launcher Pod QoS

```
oc get pod -n cpu-pinning-demo -l kubevirt.io/domain=vm-dedicated-cpu -o jsonpath='{.items[0].status.qosClass}'
```

**Guaranteed** **Burstable** VM Memory requests Memory limits

**cpu\_manager\_state** VM

CPU Manager state file virt-launcher Pod CPU kubelet **none**

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

**cpuManagerPolicy** **none** KubeletConfig MachineConfigPool

```
oc delete namespace cpu-pinning-demo
```

cpumanager=true

```
oc label node <node-name> cpumanager-
```

CPU manager policy `KubeletConfig` `MachineConfigPool`

```
oc delete kubeletconfig cpumanager-enabled
```

```
oc label machineconfigpool worker custom-kubelet-
```



`KubeletConfig` `kubelet` `MCO` `CPU Manager` `MachineConfigPool`

- `KubeletConfig` `Kubernetes CPU Manager` `static`
- `CPU`
- `dedicatedCpuPlacement: true` `Guaranteed QoS`
- `CPU Manager state file` `CPU pinning`
- `vCPU Topology`
- `isolateEmulatorThread: true` `QEMU Emulator Thread`

- `VM` `QoS` — `Guaranteed QoS`
- — `VM`
- —
- `KubeVirt` `CPU` (`)`
- `CPU Manager` `Topology Manager` `- OpenShift` (`)`
- `CPU` - `Kubernetes` (`)`

## NUMA `Hugepages`

`OpenShift` `Virtualization` `Hugepages` `NUMA`  
`VM` `4Ki` `OS` `NUMA`  
`NFV/HPC`  
2

- **TLB pressure:** 16Gi 4Ki VM 400 CPU TLB (Translation Lookaside Buffer) TLB
- **Cross-NUMA memory latency:** NUMA 1.5-3 OS NUMA

Hugepages 4Ki 2Mi 1Gi TLB pressure NUMA (guestMappingPassthrough) NUMA VM NUMA CPU VM

□□□□

- OpenShift Virtualization Operator OpenShift 4.18 (4.21)
- (system:admin)
- oc virtctl CLI
- CPU (8) Hugepages RAM Worker Node
- CPU Manager static policy (CPU)
- Kubernetes QoS (Resource limits QoS)

Hugepages □□□

Linux 4Ki 2Gi VM 524,288 TLB CPU

Hugepages

| □□□□□       | 2Gi     | TLB     | □□                   |
|-------------|---------|---------|----------------------|
| 4Ki (□□□□□) | 524,288 | 524,288 | □□□□□□□              |
| 2Mi         | 1,024   | 1,024   | □□□□□□□□□□ VM        |
| 1Gi         | 2       | 2       | □□□□□□□□□□ VM□□□□□□□ |

VM memory.hugepages.pageSize KubeVirt Hugepages QEMU (pinned)



Hugepages VM NUMA Hugepages VM NUMA VM CPU NUMA Hugepages NUMA QEMU VM Memory requests Hugepages

NUMA □□□□□□□

NUMA (Non-Uniform Memory Access) CPU (□□□□□□□□)

□□□□ 2 □□□□□□□□□□:

NUMA Node 0: CPUs 0-19, 40-59 Local Memory: 128Gi  
NUMA Node 1: CPUs 20-39, 60-79 Local Memory: 128Gi

NUMA ノードは OS の NUMA ノード 0 と 1 に対応する vCPU が割り当てられています。NUMA ノード 1 は、ノード 0 と同じように構成されています。

KubeVirt の `guestMappingPassthrough` オプションは、NUMA ノードをパッシングします (パッシングは CPU ノード) 対応する NUMA ノードを NUMA ノードにマッピングします。

NUMA ノードは 3 つあります。

1. CPU ノード (`dedicatedCpuPlacement: true`)
2. Hugepages ノード
3. KubeVirt CR の NUMA ノード (OpenShift Virtualization 4.18 からサポートされています)

## ステップ 1: Hugepages ノード

Hugepages ノードは OpenShift の Node Tuning Operator の `Tuned` ノードによって管理されます。

`Tuned` ノードは Worker Node の 2Mi の hugepages を 1024 ノード (2Gi) に設定します。

`hugepages-tuned-boottime.yaml` の内容

```
oc apply -f - <<EOF
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
 name: hugepages
 namespace: openshift-cluster-node-tuning-operator
spec:
 profile:
 - data: |
 [main]
 summary=Boot time configuration for hugepages
 include=openshift-node
 [bootloader]
 cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=1024
 name: openshift-node-hugepages
 recommend:
 - machineConfigLabels:
 machineconfiguration.openshift.io/role: "worker"
 priority: 20
 profile: openshift-node-hugepages
EOF
```



Single Node OpenShift (SNO) の `machineConfigLabels` は `worker` または `master` になります。



Node Tuning Operator  `hugepagesz=2M hugepages=1024`  MachineConfig  
 Machine Config Operator  Worker Node                  

```
oc get mcp worker -w
```

UPDATED 1 True 0000MACHINECOUNT 1 READYMACHINECOUNT 0000000000000000

| NAME     | CONFIG                                                   | UPDATED | UPDATING |       |
|----------|----------------------------------------------------------|---------|----------|-------|
| DEGRADED |                                                          |         |          |       |
| worker   | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True    | False    | False |



MachineConfigPool

## Hugepages ☐☐☐☐☐☐

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-2Mi}{"\n"}'
```

□□□□□□:

2Gi

```
00000000000000000000000000000000/proc/meminfo 00000000
```

```
oc debug node/<node-name>
```

□□□□□□□□:

```
chroot /host grep -i hugepages /proc/meminfo
```

□□□□□□:

```
AnonHugePages: 0 kB
ShmemHugePages: 0 kB
FileHugePages: 0 kB
```



```
chroot /host cat /sys/devices/system/node/node0/meminfo
```

結果 (結果) :

```
Node 0 MemTotal: 131072000 kB
Node 0 MemFree: 125000000 kB
Node 0 HugePages_Total: 512
Node 0 HugePages_Free: 512
```

NUMA 確認 CPU-to-node 確認 `lscpu` 確認

```
chroot /host lscpu | grep -i numa
```

結果:

```
NUMA node(s): 2
NUMA node0 CPU(s): 0-9,40-49
NUMA node1 CPU(s): 20-29,60-69
```



Single Node OpenShift NUMA ノード 1 確認 NUMA  
CPU 確認

確認

```
exit
```

### ステップ 3: Namespace 作成

```
oc create namespace numa-hugepages-demo
```

### ステップ 4: Hugepages 設定 VM 作成

VM NUMA 設定 CPU 設定 Hugepages 設定 TLB 設定 VM  
設定

設定:

- `dedicatedCpuPlacement: true` — vCPU 専用 CPU (KubeVirt 専用 Hugepages 専用) □
- `memory.hugepages.pageSize: 2Mi` — 2Mi Hugepages 設定
- 専用 `requests` □ `limits` (Guaranteed QoS) □





cloud-init をインストールして SSH をインストールする

## vm-hugepages.yaml の内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-hugepages
 namespace: numa-hugepages-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-hugepages
 spec:
 nodeSelector:
 cpumanager: "true"
 domain:
 cpu:
 sockets: 2
 cores: 1
 threads: 1
 dedicatedCpuPlacement: true
 memory:
 hugepages:
 pageSize: 2Mi
 resources:
 requests:
 memory: 2Gi
 limits:
 memory: 2Gi
 devices:
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
```

```
image: quay.io/containerdisks/fedora:latest
- name: cloudinitdisk
 cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
```

EOF

VM is Running

```
oc get vm vm-hugepages -n numa-hugepages-demo -w
```

Output:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| vm-hugepages | 45s | Running | True  |

## 5: Hugepages

Output

VM

```
oc get vmi vm-hugepages -n numa-hugepages-demo -o wide
```

Output

```
oc debug node/<node-name>
```

Hugepages HugePages\_Free VM HugePages\_Total

```
chroot /host grep -i hugepages /proc/meminfo
```

2Mi VM 1024 HugePages\_Free 1024

Output

```
exit
```



```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-numa-hugepages
 namespace: numa-hugepages-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-numa-hugepages
 spec:
 nodeSelector:
 cpumanager: "true"
 domain:
 cpu:
 sockets: 1
 cores: 2
 threads: 1
 dedicatedCpuPlacement: true
 isolateEmulatorThread: true
 numa:
 guestMappingPassthrough: {}
 memory:
 hugepages:
 pageSize: 2Mi
 resources:
 requests:
 memory: 1Gi
 limits:
 memory: 1Gi
 devices:
 autoattachMemBalloon: false
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:
 - name: default
 masquerade: {}
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - name: cloudinitdisk

```

```
cloudInitNoCloud:
 userData: |
 #cloud-config
 password: redhat
 chpasswd:
 expire: false
EOF
```

VM is Running successfully

```
oc get vm vm-numa-hugepages -n numa-hugepages-demo -w
```

Output:

| NAME              | AGE | STATUS  | READY |
|-------------------|-----|---------|-------|
| vm-numa-hugepages | 60s | Running | True  |

Section 7: VM is NUMA enabled

VMI is NUMA enabled

Check VMI is NUMA enabled

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o
jsonpath='{.spec.domain.cpu.numa}{"\n"}'
```

Output:

```
{"guestMappingPassthrough":{}}
```

Check NUMA is

VM is NUMA enabled

```
virtctl console vm-numa-hugepages -n numa-hugepages-demo
```

Output **fedora** is **redhat** is NUMA enabled

```
lscpu | grep -i numa
```

Check NUMA is CPU NUMA is NUMA is NUMA 1  
is NUMA

看看 CPU 配置信息

```
lscpu
```

看看 NUMA 配置信息

```
ls /sys/devices/system/node/
```

看看:

```
node0
```

看看 NUMA 配置信息 CPU 配置信息

```
cat /sys/devices/system/node/node0/cpulist
```

看看:

```
0-1
```

看看配置信息

```
cat /sys/devices/system/node/node0/meminfo | head -5
```

看看:

```
Node 0 MemTotal: 1048576 kB
Node 0 MemFree: 800000 kB
Node 0 MemUsed: 248576 kB
Node 0 Active: 120000 kB
Node 0 Inactive: 80000 kB
```

看看 NUMA 配置信息 CPU 配置信息

**Ctrl+] 看看配置信息**

看看 NUMA 配置

看看 CPU Manager 配置 CPU pinning 配置

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o wide
```

```
oc debug node/<node-name>
```

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

**entries** **compute** **isolateEmulatorThread: true** 1 **CPU** (3 **vCPU** 2 + 1)

```
exit
```

2Mi 1Gi Hugepages

|     | 2Mi       | 1Gi           |
|-----|-----------|---------------|
| TLB | 2Mi       | 1Gi (512)     |
|     | (: 100Mi) | ( 1Gi)        |
|     |           | 1Gi VM 1Gi    |
|     |           | Hugepages     |
|     | VM TLB    | VM (64Gi) TLB |

1Gi **Tuned**

```
[bootloader]
cmdline_openshift_node_hugepages=hugepagesz=1G hugepages=16
```

VM

```
memory:
 hugepages:
 pageSize: 1Gi
resources:
 requests:
 memory: 16Gi
 limits:
 memory: 16Gi
```



Memory requests **Hugepages** 1Gi **Hugepages** 1.5Gi







# VM

VM

OpenShift Virtualization VM 5G RAN (vDU/vCU)(tick-to-trade)

CPU VM VM

| CPU       | CPU steal | VM (dedicatedCpuPlacement)          |
|-----------|-----------|-------------------------------------|
|           | vCPU QEMU | VM (isolateEmulatorThread)          |
| NUMA      | NUMA      | VM (guestMappingPassthrough)        |
| Hugepages | TLB       | VM (memory.hugepages)               |
| RT        |           | PerformanceProfile (realTimeKernel) |
| CPU       | IRQ       | PerformanceProfile (cpu.isolated)   |
| RT        | vCPU      | VM (cpu.realtime)                   |
|           |           | VM (autoattachMemBalloon: false)    |

3 CPU NUMA/Hugepages

VM

- OpenShift Virtualization Operator OpenShift 4.18 (4.21)
- (system:admin)
- oc virtctl CLI
- 8 CPU 16Gi RAM Worker Node
- CPU Manager static policy (CPU)
- Hugepages NUMA (NUMA Hugepages)
- Node Tuning Operator (OpenShift)

VM

VM

VM 配置を定義する MachineConfigPool を作成します。

MachineConfigPool は、PerformanceProfile を指定し、RT CPU Hugepages を指定します。



MachineConfigPool VM 配置を定義する MachineConfigPool を作成します (RT CPU Hugepages)。

## ステップ 1: MachineConfigPool を作成

MachineConfigPool PerformanceProfile RT CPU Hugepages を指定します。

Worker Node `worker-rt` を指定します。

```
oc label node <node-name> node-role.kubernetes.io/worker-rt=
```

`worker` を指定し `worker-rt` MachineConfigPool を作成します。

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
 name: worker-rt
 labels:
 machineconfiguration.openshift.io/role: worker-rt
spec:
 machineConfigSelector:
 matchExpressions:
 - key: machineconfiguration.openshift.io/role
 operator: In
 values:
 - worker
 - worker-rt
 nodeSelector:
 matchLabels:
 node-role.kubernetes.io/worker-rt: ""
EOF
```

MachineConfigPool を確認します。

```
oc get mcp worker-rt
```

出力:

| NAME | CONFIG | UPDATED | UPDATING | DEGRADED | MACHINECOUNT | READYMACHINECOUNT |
|------|--------|---------|----------|----------|--------------|-------------------|
|------|--------|---------|----------|----------|--------------|-------------------|

|           |      |       |       |   |   |
|-----------|------|-------|-------|---|---|
| worker-rt | True | False | False | 1 | 1 |
|-----------|------|-------|-------|---|---|

## 2: PerformanceProfile

PerformanceProfile Node Tuning Operator

- (kernel-rt)
- CPU 2-7 (IRQ )
- CPU 0-1
- 4 x 1GiHugepages
- 

performance-profile.yaml

```
oc apply -f - <<EOF
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
 name: realtime-workers
spec:
 cpu:
 isolated: "2-7"
 reserved: "0-1"
 hugepages:
 defaultHugepagesSize: 1G
 pages:
 - size: 1G
 count: 4
 realTimeKernel:
 enabled: true
 additionalKernelArgs:
 - nmi_watchdog=0
 - audit=0
 - mce=off
 - processor.max_cstate=1
 - idle=poll
 - intel_idle.max_cstate=0
 nodeSelector:
 node-role.kubernetes.io/worker-rt: ""
 machineConfigPoolSelector:
 machineconfiguration.openshift.io/role: worker-rt
EOF
```



Single Node OpenShift (SNO) 1 MachineConfigPool  
 nodeSelector node-role.kubernetes.io/master:  
 machineConfigPoolSelector  
 pools.operator.machineconfiguration.openshift.io/master:

machineConfigPoolSelector machineConfigSelector MCP metadata.labels VM nodeSelector 1 worker-rt (oc label node <node-name> node-role.kubernetes.io/worker-rt=)



PerformanceProfile API Hugepages IEC (1Gi, 2Mi) SI (1G, 2M) VM Kubernetes (1Gi, 2Mi)

PerformanceProfile Tuned MachineConfig CPU Manager KubeletConfig Machine Config Operator



PerformanceProfile RT

oc get mcp worker-rt -w

SNO master

oc get mcp master -w

UPDATED True READYMACHINECOUNT MACHINECOUNT RT 10-20 MCP UPDATING: True

| nmi_watchdog=0          | NMI                                |
|-------------------------|------------------------------------|
| audit=0                 |                                    |
| mce=off                 |                                    |
| processor.max_cstate=1  | CPU (C-states > C1)                |
| idle=poll               | CPU (halt)                         |
| intel_idle.max_cstate=0 | Intel C-state processor.max_cstate |



RT CPU

### 3: RT

MachineConfigPool

確認コマンド

```
oc debug node/<node-name>
```

確認コマンド

```
chroot /host uname -r
```

確認コマンド **rt** 確認コマンド

```
5.14.0-427.55.1.el9_4.x86_64+rt
```

確認コマンド **CPU** 確認コマンド

```
chroot /host cat /proc/cmdline | grep -o 'isolcpus=managed_irq,[^]*'
```

確認コマンド:

```
isolcpus=managed_irq,2-7
```

確認コマンド **CPU** **tick-less (nohz)** 確認コマンド

```
chroot /host cat /sys/devices/system/cpu/nohz_full
```

確認コマンド (PerformanceProfile **cpu.isolated** 確認):

```
2-7
```

確認コマンド **IRQ** 確認コマンド

```
chroot /host cat /proc/irq/default_smp_affinity
```

**managed\_irq** 確認コマンド **CPU** 確認コマンド **IRQ** 確認コマンド **Tuned** 確認コマンド **irqbalance** 確認コマンド

Hugepages 確認コマンド

```
chroot /host grep -i hugepages /proc/meminfo
```

**HugePages\_Total** 確認コマンド (1Gi 確認コマンド 4) 確認コマンド

kubvirt.io/realtime

```
exit
```

```
oc get node <node-name> -L kubvirt.io/realtime
```

KubeVirt      `kernel.sched_rt_runtime_us`      -1      (      RT      )

```
oc debug node/<node-name> -- chroot /host sysctl kernel.sched_rt_runtime_us
```

```
kernel.sched_rt_runtime_us = -1
```

## 4: Namespace

```
oc create namespace realtime-demo
```

## 5: VM

VM      Fedora      RT

- `cpu.model: host-passthrough` — CPU
- `cpu.dedicatedCpuPlacement: true` — vCPU CPU
- `cpu.isolateEmulatorThread: true` — QEMU CPU
- `cpu.ioThreadsPolicy: auto` — I/O CPU
- `cpu.numa.guestMappingPassthrough: {}` — NUMA
- `cpu.realtime.mask: "0"` — vCPU 0 SCHED\_FIFO vCPU 1
- `memory.hugepages.pageSize: 1Gi` — 1GiHugepages
- `autoattachMemBalloon: false` —
- `autoattachGraphicsDevice: false` —



cloud-init SSH

vm-realtime.yaml

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-realtime
 namespace: realtime-demo
spec:
 runStrategy: Always
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-realtime
 spec:
 nodeSelector:
 node-role.kubernetes.io/worker-rt: ""
 domain:
 cpu:
 model: host-passthrough
 sockets: 1
 cores: 2
 threads: 1
 dedicatedCpuPlacement: true
 isolateEmulatorThread: true
 ioThreadsPolicy: auto
 numa:
 guestMappingPassthrough: {}
 realtime:
 mask: "0"
 memory:
 hugepages:
 pageSize: 1Gi
 resources:
 requests:
 memory: 2Gi
 cpu: 2
 limits:
 memory: 2Gi
 cpu: 2
 devices:
 autoattachSerialConsole: true
 autoattachMemBalloon: false
 autoattachGraphicsDevice: false
 disks:
 - name: rootdisk
 disk:
 bus: virtio
 - name: cloudinitdisk
 disk:
 bus: virtio
 interfaces:

```



EOF

```
oc get vm vm-realtime -n realtime-demo -w
```

| NAME        | AGE | STATUS  | READY |
|-------------|-----|---------|-------|
| vm-realtime | 90s | Running | True  |

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

**entries** 2 vCPU ID 1 CPU (3) **compute** CPU (PerformanceProfile 2-7)

### Hugepages

```
chroot /host grep HugePages /proc/meminfo
```

**HugePages\_Free** VM (1GiHugepages 2Gi VM 2) **HugePages\_Total**



**guestMappingPassthrough** vCPU NUMA Hugepages  
PerformanceProfile Hugepages NUMA  
VM NUMA  
Hugepages

```
exit
```

## 7: VM

VM

```
virtctl console vm-realtime -n realtime-demo
```

**fedora** **redhat**

```
uname -r
```

(**rt**) vCPU RT

### CPU

```
lscpu | grep -E "^CPU(s)|^Socket|^Core|^Thread"
```



VM をインストール

```
virtctl console vm-realtime -n realtime-demo
```

realtime-tests をインストール (cyclictest を含む)

```
sudo dnf install -y realtime-tests
```

cyclictest を 3 回実行して 60 秒間実行

```
sudo cyclictest --mlockall --smp --priority=80 --interval=200 --distance=0
--duration=60
```

結果:

```
/dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 0.92 0.42 0.18 1/148 1599

T: 0 (1597) P:80 I:200 C: 156868 Min: 3 Act: 5 Avg: 191 Max: 9501
T: 1 (1598) P:80 I:200 C: 277750 Min: 3 Act: 7 Avg: 36 Max: 1406
```

Max 実行時間 (秒) (RT)  
実行時間 (秒) RT  
実行時間 (秒)



RT Worker Node をインストール OpenShift Virtualization  
Red Hat をインストール 54-55 章 [Benchmarking Red Hat OpenShift Virtualization for latency-sensitive workloads](#) を参照



BIOS  
BIOS をインストール  
BIOS

Ctrl+] を押して終了

インストール

VM をインストール

インストール [kubevirt.io/realtime](https://kubevirt.io/realtime) を参照

```
oc get nodes -L kubevirt.io/realtime
```

RT をインストール kernel.sched\_rt\_runtime\_us を -1 PerformanceProfile

MachineConfigPool

**PerformanceProfile** (Degraded)

MachineConfigPool

```
oc get mcp worker-rt
```

```
oc describe mcp worker-rt
```

DEGRADED **True** machine-config-daemon

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-daemon --tail=50
```

RT RPM MachineConfig

**cyclictest**

**cyclictest** 100

- QEMU vCPU **SCHED\_FIFO** (7 )
- CPU IRQ RT

```
oc debug node/<node-name> -- chroot /host cat /proc/interrupts
```

CPU (27 ) IRQ

- BIOS C1 C
- RT

**VM insufficient hugepages (Hugepages )**

PerformanceProfile Hugepages VM Pod Hugepages

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-1Gi}{"\n"}'
```

Hugepages PerformanceProfile **pages.count**

VM Namespace

```
oc delete namespace realtime-demo
```

PerformanceProfile `worker-rt` `worker` CPU `worker-rt`

```
oc delete performanceprofile realtime-workers
```

MachineConfigPool `worker-rt`

```
oc delete mcp worker-rt
```

`worker-rt` `worker-rt`

```
oc label node <node-name> node-role.kubernetes.io/worker-rt-
```



PerformanceProfile



MachineConfigPool

worker-rt

worker

MachineConfigPool

worker-rt

worker-rt

worker-rt

- Worker Node `worker-rt` MachineConfigPool `worker-rt`
- PerformanceProfile `worker-rt` RT `worker-rt` CPU `worker-rt` Hugepages `worker-rt`
- RT `worker-rt`: `worker-rt` CPU `worker-rt` Hugepages `worker-rt` `kubvirt.io/realtime` `worker-rt`
- `worker-rt` VM `worker-rt`
- `worker-rt` CPU pinning `worker-rt` Hugepages `worker-rt` SCHED\_FIFO `worker-rt`
- `cyclictest` `worker-rt`
- `worker-rt`

worker-rt

- CPU `worker-rt` — CPU Manager static policy `worker-rt` vCPU
- NUMA `worker-rt` Hugepages — Hugepages `worker-rt` NUMA `worker-rt`
- VM `worker-rt` QoS — Guaranteed QoS `worker-rt`
- KubeVirt Real-Time Workloads
- Tuning Nodes for Low Latency - OpenShift Documentation
- Provisioning Real-Time Workloads - OpenShift Documentation
- KVM Real-Time Tuning - libvirt

# VM 仮想マシン

## 概要

OpenShift

Virtualization

OpenShift 仮想マシン (VM) を利用して、アプリケーションを仮想化環境で実行します。

## 前提条件

- `qcow2` 形式のイメージファイルが OpenShift Virtualization 環境に存在すること
- 仮想マシン (VM) の作成に必要なリソースが確保されていること
- `namespace` として `namespace` という VM の名前空間が設定されていること
- VM の起動に必要なパラメータが設定されていること
- UEFI/Secure Boot の設定が正しいこと

## 手順

VM の作成と起動の手順を説明します。

- OpenShift 4.18 以降は OpenShift Virtualization Operator をインストールする必要があります。
- `ReadWriteMany` アクセスモードの `StorageClass` が設定されていること。
- CLI ツールとして `oc`, `virtctl`, `kubectl` を使用します。
- `PersistentVolumeClaims` と `DataVolumes` を作成します。

## インストール

### qcow2 イメージの準備

KVM 環境に `qcow2` 形式のイメージファイルが OpenShift Virtualization 環境に存在することを確認します。

### ストレージの設定

仮想マシン (VM) の作成に必要なストレージ設定を確認します。

### 名前空間の設定

仮想マシン (VM) の作成に必要な名前空間設定を確認します。

### VM の起動

仮想マシン (VM) を起動し、`namespace` として `cloud-init` を設定します。

### VM の管理

仮想マシン (VM) の管理と監視を行います。

## VM 設定オプション

VM 設定オプション (PVCContainerDiskHTTP) の設定は UEFI の BIOS の Secure Boot の設定を参照してください。

### 設定オプション

- [設定オプション](#) - VM 設定オプション
- [設定オプション](#) - VM 設定オプション
- [VM 設定オプション](#) - 設定オプションCPU 設定オプションNUMA

### 設定オプション

- [OpenShift Virtualization Documentation](#)
- [KubeVirt User Guide](#)

## qcow2 設定オプション

### 設定

設定オプション `qcow2` の設定は KVM の設定オプション `OpenShift Virtualization` の設定オプション (VM) の設定オプションOVA の設定オプション OS の設定オプションを参照してください。

`qcow2` (QEMU Copy-On-Write version 2) の KVM の設定オプション QEMU の設定オプション `OpenShift Virtualization` の設定オプション



設定オプション `OpenShift Virtualization` の設定オプション `OpenShift 4.18.33` の設定オプションAWS EBS gp3-csi の設定オプション

### 設定

- OpenShift 4.18 の [OpenShift Virtualization Operator](#) の設定オプション
- CLI の設定: `oc, virtctl`
- `qemu-img` (設定オプション) - `qcow2` の設定オプション
- 設定オプション (設定オプション HTTP/S3 の設定オプション) `qcow2` の設定オプション
- [設定オプション](#) - 設定オプション
- namespace の設定オプション

### 設定オプション

`qcow2` の設定オプション `OpenShift Virtualization` の設定オプション 4 の設定オプション



| コマンド                              | 説明                               | 関連リソース                           |
|-----------------------------------|----------------------------------|----------------------------------|
| <b>virtctl image-upload</b>       | イメージをアップロード                      | DataVolume/PVC を作成               |
| <b>DataVolume with HTTP/S3</b>    | Web から取得                         | Web から取得した DataVolume            |
| <b>ContainerDisk</b>              | ContainerDisk から取得               | ContainerDisk から取得した qcow2       |
| <b>VM with DataVolumeTemplate</b> | VM spec に DataVolumeTemplate を指定 | VM spec に DataVolumeTemplate を指定 |

virtctl image-upload コマンドは、イメージをアップロードするためのコマンドです。

## 1: qcow2 イメージ

qcow2 イメージは OpenShift Virtualization で使用できます。

qcow2 イメージの例

Fedora Cloud の qcow2 イメージ

```
Download Fedora Cloud image (example)
curl -LO
https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2

Verify the image
ls -lh Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

qemu-img (インストール)

qemu-img コマンドは、イメージを操作するためのコマンドです。

```
qemu-img info Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

例:

```
image: Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
file format: qcow2
virtual size: 5 GiB (5368709120 bytes)
disk size: 420 MiB
```

Virtual size は DataVolume のサイズを示します。

## ステップ 2: namespace 作成

OpenShift VM の namespace を作成

```
oc create namespace imported-vms
```

## ステップ 3: virtctl を使って qcow2 をアップロード

virtctl を使って qcow2 をアップロードし、PVC を作成

```
virtctl image-upload pvc fedora-imported-disk \
 --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind
```

オプション:

- **pvc fedora-imported-disk** - PVC の名前
- **--image-path** - qcow2 のパス
- **--size** - PVC のサイズ (qcow2 の Virtual size を指定)
- **--namespace** - namespace
- **--insecure** - TLS を無視 (デフォルトは有効)
- **--force-bind** - PVC を作成する (WaitForFirstConsumer を指定)

例: **--force-bind** を指定して **volumeBindingMode: WaitForFirstConsumer** を指定する StorageClass (AWS EBS/Azure Disk をサポート) を作成し、PVC を作成 (pending) を確認

確認:

```
PVC imported-vms/fedora-imported-disk created
Uploading data to https://cdi-uploadproxy-openshift-cnv.apps...

420.00 MiB / 420.00 MiB [=====] 100.00% 1m30s

Uploading data completed successfully, waiting for processing to complete
Processing completed successfully
```



アップロードされたイメージのサイズは 1GB を超えるため、103 秒かかります

確認:

PVC が作成されたことを確認

```
oc get pvc fedora-imported-disk -n imported-vms --watch
```

~~~~~PVC ~ Pending ~ Bound ~~~~~

| NAME                 | STATUS  | VOLUME    | CAPACITY |
|----------------------|---------|-----------|----------|
| ACCESS MODES         |         |           |          |
| fedora-imported-disk | Pending |           |          |
| fedora-imported-disk | Bound   | pvc-xxxxx | 10Gi     |
| RWO                  |         |           |          |

~~~~~ **cdi-upload** Pod ~~~~~

```
oc get pods -n imported-vms
oc logs -n imported-vms cdi-upload-fedora-imported-disk -f
```

~~~~~ **4:** ~~~~~ **VM** ~~~~

~~~~~ PVC ~~~~~ VM ~~~~~



~~~~~ **cloud-init** ~~~~~SSH ~~~~~

**vm-from-imported-qcow2.yaml** ~~~~~

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: fedora-imported-vm
 namespace: imported-vms
 labels:
 app: fedora-imported
spec:
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: fedora-imported-vm
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
```

```

 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: fedora-imported-disk
 - name: cloudinitdisk
 cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true

```

VM 创建成功

```
oc apply -f vm-from-imported-qcow2.yaml
```

VM 启动

```
virtctl start fedora-imported-vm -n imported-vms
```

## 5: VM 部署

VM 部署

```
oc get vmi -n imported-vms --watch
```

VM 部署结果:

| NAME               | AGE | PHASE      | IP | NODENAME  | READY |
|--------------------|-----|------------|----|-----------|-------|
| fedora-imported-vm | 15s | Scheduling |    |           | False |
| fedora-imported-vm | 30s | Scheduled  |    | worker-01 | False |



storage: 10Gi



cdi.kubevirt.io/storage.bind.immediate.requested: "true" PVC  
WaitForFirstConsumer StorageClass

DataVolume CDI

```
oc apply -f datavolume-http-import.yaml
oc get dv -n imported-vms --watch
```

| NAME                 | PHASE            | PROGRESS | RESTARTS | AGE   |
|----------------------|------------------|----------|----------|-------|
| fedora-http-imported | ImportScheduled  | N/A      |          | 5s    |
| fedora-http-imported | ImportInProgress | 12.5%    |          | 15s   |
| fedora-http-imported | ImportInProgress | 67.3%    |          | 45s   |
| fedora-http-imported | Succeeded        | 100.0%   |          | 1m30s |

DataVolume Succeeded VM

```
volumes:
- name: rootdisk
 dataVolume:
 name: fedora-http-imported
```

: CI/CD S3 URL

### 3: ContainerDisk

qcow2

Containerfile

```
FROM scratch
ADD Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 /disk/
```

```
podman build -t quay.io/youruser/fedora-disk:v1 .
podman push quay.io/youruser/fedora-disk:v1
```

VM spec

```
volumes:
```

```
- name: rootdisk
 containerDisk:
 image: quay.io/youruser/fedora-disk:v1
```



ContainerDisk    VM    Persistent    Volume

#### 4: DataVolumeTemplate    VM

DataVolumeTemplate    VM    DataVolume    VM    spec

- VM    - GitOps    - DataVolume    VM

[vm-with-upload-datavolume.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-with-upload
 namespace: imported-vms
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-with-upload-disk
 spec:
 source:
 upload: {}
 storage:
 resources:
 requests:
 storage: 10Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-with-upload
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 interfaces:
```

```

 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-with-upload-disk
 name: rootdisk

```

VM `oc apply -f vm-with-upload-datavolume.yaml`

```
oc apply -f vm-with-upload-datavolume.yaml
```

DataVolume `virtctl image-upload dv vm-with-upload-disk \`

```

virtctl image-upload dv vm-with-upload-disk \
 --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind

```

`virtctl start vm-with-upload -n imported-vms`

```
virtctl start vm-with-upload -n imported-vms
```

**ステップ 1:** - `oc apply -f vm-with-upload-datavolume.yaml` - DataVolume `oc apply -f vm-with-upload-datavolume.yaml` - Infrastructure-as-Code `oc apply -f vm-with-upload-datavolume.yaml`

**ステップ 2:** - PVC `oc apply -f vm-with-upload-datavolume.yaml` - VM `oc apply -f vm-with-upload-datavolume.yaml` - DataVolume/virtctl `oc apply -f vm-with-upload-datavolume.yaml`

`oc apply -f vm-with-upload-datavolume.yaml`

`oc apply -f vm-with-upload-datavolume.yaml`

- `oc apply -f vm-with-upload-datavolume.yaml`
- `oc apply -f vm-with-upload-datavolume.yaml` - `virt-sparsify` `oc apply -f vm-with-upload-datavolume.yaml` `qcow2` `oc apply -f vm-with-upload-datavolume.yaml`
- `oc apply -f vm-with-upload-datavolume.yaml` - `oc apply -f vm-with-upload-datavolume.yaml`
- **cloud-init** `oc apply -f vm-with-upload-datavolume.yaml` - `oc apply -f vm-with-upload-datavolume.yaml` `cloud-init` `oc apply -f vm-with-upload-datavolume.yaml`



□□□□□□□□□□

- □□□□□□ - VM □□□□□□□□Virtual size □□□□□□□□□□□□□□
- □□□□□□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□ - □□□□ PVC □□□□□□□□

□□□□□□

- □□□□□□□□ - □□□□□□□□ qcow2 □□□□□□□□□□□□□□
- **TLS** □□□ - □□□□□□ **--insecure** □□□□□□□□□□□□□□□□□□
- □□□□□□ - RBAC □□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□□□

- □□□□□□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□□ - □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□

□□□□□□□□ **"Waiting for PVC upload pod to be ready"** □□□□□□

□□□□□□□□ Pod □□□□□□□□□□□□PVC □ **"WaitForFirstConsumer"** □□□□□□□□□□□□StorageClass □ binding mode □□□□□□□□

PVC □□□□□□□□□□□□□□

```
oc get pvc -n imported-vms
oc describe pvc fedora-imported-disk -n imported-vms
```

□□□□□□□□ **"waiting for first consumer to be created before binding"** □□□□□□□□□□□□StorageClass □ **volumeBindingMode: WaitForFirstConsumer** □□□□□□□□□□□□

□□□□: virtctl □□□□□□ **--force-bind** □□□□□□□□□□□□

```
virtctl image-upload pvc fedora-imported-disk \
 --image-path=./image.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure \
 --force-bind
```

□□□□□□□□ PVC □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
oc delete pvc fedora-imported-disk -n imported-vms
```

🔴`--force-bind` 🟢🟢🟢🟢🟢🟢

🟢🟢🟢🟢🟢 **"Unable to get PVC Prime name from PVC"** 🟢🟢🟢

🟢🟢🟢🟢🟢**DataVolume** 🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢 **CDI** 🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢 **PVC** 🟢🟢🟢🟢🟢🟢🟢  
(🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢) 🟢

```
Use PVC upload (not DataVolume)
virtctl image-upload pvc <pvc-name> \
 --image-path=./image.qcow2 \
 --size=10Gi \
 --namespace=imported-vms \
 --insecure
```

🟢🟢🟢🟢🟢: \* **CDI Operator** 🟢🟢: **cdi-deployment** 🟢 **cdi-operator** Pod 🟢🟢🟢🟢🟢🟢🟢 \* **Storage Class** 🟢🟢:  
**StorageClass** 🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢

CDI Pod 🟢🟢🟢🟢🟢🟢

```
oc get pods -n openshift-cnv | grep cdi
```

🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢

CDI upload proxy Pod 🟢🟢🟢🟢🟢🟢

```
oc get pods -n openshift-cnv | grep cdi-uploadproxy
oc logs -n openshift-cnv <cdi-uploadproxy-pod>
```

🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢

**PVC** 🟢🟢🟢🟢🟢

🔴`--size` 🟢🟢🟢🟢🟢🟢qcow2 🟢🟢🟢 Virtual size 🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢

```
qemu-img info image.qcow2 | grep "virtual size"
```

🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢**PVC** 🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢

```
oc delete pvc fedora-imported-disk -n imported-vms
```

```
Retry upload with larger --size parameter
virtctl image-upload pvc fedora-imported-disk \
```

```
--image-path=./image.qcow2 \
--size=20Gi \
--namespace=imported-vms \
--insecure
```

VM を起動する

VM を確認する

```
oc describe vm fedora-imported-vm -n imported-vms
oc get events -n imported-vms --field-selector involvedObject.name=fedora-imported-vm
```

VM のコンソールに接続する

```
virtctl console fedora-imported-vm -n imported-vms
```

確認: \* 起動した VM - 起動した VM \* 起動した VM - VM spec の `machine.type` が \* 起動した VM - 起動した VM

確認

確認

```
Delete the VM
oc delete vm fedora-imported-vm -n imported-vms

Delete the PVC
oc delete pvc fedora-imported-disk -n imported-vms

Delete the namespace
oc delete namespace imported-vms
```



PVC を削除する

確認

確認

- OpenShift Virtualization をインストールして `qcow2` をインストール
- `virtctl image-upload pvc` を使って `qcow2` をインストール
- 起動した VM を確認
- 起動した VM
- 起動した VM (HTTPContainerDiskDataVolumeTemplate) を確認
- 起動した VM を確認

- □□□□□□□□□□□□□□□□□□□□□□

qcow2 KVM OpenShift Virtualization

□ □ □ □ □ □

- **虚拟机网络配置** - 配置 VM 网络接口
- **IP 地址** - VM 的 IP 地址
- **SSH 连接** - 通过 VM 连接

11

- Containerized Data Importer (CDI)
- virtctl □□□□□□

□ □ □ □ □ □ □ □ □ □ □ □

11

OpenShift

## Virtualization

[illegible]

VM

VM

□ □ □ □ □ □ □ □ □ □

00000000000000000000000000000000 VM 00000000 - 0000000000000000 - 00000000000000000000 - 00000000 -  
 0000000000 - 0000

000000000000000000000000 - VM 00000000 (000000000000000000000000) - 00000 VM 00000000 - 0000000000 - 000000000000000000000000 - VM 000000000000

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18](#)
- CLI `oc`, `virtctl`, `kubectl`
- [OpenShift Virtualization](#) (includes `lvms-vg1`)
- Linux [OpenShift Virtualization](#)
- `cloud-init` [OpenShift Virtualization](#)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

00000000000000000000 . 00 OS 0000000000 VM 000000 . VM 0000000000 (00000000000000000000) . VM 00000000  
 (000000000000) . VM 00000000000000000000 . 000000000000 DataSource 000000 . 0000000000000000 VM 000000

## 例 1: VM 作成

namespace 作成

```
oc create namespace golden-images
```

Fedora VM 作成



cloud-init

cloud-init

SSH

SSH

**golden-image-base-vm.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: base-fedora-vm
 namespace: golden-images
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: base-fedora-vm-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: base-fedora-vm
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
```



base-fedora-vm 4m Running 10.128.1.151 ocplab01 True



この VM は、305 のイメージから作成されました。

## ステップ 2: VM を起動する

VM を起動します。

```
virtctl console base-fedora-vm -n golden-images
```

この VM は、**fedora** イメージから作成されました。

この VM は、**fedora123** イメージから作成されました。

```
インストール
sudo dnf update -y

nginx Webサーバーをインストール
sudo dnf install -y nginx vim htop curl wget git

nginx サービスを有効にする
sudo systemctl enable nginx

インデックス HTML ファイルを作成
sudo bash -c 'cat > /usr/share/nginx/html/index.html << EOF
<!DOCTYPE html>
<html>
<head>
 <title>Golden Image VM</title>
</head>
<body>
 <h1>VM from Golden Image</h1>
 <p>This VM was deployed from a golden image with pre-installed software.</p>
 <p>Nginx is pre-configured and ready to use.</p>
</body>
</html>
EOF'

ファイアウォールを設定
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

## ステップ 3: VM を確認する

この VM は、**fedora123** イメージから作成されました。

```
インストール
```

```

sudo dnf clean all

清理日志
sudo find /var/log -type f -exec truncate -s 0 {} \;

清理ID
sudo rm -f /etc/machine-id
sudo touch /etc/machine-id

cloud-init清理
sudo cloud-init clean --logs --seed

bash清理
history -c
cat /dev/null > ~/.bash_history

同步并退出
sync
exit

```

**Ctrl+] 退出虚拟机**

## 步骤 4: VM 停止

停止虚拟机 VM

```
virtctl stop base-fedora-vm -n golden-images
```

查看虚拟机状态

```
oc get vm -n golden-images
```

输出

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 15m | Stopped | False |

## 步骤 5: 创建 DataSource

创建 VM 需要 DataSource 来指定 VM 的镜像

DataSource 配置

**golden-image-datasource.yaml** 配置

```
apiVersion: cdi.kubevirt.io/v1beta1
```



```
kind: DataSource
metadata:
 name: fedora-nginx-golden
 namespace: golden-images
spec:
 source:
 pvc:
 name: base-fedora-vm-disk
 namespace: golden-images
```

`fedora-golden-datasource.yaml` を作成

```
oc apply -f fedora-golden-datasource.yaml
```

DataSource を確認

```
oc get datasource -n golden-images
```

結果

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 5s  |

ステップ 6: VM を作成

VM を作成する前に、VM のイメージを準備する必要があります。

VM のイメージを準備

`webserver-vm-from-golden.yaml` を作成

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-vm-1
 namespace: golden-images
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: webserver-vm-1-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
```

```

 namespace: golden-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-vm-1
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: webserver-vm-1-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-vm-1
 fqdn: webserver-vm-1.example.com
 name: cloudinitdisk

```

2 VM 创建

**webserver-vm-2.yaml** 创建

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine

```

```

metadata:
 name: webserver-vm-2
 namespace: golden-images
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: webserver-vm-2-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
 namespace: golden-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-vm-2
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: webserver-vm-2-disk
 name: rootdisk
 - cloudInitNoCloud:

```

```
userData: |-
 #cloud-config
 hostname: webserver-vm-2
 fqdn: webserver-vm-2.example.com
 name: cloudinitdisk
```

#### VM #####

```
oc apply -f webserver-vm-1.yaml
oc apply -f webserver-vm-2.yaml
```

#####

```
oc get dv -n golden-images
```

####

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded       | 100.0%   |          | 15m |
| webserver-vm-1-disk | CloneInProgress | 72.5%    |          | 3m  |
| webserver-vm-2-disk | CloneInProgress | 71.8%    |          | 3m  |

DataVolume #####VM #### 305 #####

#### **7: VM** ####

#### VM #####

```
oc get vmi -n golden-images
```

####

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| webserver-vm-1 | 5m  | Running | 10.128.1.152 | ocplab01 | True  |
| webserver-vm-2 | 5m  | Running | 10.128.1.153 | ocplab01 | True  |

##### VM #####nginx #####

```
virtctl console webserver-vm-1 -n golden-images
```

nginx #####

```
sudo systemctl status nginx
```

nginx.service (nginx) の状態を調査

Web サービスのインストール

```
curl localhost
```

nginx がインストールされた状態で HTML ページを返すように設定

**Ctrl+]** nginx の管理画面へ移動

nginx のインストール

nginx のインストールと起動

1. 仮想マシン (VM) のインストール
2. 仮想マシンの起動
3. VM の設定
4. 仮想マシンの Data Source を設定 (例: **fedora-nginx-golden-v2**)
5. 仮想マシンの VM 名を設定

Data Source を設定 - **fedora-nginx-golden-v1** - **fedora-nginx-golden-v2** - **fedora-nginx-golden-2025-10**

nginx のインストール

nginx のインストール

- nginx のインストール
- nginx のインストールと起動
- nginx のインストールと起動
- nginx のインストールと起動
- nginx のインストールと起動 (changelog) を確認

nginx

- nginx のインストール
- SSH のインストール
- nginx のインストール
- cloud-init のインストール
- bash のインストール



## OpenShift Container Platform

OpenShift

Virtualization

Container

OS

OpenShift Container Platform is a Kubernetes-based container management platform that runs on top of a Linux operating system.

❏

- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)
- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)
- [\[vm-lifecycle:golden-images-troubleshooting:::troubleshooting\]](#)

OpenShift Container Platform

**namespace** ❏

```
oc get namespace golden-images
```

Output:

| NAME          | STATUS | AGE |
|---------------|--------|-----|
| golden-images | Active | 5m  |

OpenShift Container Platform

```
oc get all -n golden-images
```

**VM** ❏ **VMI** ❏

```
oc get vm,vmi -n golden-images
```

**DataVolume** ❏ **PVC** ❏

```
oc get dv,pvc -n golden-images
```

**DataSource** ❏

```
oc get datasource -n golden-images
```

OpenShift Container Platform

□□□□ 1: □□□ VM □□□□□

VM □□□□□□□□

```
oc get vm base-fedora-vm -n golden-images
```

□□□□□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 2m  | Running | True  |

VMI □□□□□

```
oc get vmi base-fedora-vm -n golden-images -o yaml
```

□□□□□□□□□□□□: - **status.phase: Running** - **status.interfaces[].ipAddress** (VM □ IP □□□□□□□□□□□□) - **status.guestOSInfo.id: fedora** (guest agent □□□□□□□□□□)

□:

```
status:
 guestOSInfo:
 id: fedora
 kernelRelease: 6.11.4-301.fc41.x86_64
 kernelVersion: '#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024'
 name: Fedora Linux
 prettyName: Fedora Linux 41 (Cloud Edition)
 version: '41'
 versionId: '41'
 interfaces:
 - infoSource: domain, guest-agent, multus-status
 interfaceName: eth0
 ipAddress: 10.128.0.168
 ipAddresses:
 - 10.128.0.168
 - fd02::76
 mac: 02:92:c2:00:00:06
 name: default
 queueCount: 1
 phase: Running
```

□□□ VM □ **DataVolume** □□

```
oc get dv base-fedora-vm-disk -n golden-images
```



□□□□□□:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded | 100.0%   |          | 5m  |

□□□ **VM** □ **PVC** □□□

```
oc get pvc base-fedora-vm-disk -n golden-images
```

□□□□□□□□ **Bound** □□□□□□□□ Storage Class □□□□□□□□:

| NAME                | STATUS       | VOLUME        | CAPACITY |
|---------------------|--------------|---------------|----------|
| ACCESS MODES        | STORAGECLASS | AGE           |          |
| base-fedora-vm-disk | Bound        | pvc-abc123... | 30Gi     |
| RWO                 | lvms-vg1     | 5m            |          |

□□□□ **2: VM** □□□□□□□□□□

**VM** □□□□□□□□□□

```
virtctl console base-fedora-vm -n golden-images
```

□□□□□□□□□□□□□□□□□□□□

**VM** □□□□:

```
For nginx
systemctl status nginx
nginx -v

For custom configuration
cat /etc/nginx/nginx.conf
ls -la /usr/share/nginx/html/
```

**cloud-init** □□□□□ (□□□□□□□□□□)

```
cat /var/log/cloud-init.log
cat /var/log/cloud-init-output.log
```

□□□□ **3: VM** □□□□□□□□□□

**VM** □□□□□□□□□□

```
oc get vm base-fedora-vm -n golden-images
```

□□□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 10m | Stopped | False |

**VMI** □□□□□□□□□□

```
oc get vmi base-fedora-vm -n golden-images 2>&1
```

□□□□□□:

Error from server (NotFound): virtualmachineinstances.kubevirt.io "base-fedora-vm" not found

□□□□ **4: DataSource** □□□□□

**DataSource** □□□

```
oc get datasource fedora-nginx-golden -n golden-images
```

□□□□□□:

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 1m  |

**DataSource** □□□□□

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

□□□□□□□□□□:

```
spec:
 source:
 pvc:
 name: base-fedora-vm-disk
 namespace: golden-images
```

□□□□ 5: □□□□□□□□□□□□□□□□ VM □□□

□□□□□□ VM □□□

```
oc get vm webserver-vm-1 webserver-vm-2 -n golden-images
```

□□□□□□□:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| webserver-vm-1 | 2m  | Running | True  |
| webserver-vm-2 | 2m  | Running | True  |

□□□□□□ VM □ **DataVolume** □□□

```
oc get dv webserver-vm-1-disk webserver-vm-2-disk -n golden-images
```

□□□□□□□:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| webserver-vm-1-disk | Succeeded | 100.0%   |          | 2m  |
| webserver-vm-2-disk | Succeeded | 100.0%   |          | 2m  |

**VM** □□□□□□□□□□□□

```
oc get dv webserver-vm-1-disk -n golden-images -o jsonpath='{.spec.sourceRef}'
```

□□□□□□□□ **DataSource** □□□□□□□□□□□:

```
{"kind":"DataSource","name":"fedora-nginx-golden","namespace":"golden-images"}
```

**VM** □□□□□

**VM** □ **IP** □□□□□□□□□□□:

```
oc get vmi webserver-vm-1 webserver-vm-2 -n golden-images -o jsonpath='{range .items[*]}{.metadata.name}: {.status.interfaces[0].ipAddress}{"\n"}{end}'
```

□□□:

```
webserver-vm-1: 10.128.0.169
webserver-vm-2: 10.128.0.170
```

Pod nginx 실행:

```
oc run test-curl --image=curlimages/curl:latest --rm -it --restart=Never -- curl
http://10.128.0.169
```

6: 실행

PVC 실행

```
oc get pvc -n golden-images
```

VM 실행 PVC 실행:

| NAME                | STATUS | VOLUME        | CAPACITY | ACCESS MODES |
|---------------------|--------|---------------|----------|--------------|
| base-fedora-vm-disk | Bound  | pvc-abc123... | 30Gi     | RWO          |
| webserver-vm-1-disk | Bound  | pvc-def456... | 30Gi     | RWO          |
| webserver-vm-2-disk | Bound  | pvc-ghi789... | 30Gi     | RWO          |

(LVM 실행)

```
Get the LVM thin pool usage
oc get lvmvolumegroup -n openshift-storage -o yaml | grep -A 10 status
```

실행

1: VM DataVolume ImportInProgress 실행

:

```
oc get dv base-fedora-vm-disk -n golden-images
NAME PHASE PROGRESS AGE
base-fedora-vm-disk ImportInProgress 10% 10m
```

:

```
Check CDI importer pod
oc get pods -n golden-images | grep importer

Check importer logs
```

```
oc logs -n golden-images <importer-pod-name>
```

データベース: - データ DataSource データベース - データベースデータベースデータベース - データベースデータベース - Storage Class データ

- ### 1. DataSource 如何實現數據源:

```
oc get datasource fedora -n openshift-virtualization-os-images
```

- ## 2. CDI Operator 関数定義:

```
oc get pods -n openshift-cnv | grep cdi
```

3. 00000000000000:

```
oc get pvc -n golden-images
oc get lvmvolumegroup -n openshift-storage
```

## 2: VM 数据卷 - DataVolume 数据卷

```
oc get vm base-fedora-vm -n golden-images
```

| NAME           | AGE | STATUS                  | READY |
|----------------|-----|-------------------------|-------|
| base-fedora-vm | 5m  | WaitingForVolumeBinding | False |

```
Check DataVolume status
oc get dv base-fedora-vm-disk -n golden-images

Check events
oc get events -n golden-images --sort-by='.lastTimestamp' | grep -i error
```

Pod: - **DataVolume** - **PendingPopulation** - **Storage Class** - **PVC**

1. **WaitForFirstConsumer** `dataVolume` VM `dataVolume`

- ## 2. Storage Class `volatile`:

```
oc get storageclass
```

### 3. `default StorageClass` `oc`:

```
oc get storageclass -o
jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-
class=="true")].metadata.name}'
```

### `3: guest agent` `oc`:

`oc`:

```
oc get vmi base-fedora-vm -n golden-images -o jsonpath='{.status.guestOSInfo}'
Returns empty
```

`oc`:

```
Access VM console
virtctl console base-fedora-vm -n golden-images

Check if qemu-guest-agent is installed
rpm -qa | grep qemu-guest-agent

Check service status
systemctl status qemu-guest-agent
```

`oc`:

#### 1. `guest agent` `oc`:

```
sudo dnf install -y qemu-guest-agent
sudo systemctl enable --now qemu-guest-agent
```

#### 2. `guest agent` `oc`

### `4: VM` `oc`

`oc`: `VM` `nginx` `oc`

`oc`:

```
Access the VM
virtctl console webserver-vm-1 -n golden-images

Check for nginx
which nginx
systemctl status nginx
```



```
Check PVC requests vs available
oc get pvc -n golden-images
```

□□□:

1. □□□ PVC □□□□□□□□□□:

```
oc delete pvc <unused-pvc> -n golden-images
```

2. LVM □□□□□□□□□□□□ (□□□□□□□□□□□□)
3. VM/DataVolume spec □□□□□□□□□□□□□□□□

□□ 7: **VM** □□□□□□

□□: □□□□□□ VM □ DataVolume □ **Succeeded** □□□□□□□□□□□□□□

□□:

```
Check DataVolume progress
oc get dv webserver-vm-1-disk -n golden-images -w

Check CDI pod logs
oc logs -n golden-images -l app=containerized-data-importer --tail=100
```

□□□□□□: - □□□□□□□□□□ - □□□□□□□□□□□□□□□□□ - □□□□□□□□□□□□□□□□ (□: DataSource □□□□□□□□)

□□□:

1. □□□□□□□□□□ DataSource □□□□□□□□□□□□□□:

```
spec:
 sourceRef:
 kind: DataSource
 name: fedora-nginx-golden
```

2. □□□□□□□□□□□□ Storage Class □□□□□□ (□: LVM □□□□□□□□□□□□□□□□)
3. VM □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□

□□□□□□□□□□

```
Everything in namespace
oc get all,dv,pvc,datasource -n golden-images

With more details
```



```
oc get all,dv,pvc,datasource -n golden-images -o wide

Watch for changes
oc get vm,vmi,dv -n golden-images -w
```

```
oc get vm,vmi,dv -n golden-images -w
```

```
Recent events
oc get events -n golden-images --sort-by='.lastTimestamp' | tail -20

VM events
oc describe vm base-fedora-vm -n golden-images | tail -20

CDI operator logs
oc logs -n openshift-cnv -l app=cdi-operator --tail=50
```

```
oc get events -n golden-images --sort-by='.lastTimestamp' | tail -20
```

```
oc describe vm base-fedora-vm -n golden-images | tail -20
```

```
oc logs -n openshift-cnv -l app=cdi-operator --tail=50
```

```
PVC details
oc describe pvc <pvc-name> -n golden-images

Storage class configuration
oc get storageclass -o yaml

LVM status (if using LVM)
oc get lvmvolumegroup,lvmvolumegroupnodestatus -n openshift-storage
```

```
oc describe pvc <pvc-name> -n golden-images
```

```
oc get storageclass -o yaml
```

```
oc get lvmvolumegroup,lvmvolumegroupnodestatus -n openshift-storage
```

- OpenShift Virtualization
- KubeVirt - DataVolumes
- Containerized Data Importer (CDI)
- LVM

- VM** □□□□□□

|           |                |  |    |
|-----------|----------------|--|----|
| OpenShift | Virtualization |  | VM |
|           | VM             |  | VM |
|           |                |  |    |

VM

VM

VM 000000

VM OpenShift Virtualization VM OpenShift Virtualization OpenShift Virtualization 2

- **Web** 인터페이스를 통해: VM 프로비저닝을 위한 템플릿 & 이미지 관리
- **CLI** 인터페이스를 통해: PVC 프로비저닝을 위한 VM 프로비저닝

VM 프로비저닝:

- 프로비저닝을 위한 템플릿
- 프로비저닝을 위한 이미지
- 프로비저닝을 위한 스토리지
- 프로비저닝을 위한 네트워크
- 프로비저닝을 위한 보안

프로비저닝을 위한 템플릿

프로비저닝을 위한 VM 프로비저닝을 위한 템플릿

| 구분    | VM 프로비저닝                      | 프로비저닝 방법           |
|-------|-------------------------------|--------------------|
| 프로비저닝 | 프로비저닝을 위한 VM 프로비저닝            | 프로비저닝을 위한 VM 프로비저닝 |
| 프로비저닝 | 프로비저닝을 위한 cloud-init<br>프로비저닝 | 프로비저닝을 위한 VM 프로비저닝 |
| 프로비저닝 | 프로비저닝을 위한 VM 프로비저닝            | 프로비저닝을 위한 VM 프로비저닝 |
| 프로비저닝 | 프로비저닝을 위한 VM 프로비저닝            | 프로비저닝을 위한 VM 프로비저닝 |
| 프로비저닝 | 프로비저닝을 위한 VM 프로비저닝            | 프로비저닝을 위한 VM 프로비저닝 |

프로비저닝

- OpenShift 4.18 프로비저닝 OpenShift Virtualization Operator 프로비저닝
- OpenShift Web 인터페이스를 통해 CLI (oc, virtctl) 프로비저닝
- 프로비저닝을 위한 Storage Class
- 프로비저닝을 위한 VM (프로비저닝 VM)
- PersistentVolumeClaim (PVC) 프로비저닝

프로비저닝을 위한 VM

OpenShift Virtualization 프로비저닝을 위한 VM 프로비저닝을 위한 VM

**Web** 인터페이스를 통해 (oc)

Web 인터페이스를 통해 VM 프로비저닝을 위한 VM

- 프로비저닝을 위한 PVC 프로비저닝
- 프로비저닝을 위한 VM 프로비저닝
- 프로비저닝을 위한 VM 프로비저닝

- VM 映像 (CPU/メモリ/ディスク) 共有

以下の CLI コマンドを実行

以下のコマンドを実行:

- 既存 VM の PVC を作成
- 作成した PVC を既存 VM へマウント
- 既存 VM のディスクを複製
- namespace を作成

以下のコマンドを実行

以下のコマンドを実行:

- **Smart cloning:** 既存 VM のディスクを複製 (既存 CSI の Storage Class)
- **Host-assisted cloning:** Pod を作成して複製 (既存 CSI の Storage Class)

## Web 環境を VM として実行

Web 環境を VM として実行

以下のコマンドを実行

以下の namespace を作成

```
oc create namespace vm-cloning-demo
```

nginx を VM として実行



cloud-init を利用して VM 内に SSH をインストール

source-webserver-vm.yaml を作成

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: source-webserver
 namespace: vm-cloning-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: source-webserver-disk
 spec:
 sourceRef:
```

```

 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: source-webserver
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: source-webserver-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - dnf install -y nginx
 - systemctl enable --now nginx
 - echo "<h1>Source Webserver VM</h1><p>Hostname: $(hostname)</p>" >
 /usr/share/nginx/html/index.html

```

```
- firewall-cmd --permanent --add-service=http
- firewall-cmd --reload
name: cloudinitdisk
```

## VM 虚拟机

```
oc apply -f source-webserver-vm.yaml
```

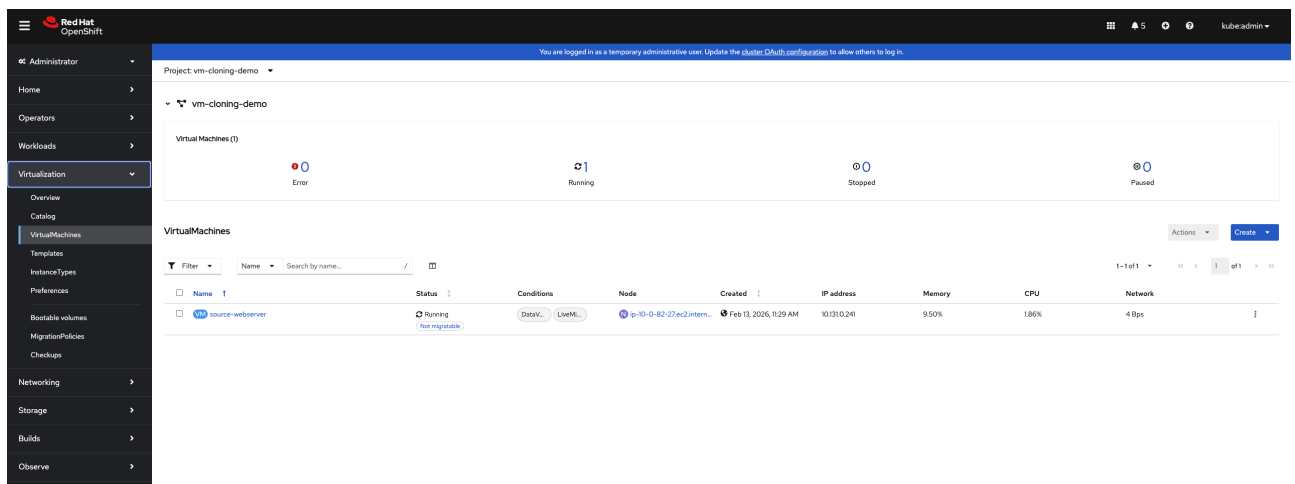
VM 0000000000000000

```
oc get vmi -n vm-cloning-demo
```

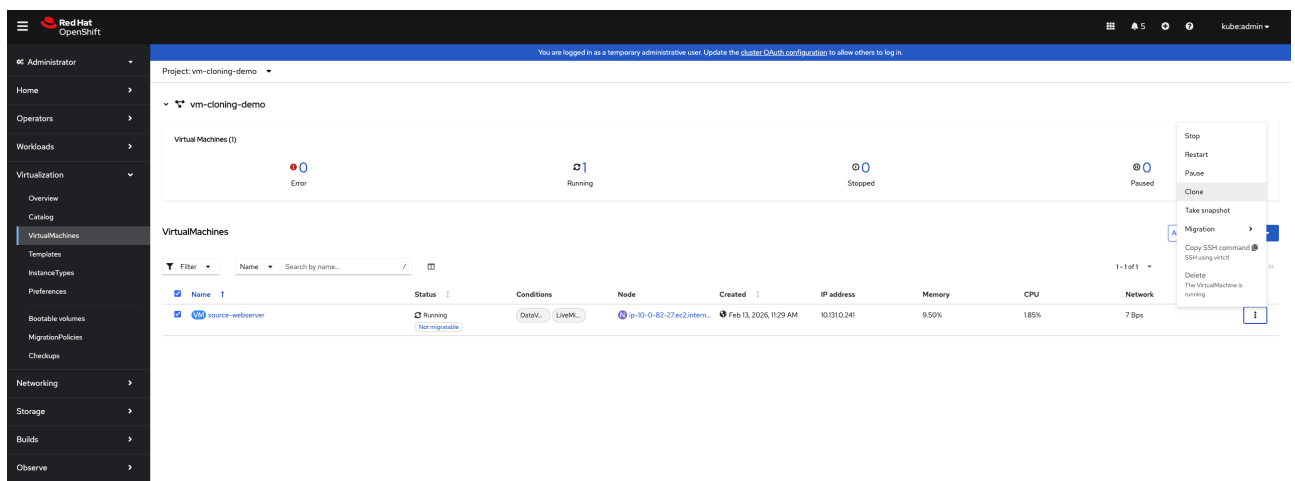
**Web** 0000000000000000

OpenShift Web コンソールから VM をインストール:

1.  $\mathbb{R}^n \rightarrow \mathbb{R}^n$  変換



1. 了解 VM (source-webserver) 的用途
2. VM 的 IP 地址 (source-webserver) 的用途
3. VM 的 IP 地址 (source-webserver) 的用途
4. 了解 VM 的用途



## 5. VM 클론 생성:

- VM 클론 생성 (이름: `webserver-clone-1`)
- 클론 생성 시 VM의 구성 (CPU, Memory, NIC, Disks)을 복사
- 클론 (클론 생성 시 CPU, Memory, NIC, Disks)을 생성

## 6. 클론 생성

Clone VirtualMachine

Name \*

webserver-clone-1

Start VirtualMachine once created

☒

Configuration

Operating system

Fedora Linux 43 (Cloud Edition)

Flavor

2 CPU | 4 GiB Memory

Workload profile

Not available

NICs

default

Disks

rootdisk  
cloudinitdisk

Request sent

In process

Created

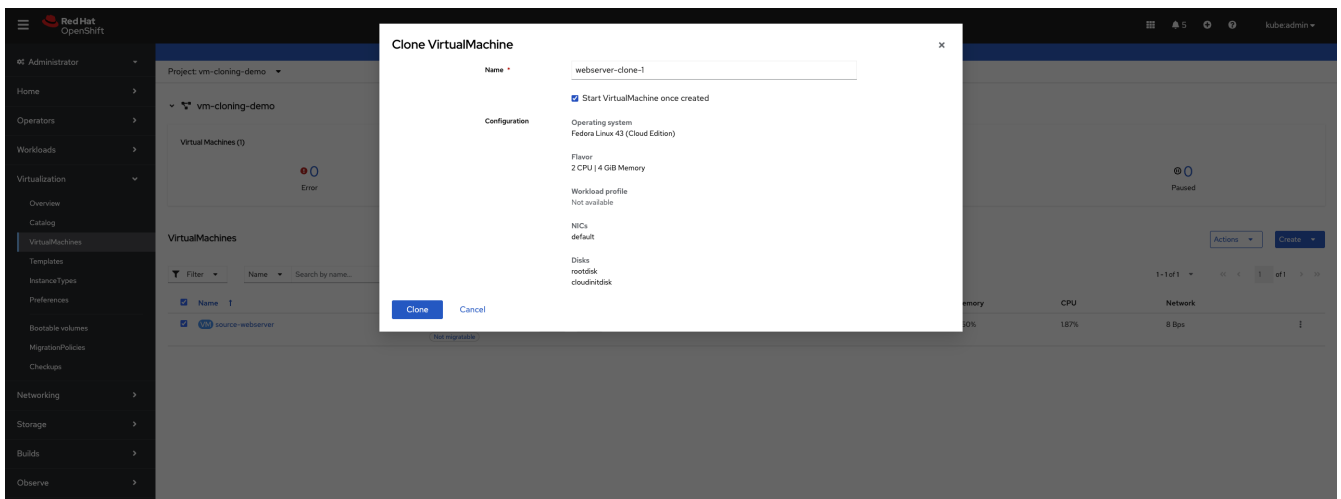
Clone

Cancel

## Web 클론 생성

- VM 클론 생성 PVC 생성
- 클론 생성 시 VM의 구성 (CPU, Memory, NIC, Disks)을 복사
- VM 클론 (CPU, Memory, NIC, Disks)을 생성
- 클론 생성 시 VM 클론 생성

클론 생성 시 VM 클론 생성 PVC 생성 Provisioning 생성



## Web ブラウザからアクセス

作成した VM を確認する

```
oc get vm -n vm-cloning-demo
oc get vmi -n vm-cloning-demo
```

作成した VM を確認する

```
virtctl console webserver-clone-1 -n vm-cloning-demo
```

作成した VM が nginx を起動していることを確認する (確認する) 確認する

## CLI から VM を作成する

namespace を作成する CLI から作成する

1. PVC を作成する VM を作成 (1)

CLI から作成する VM の PVC を作成する

```
oc get pvc -n vm-cloning-demo
```

例:

| NAME                  | STATUS       | VOLUME                                   | CAPACITY |
|-----------------------|--------------|------------------------------------------|----------|
| ACCESS MODES          | STORAGECLASS | AGE                                      |          |
| source-webserver-disk | Bound        | pvc-1234abcd-5678-efgh-9012-ijklmnop3456 | 30Gi     |
| RWO                   | lvms-vg1     | 10m                                      |          |

作成した PVC を確認する DataVolume を作成する

clone-datavolume.yaml を作成する

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-clone-2-disk
 namespace: vm-cloning-demo
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true" ①
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
```

```
storage:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 30Gi
 storageClassName: gp3-csi ②
```

① `WaitForFirstConsumer` Storage Class

② Storage Class (PVC)



`cdi.kubevirt.io/storage.bind.immediate.requested:` `"true"`  
`WaitForFirstConsumer` Storage Class (AWS EBS gp3-csi Azure Disk) VM `DataVolume` `PendingPopulation`

`DataVolume`

```
oc apply -f clone-datavolume.yaml
```

```
oc get dv webserver-clone-2-disk -n vm-cloning-demo -w
```

| NAME                   | PHASE                             | PROGRESS | RESTARTS | AGE |
|------------------------|-----------------------------------|----------|----------|-----|
| webserver-clone-2-disk | CloneFromSnapshotSourceInProgress | N/A      |          | 4s  |
| webserver-clone-2-disk | PrepClaimInProgress               | N/A      |          | 68s |
| webserver-clone-2-disk | RebindInProgress                  | N/A      |          | 78s |
| webserver-clone-2-disk | Succeeded                         | 100.0%   |          | 79s |

`DataVolume` `Succeeded` VM

`vm-from-cloned-pvc.yaml`

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-clone-2
 namespace: vm-cloning-demo
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
```



```

 kubevirt.io/domain: webserver-clone-2
spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: webserver-clone-2-disk

```

VM を適用

```
oc apply -f vm-from-cloned-pvc.yaml
```

VM を起動

```
virtctl start webserver-clone-2 -n vm-cloning-demo
```

2つのVM を確認 (確認)



確認が完了したら、1つのVM を確認

確認が完了したら、VM を確認

```
virtctl stop source-webserver -n vm-cloning-demo
```

VM を確認

```
oc get vm source-webserver -n vm-cloning-demo
```

创建 VM 模板

```
oc get vm source-webserver -n vm-cloning-demo -o yaml > clone-base.yaml
```

使用 `clone-base.yaml` 创建模板

### 1. VM 模板:

```
metadata:
 name: source-webserver # 名称: webserver-manual-clone
 namespace: vm-cloning-demo
```

### 2. DataVolume 模板PVC 模板:

```
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: source-webserver-disk # 名称: webserver-manual-clone-disk
 spec:
 # 源 Ref 模板
 # 模板:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 resources:
 requests:
 storage: 30Gi
```

### 3. 模板模板:

```
spec:
 template:
 metadata:
 labels:
 kubevirt.io/domain: source-webserver # 名称: webserver-manual-clone
 spec:
 domain:
 # ... 模板
 volumes:
 - dataVolume:
 name: source-webserver-disk # 名称: webserver-manual-clone-disk
 name: rootdisk
```

#### 4. 環境変数:

環境変数 \* `metadata.uid` \* `metadata.resourceVersion` \* `metadata.creationTimestamp` \* `metadata.generation` \* `status` (環境変数)

#### 5. MAC 環境変数 (環境):

環境変数 `macAddress` 環境変数

```
spec:
 template:
 spec:
 domain:
 devices:
 interfaces:
 - masquerade: {}
 name: default
 macAddress: "52:54:00:xx:xx:xx" # 環境変数
```

MAC 環境変数 Failed to allocate mac to the vm object: failed to allocate requested mac address

環境変数 YAML 環境変数

```
oc apply -f clone-base.yaml
```

### cloud-init 環境変数 VM 環境変数

環境変数 VM 環境変数 cloud-init 環境変数

環境変数

DataVolume 環境変数 (2 環境)

```
cat <<EOF | oc apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-custom-disk
 namespace: vm-cloning-demo
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
```

```

 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 30Gi
EOF

```

□□□□□□□□□□□□

```
oc get dv webserver-custom-disk -n vm-cloning-demo -w
```

cloud-init □□□□□□□□ VM □□□□□□□□

**cloned-vm-with-cloudinit.yaml** □□□□□□□□

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-custom
 namespace: vm-cloning-demo
spec:
 runStrategy: Manual
 template:
 metadata:
 labels:
 kubevirt.io/domain: webserver-custom
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}

```

```

volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: webserver-custom-disk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-custom
 fqdn: webserver-custom.local
 manage_etc_hosts: true
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - hostnamectl set-hostname webserver-custom
 - echo "<h1>Customized Clone</h1><p>Hostname: $(hostname)</p>" >
 /usr/share/nginx/html/index.html
 - systemctl restart nginx
 name: cloudinitdisk

```

❏ cloud-init ❏❏❏❏:

- ❏❏❏❏❏❏ (**webserver-custom**) ❏❏❏
- fedora ❏❏❏❏❏❏❏❏❏❏❏❏
- nginx ❏❏❏❏❏❏❏❏❏❏
- ❏❏❏ VM ❏❏ nginx ❏❏❏❏❏❏❏❏❏❏❏❏

❏❏❏❏❏❏❏❏❏

```

oc apply -f cloned-vm-with-cloudinit.yaml
virtctl start webserver-custom -n vm-cloning-demo

```

❏❏❏❏❏❏❏❏❏❏❏❏

```

virtctl console webserver-custom -n vm-cloning-demo

```

VM ❏:

```

hostname
❏❏: webserver-custom

systemctl status nginx
nginx❏❏❏❏❏❏❏❏❏❏❏❏ (❏❏❏❏❏❏❏❏)

curl localhost

```

```
HTML
```

## namespace

namespace (namespace) namespace VM

namespace

```
oc create namespace vm-cloning-production
```

## namespace RBAC

namespace namespace PVC

[cross-namespace-clone-rbac.yaml](#)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: datavolume-cloner
 namespace: vm-cloning-demo
rules:
- apiGroups: [""]
 resources: ["persistentvolumeclaims"]
 verbs: ["get", "list"]
- apiGroups: ["cdi.kubevirt.io"]
 resources: ["datavolumes"]
 verbs: ["get", "list"]

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: allow-clone-from-demo
 namespace: vm-cloning-demo
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: Role
 name: datavolume-cloner
subjects:
- kind: ServiceAccount
 name: default
 namespace: vm-cloning-production
```

RBAC

```
oc apply -f cross-namespace-clone-rbac.yaml
```

## namespace 名前空間

demo namespace 名前空間 DataVolume 1 production namespace 名前空間

### cross-namespace-clone-datavolume.yaml 名前空間

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: webserver-production-disk
 namespace: vm-cloning-production
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 pvc:
 name: source-webserver-disk
 namespace: vm-cloning-demo
 storage:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 30Gi
```

DataVolume 名前空間

```
oc apply -f cross-namespace-clone-datavolume.yaml
```

名前空間

```
oc get dv -n vm-cloning-production -w
```

1 VM 名前空間

### cloned-webserver-production.yaml 名前空間

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: webserver-production
 namespace: vm-cloning-production
 labels:
 environment: production
spec:
 runStrategy: Manual
 template:
```

```

metadata:
 labels:
 kubevirt.io/domain: webserver-production
spec:
 domain:
 cpu:
 cores: 4
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 8Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - name: rootdisk
 persistentVolumeClaim:
 claimName: webserver-production-disk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 hostname: webserver-prod
 fqdn: webserver-prod.production.local
 manage_etc_hosts: true
 password: fedora123
 chpasswd: { expire: false }
 ssh_pwauth: true
 runcmd:
 - hostnamectl set-hostname webserver-prod
 - echo "<h1>Production Webserver</h1><p>Environment: Production</p>" >
 /usr/share/nginx/html/index.html
 - systemctl restart nginx
 name: cloudinitdisk

```

□□□□□□□□:

- □□□ namespace
- □□□□□□□□ (4 □□□8Gi RAM)



- `oc apply -f cloned-webserver-production.yaml`
- `virtctl start webserver-production -n vm-cloning-production`
- `oc apply -f cloned-webserver-production.yaml`

oc apply -f cloned-webserver-production.yaml

```
oc apply -f cloned-webserver-production.yaml
virtctl start webserver-production -n vm-cloning-production
```

oc apply -f cloned-webserver-production.yaml

virtctl start webserver-production -n vm-cloning-production

oc apply -f cloned-webserver-production.yaml

- **Smart cloning** (CSI `SnapshotForSmartCloneInProgress`): `SmartClonePVCInProgress`
  - Ceph RBD, AWS EBS, Azure Disk, GCP PD
  - `SnapshotForSmartCloneInProgress` → `SmartClonePVCInProgress`
- **Host-assisted cloning**: `HostAssistedCloneInProgress`
  - `HostAssistedCloneInProgress` Storage Class
  - `HostAssistedCloneInProgress` `CloneInProgress`

oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType

```
oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType
```

oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType

```
cdi.kubevirt.io/cloneType: snapshot # Smart cloning
Host-assisted cloning
cdi.kubevirt.io/cloneType: copy # Host-assisted cloning
```

oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType

oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType

- `oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType`
- `oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType`
- `oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType`
- `namespace <namespace> Storage Class <storage-class>`

oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType

```
oc get sc
oc describe sc <storage-class-name>
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- Storage Class
- namespace
- namespace
- namespace
- namespace
- namespace

□ □ □ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □

DataVolume 00000000000000

```
oc get dv -n vm-cloning-demo
```

Success

| NAME                  | PHASE     | PROGRESS | AGE |
|-----------------------|-----------|----------|-----|
| webserver-custom-disk | Succeeded | N/A      | 5m  |

□ □ □ □ □ □ □ □ □ □ □ □

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

```
oc describe dv <datavolume-name> -n <namespace>
```

```
Events:
 Type Reason Age From Message
 ---- -
 Normal CloneInProgress 5m datavolume-controller Cloning from vm-cloning-
demo/source-webserver-disk in progress
 Normal CloneSucceeded 2m datavolume-controller Successfully cloned from vm-
cloning-demo/source-webserver-disk
```

```
Events:
 Type Reason Age From Message
 ---- -
 Normal CloneInProgress 5m datavolume-controller Cloning from vm-cloning-
demo/source-webserver-disk in progress
 Normal CloneSucceeded 2m datavolume-controller Successfully cloned from vm-
cloning-demo/source-webserver-disk
```

## VM 操作

```
oc get vmi -n vm-cloning-demo
```

操作

VM 操作

```
virtctl console <vm-name> -n <namespace>
```

VM 内:

```
操作
hostname

VM 内操作
systemctl status nginx

cloud-init 操作
cloud-init status

cloud-init 操作
sudo cat /var/log/cloud-init-output.log

VM ID (UUID)
cat /etc/machine-id
```

操作

操作 **Provisioning** 操作 **CloneInProgress** 操作

操作: DataVolume 操作 **CloneInProgress** 操作

操作: \* 操作 \* 操作 \* 操作 Host-assisted cloning

操作:

```
操作
oc describe dv <datavolume-name> -n <namespace>

CDI Pod
oc get pods -n <namespace> | grep cdi

CDI 操作
oc logs <cdi-pod> -n <namespace>
```

Host-assisted cloning

**permission denied**

namespace RBAC

RBAC

```
#
oc get role datavolume-cloner -n <source-namespace>

#
oc get rolebinding -n <source-namespace>

#
oc auth can-i get pvc --as=system:serviceaccount:<target-namespace>:default -n
<source-namespace>
```

yes

VM

VM cloud-init

- cloudinitdisk
- cloud-init userData hostname
- runCmd: hostnamectl set-hostname <new-name>
- cloud-init: sudo cat /var/log/cloud-init.log

```
#
oc get resourcequota -n <namespace>
oc describe resourcequota -n <namespace>

#
oc edit resourcequota <quota-name> -n <namespace>
```

PVC

DataVolume PVC

□□□:

```
PVC
oc get pvc -n <source-namespace>

DataVolume
oc get dv <datavolume-name> -n <namespace> -o yaml
```

```
spec.source.pvc.name spec.source.pvc.namespace
```

□□□□□□□□□□ **VM** □□□□□□

□□: DataVolume □□□□□□VM □□Scheduling□□□□□□Starting□□□□□□□□

□□□□□□: \* PVC □□□□□□□□□□ \* □□□□□□ \* □□□□□□□□□□□□

000:

```
PVC操作
oc get pvc -n <namespace>

VM操作
oc describe vm <vm-name> -n <namespace>

VMI操作 (操作)
oc describe vmi <vm-name> -n <namespace>
```

□ □ □ □ □ □ □

[illegible]

```
demo namespaceVM
oc delete vm --all -n vm-cloning-demo

demo namespaceDataVolume
oc delete dv --all -n vm-cloning-demo

VM
oc delete vm webserver-production -n vm-cloning-production
oc delete dv --all -n vm-cloning-production

RBAC
oc delete role datavolume-cloner -n vm-cloning-demo
oc delete rolebinding allow-clone-from-demo -n vm-cloning-demo

namespace
oc delete namespace vm-cloning-demo
```

```
oc delete namespace vm-cloning-production
```

~~~~~

```
oc get namespace | grep vm-cloning
```

~~~

~~~~~:

- OpenShift Virtualization ~~~~ VM ~~~~~
- Web ~~~~~ CLI ~~~~~
- OpenShift Web ~~~~~ VM ~~~~~
- ~~~~~ DataVolume ~~~~~ CLI ~~~~~
- cloud-init ~~~~~
- RBAC ~~~~~ namespace ~~~~~ VM ~~~~~
- ~~~~~
- ~~~~~Smart cloning ~ Host-assisted cloning ~~~
- ~~~~~
- ~~~~~ ID ~~~~~
- ~~~~~
- VM ~~~~~

VM ~~~~~OpenShift Virtualization ~~~~~

~~~

- [PVC ~~~~~ VM ~~~](#)

## VM ~~~~~

~~~

VM ~~~~~ (~~~~~)  
~~~~~  
~~~~~

~~~~~OpenShift Virtualization ~ VM ~~~~~

~~~~~

- VM ~~~~~

- Web コンソール CLI を使用してインストールを実行します
- インストールする VM のタイプ
- インストールするストレージプロバイダー

インストール:

OpenShift 4.20

インストール

- OpenShift Virtualization をインストールして OpenShift 4.18 以降
- インストールするストレージプロバイダー VM
- CSI をインストールしてストレージプロバイダー
- インストールする `oc` ではなく `virtctl` CLI を使用

インストールを実行する前に

インストールを実行する前に CSI をインストールしてストレージプロバイダー

```
oc get volumesnapshotclass
```

出力:

NAME	DRIVER	DELETIONPOLICY	AGE
csi-hostpath-snapclass	hostpath.csi.k8s.io	Delete	30d
ocs-storagecluster-rbdplugin-snapclass	openshift-storage.rbd.csi.ceph.com	Delete	30d

VolumeSnapshotClass をインストールしてストレージプロバイダー

**VM** インストール

インストール

OpenShift Virtualization をインストールして 2 インストール

**VirtualMachineSnapshot**

VM をインストールしてストレージプロバイダー VM をインストール

**VirtualMachineSnapshotContent**

インストールして VirtualMachineSnapshot インストールして VolumeSnapshot  
インストール

1. 检查虚拟机快照

状态	原因	解决方法
虚拟机快照失败	VM 快照失败	VM 快照失败
虚拟机快照失败	VM 快照失败 (quiescing) 失败	VM 快照 QEMU guest agent 失败



虚拟机快照失败时，QEMU guest agent 可能会失败。

虚拟机快照失败时，5 分钟内会再次尝试快照。如果失败 (thaw) 虚拟机快照失败，VirtualMachineSnapshot 的 spec 中的 failureDeadline 属性会设置失败时间 (例如: failureDeadline: 10m)。

虚拟机快照失败 (Indication)

虚拟机快照失败时，status.indications 属性会返回失败原因。

状态	原因
Online	虚拟机快照成功
GuestAgent	QEMU guest agent 失败
NoGuestAgent	QEMU guest agent 失败
QuiesceFailed	guest agent 失败

虚拟机快照失败 Indication 属性

```

oc get virtualmachinesnapshot <snapshot-name> -n vm-snapshots -o json | jq
'.status.indications'
```

输出:

```

[
 "GuestAgent",
 "Online"
]
```

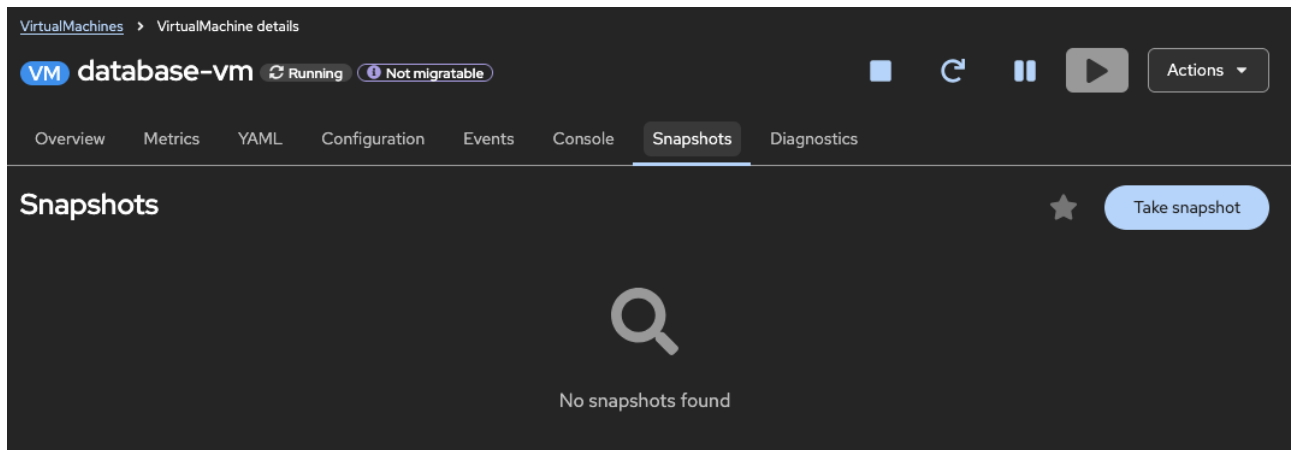
虚拟机快照失败

Web 虚拟机快照失败

1. Virtualization → VirtualMachines 虚拟机快照
2. 虚拟机快照失败 VM 虚拟机快照



### 3. Snapshots



### 4. Take snapshot

Take snapshot

×

ⓘ

Taking snapshot of running VirtualMachine.

Name \*

database-vm-snapshot-20260304-155755

Description

Deadline

600

Seconds (s) ▼

▼ Disks included in this snapshot (1)

rootdisk

⚠

The following disk will not be included in the snapshot

cloudinitdisk - Snapshot is not supported for this volumeSource type [cloudinitdisk]

Edit the disk or contact your cluster admin for further details.

[Learn more about snapshots](#)

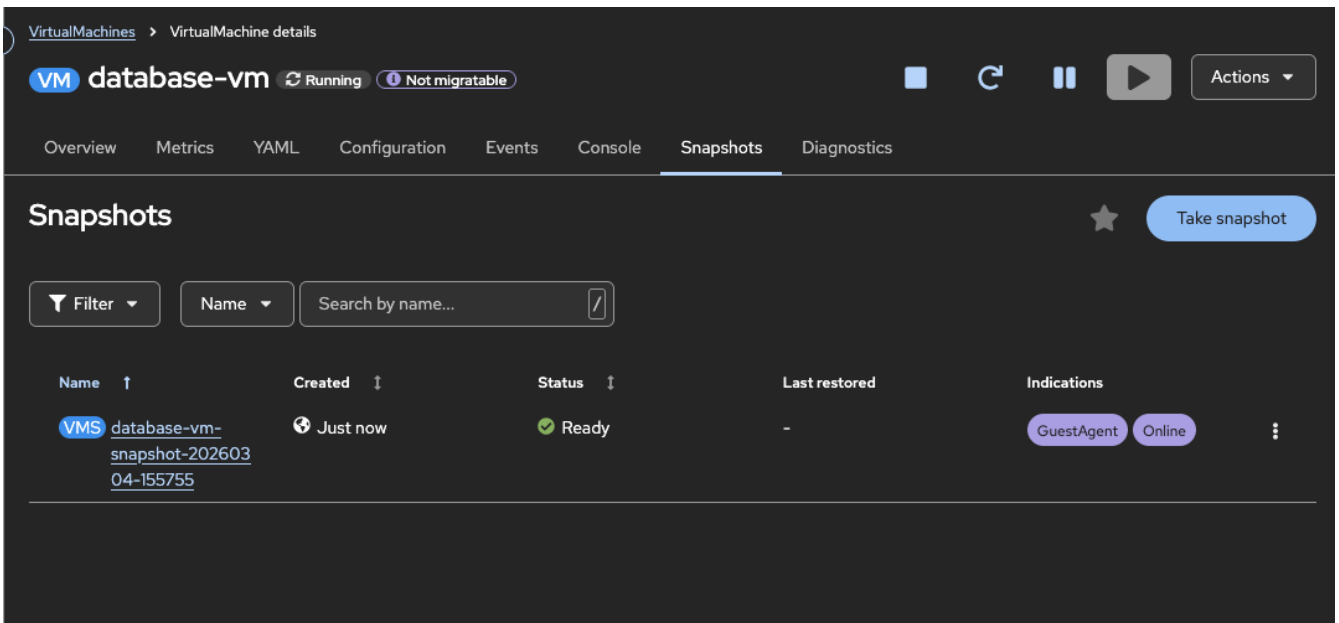
Save

Cancel

5. 名前を入力 (任意) を入力してください

6. **Save** をクリックしてください

作成されたSnapshots は、左側のナビゲーションメニューから確認できます



CLI

VirtualMachineSnapshot

vm-snapshot.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
 name: database-vm-snapshot-01
 namespace: vm-snapshots
spec:
 source:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: database-vm
```

```
oc apply -f vm-snapshot.yaml
```

```
oc get virtualmachinesnapshot -n vm-snapshots
```

NAME	SOURCEKIND	SOURCENAME	PHASE
------	------------	------------	-------





NAME ERROR	READYTOUSE	CREATIONTIME
vmsnapshot-content-cd166db8-00be-4aa9-972e-dea669527bfd	true	8m

```
oc get volumesnapshot -n vm-snapshots
```

결과:

NAME				READYTOUSE
SOURCEPVC	RESTORESIZE	SNAPSHOTCLASS	CREATIONTIME	AGE
vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk				true
database-vm-disk	30Gi	lvms-vg1	8m	8m

데이터 백업 완료

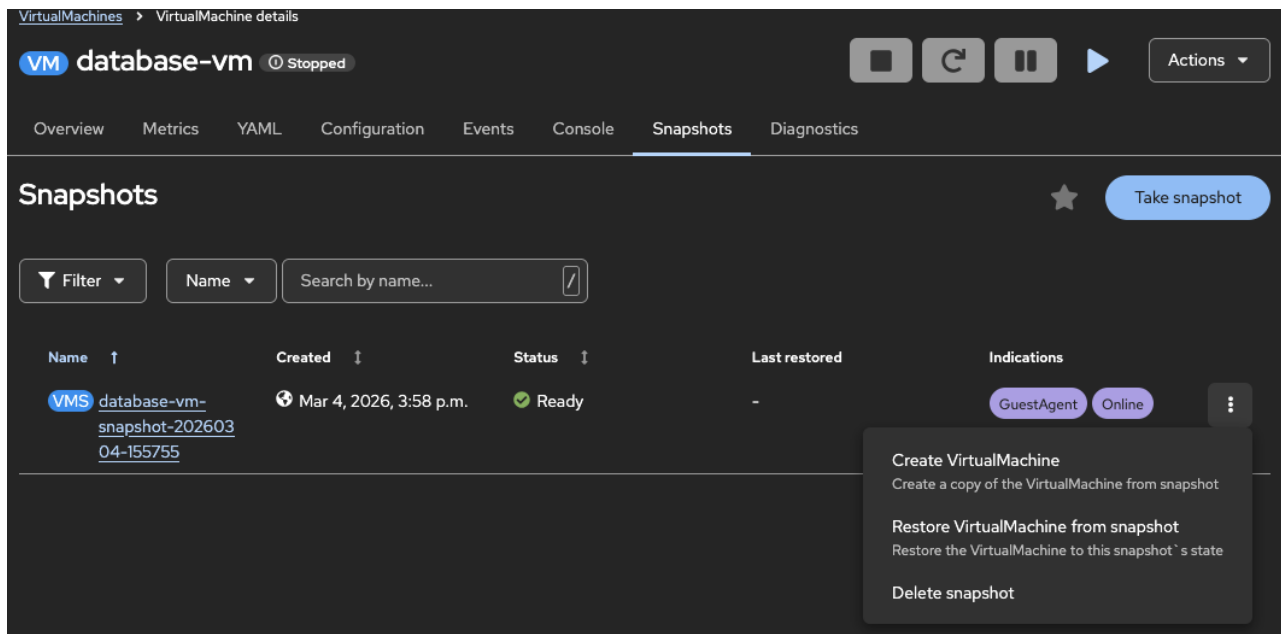
데이터 백업 완료 VM 데이터 백업 완료 데이터 백업 완료



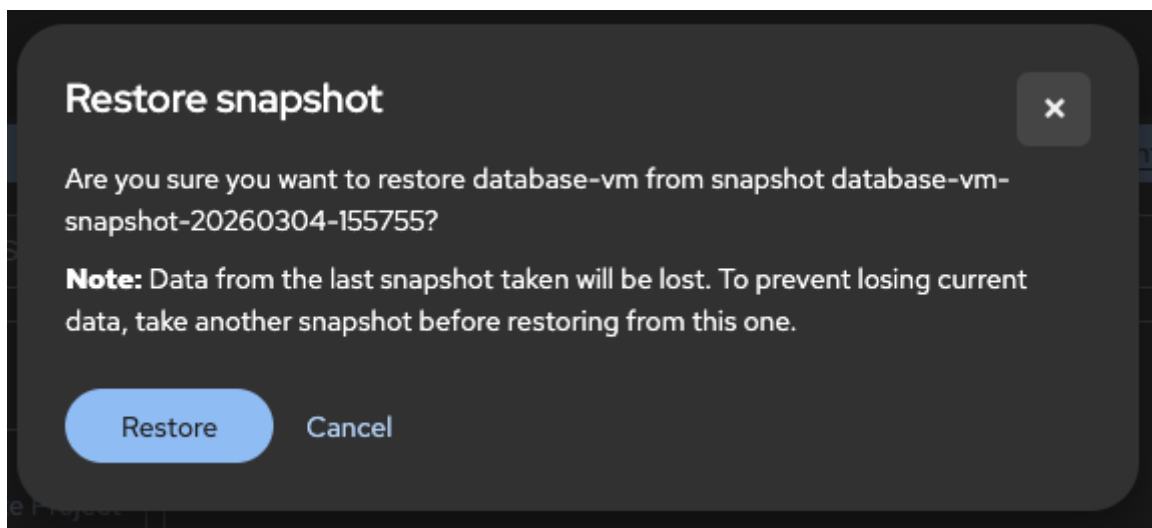
데이터 백업 완료 VM 데이터 백업 완료 VirtualMachineRestore spec targetReadinessPolicy: StopTarget 데이터 백업 완료 FailImmediate (데이터) WaitGracePeriod (5 초) WaitEventually (데이터) 데이터 백업 완료 데이터 백업 완료

**Web** 데이터 백업 완료

1. VM 데이터 백업 완료
2. VM 데이터 백업 완료 (데이터 백업 완료) 데이터 백업 완료
3. **Snapshots** 데이터 백업 완료
4. 데이터 백업 완료 (3 초) 데이터 백업 완료
5. **Restore VirtualMachineSnapshot** 데이터 백업 완료



6.



CLI

VirtualMachineRestore

vm-restore.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineRestore
metadata:
 name: database-vm-restore-01
 namespace: vm-snapshots
spec:
 target:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: database-vm
 virtualMachineSnapshotName: database-vm-snapshot-01
```

確認コマンド

```
oc apply -f vm-restore.yaml
```

確認

仮想マシンリストを確認

```
oc get virtualmachinerestore -n vm-snapshots
```

結果:

NAME	TARGETKIND	TARGETNAME	COMPLETE	RESTORETIME	ERROR
database-vm-restore-01	VirtualMachine	database-vm	true	2m	

仮想マシンを再起動:

```
oc describe virtualmachinerestore database-vm-restore-01 -n vm-snapshots
```

再起動されたVMを確認

```
virtctl start database-vm -n vm-snapshots
```

VMの状態を確認

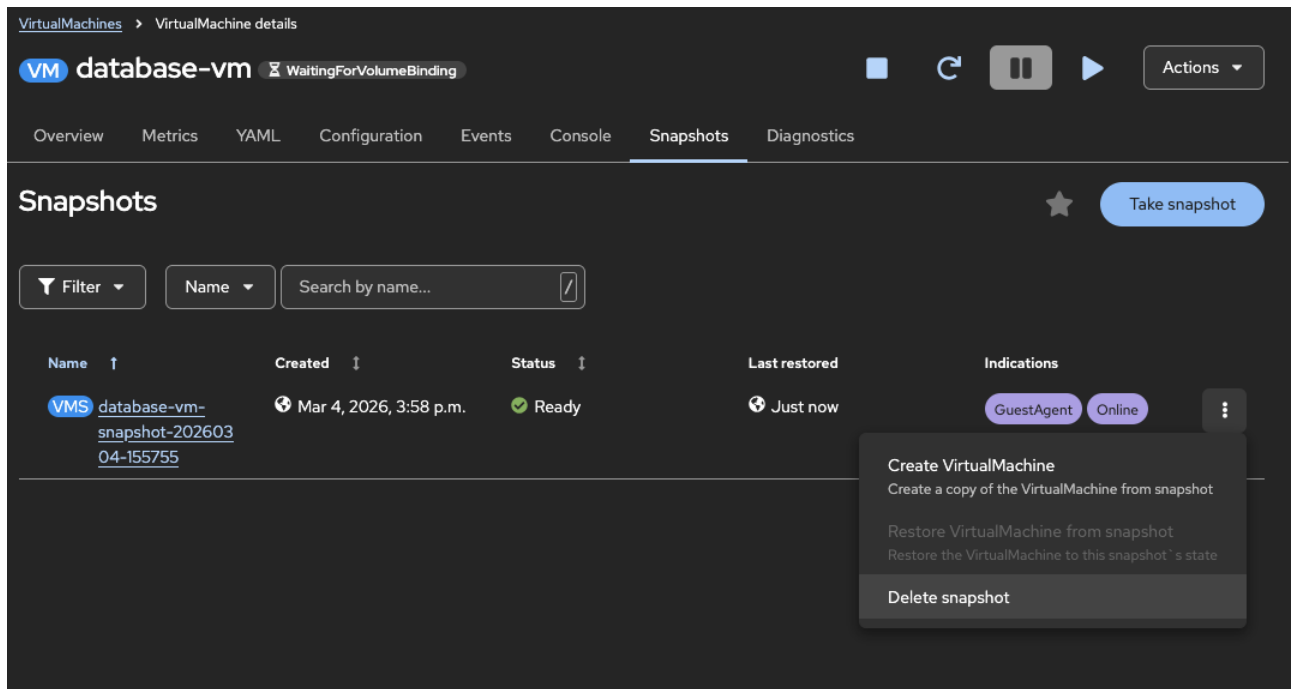
```
oc get vm database-vm -n vm-snapshots
```

確認コマンド

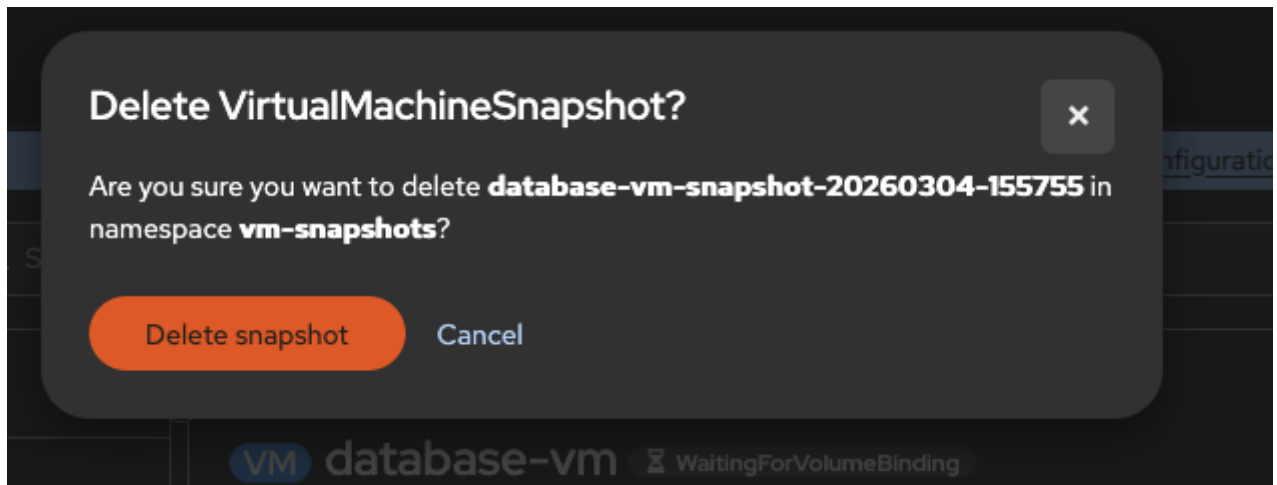
**Web** 操作手順

1. VM の **Snapshots** を確認
2. 仮想マシンを再起動
3. **Delete snapshot** を実行





#### 4. 削除



#### CLI 実行

```
oc delete virtualmachinesnapshot database-vm-snapshot-01 -n vm-snapshots
```



VirtualMachineSnapshot 削除 VirtualMachineSnapshotContent 削除  
VolumeSnapshot 削除

#### 関連項目

#### CSI 概要

VM 仮想マシン (Container Storage Interface) (Container Storage Interface)

- Red Hat OpenShift Data Foundation (ODF)

- AWS EBS CSI
- Microsoft Azure Disk CSI
- Google Cloud Persistent Disk CSI
- LVM Storage (LVMS) (aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)
- VMware vSphere CSI
- NetApp Trident
- Pure Storage
- Dell CSI aaaaaa

## VolumeSnapshotClass aaaa

VolumeSnapshotClass aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa snapshot class aaaaaaaaaaaaaaaaaaaaaa

```
aaaaaaaaaaaaaa
oc get storageclass

aaaaa snapshot class aaaa
oc get volumesnapshotclass
```

aaaaaaaaaaaaaaaaaaaaa snapshot class a storage provisioner aaaaaaaaaaaaaaaaaaaaaa

aaaaaaaaaa

aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

- aaaaaaaa: aaaaa CSI aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaaa: VM aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaaaaaaaaa: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaaaaaaaaa: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

```
oc get volumesnapshot -n vm-snapshots -o wide
```

aaaa:

NAME				READYTOUSE
SOURCEPVC	RESTORESIZE	SNAPSHOTCLASS	CREATIONTIME	AGE
vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk				true
database-vm-disk	30Gi	lvms-vg1	9m	9m

aaaaaaaaaaaaaa





# VM 安装指南

简介

OpenShift Virtualization 为 VM 提供了在 Kubernetes 集群中部署和管理的能力。OS 提供了虚拟机镜像和存储管理。

OpenShift Virtualization 支持 BIOS 和 UEFI 固件。OpenShift Virtualization 提供了虚拟机生命周期管理。

前提条件

- OpenShift Virtualization Operator 部署在 OpenShift 4.18 以上
- CLI 工具: `oc`, `virtctl`, `kubectl`
- VM 镜像文件
- Storage Class
- 镜像仓库: Quay.io 或 Docker Hub

## OpenShift Virtualization 部署

OpenShift Virtualization 部署步骤

配置项	说明	默认值
PersistentVolume Claim (PVC)	持久化存储	VM 镜像和 OS 镜像
ContainerDisk	容器镜像存储	VM 镜像和 CI/CD 镜像
DataVolume	Containerized Data Importer (CDI)	HTTP/S3 镜像仓库
CloudInitNoCloud	虚拟机初始化 (无云)	VM 镜像

部署步骤

OpenShift Virtualization 2.0 部署指南

### BIOS (Legacy Mode)

- 支持 PC BIOS 模式
- 支持 32 位和 64 位系统
- 支持 MBR 分区 (MBR 最大 2TB)
- 支持 Linux 操作系统

## UEFI (Unified Extensible Firmware Interface)

- 64-bit architecture
- GUID Partition Table (GPT) (2TB 이상)
- Windows 11, Windows Server 2022
- Secure Boot

### UEFI VM 설정

UEFI VM 설정 시 `bootOrder` 설정이 필요합니다. KubeVirt에서는 `bootOrder` 설정을 통해 VM의 부팅 순서를 지정할 수 있습니다.

예시:

- `bootOrder` 설정 (1, 2 순서)
- UEFI 설정
- `bootOrder` 설정
- `bootOrder: 1` 설정

## Chapter 1: PersistentVolumeClaim (PVC) 설정 (UEFI)

UEFI VM 설정 시 PVC 설정이 필요합니다.

namespace 설정

```
oc create namespace boot-demo
```



cloud-init 설정을 통해 SSH 접속이 가능합니다.

`vm-boot-from-pvc.yaml` 설정

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-pvc
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-pvc-disk
 spec:
 sourceRef:
 kind: DataSource
```

```

 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-pvc
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-boot-pvc-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□:

- :

□□□□□□:

- 451





### 例 3: HTTP を用いた DataVolume の作成

DataVolume を用いて HTTP/HTTPS からイメージをダウンロードし、VM にインストールする

[vm-boot-from-http.yaml](#) の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-http
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-http-disk
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
 spec:
 source:
 http:
 url:
 "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-http
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
```



```

kind: VirtualMachine
metadata:
 name: vm-multi-boot
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-multi-boot-primary
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-multi-boot-secondary
 spec:
 source:
 blank: {}
 storage:
 resources:
 requests:
 storage: 20Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-multi-boot
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: primarydisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: secondarydisk
 bootOrder: 2
 - cdrom:
 bus: sata

```

```

 name: installedcdrom
 bootOrder: 3
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-multi-boot-primary
 name: primarydisk
 - dataVolume:
 name: vm-multi-boot-secondary
 name: secondarydisk
 - name: installedcdrom
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

bootOrder:

1. **bootOrder: 1** - primarydisk (cloudinitdisk)
2. **bootOrder: 2** - secondarydisk (cloudinitdisk)
3. **bootOrder: 3** - CD-ROM (cloudinitdisk)

oc:

```
oc apply -f vm-multiple-boot-devices.yaml
```

oc:

- **oc (DR) oc**

- 安装 OS 虚拟机
- 安装 OS 虚拟机
- 安装操作系统虚拟机

## 第 5: UEFI 虚拟机

BIOS 虚拟机 UEFI 虚拟机 VM 虚拟机 Windows 虚拟机

**vm-boot-uefi.yaml** 虚拟机

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-uefi
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-uefi-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-uefi
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
```

```

 - masquerade: {}
 name: default
 rng: {}
 firmware:
 bootloader:
 efi:
 secureBoot: false
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-boot-uefi-disk
 name: rootdisk
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□:

- `firmware.bootloader.efi` □ UEFI □□□□□□□□□□
- `secureBoot: false` - □□□□□□□□□□ (□□□□□□□□ OS □□□□□□)
- □□□□□□ `pc-q35-rhel9.4.0` □ UEFI □□□□□□□□□□

□□□□:

```
oc apply -f vm-boot-uefi.yaml
```

□□□□□□:

- Windows 11 □□□□□□ Windows Server VM
- □□□□□□□□□□□□□□□□□□
- □□□□□□□□ (2TB □) □□□□□□
- □□□□□□□□□□□□□□
- UEFI □□□□□□□□□□ Linux □□□□□□□□□□□□

## 例 6: UEFI (セキュアブート)

この例では、UEFI セキュアブートを有効にするための VM 設定を示しています。

**vm-boot-uefi-secureboot.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-secureboot
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-secureboot-disk
 spec:
 sourceRef:
 kind: DataSource
 name: fedora
 namespace: openshift-virtualization-os-images
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-secureboot
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 1
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 rng: {}
 firmware:
 bootloader:
 efi:
```





## 例 7: PXE (PXE)

PXE (Preboot Execution Environment) 環境で VM を起動する OS

vm-boot-pxe.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
 name: vm-boot-pxe
 namespace: boot-demo
spec:
 dataVolumeTemplates:
 - apiVersion: cdi.kubevirt.io/v1beta1
 kind: DataVolume
 metadata:
 name: vm-boot-pxe-disk
 spec:
 source:
 blank: {}
 storage:
 resources:
 requests:
 storage: 30Gi
 runStrategy: RerunOnFailure
 template:
 metadata:
 labels:
 kubevirt.io/domain: vm-boot-pxe
 spec:
 domain:
 cpu:
 cores: 2
 devices:
 disks:
 - disk:
 bus: virtio
 name: rootdisk
 bootOrder: 2
 - disk:
 bus: virtio
 name: cloudinitdisk
 interfaces:
 - masquerade: {}
 name: default
 - bridge: {}
 name: pxe-net
 bootOrder: 1
 rng: {}
```





```

 rng: {}
 firmware:
 bootloader:
 bios:
 useSerial: true
 machine:
 type: pc-q35-rhel9.4.0
 memory:
 guest: 4Gi
 networks:
 - name: default
 pod: {}
 volumes:
 - dataVolume:
 name: vm-boot-menu-disk
 name: rootdisk
 - name: installedcdrom
 containerDisk:
 image: quay.io/containerdisks/fedora:latest
 - cloudInitNoCloud:
 userData: |-
 #cloud-config
 user: fedora
 password: fedora123
 chpasswd: { expire: False }
 ssh_pwauth: True
 name: cloudinitdisk

```

□□□□□□□□□□□□:

```

oc apply -f vm-boot-menu.yaml

VNC□□□□□□□□□□□□□□□□
virtctl vnc vm-boot-menu -n boot-demo

□□□□□□□□□□
virtctl console vm-boot-menu -n boot-demo

```

- □□□□ ESC □□□ F12 □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□
- **useSerial: true** □□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□□□□□

## VM 起動確認

```
VM起動確認
oc get vm -n boot-demo

VM起動確認
oc get vmi -n boot-demo
```

```
VM起動確認 (vm-boot-pvc起動VM起動確認)
oc describe vm vm-boot-pvc -n boot-demo
```

## 起動確認

```
VM起動確認 spec 確認 (vm-boot-pvc起動VM起動確認)
oc get vm vm-boot-pvc -n boot-demo -o yaml | grep -A 5 bootOrder

VM起動確認 (vm-boot-uefi起動VM起動確認)
oc get vm vm-boot-uefi -n boot-demo -o yaml | grep -A 10 firmware
```

## VM 起動確認

```
VM起動確認 (vm-boot-pvc起動VM起動確認)
virtctl console vm-boot-pvc -n boot-demo

VNC起動確認 (vm-boot-menu起動VM起動確認)
virtctl vnc vm-boot-menu -n boot-demo
```

## 起動確認

### VM 起動確認:

```
VM起動確認
dmesg | head -50

VM起動確認
systemd-analyze

VM起動確認
journalctl -b

UEFI起動確認
[-d /sys/firmware/efi] && echo "UEFI" || echo "BIOS"
```

## VM 启动顺序

步骤: 配置 VM 启动顺序

步骤: VM 的 `bootOrder: 1` 配置项

步骤:

- 配置 VM 的 `bootOrder` 属性
- 配置 VM 的 `bootOrder` 属性
- VM 的 `bootOrder`:

```
oc describe vmi vm-boot-pvc -n boot-demo
```

- 配置 VM 的 `bootOrder` 属性:

```
virtctl console vm-boot-pvc -n boot-demo
```

步骤: 配置 VM 启动顺序

步骤: `secureBoot: true` 配置 VM 启动顺序

步骤:

- OS 启动顺序
- 配置 `smm.enabled: true` 属性
- 配置 `smm.enabled: true` 属性
- 配置 `smm.enabled: true` 属性

```
VM 启动顺序配置
oc edit vm vm-boot-secureboot -n boot-demo
secureBoot: true 或 secureBoot: false
```

步骤: **PXE** 启动顺序

步骤: PXE 启动顺序 VM 启动顺序

步骤:

- PXE 启动顺序
- DHCP 或 PXE 启动顺序
- VM 启动顺序
- 配置 PXE 启动顺序 (DHCP/TFTP) 属性
- 配置 `bootOrder` 属性





- 10203 000000000000 (00000000)
- VM 00000000000000000000000000000000
- 00000000 BIOS/UEFI 000000000000000000000000

#### 0000000000

- 0000000000 UEFI 000000 (0000000000000000)
- BIOS 00000000000000000000000000000000
- 0000000000000000OS 00000000000000000000
- 0000000000000000 SMM 000000

#### 0000000000

- 000000000000000000 PVC 000000
- 0000 Immutable 0000000000 ContainerDisk 000000
- 000000000000000000 HTTP DataVolume 000000
- 00000000000000000000 ContainerDisk 000000000000

#### 00000000

- Windows 11 00000000000000000000000000000000
- 000000000000000000 BIOS 0000 UEFI 000000
- 00000000000000000000000000000000000000
- 00000000000000000000000000000000000000

#### 00000000

- 000000000000000000 virtio 00000000
- 00000000000000000000000000000000000000
- 000 Storage Class 000000 (0000000000 SSD)
- VM 000000000000 PVC 0000000000000000

#### 00000000

00000000000000000000000000000000000000:

```
0000VM0000
oc delete vm --all -n boot-demo

namespace0000
oc delete namespace boot-demo
```



VM 0000000000000000 DataVolume 0 PVC 00000000000000000000



目 次

目次は、本書の構成を YAML の形式で記述しています。

目 次

目次	章	目次
<a href="#">Storage</a>	7	LVM Operator のインストール
<a href="#">Networking</a>	8	LocalnetとUDNとBridge mapping のインストール
<a href="#">Virtual Machines</a>	14	VM のインストール
<a href="#">Services</a>	3	VM のインストールと Kubernetes のインストール

Storage

LVM Storage Operator のインストール

目次	Kind	目次	目次	目次
lvm-namespace	Namespa ce	openshift-lvm-storage namespace のインストール	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
lvm-operatorgroup	Operator Group	LVM Operator の namespace のインストール	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
lvm-subscription	Subscript ion	OperatorHub から LVM Storage Operator をインストール	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
lvmcluster-with- wipe	LVMClust er	インストール時に forceWipe オプションを LVM に指定	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
lvmcluster-clean- disk	LVMClust er	インストール (初回) 時に LVM のクリーンアップ	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
test-pvc	PVC	インストール時に PVC を作成	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>
test-pod	Pod	PVC をインストール時に Pod にインストール	<a href="#">LVM Operator</a> のインストール	<a href="#">YAML</a>

Networking

localnetとUDNとBridge mapping のインストール

目次	Kind	目次	目次	目次
localnet-secondary- namespace	Namespa ce	localnet のインストール VM namespace	<a href="#">CUDN</a> のインストール <a href="#">Localnet</a> のインストール	<a href="#">YAML</a>

名前	Kind	説明	関連リンク	ファイル
localnet-bridge-mapping-nncp	NNCP	localnet を localnet1 に bridge 接続	<a href="#">CUDN を Localnet で使用する</a>	<a href="#">YAML</a>
cuda-localnet	CUDN	localnet を ClusterUserDefinedNetwork	<a href="#">CUDN を Localnet で使用する</a>	<a href="#">YAML</a>
localnet-vlan-namespace	Namespace	VLAN localnet VM の namespace	<a href="#">NAD を VLAN localnet で使用する</a>	<a href="#">YAML</a>
localnet-vlan-bridge-mapping-nncp	NNCP	VLAN を localnet-vlan に bridge 接続	<a href="#">NAD を VLAN localnet で使用する</a>	<a href="#">YAML</a>
localnet-vlan-100-nad	NAD	VLAN 100 localnet の NetworkAttachmentDefinition	<a href="#">NAD を VLAN localnet で使用する</a>	<a href="#">YAML</a>
udn-primary-namespace	Namespace	UDN の namespace	<a href="#">UDN を使用する</a>	<a href="#">YAML</a>
udn-primary	UDN	VM の Layer 2 UserDefinedNetwork	<a href="#">UDN を使用する</a>	<a href="#">YAML</a>

## Virtual Machines

以下の VM を作成します。

名前	Kind	説明	関連リンク	ファイル
fedora-localnet-vm	VirtualMachine	localnet を Fedora VM	<a href="#">CUDN を Localnet で使用する</a>	<a href="#">YAML</a>
fedora-vlan-vm	VirtualMachine	VLAN 100 を Fedora VM	<a href="#">NAD を VLAN localnet で使用する</a>	<a href="#">YAML</a>
fedora-vm-with-udn	VirtualMachine	UDN を Fedora VM	<a href="#">UDN を使用する</a>	<a href="#">YAML</a>
fedora-dhcp-vm	VirtualMachine	DHCP を Fedora VM	<a href="#">IP 設定</a>	<a href="#">YAML</a>
fedora-multi-network-vm	VirtualMachine	DHCP + IP を Fedora VM	<a href="#">IP 設定</a>	<a href="#">YAML</a>
fedora-vlan-static-ip-vm	VirtualMachine	VLAN を IP を Fedora VM	<a href="#">IP 設定</a>	<a href="#">YAML</a>
golden-image-base-vm	VirtualMachine	ベース VM	<a href="#">イメージ</a>	<a href="#">YAML</a>
webserver-vm-from-golden	VirtualMachine	ベース VM	<a href="#">イメージ</a>	<a href="#">YAML</a>
webserver-vm-2	VirtualMachine	2 番目の VM	<a href="#">イメージ</a>	<a href="#">YAML</a>

이름	Kind	설명	참고 링크	파일 형식
fedora-access-vm	VirtualMachine	SSH 접속을 위한 Fedora VM	<a href="#">Linux</a> <a href="#">인스턴스 생성 가이드</a>	YAML
fedora-vlan-access-vm	VirtualMachine	SSH 접속을 위한 VLAN 기반 Fedora VM	<a href="#">Linux</a> <a href="#">인스턴스 생성 가이드</a>	YAML
fedora-app-vm	VirtualMachine	DNS 설정을 위한 Fedora VM	<a href="#">VM DNS</a>	YAML
golden-image-datasource	DataSource	인스턴스 생성을 위한 PVC 기반 DataSource	<a href="#">인스턴스 생성 가이드</a>	YAML

## Services

VM 인스턴스를 Kubernetes 클러스터에 배포

이름	Kind	설명	참고 링크	파일 형식
fedora-vm-web-service	Service	VM에 Web 트래픽을 위한 LoadBalancer 설정	<a href="#">UDN</a> <a href="#">인스턴스 생성 가이드</a>	YAML
ssh-nodeport-service	Service	VM에 SSH 접속을 위한 NodePort 설정	<a href="#">Linux</a> <a href="#">인스턴스 생성 가이드</a>	YAML
headless-service	Service	DNS 설정을 위한 Headless 설정	<a href="#">VM DNS</a>	YAML

## Usage

인스턴스 생성

1. YAML 파일을 클러스터에 업로드
2. 클러스터에 인스턴스 생성
3. `oc apply -f <filename>.yaml` 명령어를 실행

인스턴스 확인

```
oc apply --dry-run=client -f <filename>.yaml
```

## See Also

- [인스턴스 생성](#)
- [인스턴스 생성](#)
- [인스턴스 생성](#)
- [인스턴스 생성](#)
- [VM 인스턴스 생성 가이드](#)

- VM 0000000000

# API

## 

OpenShift Virtualization API API

API OpenShift Virtualization OpenShift Container Platform

## 

- API
- API
- API
- API
- API
- API
- API
- API

## 

- OpenShift Virtualization OpenShift v4.18
- kubeadmin
- REST API HTTP CRUD (GETPUTPOSTPATCHDELETE) JSON
- REST API (PostmancURL)

## 

API

OpenShift Virtualization API

## 

OpenShift Virtualization





- [Storage APIs](#)
  - [PersistentVolumes](#)
  - [PersistentVolumeClaims](#)
  - [VolumeSnapshots](#)
  - [DataVolumes](#)
  - [DataSources](#)
  - [StorageClasses](#)
  - [VirtualMachineSnapshots](#)
  - [VirtualMachineRestores](#)
- [Clone API](#)
  - [VirtualMachineClones](#)
  - [Mutating a Cloned VM](#)
  - [Patching a Cloned VM](#)
  - [Cloning Strategies](#) & [StorageProfiles](#)
- [Export API](#)
  - [VirtualMachineExports](#)

## API

### VirtualMachines

`VirtualMachine` is OpenShift Virtualization API that integrates with Kubernetes

`VirtualMachine` is `VirtualMachineInstance` (template) and `Status` (current state)

`VirtualMachine` has `spec` and `VM` (template) fields

- `instanceType` - `VirtualMachine` template `instanceType` (RAM & CPU configuration)
- `volumes` - `VirtualMachine` template `volumes` field contains `dataVolume` and `persistentVolumeClaims`
  - `dataVolume` - `VirtualMachine` template `DataVolume (s)` field contains `DataVolume (s)` and `dataVolumeTemplates`
  - `persistentVolumeClaim` - `PersistentVolumeClaim` template `VM` template
- `networks` - `VirtualMachine` template `networks` field contains `Pod/VM` network configuration
- `interfaces` - `VirtualMachine` template `interfaces` field contains `NIC` configuration `Pod/VM` network configuration
- `dataVolumeTemplates` - `VirtualMachine` template `DataVolume` field contains `DataVolume` and `volumes` configuration

`VirtualMachine` has `spec.runStrategy` field

- **Always** - 常に再起動する (失敗した場合) 常に VM を再起動
- **RerunOnFailure** - Always 再起動 VM 失敗した場合 (失敗した場合) 再起動
- **Once** - VM 一度だけ再起動する
- **Manual** - 手動で VM を再起動する POST 操作が必要
- **Halted** - `spec.runStrategy` 再起動 VM 再起動 `start` 再起動 VM 再起動 (再起動) 再起動



`spec.running` 再起動 `spec.runStrategy` 再起動

`virtctl` 再起動 `VirtualMachine` 再起動 `start` `stop` `restart` 再起動  
KubeVirt 再起動 VM 再起動 `../status` API 再起動

再起動 `VirtualMachine` 再起動 `runStrategy` 再起動

- **start** - VM 再起動 `Halted` run strategy 再起動 `Always` 再起動 `RerunOnFailure` 再起動 `Manual` run strategy 再起動
- **restart** - 再起動 VM 再起動 run strategy 再起動 `Halted` VM 再起動
- **stop** - VM 再起動 `Always` run strategy 再起動 `Halted` 再起動 `RerunOnFailure` 再起動 `Manual` run strategy 再起動

API 再起動 `Manual` run strategy 再起動 VM 再起動 `stateChangeRequests[]` 再起動 `VirtualMachine` 再起動

```
status:
 stateChangeRequests:
 - action: <Stop|Start> ①
 uid: <uid-of-the-virtual-machine-instance> ②
```

① 再起動 (Stop, Start) 再起動

② 再起動 `VirtualMachineInstance` (`VirtualMachine` 再起動) 再起動 `UID` VM 再起動 `VMI` 再起動 `Start` 再起動

`VirtualMachine` 再起動 API 再起動 VM `status` 再起動

- **status.printableStatus** - VM 再起動 `Running` (再起動/再起動) `Starting` (再起動/再起動) `Stopped` 再起動
- **status.runStrategy** - 再起動 run strategy 再起動 `spec.runStrategy` 再起動 `Always` `RerunOnFailure` `Once` `Manual` `Halted` 再起動
- **status.conditions["type": "Ready"]** - `conditions` 再起動: `type` - 再起動 (再起動 `Ready`) `status` - 再起動 `Reason` 再起動 `Message` - 再起動 `reason code` 再起動

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
```

```
metadata:
 name: vm-status-example
spec:
 ...
status:
 conditions:
 - lastProbeTime: "2025-11-10T15:00:46Z"
 lastTransitionTime: "2025-11-10T15:00:46Z"
 message: Guest VM is not reported as running
 reason: GuestNotRunning
 status: "False"
 type: Ready
```

## VirtualMachineInstances

## VirtualMachineInstance (VMI) OpenShift Virtualization VM

1. `VirtualMachineInstances` → `Kubernetes Pod` → `Pod` → `QEMU / KVM`  
 2. `VM` (PersistentVolumeClaim) → `PersistentVolumeClaim` (Persistent)

```
VirtualMachine → VirtualMachineInstance → VM → VM template → spec.template →
yaml (json) →
```

VM のデータは VMI を介して VM と共有され、VMI は DataVolumes (DataVolumes=PVC) を介して PV と共有される。

VM 000000000000 VM 1 yaml/json 00000000000000000000 VMI 00000000000000000000 VMI 0000000000

## Instance Types

OpenShift	Virtualization	CPU	
		(: u1)	
T	(smallmediumlargexlarge2xlarge)		
	(: u1.xlarge)		
(vs CPU )			

# OpenShift Virtualization

- **Network (N)** - DPDK CPU/RAM 1:2
- **Overcommitted (O)** - RAM CPU/RAM 1:4
- **Compute Exclusive (CX)** - CPU CPU/RAM 1:2
- **General Purpose (U)** - CPU CPU/RAM 1:4
- **Memory Intensive (M)** - CPU/RAM 1:8

VirtualMachineClusterInstanceType

## VirtualMachineClusterInstanceTypes

VirtualMachineClusterInstanceType      OpenShift  
Virtualization      VM      spec      ( )

VirtualMachineClusterInstanceTypes      spec      ( )

- **annotations** - annotations
- **cpu** ( ) - CPU
  - **dedicatedCPUPlacement** - VM vCPU CPU / **false**
  - **guest** ( ) - vCPU **0**
  - **isolateEmulatorThread** - VMI CPU / **false**
  - **maxSockets** -
  - **model** - VMI CPU **host-model**
  - **numa** - NUMA
    - **guestMappingPassthrough** - NUMA CPU NUMA { }
  - **realtime** - virt-launcher Pod VMI ( )
    - **mask** - vCPU "0-3,^1", "1,2,3", "1-2" libvirt
- **gpu** - GPU
  - **deviceName** - ""
  - **name** ( ) - GPU ""
  - **tag** -
  - **virtualGPUOptions** - vGPU
    - **display** - vGPU
      - **enabled** - vGPU **true**
      - **ramFB** - OS GPU
        - **enabled** - **true**
- **hostDevices** -
  - **deviceName** ( ) - ""
  - **name** ( ) - ""
  - **tag** -
- **memory** ( ) -
  - **guest** ( ) - OS ("4096Mi" "6Gi")
  - **hugepages** - VMI Hugepage
    - **pageSize** - Hugepage amd64 1Gi 2Mi
  - **maxGuest** - maxGuest guest

- **overcommitPercent** - 0% (virt-launcher Pod 0%)
- **nodeSelector** - Kubernetes selector label
- **schedulerName** - Kubernetes scheduler (K8s)

## VirtualMachineInstanceTypes

VirtualMachineInstanceType namespace (project/namespace)



specinstancetype Openshift Virtualization Web  
spec.template.spec.domain.cpu spec.template.spec.domain.memory  
CPU VirtualMachine  
VM VirtualMachineInstance  
2

## VirtualMachinePreferences VirtualMachineClusterPreferences

VirtualMachineInstanceTypes VirtualMachinePreferences  
(preference)VirtualMachine

spec VirtualMachineClusterPreferences

OpenShift Virtualization OS OS  
(fedora rhel10 win11) OS UEFI BIOS  
OS VM VirtualMachineClusterPreference

SecureBoot EFI VirtualMachineClusterPreference

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterPreference
metadata:
 name: rhel10-custom
spec:
 firmware:
 preferredUseEfi: true
 preferredUseSecureBoot: true
```



VirtualMachinePreference spec VM BIOS preferredUseEfi  
spec.firmware.preferredUseBios SecureBoot UEFI

## PreferredCPUTopology

VirtualMachinePreferences 1 VirtualMachineInstanceType vCPU OS  
(CPU) CPU vCPU

VM の CPU 構成を指定する (オプション) 構成を指定する CPU 構成を指定する

preferredCPUTopology オプションを指定する

- **sockets** (オプション) - vCPU 単位 OS 構成を指定する
- **cores** - vCPU 単位 OS 構成を指定する
- **threads** - vCPU 単位 OS 構成を指定する
- **spread** - vCPU 構成を指定する **spreadOptions** オプションを指定する (オプション)
- **any** - vCPU 構成を指定する OS 構成を指定する vCPU 構成を指定する preference (オプション) オプションを指定する

## vCPU SpreadOptions

preferredCPUTopology オプションを指定する **spread** オプションを指定する **spreadOptions** オプションを指定する vCPU 構成を指定する

**spreadOptions** オプションを指定する 2 オプションを指定する

- **across** - vCPU 構成を指定する
- **ratio** (オプション 2) - オプションを指定する (オプション N オプション)

**across** オプションを指定する

- **SocketsCores** (オプション) - vCPU 単位 1 オプションを指定する N オプションを指定する
- **SocketsCoresThreads** - vCPU 単位 1 オプションを指定する N オプションを指定する 2 オプションを指定する
- **CoresThreads** (2 オプション vCPU 単位) - vCPU 構成を指定する 1 オプションを指定する 2 オプションを指定する

オプション **preferredCPUTopology** オプション VirtualMachinePreference オプションを指定する

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachinePreference
metadata:
 name: cpu-topology-example
spec:
 cpu:
 preferredCPUTopology: spread
 spreadOptions:
 across: SocketsCoresThreads
 ratio: 6
```

## オプション API

オプション VirtualMachine オプション VirtualMachineInstances オプション API オプション Multus CNI オプション NMState Operator オプション VM/VMi オプション

VirtualMachine オプション 2 オプション (オプション) オプション **networks** オプション **interfaces** オプション

## Networks

VirtualMachine 对象包含 Pod 配置 (`spec.template.spec`) 的 `networks[]` 属性 ([]) 包含网络配置 (网络 `kind` 属性 Kubernetes 网络配置) 包含 VirtualMachine 网络配置 (网络) 属性

网络配置 `spec.template.spec.networks[]` 包含 2 个网络配置属性

- **pod** - 网络配置 Kubernetes 网络配置
- **multus** - Multus CNI 网络配置

## Interfaces

VirtualMachine 对象包含 VM 网络配置 (NIC) 属性 包含 Pod 网络配置 VM 网络配置 `virtio` 属性 (Windows 网络) 包含 OpenShift Virtualization 网络配置 (网络 NIC) 属性

网络配置 VirtualMachine 对象包含 `spec.domain.devices.interfaces[]` 属性 网络配置 `network` 属性

网络配置属性包含 2 个网络配置属性

- **name** (网络) - VirtualMachine 的 `spec.template.spec.networks[]` 属性 `network` 属性
- (网络) - 网络配置 `binding` 属性:
  - **bridge** - Linux bridge 网络配置
  - **sriov** - 网络 SR-IOV VF (`vfi` 网络) 网络配置
  - **masquerade** - 网络 NAT 网络 IP 网络 (`nftables` 网络) 网络配置
  - **binding** - 网络 (网络) 网络配置:
    - **name** - 网络配置:
      - **l2bridge** - VM `interface` 网络 `network` 网络 Layer 2 网络配置
      - **passt** (网络) - Plug a Simple Socket Transport Pod 网络配置 OpenShift 网络配置
- **model** - 网络配置 (10/100 网络配置)
  - **e1000** - Intel 82545EM PCI 网络配置
  - **e1000e** - Intel 82574 PCI-E 网络配置
  - **virtio** - 网络配置 Linux 网络 (网络) 网络配置
- **macAddress** - 网络配置 MAC 网络
- **ports** - VM 网络配置
  - **name** - 网络配置
  - **port** - 网络 (0 网络 65535 网络)
- **pciAddress** - `0000:81:00.1` 网络配置 PCI 网络

VM 网络配置





OpenShift Virtualization

- **bridge** - Pod network namespace is connected to the host network namespace
- **host-device** - Pod network namespace is connected to the host network namespace
- **ipvlan** - Pod has its own MAC address (VPC interface) and is connected to the host network namespace
- **macvlan** - Pod has its own MAC address and is connected to the host network namespace
- **SR-IOV** - single root I/O virtualization (SR-IOV) interface is connected to the host network namespace

OpenShift (Kubernetes) uses Multus to create NetworkAttachmentDefinition objects in the project/namespace.

OpenShift (Kubernetes) uses the `cluster` Network namespace to create `spec.additionalNetworks[]` objects in the project/namespace. NetworkAttachmentDefinition objects are created in the NetworkAttachmentDefinition namespace.

## NMState Operator

NMState ("NM" is NetworkManager) is a Linux (RHEL, Fedora, CentOS, Ubuntu, etc.) Linux distribution that provides a NetworkManager API.

NMState Operator uses the API (NodeNetworkConfigurationPolicy) to create Kubernetes Operator objects in the NMState namespace (Kubernetes/OpenShift) and create NetworkAttachmentDefinition objects.

Multus uses the `linux-bridge` interface to create Pod (VM) Bridge Interface objects in the NMState Operator namespace.

Multus uses the `NetworkAttachmentDefinition` interface to create Pod/VM objects in the Pod/VM namespace.

## OpenShift API

OpenShift Virtualization uses the `Kind` API to create Kubernetes objects in the OpenShift namespace.

OpenShift Virtualization uses the `API` to create Kubernetes objects in the OpenShift namespace.

- **VolumeSnapshotContents** - VolumeSnapshotContents objects are created in the VolumeSnapshot namespace.
- **VirtualMachineSnapshotContents** - VirtualMachineSnapshotContents objects are created in the VirtualMachineSnapshot namespace.
- **VolumeSnapshotClasses** - StorageClasses objects are created in the VolumeSnapshotClasses namespace.

OpenShift Virtualization uses the `API` to create Kubernetes objects in the OpenShift namespace.

Virtualization と Kubernetes の関係性を整理する

Kubernetes API (Kubernetes API) と Kubernetes (Kubernetes) の関係性を整理する

## PersistentVolumes

PersistentVolume (PV) は Kubernetes の PersistentVolumeClaim (PVC) と Pod の PersistentVolume (PV) の関係性を整理する (ReadWriteOnce/ReadWriteMany など)の関係を整理する

## PersistentVolumeClaims

PersistentVolumeClaim (PVC) は Pod の PersistentVolume (PV) の関係性を整理する Kubernetes の PersistentVolume (PV) と PVC の 1:1 の関係を整理する PV は ReadWriteMany の関係を整理する PVC は ReadWriteOnce/ReadWriteMany の関係を整理する

OpenShift は Kubernetes の (SAN / NAS) PersistentVolumes の関係を整理する PersistentVolumes の関係を整理する StorageClasses PersistentVolumes の関係を整理する

OpenShift Virtualization は DataSources と DataVolumes の関係を整理する kubernetes の PV と PVC の関係を整理する VirtualMachines の関係を整理する

StorageClasses は PersistentVolumeClaim の関係を整理する dataSource は VolumeSnapshot の関係を整理する PVC は dataSource の関係を整理する PVC は PVC の関係を整理する

## VolumeSnapshots

VolumeSnapshots は PersistentVolume の関係を整理する VM の関係を整理する PersistentVolume の関係を整理する

VolumeSnapshots は VirtualMachine の関係を整理する (KubeVirt) VirtualMachineSnapshots の関係を整理する KubeVirt DataVolume は DataSource の関係を整理する PersistentVolume の関係を整理する

## StorageClasses

StorageClass は PersistentVolumeClaim の関係を整理する PersistentVolumes の関係を整理する OpenShift の Kubernetes の関係を整理する

StorageClasses は CSI の関係を整理する OpenShift (Amazon Elastic Block Store など) API の関係を整理する kubernetes API (PV と PVC) の関係を整理する

StorageClasses は OpenShift Virtualization の関係を整理する OpenShift Virtualization の関係を整理する

## DataVolumes

DataVolumes 与 PersistentVolumeClaims 一起使用以声明和请求持久化存储资源

DataVolumes 由 VM 使用以声明所需的 PVC。DataVolumes 由 PVC 通过 Containerized Data Importer (CDI) 创建。

DataVolumes (通过 CDI) 支持两种格式 (qcow2 和 raw) 的 PVC。DataVolumes 支持 3 种不同的数据源。

- 本地存储
- 网络 (Web 存储) (网络存储)
- 本地存储 (本地存储 PVC)

CDI (通过 DataVolumes) 支持以下数据源：

- **blank** - 本地存储
- **gcs** - GCS 存储
- **http** - HTTP/S URL 存储
- **imageio** - ovirt-imageio 存储
- **pvc** - 本地 PersistentVolumeClaim 存储
- **registry** - 本地 registry 存储
- **s3** - AWS 存储 S3 (MinIO) 存储
- **snapshot** - 本地 VolumeSnapshot 存储
- **upload** - 本地 upload 存储
- **vddk** - VMware 存储

## DataSources

DataSources 通过 DataVolume 声明持久化存储资源。PersistentVolumeClaim 通过 VolumeSnapshot 声明持久化存储资源。OpenShift Virtualization Web 存储 DataSource 通过 Bootable Volumes 声明持久化存储资源。

DataSource **spec** 支持以下 1 种格式：

- **source** - **pvc** 和 **snapshot** (本地存储)
  - **pvc** 和 **snapshot** - 本地 PersistentVolumeClaim 和 VolumeSnapshot 存储
  - **name** - pvc 和 snapshot 名称
  - **namespace** - pvc 和 snapshot 命名空间

## VirtualMachineSnapshots

VirtualMachineSnapshot 由 VM 使用以声明所需的/持久化存储资源。

VirtualMachineSnapshot 对象与 VolumeSnapshot 对象 namespace 相同  
VirtualMachineSnapshot 对象 Status 属性

VirtualMachineSnapshot 对象的 **source** VM 对象的 **1m3600s1h30m** 属性  
对象的 **failureDeadline** 属性 5m

## VirtualMachineRestores

对象 (对象) 对象 VM  
VirtualMachineRestore 对象

VirtualMachineRestore 对象 VM 的 **apiGroup****kind****name** 属性 **target** VM 对象的 **virtualMachineSnapshotName** 属性

## API

VM 对象 3 个

- VM 对象 (VolumeSnapshots 对象) PVC 对象 VM 对象 VirtualMachine 对象
- VirtualMachineSnapshot 对象 (对象 VirtualMachine 对象 VM/对象 MAC 对象)
- API 对象 VirtualMachine (对象 VirtualMachineSnapshot) 对象 VirtualMachineClone 对象

VirtualMachine 对象 **VirtualMachineClones** 对象

## VirtualMachineClones

API 对象 VirtualMachineClone 对象 VirtualMachine 对象 VM 对象 (对象 VM 对象)

对象 VirtualMachineClone 对象的 **source** 属性 (对象) **target** 对象 VirtualMachine 对象的 **apiGroup****kind****name** 属性

对象 VM 对象

VM 对象 (**labels** 对象/对象 **annotations**) MAC 对象 BIOS / UEFI 对象 VM 对象 CPU 对象 RAM 对象 VM 对象

对象 (对象) 对象

- **labelFilters** - 对象 VM 对象 label (**metadata.labels** 对象) 对象 VM 对象
- **annotationFilters** - 对象 VM 对象 annotation (**metadata.annotations** 对象) 对象 VM 对象
- **template** - VM / VMI **template** 对象 2 对象
  - **labelFilters** - 对象 VM 对象 **template.metadata.labels** 对象 VM 对象 label 对象

- **annotationFilters** - 指定 VM の `template.metadata.annotations` フィルタ VM の annotation をフィルタリング/変更する

- **newMacAddresses** - 指定 VM の `interface` 指定 MAC 指定 `<interface-name>` 指定 (00-01-ff-00-01-00) MAC 指定 1 から 1 まで指定

- **newSMBiosSerial** - 指定 SMBios 指定

- **patches** (v4.20+) - 指定 VM の `patches` 指定 json 指定



- 指定 (label-or-annotation-name/\*) 指定 (!label-or-annotation-name) 指定
- 指定 VM の MAC 指定

指定 `patches` 指定 `op` 指定 `path` 指定 `value` 指定 json 指定 (json 指定) 指定

指定 `VirtualMachineClone spec` 指定

```
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
 name: example-vm-clone
 namespace: my-namespace
spec:
 source:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: my-virtual-app
 target:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: my-clone-app
 labelFilters:
 - "metadata-labels-to-include/*"
 - "!metadata-labels-to-omit/*"
 annotationFilters:
 - "metadata-annotations-to-include/*"
 - "!metadata-annotations-to-omit/*"
 template:
 labelFilters:
 - "template-labels-to-include/*"
 - "!template-labels-to-omit/*"
 annotationFilters:
 - "template-annotations-to-include/*"
 - "!template-annotations-to-omit/*"
 newMacAddresses:
 nameOfInterface: "ff-00-00-bb-aa-aa"
 newSMBiosSerial: "S3R14LNUM83R"
 patches:
```

```
- '{ "op": "add", "path": "/metadata/labels/my-app", "value": "cloned" }'
```

```
- '{ "op": "replace", "path": "/spec/instanceType/name", "value": "u1.xlarge" }'
```

VirtualMachine

VirtualMachine

VirtualMachineClone

VirtualMachineClone

```
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
 name: missing-parent-patch-example
 namespace: my-namespace
spec:
 source:
 apiGroup: kubevirt.io
 kind: VirtualMachine
 name: example-source-vm
 patches:
 - '{ "op": "add", "path": "/spec/template/spec/domain/cpu", "value": {} }'
```

```
- '{ "op": "add", "path": "/spec/template/spec/domain/cpu/cores", "value": 6 }'
```



- VirtualMachine `cpu` `memory` `spec.instanceType`
- `spec.instanceType` VM UI
- CPU `InstanceTypes` CPU `VirtualMachine[Cluster]Preferences`
- `spec.instanceType` VM `cpu` `memory` VM

StorageProfiles

OpenShift Virtualization `cloneStrategy` (OpenShift/Kubernetes `StorageClass`) `OpenShift Virtualization` (CDI) `StorageClass` (CSI) `StorageProfile` `StorageProfile` `StorageClass`



CDI `StorageClass` `StorageProfile` `StorageClass` `DataVolumes` `PVC`

OpenShift Virtualization `VirtualMachine` `VM` (VolumeSnapshot) `PVC` `DataVolumes`

OpenShift Virtualization (CSI)

OpenShift/Kubernetes CSI CoW (Copy on Write)

AWS EBS fast snapshot restoration (GitHub Issue)

StorageProfile cloneStrategy

- snapshot (CSI VolumeSnapshotClass CDI)
- copy Pod Pod
- csi-clone CSI API StorageClass StorageProfile

cloneStrategy csi-clone StorageProfile

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: StorageProfile
metadata:
 name: example-provisioner
spec:
 claimPropertySets:
 - accessModes:
 - ReadWriteOnce
 volumeMode: Filesystem
 cloneStrategy: csi-clone
```

## API

OpenShift Virtualization VM API

VM OS OpenShift API IngressRouteNodePort



Ingress Route virtctl port-forward export proxy server

## Export Tokens

VM namespace export token K8s export proxy endpoint (x-kubevirt-export-token)



12

export token

```
apiVersion: v1
kind: Secret
metadata:
 name: export-token
stringData:
 token: abc098765421def
```

VirtualMachineExport

## VirtualMachineExports

VirtualMachineExport  
VirtualMachineVirtualMachineSnapshotPersistentVolumeClaim

export token tokenSecretRef source

VirtualMachine VM yml VM PVCDataVolumesMemoryDump  
volume VirtualMachineSnapshot PVC VM

VirtualMachineExport  
URLVM

4

- **raw** - qcow2
- **gzip** - gzip
- **dir** - PVC URL /dir /path/to/file.txt
- **tar.gz** - gzip tar PVC

- **KubeVirt content-type** - PVC KubeVirt raw gzip
- **Archive content-type** - PVC KubeVirt dir tar.gz

VirtualMachineExport

```
apiVersion: export.kubevirt.io/v1beta1
kind: VirtualMachineExport
metadata:
 name: export-example ①
spec:
 tokenSecretRef: export-token ②
 source: ③
 apiGroup: "kubevirt.io"
 kind: VirtualMachine
```



name: virtual-machine-example

- ① 指定 VirtualMachineExport の名前
- ② export token を指定
- ③ 指定/指定 kind を指定

指定/指定

apiGroup	kind
kubevirt.io	VirtualMachine
snapshot.kubevirt.io	VirtualMachineSnapshot
""	PersistentVolumeClaim

# Glossary

## OpenShift Virtualization

### CNV

Container Native Virtualization (CNV) Red Hat OpenShift Virtualization

### NAD

NetworkAttachmentDefinition (NAD) Pod VM

### NNCP

NodeNetworkConfigurationPolicy (NNCP) NMState (CR)

### UDN

User Defined Network (UDN) 2 3

### VLAN

Virtual LAN (VLAN) VLAN

### OVS

Open vSwitch (OVS)

### IPAM

IP Address Management (IPAM) IP

### PVC

PersistentVolumeClaim (PVC)

### DataVolume

VM CDI (CR)

### VMI

VirtualMachineInstance (VMI) VM

### SCC

SecurityContextConstraint (SCC) OpenShift Pod

### RBAC

Role-Based Access Control (RBAC) Kubernetes

### Localnet

### cloud-init

VM

## Virtctl

VM [CLI](#) [CLI](#)

### 📄

- [OpenShift Virtualization](#) [CLI](#)
- [CLI](#) - [CLI](#)